December 2023 Marcelo de Freitas Marreiros FMdeploy - Simplifying Machine Learning Deployment for Academia via Web Interfaces

Marcelo de Freitas Marreiros

# FMdeploy - Simplifying Machine Learning Deployment for Academia via Web Interfaces

December 2023

**Universidade do Minho**
Escola de Engenharia

Marcelo de Freitas Marreiros

# FMdeploy - Simplifying Machine Learning Deployment for Academia via Web Interfaces

Dissertation
Integrated Master's in Biomedical Engineering

Dissertation supervised by
**Victor Manuel Rodrigues Alves**

December 2023

# DECLARATION

**Name**: Marcelo de Freitas Marreiros

**Dissertation Title**: FMdeploy - Simplifying Machine Learning Deployment for Academia via Web Interfaces

**Supervisor**: Victor Manuel Rodrigues Alves

**Conclusion Year**: 2023

**Master Designation**: Integrated Master's in Biomedical Engineering

**Master Branch**: Medical Informatics

University of Minho, 06/12/2023

Signature: Assinado por: **Marcelo de Freitas Marreiros**
Num. de Identificação: 15663968

# Acknowledgments

Throughout the writing of this dissertation, I received plenty of support and assistance.

First, I would like to thank my supervisor, Professor Victor Alves, whose expertise was invaluable throughout this project's stages. Your perceptive feedback pushed me to sharpen my thinking and brought my work to a higher level.

I want to acknowledge my colleagues from my internship at Tlantic for their outstanding collaboration. I want to single out my supervisor at Tlantic, Martinho Braga. Martinho, thank you for your patient support and the opportunities given to further my research.

I also want to thank my friends who provided stimulating discussions and happy distractions to rest my mind outside my research.

In addition, I leave a special thanks to my family for their wise counsel and sympathetic ear.

# STATEMENT OF INTEGRITY

I hereby declare having conducted this academic work with integrity. I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration.

I further declare that I have fully acknowledged the Code of Ethical Conduct of the University of Minho.

University of Minho, 06/12/2023

Signature: Assinado por: **Marcelo de Freitas Marreiros**
Num. de Identificação: 15663968

(PAGE INTENTIONALLY LEFT BLANK)

# ABSTRACT

Medicine and healthcare are dynamic, continually advancing fields with much-untapped potential. A critical driving force for new knowledge in medicine is the explosive development of technology, specifically computers. Within a few decades, computers have transitioned from room-filling giants performing basic tasks to portable devices capable of executing complex operations. At the intersection of technology, information, and healthcare, a new medical discipline emerged: Medical Informatics. This discipline harnesses technology to address healthcare challenges, leveraging artificial intelligence as a potent tool. Machine learning, a subfield of artificial intelligence, has become a critical part of clinical diagnosis and decision-making and an active research area.

The machine learning workflow for healthcare applications has two main challenges: collaboration and deployment. First, in the development phase, there is a lack of communication and synergy between researchers and domain experts, such as medical practitioners, that stems from the lack of proficiency of domain experts in machine learning and coding, thereby hindering their ability to provide essential feedback on model performance. Second, research predominantly focuses on model engineering. Even when research yields highly effective machine learning models, the process typically concludes at this stage, leaving the models in a state requiring complex steps for practical utilization by those who would benefit the most, namely domain experts.

To address these challenges, this research project has developed FMdeploy, a platform optimized for the academic context while remaining versatile for broader applications. FMdeploy is designed to facilitate the deployment of pre-trained Machine Learning Models. It includes user-friendly web interfaces, enhancing interdisciplinary collaboration. This dissertation results in two key deliverables: a web application and an accompanying API that empowers the application.

**Keywords:** Machine Learning Model Deployment, Pre-trained Models, API Integration, Web Application, User-Friendly Interfaces.

# RESUMO

A medicina e os cuidados de saúde são áreas dinâmicas e em constante evolução, com muito potencial por explorar. Uma força motriz fundamental para novos conhecimentos em medicina é o desenvolvimento explosivo da tecnologia, especificamente dos computadores. Em poucas décadas, os computadores passaram de gigantes que enchem salas e executam tarefas básicas para dispositivos portáteis capazes de executar operações complexas. Na intersecção da tecnologia, informação e saúde, surgiu uma nova disciplina médica: a Informática Médica. Esta disciplina utiliza a tecnologia para enfrentar os desafios de prestação de cuidados de saúde, tirando partido da Inteligência Artificial como uma ferramenta potente. A aprendizagem de máquina, um subcampo da inteligência artificial, tornou-se uma parte essencial do diagnóstico e da tomada de decisões clínicas e, consequentemente, uma área de investigação ativa.

O fluxo de trabalho da aprendizagem automática para aplicações na área da saúde apresenta dois desafios principais: colaboração e colocação em produção. Em primeiro lugar, na fase de desenvolvimento, há uma falta de comunicação e de sinergia entre os investigadores e os especialistas na área, como os médicos, que decorre da falta de proficiência em aprendizagem automática e codificação, dificultando assim a sua capacidade de fornecer feedback essencial sobre o desempenho do modelo. Em segundo lugar, a investigação centra-se predominantemente no desenvolvimento dos modelos. Mesmo quando a investigação produz modelos de aprendizagem automática altamente eficazes, o processo termina normalmente nesta fase, deixando os modelos num estado que exige passos complexos para a sua utilização prática por aqueles que mais beneficiariam, nomeadamente os especialistas na área.

Para enfrentar estes desafios, o presente projeto de investigação desenvolveu o FMdeploy, uma plataforma otimizada para o contexto académico, mantendo-se versátil para aplicações mais abrangentes. O FMdeploy foi concebido para facilitar a colocação em produção de modelos de aprendizagem automática pré-treinados. Inclui interfaces web simples e acessíveis, melhorando a colaboração interdisciplinar. Esta dissertação resulta em dois elementos chave: uma aplicação web e uma API que a suporta.

**Palavras-Chave:** Colocação em produção de Modelos de aprendizagem Automática, Modelos Pré-treinados, Integração de API, Aplicação Web, Interfaces de fácil utilização.

# Table of Contents

# LIST OF FIGURES

# CODE SNIPPETS

# List of Tables

# LIST OF ABBREVIATIONS AND ACRONYMS

## A

| | |
|---|---|
| AI | Artificial Intelligence |
| ANN | Artificial Neural Network |
| ANSI | American National Standards Institute |
| API | Application Programming Interface |
| ASD | Autism Spectrum Disorder |
| ASGI | Asynchronous Server Gateway Interface |

## C

| | |
|---|---|
| CLI | Command Line Interface |
| CNN | Convolutional Neural Network |
| COD | Code On Demand |
| CORS | Cross-Origin Resource Sharing |
| CSS | Cascading Style Sheets |
| CT | Computer Tomography |

## D

| | |
|---|---|
| DBA | Database Administrator |
| DBMS | Database Management System |
| DDL | Data Definition Language |
| DL | Deep Learning |
| DML | Data Manipulation Language |
| DNN | Deep Neural Network |
| DNS | Domain Name System |
| DSR | Design Science Research |

## E

| | |
|---|---|
| EER | Enhanced Entity-Relationship |
| EHR | Electronic Health Records |

## F

| | |
|---|---|
| FHIR | Fast Healthcare Interoperability Resources |

## G

| | |
|---|---|
| GBDT | Gradient Boosted Decision Trees |
| GBM | Gradient Boosting Machines |
| GPU | Graphical Processing Unit |
| GUI | Graphics User Interface |

## H

| | |
|---|---|
| HDD | Hard Disk Drive |
| HL7 | Health Level Seven |
| HTML | Hypertext Markup Language |

## I

| | |
|---|---|
| ICT | Information and Communication Technology |
| IDE | Integrated Development Environment |
| ISO | International Organization of Standardization |
| IT | Information Technology |

## J

| | |
|---|---|
| JCR | Joint Research Centre |

## M

| | |
|---|---|
| MI | Medical Informatics |
| ML | Machine Learning |
| MLP | Multilayer Perceptron |
| MR | Magnetic Resonance |
| MRI | Magnetic Resonance Imaging |
| MVC | Model-View-Controller |
| MVCC | Multiversion Concurrency Control |

## O

| | |
|---|---|
| OB | Olfactory Bulb |
| ORM | Object Relational Mapper |
| OS | Operating System |
| OMOP CDM | Observational Medical Outcomes Partnership Common Data Model |

## P

| | |
|---|---|
| PITR | Point-In-Time Recovery |

## R

| | |
|---|---|
| REST | Representational State Transfer |

## S

| | |
|---|---|
| SPA | Single Page Application |
| SQL | Structured Query Language |
| SSD | Solid State Drive |

## T

| | |
|---|---|
| TFX | TensorFlow Extended |
| TLS | Transport Layer Security |

## U

| | |
|---|---|
| UI | User Interface |
| UM | University of Minho |
| URL | Uniform Resource Locator |
| UX | User Experience |

## W

| | |
|---|---|
| WSGI | Web Server Gateway Interface |

## X

| | |
|---|---|
| XGB | Extreme Gradient Boosting |
| XSS | Cross Site Scripting |

# 1 INTRODUCTION

**Key points:**

- Medical Informatics lies at the intersection of information, technology, and healthcare.
- Machine Learning and Deep Learning already play a significant role in medicine today.
- Deep learning is starting to outperform humans in some tasks.
- Despite the increasing research, few papers have a noteworthy contribution to clinical care.
- Machine learning models must be deployed to improve patient care.
- The main goal of this dissertation is to create a research platform that can deploy pre-trained machine-learning models and promote interdisciplinary collaboration.

## 1.1 CONTEXT

Medical informatics is a relatively recent discipline born from technology, information, and healthcare [1], [2]. In fact, the earliest references to any application of computers in medicine appeared in the 1950s, while other medical fields have existed for centuries [3]. The swift development of this discipline in the last decades correlates evidently with technological advancements in computers, information, and communication. As technology evolves and the needs, requirements, and expectations of human societies grow, so does the responsibility of this field to answer the increasing demands for higher-quality medicine and healthcare [1]. Fortunately, medical informatics has moved from hope to real-world application with proven results that put it in a position to be a fundamental active contributor [2], [4].

Some institutions view medical informatics only as a service. However, a science that handles how best to use the information to enhance healthcare is a better definition [5]. Simply put, medical informatics is the cross-sectional or bridging discipline committed to systematically processing data, information, and knowledge, forming one of the bases for medicine and healthcare today [1].

One of the most notable fields in Medical Informatics is Medical Imaging Informatics. Medical Imaging Informatics is a rapidly evolving field and a growing part of clinical care and medical research. The study of this field combines medical informatics and imaging, developing and adapting core techniques in informatics to enhance the use and application of imaging in healthcare and to derive new knowledge from imaging research. Medical Imaging has become such a valuable tool in modern healthcare that it is inconceivable to practice medicine without it. This is a consequence of frequently being the only in vivo set of processes and techniques that allow the visual and spatial representation of the human condition for diagnostic and research purposes [6]. Medical Imaging overlaps with many clinical disciplines, such as neurology, cardiology, surgery, and many more. Currently, almost every specialty produces many digital medical images. The significance of this field also translates to considerable attention in research, given its pervasive use in several clinical applications and the many refinements in modalities [7].

With the constant innovation of Artificial Intelligence (AI), particularly the current advancements of machine learning, applying advanced Deep Learning (DL) methods to medical image analysis, a critical part of clinical diagnosis and decision-making, has become an active research area in the medical industry and academia [8].

AI is the computer science discipline tasked with creating algorithms to solve problems that traditionally require human intelligence. Machine learning is a subfield of AI that enables computers to learn from data without extensive and strict instructions provided by humans. In classical machine learning, human experts identify and select relevant imaging features that best represent the visual data. Then, algorithms are used to classify the data based on the characteristics identified in the first step. Deep learning is a subfield of machine learning. In contrast with traditional machine learning, one of the benefits of deep learning is that it does not require image feature identification by a domain expert as a first step. Instead, the algorithm identifies the best features for classifying the data as part of the learning process [9].

Machine learning is a pattern recognition technique that, when applied to a dataset and knowledge of the data, can use algorithms to learn from the data and apply that knowledge to make a prediction. In medical imaging, the dataset can be a set of tumor images and the understanding of whether these tumors are benign or malignant. Applying the algorithm to a new tumor image predicts whether the tissue is benign or malignant. If the algorithm

optimizes its parameters to increase the number of test cases diagnosed correctly, it means the task is being learned [10].

Machine learning has been a target of an incredible amount of engagement in the last few years. The recent developments of deep learning started around 2009 when artificial neural networks (ANN) began surpassing other established models on several significant benchmarks. Deep neural networks (DNN) are now state-of-the-art machine learning models across numerous areas and are widely deployed in academia and industry [11].

The ANN is an algorithm inspired by the multilayered human brain introduced in the 1950s [12]. An artificial neural network connects layers with artificial neurons, also called nodes. These networks are composed of input and output layers with hidden layers. However, the ANN's ability to solve problems was limited by overfitting and vanishing gradient problems caused by low computing power and primarily insufficient learnable data. The driving forces of the recent breakthroughs in artificial neural networks are the increase in data accessibility, novel algorithms, and enhanced computing power with graphics processing units (GPUs), memory, and storage [12], [13]. These advancements expanded ANN into DNN by stacking multi-hidden layers with connected nodes between input and output layers. These inputs can be radiomic features extracted from images or, in the case of convolutional neural networks (CNN), the images themselves. ANN is generally associated with CNN to identify and extract features directly from images [14].

AI, in general, and DL, in particular, are starting to outperform humans in a growing number of tasks, from computer vision to speech recognition [13]. Accordingly, many applications that recently seemed to be years away from human performance have become viable. Most significantly, deep learning has been quite successful in numerous medical imaging applications [15]. In radiology, visual recognition using AI has produced lower error rates than the human observer since 2015 [14]. Even more exciting is that some studies indicate that computers can identify patterns beyond human perception [10].

## 1.2 MOTIVATION

With the progress in science and technology, there has been an explosion of research papers published on AI in healthcare, as Figure 1 suggests. From 2015 to 2019, the growth rate of publications reached 46.27%. Furthermore, technical reports by the Joint Research Centre (JRC), the European Commission's science and knowledge service, observe a definite increase in the scientific medical literature on AI and healthcare. It further reports accelerating publications because of the COVID-19 pandemic in 2020 [16]. This exponential growth pattern of healthcare-related AI research indicates that the number of publications will continue to grow. Accordingly, estimations predict that the publication volume may double every two years [17].

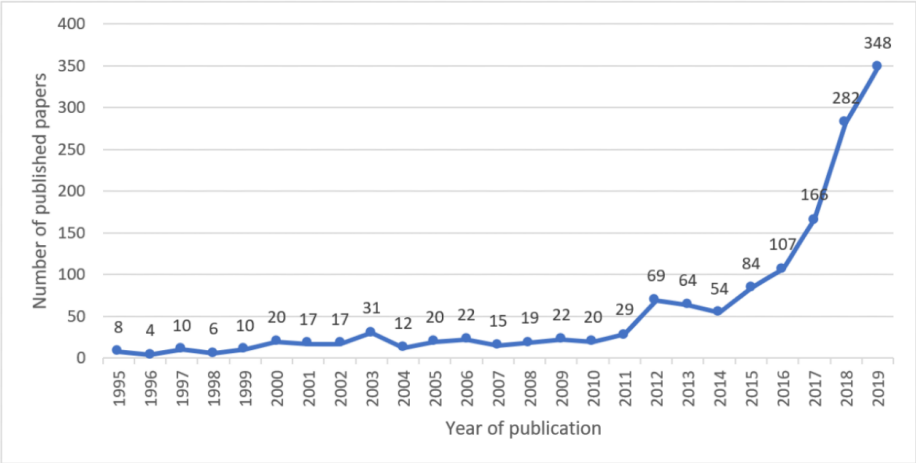



Figure 1 - Distribution of bibliographic records per year retrieved from [17].

All these publications translate to advancements in AI and healthcare. This progress, along with other breakthroughs in biotechnology and nanotechnology, is leading to an age of unparalleled rapid innovations characterized by the integration of various disciplines and technologies and a tightening of the gap between humans and machines [15], [18]. This period of rapid changes with evident implications in human society has already been termed the fourth industrial revolution [9], [19].

In several publications, numerous applications of AI to healthcare are explored [11], [12], [15], [20]. More specifically, applications of deep learning in healthcare range from one-dimensional biosignal analysis and the prediction of medical events, e.g., seizures and cardiac arrests, to computer-aided detection and diagnosis supporting clinical decision-making and survival analysis, to drug discovery and as an aid in therapy selection and pharmacogenomics, to increased operational efficiency, stratified care delivery, analysis of electronic health

records and many more [11]. Additionally, deep learning algorithms have been used in the context of medical imaging for several challenging tasks, such as pulmonary embolism segmentation with computed tomographic (CT) angiography, polyp detection with virtual colonoscopy or CT in the setting of colon cancer, breast cancer detection, and diagnosis with mammography, brain tumor segmentation with magnetic resonance (MR) imaging, and detection of the cognitive state of the brain with functional MR imaging to diagnose neurologic disease (e.g., Alzheimer disease) [10]. In medical imaging, classification, detection, and segmentation tasks are beginning to make their way into commercial applications. Consequently, these applications constitute a significant and growing financial market [9].

Medical informatics and Information and Communication Technology (ICT) have become central elements for the quality and efficiency of healthcare and have emerged as paramount contributors to the global ICT market. According to the US Census Bureau, the estimated total expenditure of ICT equipment and software in health care was about 28.2 billion US$ in 2013 [21]. Reports from the European Commission estimate that in 2006, the eHealth industry was worth around 21 billion EUR. Moreover, the same report suggested that the health ICT industry in Europe represents one-third of the global turnover of 50 to 60 billion EUR [22]. Multiple countries have invested between 50 million and 11.5 billion USD to stimulate ICT applications in the eHealth sector [1]. One report predicts that global spending on AI will grow from 50.1 billion USD in 2020 to around 110 billion in 2024 [23]. By 2025, research indicates that the AI healthcare market will grow to 36.1 billion USD [24]. These multi-billion-dollar investments have created a cycle of advancements and investments in this market.

Simultaneously, the number of companies applying data analysis to diverse fields has boomed, and predictably, some are turning their attention to healthcare problems [4]. Some examples are Google Brain, Deep Mind, Microsoft, and IBM [11].

Observing the global AI and health landscape, China is the leading player, followed by Europe and the USA. The characteristics of these regions are distinct. The EU has a robust research vocation, with pure research constituting almost two-thirds of all EU players, followed by China with about one-third, while the proportion of research in the US is relatively small. However, in the US, developments are strongly dominated by commercial firms. Regarding specialization, China focuses on diagnostics with the leading image recognition technologies, while in the EU and the USA, most action concentrates on diagnostics and health technology assessment. [16]

Despite all this research on applying machine learning to medical imaging, few papers have a noteworthy contribution to clinical care [4]. Many academic research papers do not translate to meaningful applications because the development stops after achieving the desired results with a machine learning model. Models must not only be developed but also deployed to improve patient care [25]. This deployment integrates trained models into the software infrastructure necessary to run it [26].

The deployment of such models is particularly complex since it faces the challenges of a regular software system in addition to specific machine learning configurations. Mixed in as a cross-cutting factor are the societal and ethical concerns underlying the use of AI. One of the crucial points to facing these problems and improving the deployment of AI solutions is the synergy between doctors, data scientists, engineers, and anyone involved in the process. Model development in an interdisciplinary setting between computer science and medicine could drastically improve their quality [16].

Lastly, most current tools and frameworks need more user-friendliness. Multiple tools still require refined technical skills, which limit usability and acceptance among non-expert users and domain experts such as physicians [27].

For all these reasons, a research platform where researchers can deploy machine learning models, share data, and collaborate without needing advanced technical skills is very sought after. To promote collaboration and kickstart the last step of the ML workflow, the deployment, this work aims to design and develop a platform to be integrated into the current workflow of research institutions like the University of Minho (UM).

## 1.3 OBJECTIVES

The main objective of this work is to promote collaboration in research settings and kickstart the ML deployment stage with the design and development of a simple and intuitive platform. To tackle the issue of machine learning model deployment, there are three main steps that need to be undertaken: conducting research on the challenges and existing solutions in this field, establishing a streamlined process for preparing models for integration, and designing and developing a solution through an iterative process. This solution will take the form of a web application with three central components: database, backend, and frontend.

Firstly, a suitable relational database will be chosen. Secondly, for the backend, a careful analysis of Flask and FastAPI will be made to determine the best option for the project. Both

technologies are widely used for building APIs with Python, and they have their own strengths and weaknesses. The best choice will be based on the project's specific needs, considering performance, scalability, and security. Once the most appropriate technology is chosen, it will be used to make the machine-learning code accessible through an API.

Thirdly, a frontend using ReactJS will provide an interactive and intuitive web interface to communicate with the API. Additionally, as this type of platform requires multiple iterations, a prototype of the project will be hosted on a private server from the University of Minho.

Lastly, all the code will be containerized using Docker. In order to develop an efficient solution for machine learning model deployment, the following steps will be taken:

1. Research state-of-the-art solutions and the current challenges facing ML deployment.
    a. Identify and study current challenges to ML deployment.
    b. Identify and study current solutions to ML deployment.
2. Perform a detailed study of Flask and FastAPI to determine the best option for the project by reading available documentation, gathering information from previous studies, and understanding their inner workings through online courses and tutorials.
3. Perform a detailed study of ReactJS – The study of the inner workings of ReactJS.
    a. First, thorough research of available documentation.
    b. Next, information will be gathered from previous web apps that use ReactJS.
    c. Thirdly, aim to understand how ReactJS works through tutorials and workflows.
4. Develop a web application using the chosen backend technology (FastAPI or Flask) and ReactJS for the front end.
    a. Identify the inputs and outputs required for the ML models and take appropriate steps for data normalization and standardization.
    b. Study the workflow of the backend technology (FastAPI or Flask) and determine the best way to run the ML models.
5. Web App: Test the web app with real data.
6. Containerization: Containerize and test the resulting work on University of Minho servers.

## 1.4 INVESTIGATION METHODOLOGY

The deployment of machine learning models in medical imaging presents several challenges. The design of a solution must have data scientists and less informed users in mind while delivering adequate performance. The service provided can impact user behavior by reducing the knowledge barrier needed to put these machine-learning models to good use. Additionally, expert users versed in these subjects have a new way to share the models they create. Therefore, there is a need to use a research method that aims to enhance technological and scientific knowledge by designing innovative artifacts [28].

The research methodology chosen to guide this dissertation was Design Science Research (DSR) since many researchers have used this iterative paradigm to develop information technology (IT) artifacts [29]. DSR has also proven effective in computer science and medical informatics research [30], [31]. DSR involves creating new knowledge through the design of new or innovative artifacts, analysis of use, and performance of these artifacts that address a specific problem. DSR establishes artifact creation and research iteration [29], [32].

A typical DSR implementation proceeds in the following phases: awareness of a problem, suggestion, development, evaluation, and conclusion [32]. Firstly, the guidelines for DSR were defined in the design as an artifact, problem relevance, design evaluation, research contributions, research rigor, design as a search process, and communication research [33]. This model divides awareness of the problem into "problem identification and motivation" and "solution objectives definition"; merges the suggestion and development phases into "design and development"; breaks evaluation into "demonstration" and "evaluation"; lastly, renames conclusion to "communication" [32]. These six activities are summarized in Table 1 and are included in the DSRM process model illustrated in Figure 2. Moreover, this model presents the concept that research initiation can stem from various contexts, namely problem-centered, objective-centered, design and development-centered, and client/context-centered initiations. This dissertation primarily centers around the problem-centric approach.

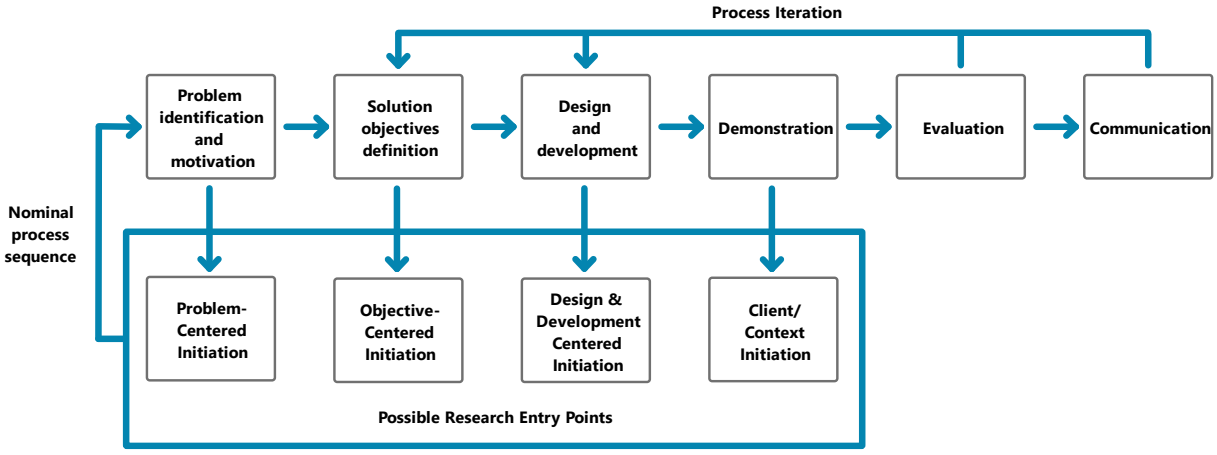



Figure 2 - DSRM Process Model.

The DSRM methodology can be divided into six main activities [30]:

Table 1 - DSRM activities.

| | |
|---|---|
| **Problem identification and motivation** | Identify the research problem and justify the value of a solution. The problem identification is crucial since it will lead to an artifact that can provide an appropriate solution. This activity requires knowledge of the state of the problem and the importance of a solution. |
| **Solution objectives definition** | Gather the objectives of a solution from the problem definition. This activity requires knowledge of the state of the problem and current solutions. |
| **Design and development** | Define the features and architecture and then build the artifact. The design research is embedded in the design of the artifact. This activity requires knowledge of the vital technologies and concepts necessary to develop a solution. |
| **Demonstration** | Experimentation, simulation, case study, proof, or other suitable activity to exhibit the use of the artifact to solve one or more instances of the problem. This activity requires knowledge of the tools the artifact provides to solve the problem. |
| **Evaluation** | Assessment of the efficacy of the artifact. This assessment involves comparing the objectives of the solution and existing solutions with the actual observed results from the use of the artifact in the demonstration. At the end of this activity, the researchers can decide whether to iterate back to activity three or leave further improvement to subsequent projects. |

| | |
|---|---|
| **Communication** | Share the problem and its significance, the artifact, its utility and novelty, the rigor of its design, and its effectiveness with researchers and other relevant audiences. |

## 1.5 STRUCTURE OF THE DISSERTATION

This dissertation is structured into five main chapters and supplemented by three appendices. The first chapter serves as an introduction, providing context, motivations, and objectives. The subsequent chapter presents a review of existing literature on ML deployment, addressing associated challenges and highlighting prevalent tools. Chapter 3 initiates the exploration of FMdeploy, the proposed platform, focusing on the planning phase. Supporting Chapter 3, appendices 2 and 3 offer additional insights into use cases and requirements. Chapter 4 delves into the design and development of the web application and API, the core components of FMdeploy. In Appendix 1, concepts and technologies relevant to FMdeploy are discussed. Lastly, Chapter 5 encompasses a synopsis, discussion, conclusions, and potential future work.

# 2 LITERATURE REVIEW

**Key points:**

- Integrating machine learning models into applications is crucial for generating real value and tangible insights.
- The deployment phase of machine learning development is pivotal, requiring careful consideration of challenges and solutions.
- Developing user-friendly and interactive visual interfaces can enhance the usability and accessibility of machine learning platforms.

This literature review chapter compiles essential research on ML deployment in the medical field, providing vital insights for guiding artifact design and development. This section explores advancements, challenges, and case studies in ML model deployment.

## 2.1 MACHINE LEARNING MODEL DEPLOYMENT

Understanding the function of a machine learning model is essential, but equally crucial is comprehending its role within the broader application. Whether employed for prediction, recommendation, classification, or segmentation, it should make the application more effective. For instance, consider the deployment of an ML model that can accurately segment brain tumors in medical images [34]. Regardless of its specific role, deployment is essential for the ML model to interact with the application.

Why is deploying the ML model so crucial? The answer is simple: it is a necessary step to extract real value from the model. Integrating the model within the application framework is essential for providing valuable insights and predictions, effectively assimilating it into the application. Failing to deploy a model is akin to training for a sports event but not participating in it, limiting the impact it could have had if embedded in the application [35].

Moreover, if there are numerous users, multiple instances of the same model should run in parallel to serve their requests [35]. It is essential to bear in mind that deploying an ML model does not ensure consistent prediction quality.

In conclusion, deploying an ML model is crucial to extracting its real value by integrating it into the application to generate tangible insights. This also means that predictions can be made in real-time, increasing the model's usefulness. However, deploying a model has its challenges.

## 2.2 MACHINE LEARNING WORKFLOW

The development of intelligent systems involves various tasks such as data collection and curation, as well as machine learning model development and tuning [36]. While the concept of applying machine learning to a specific use case appears straightforward—where the learner trains on a dataset and produces a model, and the model utilizes features to make predictions during inference—the actual workflow becomes more intricate when it comes to machine learning deployment [37].

Ashmore et al. provide a valuable framework for ML-based solutions comprising four stages: data management, model learning, model verification, and model deployment [38]. A critical stage is model deployment, which involves integrating the trained model into the necessary software infrastructure for execution.

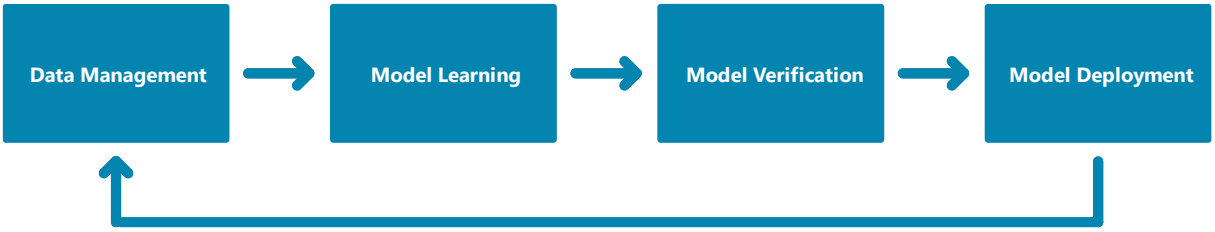



Figure 3 - Machine Learning workflow steps.

Figure 3 illustrates the steps in the machine learning workflow. It is essential to note that these stages may not always occur in a strict sequence; they may overlap or inform each other via feedback loops [26], [39]. For example, feedback from the deployment stage may inform changes to the data management or model learning stages and vice versa. Nonetheless, the workflow proposed by Ashmore et al. [38] provides a useful framework for understanding the different components of the machine learning development process.

To summarize, the process of deploying machine learning models is multifaceted and presents challenges for practitioners at each stage. This dissertation will primarily focus on the challenges specific to the deployment stage.

## 2.3 CHALLENGES

### 2.3.1 ACCESSIBILITY CHALLENGES IN COLLABORATIVE ML

Collaboration in machine learning is vital and requires active participation from domain experts, such as medical doctors, for instance, whose domain-specific knowledge greatly enhances model quality. The design of an ML model must, therefore, incorporate the

expertise of computer scientists and medical practitioners, leading to improved model quality and, consequently, better deployment efficiency [16]. To achieve this, it is essential to establish a dynamic interdisciplinary setting between computer science and medicine [40]–[42].

An illustrative example of successful synergy emerged among COVID-19 AI applications. Shan et al. demonstrated the effectiveness of a hybrid approach [43]. By combining the expertise of various teams, a hybrid system can effectively manage data science projects across multiple groups, leading to optimal utilization of resources and collaboration potential. This approach facilitates the exchange of skills and encourages innovation, which may not be possible through other means [44].

Drawing inspiration from the Society of Automotive Engineers' five levels of automation for vehicles, Topol discusses the collaboration between healthcare practitioners and technology [41]. According to this scale, Level 0 corresponds to no automation, where the human driver has full control, while Level 5 represents complete automation without human intervention. Topol predicts medicine will likely plateau at Level 3, involving conditional automation with human oversight over algorithm outputs, given the significant social and ethical implications [41]. This assessment emphasizes the need for ongoing collaboration between medical professionals and technology experts to ensure responsible AI development and deployment in healthcare.

Effective coordination between stakeholders is also a significant challenge during model deployment. Model deployment is a group task requiring constant communication and a shared understanding of the overall objective. Proper planning at the early stages of ML model development can better prepare the MLOps/DevOps teams for deployment [35].

To sum up, collaborative machine learning poses several challenges that hinder its successful implementation. Accessibility, in particular, is a significant challenge that can contribute to the complexity of the collaboration process. Typically, domain experts provide datasets for machine learning researchers to analyze, and they offer high-level feedback on the project's progress [45]. However, the lack of accessibility often limits the domain expert's ability to provide direct feedback on the model's performance, as they may lack the necessary ML and coding expertise to test the models during development. These challenges highlight the need for a more inclusive and accessible approach to collaborative machine learning, which will be critical to the success of future machine learning projects.

### 2.3.2 TRANSPARENCY AND TRUST CHALLENGES

Machine learning models have become increasingly popular in various fields, including medicine, where their predictions can significantly impact human lives [46]. However, their complexity and opaque decision-making processes have raised concerns regarding their trustworthiness and reliability. Transparency and interpretability are crucial for building trust in ML models, allowing domain experts to verify their accuracy and identify potential biases. These factors have been identified as a significant challenge, especially in medicine. To address this issue, the Explainable AI community has proposed using more interpretable models or developing visualization techniques to increase the interpretability of black-box models [47], [48].

Incorporating digital technology and AI into workflows can be challenging for practitioners in the medical field, particularly when ML is involved [49]. Medical professionals have historically resisted adopting such technology for various reasons, including a lack of trust, technophobia, uncertain liability, and fears that AI could replace human physicians [50]. This reluctance is further compounded by many AI tools being a black box to users. Concerns about the black-box nature of many ML models have persisted since the early days of artificial neural networks in the 1990s [51]. These issues have been highlighted in commentaries in prominent medical journals [52], and many ML scientists now acknowledge them as one of the primary barriers to the adoption of ML in medicine [53]. A cautionary example is the adoption of IBM Watson by an academic medical center for clinical decision-making, which cost approximately $62 million but was plagued by unexpected costs and learning difficulties due to a lack of prior use of IBM Watson for clinical decisions on actual patients [49].

The term "black box" refers to models characterized by a level of complexity that impedes their interpretation by humans. This differs from more straightforward models, such as linear and logistic regression, which are frequently employed in medical research and can be interpreted via their respective model coefficients. Not all machine learning algorithms fall into the category of black boxes; some are inherently interpretable, such as decision trees [54]. However, it is essential to recognize that specific models that are initially interpretable may become increasingly less so as their complexity increases [55]. This is true even for some state-of-the-art models, such as deep learning and ensemble methods (the latter encompassing gradient boosting and random forest) [55]. Despite the fact that these models

are founded upon interpretable mathematical functions [56], [57] their sheer complexity and scale render them opaque, necessitating the provision of explanations. Even with the use of advanced visualization tools to improve transparency, deep neural networks still pose a challenge regarding clarity and interpretation [58]. Nevertheless, results must be entirely understandable in many domains to be truly valuable [58], [59]. As a result, the low level of transparency of many model types remains an open challenge for achieving trust in machine learning [58], [60], [61].

In the healthcare sector, where numerous decisions directly affect life and death, the absence of interpretability in predictive models can erode confidence in such models [46]. Moreover, the lack of interpretability in ML models also limits clinical actionability, reducing its usefulness to healthcare practitioners [62]. Therefore, the challenges regarding interpretability have prompted a surge of research in the field of explainable machine learning. While the potential benefits of explainable machine learning are substantial, it is essential for medical professionals who may encounter these techniques in clinical decision-support tools or novel research papers to possess a discerning comprehension of their capabilities and constraints [55].

Explainability techniques can provide clear and comprehensive explanations of ML models to stakeholders, particularly end-users [63]. The overarching goal of explainability is to help users or developers of ML models comprehend the rationale behind their behavior [64]. Explainability, which has been touted as a means to enhance the transparency of ML models [65], is defined to include any method that aids this understanding [64]. Various forms of transparency can be employed, including informing patients of the symptoms that suggest a particular diagnosis [66], publishing an algorithm's code, or disclosing properties of the training procedure and datasets used [67]. However, users are typically unequipped to comprehend how unprocessed data and code translate into advantages or disadvantages that may impact them on an individual basis. In providing an account of how the model arrived at a decision, explainability techniques endeavor to furnish transparency that is explicitly tailored to human users, often with the objective of augmenting reliability [68]. These techniques aim to provide users with a high-level understanding of how ML models work and make decisions without getting bogged down in the details of their calculations [69]. This understanding is critical for users to make informed decisions about whether to follow the recommendations of a given model [53].

While it is common in the literature to observe the terms "interpretability" and "explainability" being employed interchangeably [25], [70]–[72], they are separate concepts, and conflating them can lead to confusion [73]. Although the definition of the term "interpretable" in the literature exhibits slight variability [54], [74]–[77], in this literature review, "interpretable" denotes models that can be comprehended directly by humans in terms of their operation and the causes of their decisions [78]. It is worth noting that interpretability is at least somewhat subjective since interpreting a model's decisions may necessitate expert knowledge of statistics or a specific domain [46].

Several studies have established the difficulties that end-users face in machine learning. For instance, Amershi et al. [39] and other researchers have reported issues such as users' inability to serve as accurate oracles to the system, resulting in wild fluctuations in the system's reasoning, as well as users' lack of trust in the logic behind such systems [79]–[84]. Non-experts often find using or trusting such complex systems challenging [36]. Valliani et al. [79] have identified a lack of interpretability and explainability as barriers to deploying deep learning in medical records [34]. Similarly, Xiao et al. [85] and Shickel et al. [86] have reported issues such as lack of model interpretability and transparency as challenges in deploying deep learning for EHRs [34].

In addition to the challenges of understanding and trust, the specialized and non-standardized terminology used in ML can be problematic for beginners in the field [59], [87]. This use of non-standardized language is compounded by the fact that standard ML metrics are often not easily understood by those without a background in ML, making it difficult for users to translate these metrics into meaningful outcomes [88].

In conclusion, while ML engineers increasingly use explainability techniques during development, significant challenges hinder their use in directly informing end-users. These challenges include the need for domain experts to evaluate explanations, the risk of spurious correlations in model explanations, the lack of causal intuition, and the latency in computing and displaying results in real time [64]. Regardless, these techniques hold great promise in addressing the opaqueness of black-box ML models, which has impeded their adoption in healthcare [89]. Moreover, empirical evidence suggests that explainability techniques can enhance clinicians' trust in black-box models [53]. Despite this potential, the limitations of explainability techniques must be acknowledged. Overall, explainable ML offers a promising opportunity to benefit from cutting-edge predictive methods, such as deep learning, while

avoiding the drawbacks of black-box models. However, much work still needs to be done to overcome the current limitations of these techniques.

### 2.3.3 USER FRIENDLINESS AND USER INTERFACES

Researchers are increasingly engaging in interdisciplinary collaborations, working closely with domain experts from various fields, including doctors, physicists, geneticists, and more [90]–[93]. Providing a simple user interface for deploying and monitoring with minimal configuration is essential to improve the usability and accessibility of machine learning platforms [37]. Moreover, non-technical individuals need user-friendly tools that allow them to apply models without prior data science knowledge [88]. These tools will increase the real-world impact of ML techniques [94]–[97].

However, most current tools require sophisticated technical skills, limiting usability and acceptance among non-specialist users and domain experts [27]. One effective solution to overcome this challenge is implementing an interactive and lightweight web interface. Another valuable approach is involving the end-users at an early project stage, as demonstrated by the Sepsis Watch project [98], which fosters user confidence in the end product. To overcome skepticism, trust-building mechanisms such as building strong communication channels, sharing progress toward the goal, and engaging front-line clinicians and enterprise-level decision-makers play pivotal roles [26].

As discussed in the previous chapter, the interpretability of models is not a foolproof trust-building tool, and alternative methods to attain high credibility with end-users should be explored [26]. Although this stance may seem contentious, it is in agreement with the findings of "Project explAIn," which noted that the importance of explanations for AI decisions varies based on the context [99].

Visual interfaces offer significant value in interdisciplinary collaborations, with researchers often creating customized tools for specific use cases. For example, Xu et al. [100] developed an interactive dashboard that visualized ECG data and classified heartbeats, leading to increased adoption of the ML method and improved clinical effectiveness in detecting arrhythmias [45]. Similarly, Soden et al. [101] studied the impact of ML on disaster risk management and recommended making the development of solutions transparent to the public to build trust. Personalized interfaces tailored to user needs can also improve the user

experience, as demonstrated by the developers of Firebird [102]. User interface design significantly contributes to the overall efficacy of applications [103].

In conclusion, while visual interfaces are undeniably valuable, the limitations of the highly customized tools developed by prior researchers have been noted [45]. These tools are often restricted to specific machine-learning frameworks or application domains. For instance, Klemm et al. [104] developed a visual interface that was tightly restricted to a specific machine-learning framework, while Muthukrishna et al. [105] developed a tool for a specific application domain. The limited scope of these tools renders them unsuitable for other contexts. Consequently, there is a need for more flexible visual interfaces that can be adapted to different machine-learning frameworks and application domains, ultimately improving their usefulness.

### 2.3.4 TECHNICAL CHALLENGES

#### *2.3.4.1 DEPLOYMENT*

Deploying ML models presents a unique set of challenges that require specialized skills and expertise [36]. One of the main challenges in deploying machine learning models is the need for explicit attention to software lifecycle phases [36]. Developing intelligent applications is not simply creating an application and occasionally calling ML library routines. Regular software engineering activities and skills apply to intelligent applications, but the skills needed for nurturing an intelligent application through its birth and lifecycle go far beyond this set [36].

According to Sculley et al., "changing anything changes everything" is the nature of ML models [94]. The dependencies among all parts of the ML application prevent the use of standard techniques for reducing couplings such as abstraction and information hiding. Not being able to isolate the impact of a specific change anywhere in the system of dependencies could be why practitioners often resort to ad hoc practices like trial and error and rules of thumb [36].

Another challenge in deploying ML models is the complexity of the AI domain. Three aspects of the AI domain make it fundamentally different from prior software application domains [39]:

- Managing and versioning data for ML applications is a complex task that presents unique challenges compared to other software engineering processes.

- Customizing and reusing models require specialized skills that differ from those typically found in software teams.
- AI components can be challenging to manage as separate modules and models may exhibit complex and non-monotonic error behavior due to their interconnected nature.

Furthermore, there is a necessity to apply DevOps principles to ML systems. However, although some DevOps principles apply directly, numerous challenges are unique to deploying machine learning [26]. Dang et al. [106] discuss these challenges in detail and use the term "AIOps" for DevOps tasks for ML systems. Some of the challenges mentioned include a lack of high-quality telemetry data, difficulty acquiring labels, and no agreed-upon best practices for handling machine learning models [26].

The model integration step is another important aspect of deploying machine learning models, encompassing building the infrastructure to run the model and implementing it in a form that can be consumed and supported [26]. While code reuse is a recurring theme in software engineering, it is equally applicable to machine learning, with the reuse of data and models translating into tangible time, effort, and infrastructure savings [26].

Deploying ML models remains a challenging task. The lack of standardized solutions for ML infrastructures across various domains necessitates building individual infrastructures for each project, which adds complexity to the deployment process [88]. Unsuitable or mis infrastructure is a significant challenge for any ML project, putting its success at risk [88].

In conclusion, deploying machine learning models requires specific skills and expertise beyond regular software engineering activities. To overcome the challenges in deploying machine learning models, applying DevOps, particularly AIOps principles, along with a focus on model integration, code reuse, and infrastructure development, is imperative. By addressing these challenges and adopting best practices, successful ML model deployment can be achieved.

#### *2.3.4.2 TECHNICAL DEBT*

The concept of technical debt is frequently cited in the literature as a challenge faced by practitioners in the field of machine learning [36], [37], [88], [94]. While this issue can lead to increased maintenance costs and other complications, it is typically viewed as less critical than other challenges, as it can usually be resolved with additional time investment [88].

One of the challenges ML practitioners face is the inaccessibility of necessary skills and tools, which can lead to technical debt [36]. As Sculley et al. argued, technical debt tends to compound silently, resulting in rising maintenance costs [94]. This observation is consistent with participants' experiences in various studies that have reported similar challenges [88].

Several common problems that can contribute to technical debt in ML systems are disjoint components connected through hard-to-maintain "glue code," hidden dependencies, and feedback loops. Such issues can result in significant technical debt, particularly at the system level rather than the code level [37], [94].

ML systems are particularly susceptible to technical debt because they often face many of the same maintenance issues as traditional ones, coupled with unique ML-specific challenges [94]. These issues can include erosion of abstraction boundaries and introduce coupling of otherwise unrelated systems by reusing or chaining input signals. In addition, ML packages may be treated as black boxes, creating large amounts of "glue code" or calibration layers that can be difficult to maintain [94].

To mitigate the impact of technical debt in machine learning systems, engineers and researchers alike need to recognize and prioritize this issue. It is advisable to avoid research solutions that offer only marginal improvements in accuracy but significantly increase system complexity [94]. Practitioners must actively seek practical strategies to address technical debt in machine learning to maintain the long-term health and efficiency of their systems.

#### *2.3.4.3 THE LANGUAGE BARRIER IN ML MODEL INTEGRATION*

Machine learning models are often developed using different programming languages, such as Python, TensorFlow, or PyTorch, compared to the programming languages employed by application developers, such as Java, C++, or Ruby [35]. This discrepancy can lead to increased complexity when it comes to integrating the ML model into the application, often requiring the re-coding of the ML model to match the application's native language. While migrating ML models to improve their integration with the rest of the application has become more feasible, utilizing a common language to construct the model can help alleviate integration difficulties [35].

#### *2.3.4.4 OVERLAPPING RESPONSIBILITIES*

Researchers and software engineers often work together on ML projects to achieve business objectives [26]. However, ownership can sometimes become confusing, requiring clearly

defined roles and responsibilities for each team member when it comes to model deployment. While their responsibilities may seem distinct, with researchers developing the model and engineers building the infrastructure to run it, their areas of concern often overlap when considering the development process, model inputs and outputs, and performance metrics [26].

As a result, involving researchers in the entire development journey proves advantageous. This approach ensures they take ownership of the product code base, use the same version control, and participate in code reviews with the engineers [39]. While onboarding may present initial challenges, this collaborative approach has been shown to bring long-term benefits in speed and product quality [39]. Data scientists may assume their work is done once the model is built, while developers may have little understanding of the model's inner workings, necessitating more input to integrate it properly into the application. In the end, it is the joint responsibility of the team working on the deployment to ensure that the suitable model is deployed, thereby preventing unnecessary delays [35].

#### 2.3.4.5 *UNDERSTANDING THE ROLE OF ML IN PRACTICE*

Previous research has shown that the learning algorithm is a relatively small component of an ML platform [94]. In a 2015 paper, Sculley et al. challenged the perception that machine learning code is the most critical part of the overall lifecycle, pointing out that data dependencies, model complexity, reproducibility, testing, monitoring, and version changes are also essential factors [94]. The authors demonstrated that the machine-learning code represents only a fraction of the total code in real-world ML systems, as visually depicted in Figure 4 [94]. Thus, focusing on these other components is crucial to make an application practical.

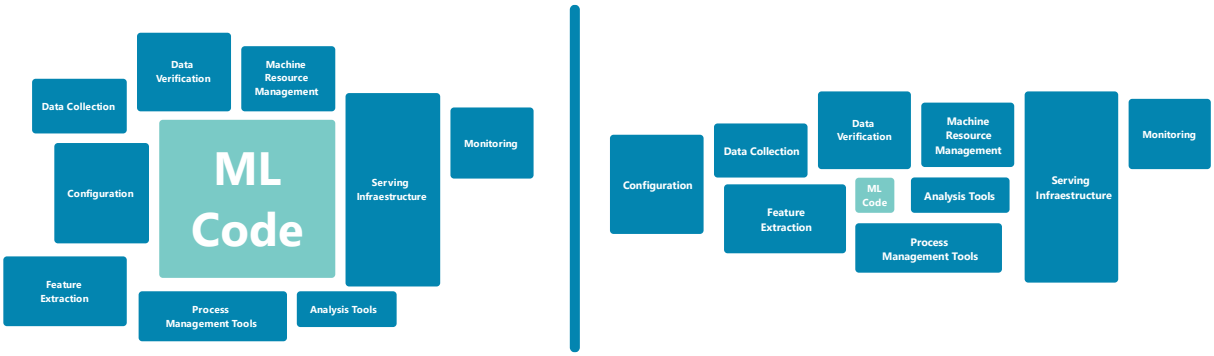



Figure 4 - Perceived vs. Real role of ML adapted from [35], [94].

#### 2.3.4.6 SCALABILITY AND INFRASTRUCTURE

Deploying ML models on large infrastructures with considerable amounts of data can be challenging. Infrastructure-related issues, such as building massive computing power and managing energy consumption, can cause difficulties [61], [107]–[109]. Additionally, applying algorithms on large amounts of data or various infrastructures requires specialized expertise, including ensuring that models process incoming data and make decisions within strict time constraints [110], [111]. Nevertheless, further difficulties are associated with scaling a model to deployment architectures, including the need for code parallelization [88]. This process is complicated by the fact that only a small number of programming languages are straightforward to parallelize [88].

A deployment platform should also be able to tolerate disruptions caused by unstable software, varying user configurations, and malfunctions in the underlying runtime environment. It should also be able to adapt to large amounts of data and growth in traffic to the serving system [37].

One of the main challenges during the deployment of ML models is infrastructure-related issues [88]. Data scientists often need to work within the existing infrastructure on the customer side, which means adapting solutions to different infrastructures [88]. Standardized architectures for local solutions are scarce, and even when customers are willing to build new infrastructure, the installation of a consistent local infrastructure proves challenging [88]. Additionally, the actual deployment environment for ML models differs significantly, requiring fundamentally different approaches for running a model on a large cluster versus a mobile device [88].

The decision to opt for a cloud-based or on-premise AI solution is influenced by factors such as preferred workflow, tool usage frequency, software and hardware expenses, and IT security teams' risk appetite toward cloud-based solutions [49]. The cost is one of the primary issues when considering a cloud-based AI solution. Vendors often charge for GPU computational use on a usage basis and for any storage costs required [112]. In addition, every time the data is handed off externally, there is an increased possibility of data breaches and unauthorized use of patient information [49]. Therefore, it is essential to carefully weigh the benefits and risks before opting for a cloud-based solution [35]. On-premises solutions may be optimal with access to robust IT infrastructure and plans to perform frequent and computationally

intensive AI calculations. However, cloud-based solutions offer the ability to outsource the AI algorithm development and expertise, as well as external hardware and software maintenance management [49]. Adopting a cloud-based or on-premises solution depends on expenses, security, and organizational preferences.

In conclusion, deploying ML models on large infrastructures with considerable amounts of data can be challenging. Infrastructure-related issues and scaling up models to deployment architectures present several challenges that must be addressed. Additionally, the deployment platform must be resilient and scalable, and the decision of whether to use cloud-based or on-premises solutions depends on several factors. Careful consideration of all the pros and cons is essential before deciding on a strategy.

### 2.3.5 CROSS-CUTTING CHALLENGES

#### *2.3.5.1 SUBPAR STANDARDS IN LITERATURE*

The potential of AI to revolutionize healthcare is widely recognized. However, many scientific papers featuring AI applications in this field suffer from methodological flaws [16], [41], [106], [107]. In addition to these technical shortcomings, the quality of the studies is a critical concern. Reviews conducted by Nagendran [108] and Liu et al. [109] on DL applications in medicine found that a majority of the studies had a poor statistical design, with flaws such as the lack of randomized trials, not following existing guidelines, and failure to share code and datasets, which hinders reproducibility [16]. Furthermore, overpromising language exacerbates this issue, leading to potential misinterpretation and contributing to the hype around AI in medicine. Thus, addressing these concerns in future research is crucial to ensure AI's responsible development and deployment in healthcare [16].

#### *2.3.5.2 REAL-WORLD VALUE*

Aligning ML research with its practical application is necessary, as a narrow focus on optimizing performance on benchmark datasets fails to create sufficient real-world value [110], [113]–[115]. More realistic evaluations of model results and rigorous assessments under real-world conditions are needed to solve the problem of ML solutions having little or no real-world value [59], [116]. Journals and editors can play a role in supporting this development [87]. Creating real-world value of ML solutions is a significant challenge that can be viewed from four different aspects [88]: expressing results in terms of real-world business

value, managing expectations effectively, communicating and explaining results adequately, and developing valuable digital services or products. Researchers should also prove the applicability of their work in real-world environments and focus on the complexity of real-world domains rather than just improving algorithms by minor percentage points on statistical metrics [88].

#### *2.3.5.3 SECURITY*

The security of data used in ML and AI is critical, as these technologies require extensive data to generate accurate predictions [35]. However, using personal and sensitive data in training models may compromise data confidentiality and security [35]. ML models may be confidential due to sensitive training data, commercial value, or security applications. Additionally, models can also be privacy-sensitive because they can leak information about training data [117]–[119]. The deployment of confidential ML models with publicly accessible query interfaces has motivated the investigation of model extraction attacks [120]. In such attacks, adversaries aim to steal a model's functionality by exploiting black-box access to the model without prior knowledge of its parameters or training data [120]. The attacks are efficient and can extract popular model classes with near-perfect fidelity, including neural networks, decision trees, and logistic regression [120]. These findings highlight the need to deploy ML models carefully and develop new extraction countermeasures [120].

The field of adversarial machine learning studies the effect of attacks against ML models and how to protect against them [121], [122]. The three most common attacks that affect deployed ML models are data poisoning, model stealing, and model inversion. Data poisoning attacks deliberately corrupt the model's integrity during the training phase to manipulate the produced results. Model stealing attacks reverse engineer a deployed model by querying its inputs and monitoring outputs to train a substitute model. Model inversion attacks aim to recover parts of the private training set, breaching its confidentiality.

Safeguarding models from adversarial attacks and preventing unauthorized access to model data during deployment pose significant challenges. Therefore, it is critical to ensure that the model remains protected from unauthorized users by operating in a secure mode. To counter extraction attacks, one possible defense is prediction API minimization by not returning confidences and not responding to incomplete queries [120]. Other defenses include using rounding confidences, differential privacy, and ensemble methods [120].

## 2.4 OVERVIEW OF DEPLOYMENT IMPLEMENTATIONS

This section briefly overviews various studies and literature on implementing machine learning models in medical and healthcare settings. It highlights the pertinent technologies and knowledge required for successful deployment in these areas. Furthermore, this overview delves into the challenges and solutions encountered in real-world scenarios by presenting practical examples of machine learning model deployment.

### 2.4.1 EPOCH® AND EPRISM®

The EPOCH® and ePRISM® software suite [123] offers a comprehensive solution for delivering complex risk-adjustment models to the clinical setting. Built on Microsoft .NET, the software features a flexible XML web services architecture, facilitating integration with existing clinical information systems and enabling remote access to the modeling engine. A user-friendly visual interface within the suite empowers researchers to translate prediction models into web-based decision support and reporting tools. Moreover, the software supports the export of models across systems using the Predictive Model Markup Language standard.

EPOCH® is a unified web portal and services application framework. At the same time, ePRISM® is a plug-in module for EPOCH® that encompasses all the functionality necessary for deploying clinical risk models as decision-support tools or embedding them within document templates. Together, EPOCH® and ePRISM® provide a framework for translating evidenced-based prognostic models into clinical tools readily available at the bedside. These tools offer a link between healthcare providers, clinical information systems, and researchers and offer a mechanism for meeting the Institute of Medicine's (IOM) vision of a healthcare system that is more evidence-based, transparent, and patient-centered.

EPOCH® and ePRISM® can help clinicians and patients make better-informed healthcare decisions by delivering sophisticated risk-adjustment models that incorporate patients' unique clinical characteristics.

### 2.4.2 TFX

TensorFlow Extended (TFX) [37] is a TensorFlow-based platform for machine learning that streamlines the production and deployment of machine learning models. The platform addresses the challenge of orchestrating many components in the machine learning workflow,

including the learner for generating models, modules for analyzing and validating data and models, and infrastructure for serving models in production.

TFX simplifies platform configuration, standardizes components, and reduces time to production. It provides automation to deal with diverse failures in production, ensures reliability and scalability, and is generic enough to handle a wide range of learning tasks.

The platform allows teams to deploy machine learning in production, ensures best practices for different platform components, and limits technical debt. TFX offers continuous training over evolving data, easy-to-use configuration and tools, and interfaces to make it easy for engineers to deploy and monitor the platform with minimal configuration.

### 2.4.3 CLINICAL PREDICTIVE MODELING DEVELOPMENT AND DEPLOYMENT THROUGH FHIR WEB SERVICES

Khalilia et al. [124] have proposed a software architecture that uses web services to develop and deploy clinical predictive models via the Health Level 7 (HL7) Fast Healthcare Interoperability Resources (FHIR) standard. The model development platform facilitates training and comparison of multiple predictive models using electronic health records (EHRs) stored in the Observational Medical Outcomes Partnership Common Data Model (OMOP CDM). The resulting predictive models are deployed as FHIR resources, designed to receive patient information requests, perform predictions using the deployed predictive model, and respond with prediction scores. The system is reasonably fast, with one second total response time per patient prediction.

The system is focused on API-based predictive modeling services that can be implemented in thin client applications, particularly in the mobile environment. The FHIR *RiskAssessment* resource is used for predictive analysis. The authors demonstrated the system using MIMIC2 ICU and *ExactData* chronic disease datasets for mortality prediction.

In conclusion, the platform leverages two fundamental technologies: the OMOP CDM database and FHIR web services. Using these technologies, the authors of the study showcase a software framework tailored for creating and implementing clinical predictive models. These services empower the development of models by utilizing electronic health records (EHRs) housed within the OMOP CDM databases and facilitate the deployment of these models to evaluate individual patient outcomes using FHIR resources.

### 2.4.4 GRADIO

Gradio [45] is an open-source Python package that creates web-based visual interfaces for machine learning (ML) models, addressing current issues faced by researchers. The package was designed with the help of interviews conducted with twelve ML researchers engaged in interdisciplinary projects. Gradio supports a range of interfaces and frameworks, input manipulation, and the sharing of models with domain experts, even without coding proficiency. It also enables domain expert feedback on individual samples, and users can embed the interfaces in websites or run them from Jupyter or Google Colab notebooks. The package can be implemented with minimal change to the existing ML workflow and includes standard interfaces for handling images, text, and audio. It supports Keras, PyTorch, scikit-learn, PyFunctions models, and custom preprocessing and postprocessing functions. Gradio creates a shareable link to the model, which is securely run on the developer's computer and encrypted before passing through a Secure Shell (SSH) tunnel to the user. Collaborators benefit from the capability to manipulate input data and flag erroneous outputs, enhancing the collaborative ML development process.

### 2.4.5 KETOS

KETOS [125] is a platform that allows researchers to perform statistical analyses and deploy resulting models in a secure environment. The system uses Docker virtualization to provide researchers with reproducible data analysis and development environments, accessible via Jupyter Notebook. It facilitates statistical analysis and the development, training, and deployment of models based on standardized input data.

The architecture supports the entire research lifecycle from creating a data analysis environment, retrieving data, and training to final deployment in a hospital setting. The KETOS platform takes a novel approach to data analysis, training, and deploying decision support models in a hospital or healthcare setting. It does so in a secure and privacy-preserving manner, combining the flexibility of Docker virtualization with the advantages of standardized vocabularies, a widely applied database schema (OMOP-CDM), and a standardized way to exchange medical data (FHIR). The platform allows researchers to use tools already part of the data-scientists workflow (Jupyter Notebook, Python, and R) to develop custom statistical computations without restrictions.

### 2.4.6 NIFTYNET

NiftyNet [126] is an open-source platform for deep learning in medical imaging, which offers specific functionality for medical image analysis. Established deep-learning platforms are not optimized for medical imaging, and adapting them for this purpose requires significant effort. This has resulted in incompatible infrastructure across many research groups. The NiftyNet platform aims to simplify the development of medical image analysis solutions, providing a standard mechanism for disseminating research outputs for the community to use, adapt, and build upon. It offers a high-level deep learning pipeline with components optimized for medical imaging applications, such as data loading, sampling and augmentation, networks, loss functions, evaluations, and a model zoo. The platform is modular, enabling the addition of new application types, and provides support for tasks such as segmentation, regression, image generation, and representation learning applications. The platform provides implementations for data loading, data augmentation, network architectures, loss functions, and evaluation metrics that are tailored for medical image analysis and computer-assisted intervention.

# 3 FMDEPLOY - PLANNING

**Key points:**

- Clear and consistent naming conventions and definitions are established, including user types (Guest, User, Admin) and their respective privileges.
- The research platform caters to diverse user groups, including students, researchers, administrators, domain experts, developers, and the general public.
- Mandated constraints outline essential requirements, such as programming languages (Python and JavaScript), database systems (MySQL or PostgreSQL), and development frameworks (FastAPI and React).
- The domain model visually represents the system's entities and concepts, including actors (Guest, User, Admin), use cases, projects, executions, and run history.
- The use case diagram categorizes use cases into Authentication, User Configurations, and Project Functionalities, defining the system's expected behavior and interactions.
- User needs are pivotal in shaping the requirements for the research platform.

This chapter focuses on the planning phase of the platform named FMdeploy, a combination of an API and a web application designed to tackle the deployment of pre-trained machine learning models and provide web-based user interaction with the deployed models. FMdeploy results from the culmination of efforts to design, develop, and implement a robust and efficient system tailored to meet specific user needs and operational requirements. For a deeper comprehension of the principles and systems underpinning FMdeploy, please refer to APPENDIX 1 - "Technologies and Concepts". This chapter outlines the strategic planning that guided the development of FMdeploy, encompassing a detailed analysis of functional and non-functional requirements, which serve as the cornerstone of the system's architecture. Planning and considering these requirements lay the groundwork for a system that addresses user expectations and adheres to high performance, security, and usability standards.

## 3.1 NAMING CONVENTIONS AND DEFINITIONS

In the naming conventions and definitions section, it is crucial to establish clear and consistent terminology to ensure effective communication and understanding within the research

platform. To achieve this, the following proposed naming conventions and definitions will be outlined:

### 3.1.1 USER

A user refers to an individual who has created an account or whose account has been created by an administrator. Accounts are categorized into three types: guest, user, and admin.

**Guest account:** A guest account is accessible to anyone who knows the platform's URL or endpoint. These accounts intentionally possess limited access rights and permissions to enhance security measures. A guest account is designed for domain experts and collaborators who require access to active projects to provide valuable feedback and insights. It allows them to interact with existing projects, run ML models, and contribute to ongoing research efforts. While guest accounts facilitate interaction with existing projects and allow users to run machine learning models, they do not grant the privilege to initiate new projects. Instead, guest accounts empower individuals to participate actively by sharing their expertise and perspectives while working alongside project creators and administrators.

**User account:** A user account is a standard account that provides full access to the functionalities and features of the research platform. Users with these accounts can create and manage projects, upload data, execute ML models, and utilize the platform's resources to conduct research or analysis.

**Admin account:** Admin accounts hold a privileged status, entailing elevated access and control over the research platform. Administrators have the authority to manage user accounts, grant permissions, and oversee the overall system administration. They have additional capabilities to configure settings, enforce security measures, and perform administrative tasks within the platform.

By distinguishing between guest, user, and admin accounts, the research platform ensures appropriate access control, ensuring users can engage in their research activities while safeguarding the platform's security and overall system integrity.

### 3.1.2 PYTHON SCRIPT

A Python script, easily recognizable by the ".py" file extension, contains essential Python code for executing ML models within the research platform. It includes essential functions such as "`load_models`" and "`run`" to facilitate the loading and execution of models on input data.

Adhering to a predefined structure and incorporating the required functions ensures seamless integration with the platform, enabling effective model execution and analysis.

### 3.1.3 MODEL

A model refers to a trained ML or DL model utilized for inference or prediction tasks. It consists of one or more files containing the necessary parameters and architecture to make predictions based on input data. The model files may have extensions such as ".h5" or ".json". The associated Python script imports the model(s) to execute and generate the desired outputs.

### 3.1.4 PROJECT

A project within this context defines a unit of work, encompassing the necessary elements to execute a specific task or study. It includes basic information such as a title, description, input type, output type, and privacy settings. These details provide a comprehensive overview of the project's purpose and scope.

Each project is centered around a Python script that follows a specified design. This script is the core component for executing ML models within the platform. It contains essential code, including the "`load_models`" and "`run`" functions, facilitating model loading and execution on input data.

Furthermore, a project may include ML models. The number of models included can be zero or more, depending on the requirements of the task at hand. These models are essential for making predictions or performing inferences based on the provided input data.

The structure and components of a project, including the Python script and optional machine learning models, are pivotal in defining the work to be performed within the research platform. By adhering to the specified design and including the necessary elements, users can effectively leverage the platform's capabilities for their research endeavors. Projects are created through the web interface, where users provide the necessary information by filling out forms and uploading files.

There are three types of projects:

**Private Projects:** Accessible exclusively to the creator, ideal for confidential research or tasks necessitating restricted access.

**Shared Projects:** These projects can be shared with specific users within the research platform, facilitating collaboration and allowing selected individuals to contribute, provide feedback, and participate in the project's progress.

**Public Projects:** Freely available to all users of the platform.

The project type selection depends on the nature of the research, the desired level of collaboration, and the need for data privacy and confidentiality.

### 3.1.5 PROJECT INFORMATION

Project information includes details such as the title, description, input type, output type, and privacy settings. These elements provide a concise overview of the project's purpose, data requirements, and access restrictions.

### 3.1.6 PROJECT ACTIONS

The research platform offers users various project-related actions, including running, editing, deleting, sharing, and managing privacy settings. The availability of project actions varies depending on the type of account.

### 3.1.7 FLAG

The research platform allows users to flag outputs generated by the project. If the output does not meet the expected result or contains valuable insights, users can flag it. Flags serve as indicators to identify potential issues or areas for improvement and may optionally include a description elaborating on their significance.

### 3.1.8 PROJECT RUN

A project run refers to the recorded information associated with the execution of a project. It encompasses essential details such as the project's unique identifier (ID), its title, and the name and email of the project creator. Additionally, a project run includes the input file used for the execution and the resulting output generated by the project. A timestamp is recorded to indicate when the execution took place. Moreover, a project run can also include flag information.

Collectively, all the gathered information within a project run provides a comprehensive overview of the project's execution, allowing for analysis, tracking, and documentation of the

project's progress and outcomes. The run history, consisting of a collection of project runs, offers a consolidated record of project executions, facilitating easy reference and review.

### 3.1.9 RUN HISTORY

The run history is a comprehensive record of executed projects. It comprises a list of project runs, each providing detailed information on project execution. This includes the project name, associated inputs, outputs, and the flag status of each output.

One of the key features of the run history is its ability to facilitate easy access to the input and output files associated with each project run. This feature lets users retrieve and download the inputs and outputs directly from the run history, even if the browser was closed after project execution. This ensures that valuable data and results are always accessible to the users.

In addition, the run history provides an alternative way to create flags for specific project runs. Users can flag previous runs directly from the run history, allowing them to provide feedback, highlight important observations, or categorize specific outputs.

## 3.2 PLANNING

### 3.2.1 IDENTIFICATION OF REQUIRED RESOURCES

To establish a successful system for deploying ML and DL, it is crucial to identify and gather the necessary resources. These resources can be broadly categorized into software and hardware components.

In terms of software, proficiency in various tools, frameworks, coding languages, and libraries is essential. Specifically, expertise in Frontend and Backend development and knowledge of containerization techniques are required. Frontend development necessitates experience in UI/UX design, while Backend development relies on a solid understanding of Python, REST API, and databases.

In addition to software resources, having access to up-to-date computer systems is essential. Furthermore, an off-site server running virtual machines is highly recommended for testing purposes.

### 3.2.2 MODEL OF THE SYSTEM TO IMPLEMENT – MOCKUP

The system will consist of two primary components: Frontend and Backend. The Frontend component will enable users to create, edit, run, delete, and share projects. On the other hand, the Backend component will support all the web application functionalities.

The Backend will continuously listen for requests from the Frontend and handle various tasks, including storing and maintaining user data, project data, service data, and managing internal system functions. Establishing a reliable connection between the Frontend and Backend is crucial to ensure the prompt and accurate delivery of data to users.

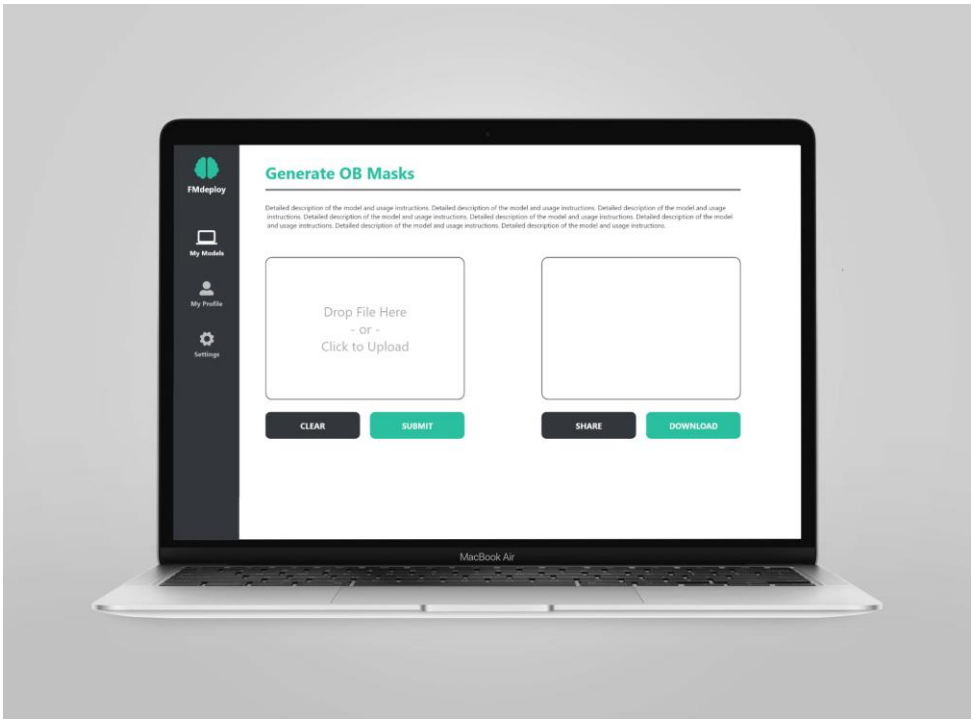



Figure 5 – Mockup of FMdeploy web application.

A mockup is a visual representation or prototype of a system demonstrating its structure and functionality. In Figure 5, the mockup created with Adobe XD illustrates the web application's central "Run Project" screen. This screen showcases the title and description of the project, as well as a section for users to submit inputs and download outputs associated with the project. This mockup helps visualize the user interface and interaction flow of the system.

### 3.2.3 DEVELOPMENT PLAN

The development plan consists of two main phases: Specification and Implementation, which will ensure the successful progression of the project.

During the specification phase, the requirements will be gathered, and a domain model along with essential diagrams such as Use Cases will be created. This phase will also include database modeling and validation.

In the subsequent implementation phase, the focus will shift towards designing the system architecture, creating the database, and modeling the software and user interface. Attention will be given to validating the construction of the software and ensuring its functionality aligns with the specified requirements.

By following this structured development plan, the aim is to efficiently progress from requirement gathering and specification to successfully implementing the research platform.

### 3.2.4 DEFINITION OF SUCCESS MEASURES

The primary measure of success revolves around the seamless deployment of the research platform to a server in a fully functional state. This entails ensuring that all components, including the Frontend and Backend, are implemented and function correctly. The platform should be accessible, responsive, and capable of running models and producing accurate results.

## 3.3 SYSTEM USERS

The research platform is designed to cater to diverse users, each with specific roles and purposes in mind. The following user groups will be considered when developing the platform:

**Students and Researchers:** Individuals with ML expertise use the platform to deploy and test their ML models. They benefit from the platform's features, collaborate with other users, and leverage the insights gained from model outputs.

**Teachers and Admins:** Teachers and administrators serve as key stakeholders. They provide guidance, support, and domain expertise to students and researchers using the platform. Admins have additional privileges, such as managing user accounts, projects, and system configurations.

**Domain Experts:** Domain experts, such as medical practitioners, contribute valuable knowledge and insights while developing ML models. They collaborate with students and researchers, providing essential expertise from their respective fields. Domain experts may interact with the platform to review and provide feedback on the models' performance.

**Developers:** Developers have a deep understanding of ML and software development. They may use the platform's RESTful API to interact with the system programmatically, integrate it into their own applications, or develop mobile apps that utilize its functionalities.

**General Public (Layman Users):** The research platform also welcomes non-specialist users from the public who may not possess expertise in ML, development, or medical concepts. Although not actively involved in model development, Layman users can explore publicly shared projects and review model outputs. They are expected to have a good understanding of web interactions and can contribute to the platform's improvement through their perspectives and usability feedback.

Table 2 presents an overview of the different user types, roles, priorities, and their characteristics within the system, providing insights into their diverse profiles, contributions, and requirements.

Table 2 - Types of system users

| Name | Role | Priority | Characteristics |
|---|---|---|---|
| Students and Researchers | User | Key User | - good to expert grasp of ML<br>- beginner to good knowledge of domain-specific concepts (e.g., medicine)<br>- well-versed in web technologies<br>- creates projects |
| Teachers and Admins | Admin | Key User | - expert grasp of ML<br>- good to expert knowledge of domain-specific concepts (e.g., medicine)<br>- well-versed in web technologies<br>- manages projects and users |
| Domain experts (e.g., doctors) | Guest | Key User | - beginner to good grasp of Machine Learning concepts<br>- expert knowledge of domain-specific concepts (e.g., medicine)<br>- well-versed in web technologies<br>- interacts with projects and provides feedback<br>- does not create projects |
| Developers | Admin | Secondary User | - good to expert grasp of ML concepts |

| | | | |
|---|---|---|---|
| | | | - exceptionally well-versed in web technologies<br>- proficient in programming languages and frameworks<br>- familiarity with API integration and development |
| General public | Guest | Secondary User | - comfortable with basic web navigation and interaction<br>- limited to no proficiency in ML and domain-specific concepts<br>- does not create projects<br>- may run projects and provide feedback |

## 3.4 MANDATED CONSTRAINTS

The mandated constraints chapter outlines the essential requirements and limitations that guide the design and implementation of the product. These constraints cover aspects such as programming languages, frameworks, technologies to be used, and the infrastructure where the platform will be deployed. Adhering to these constraints ensures the product aligns with project objectives and meets the necessary technical requirements.

### 3.4.1 SOLUTION CONSTRAINTS

**Requirement:** R01

**Description:** The system's backend development shall employ Python.

**Reasoning:** Python, an object-oriented, high-level programming language, is deliberately selected for its suitability for AI and ML application development.

**Fit Criterion:** The system's backend codebase shall consist mostly of Python code segments.

**Requirement:** R02

**Description:** JavaScript shall be the primary programming language for frontend web development.

**Reasoning:** JavaScript is deliberately chosen as it is widely recognized as one of the most optimal languages for crafting frontend web applications.

**Fit Criterion:** All frontend web development work shall be conducted mainly using JavaScript.

**Requirement:** R03

**Description:** The database management system for this project shall be a suitable relational database with flexibility of choice enabled using the ORM bundled with the FastAPI framework, SQLAlchemy.

**Reasoning:** SQLAlchemy ensures compatibility with the backend codebase and provides the flexibility to choose from a range of database systems.

**Fit Criterion:** The system shall effectively function with the chosen relational database managed through SQLAlchemy.

**Requirement:** R04

**Description:** The backend development framework shall be based on FastAPI.

**Reasoning:** FastAPI, recognized for its modernity, efficiency, and ease of use, is chosen as the optimal Python web framework for constructing APIs and backend services.

**Fit Criterion:** The entire backend shall be developed using the FastAPI framework.

**Requirement:** R05

**Description:** The frontend development framework shall exclusively rely on React.

**Reasoning:** React, a widely adopted JavaScript library, is chosen for its capacity to build user interfaces effectively through its component-based architecture and efficient rendering.

**Fit Criterion:** All frontend development shall utilize the React framework.

**Requirement:** R06

**Description:** The system shall be containerized using Docker technology.

**Reasoning:** Docker is adopted for its provision of a standardized, portable environment capable of packaging and deploying applications across various platforms, including personal use, in-house servers, and cloud solutions.

**Fit Criterion:** The system's deployment shall utilize Docker containers.

**Requirement:** R07

**Description:** The system shall employ HTTPS for secure data transmission.

**Reasoning:** HTTPS is a standard for secure communication over computer networks, ensuring data confidentiality and integrity during transmission.

**Fit Criterion:** The system shall successfully establish HTTPS connections for secure data transmission, and data integrity and confidentiality shall be maintained during communication.

## 3.5 RELEVANT FACTS AND ASSUMPTIONS

### 3.5.1 FACTS

The relevant facts in the context of this work are:

- **Platform Accessibility:** The research platform will be accessible to individuals who know the URL; however, access will be limited to a guest account, restricting certain functionalities.
- **Account Approval:** Only individuals approved by an administrator will be granted admin or user accounts on the platform, ensuring authorized access and proper user management.
- **Docker Containers:** Each student is assigned a dedicated Docker container by a professor, where they conduct their research in AI, ML, and DL. It is important to note that all personal Docker containers maintain consistent Python and library versions across the entire server.
- **Server Location:** FMdeploy will be hosted on a server located at the University of Minho.
- **Consistent Python Environment:** The proposed research platform will adopt the Python and library versions from the students' existing Docker containers, ensuring compatibility and minimizing potential version conflicts.

### 3.5.2 ASSUMPTIONS

The relevant assumptions in the context of this work are:

- **User Familiarity:** Users of the research platform are assumed to have basic familiarity with web interactions, including filling forms, using text boxes, and performing tasks such as dragging and dropping files.
- **Supported Python and Library Versions:** Users who create projects on the platform are assumed to use only the officially supported Python and library versions. They are

assumed to adhere to the recommended versions to ensure compatibility and avoid any unforeseen issues.

- **Default Private Projects:** It is assumed that all projects created on the platform will be set as private by default. Users will have control over their projects' visibility and sharing settings and can choose to make them public if desired.
- **Account Registration:** Users are assumed to register for an account on the research platform using valid and unique credentials, as required during registration.
- **File Size and Type:** Users are assumed to adhere to the platform's file size and file type restrictions when uploading files, ensuring they are within the allowed limits and compatible formats.
- **Compliance with Ethical Guidelines:** It is assumed that users will comply with ethical guidelines and regulations when conducting research or utilizing the research platform. They will respect data privacy, intellectual property rights, and other relevant ethical considerations.
- **Adequate Network Connectivity:** Users are assumed to have a stable and reliable internet connection to access the research platform and utilize its features without interruptions or significant latency issues.
- **Basic Data Backup:** Users are assumed to maintain backup copies of important project data and files, as the platform may not provide comprehensive data backup services.

## 3.6 DOMAIN MODEL

The domain model serves as a visual representation of the system's entities and concepts, aiming to provide clear information to users. It facilitates the understanding of the connections between users and the model. Figure 6 presents a domain model that illustrates these key concepts and entities.

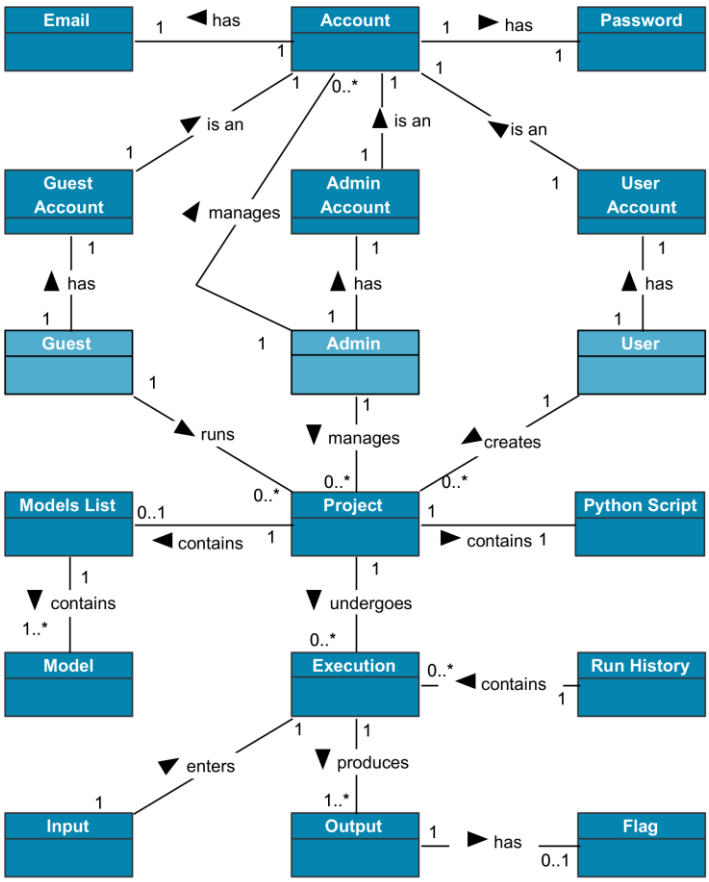



Figure 6 - Domain Model.

Three roles are depicted: Guest, Admin, and User (in light blue). All users require an account to access the system. Admins are vital in managing accounts and projects, ensuring smooth operation, and providing support when needed.

Users can create, share, and run projects. On the other hand, Guests can run and provide feedback on public projects and projects shared with them but cannot create new projects.

A Project created by an Admin or User consists of one Python script and may contain zero or more ML models. Projects undergo Execution, where an Input is provided, and one or more Outputs are generated.

Information about each Execution is stored in the Run History, including timestamps, the initiator of the Execution, project details, Input, Output, and Flags. An Execution may have an optional flag associated with its Output.

In summary, the domain model demonstrates the relationships and functionalities of the entities within the system, enabling effective management and performance of projects.

## 3.7 THE SCOPE OF THE PRODUCT

The use case diagram defines the boundaries between the actors (user, admin, and guest) and the software application. A crucial component in specifying the system requirements, a use case diagram presents an overview of the expected behavior without delving into implementation details.

Within this context, the use cases illustrated in Figure 7 can be categorized into Authentication, User Configurations, and Project Functionalities. The Authentication category encompasses the Registration and Authentication use cases, which involve creating and accessing user accounts. Additionally, role verification is performed during authentication.

User Configurations focus on updating user information, enabling actions such as modifying account details like name, email, and password. It also includes the option for account termination, granting users control over their account settings.

Lastly, the Project Functionalities encompass the various interactions associated with projects, including creating, deleting, updating, sharing, running, and flagging. These functionalities allow users to manage their projects effectively.

The arrows connecting the Guest and Admin actors in the use case diagram represent their relationship. They signify that the User inherits the properties of the Guest, and the Admin inherits the properties of the User. This generalization relationship implies that the descendant actor can utilize all the use cases defined for his ancestor.

The generalization relationship represents the inheritance between two actors, wherein one actor inherits all the properties and relationships of another actor. In this scenario, the Admin should be able to perform all the functions available to Users and the specific functions associated with their role, such as managing users. Similarly, a User can perform all the functions accessible to a Guest and the specific functions related to creating and managing projects.

By utilizing the generalization relationship between the User and Admin, with the Admin inheriting from the User, and between the Guest and User, with the User inheriting from the Guest, the use case diagram captures the hierarchical relationship and the inheritance of functionalities among the actors.

Figure 7, the Use Cases Diagram, visually represents the system's actors and their interactions with the platform. The actors are positioned outside the system boundary, while the ellipses within represent the use cases. The lines connecting the actors to the use cases depict their usage.

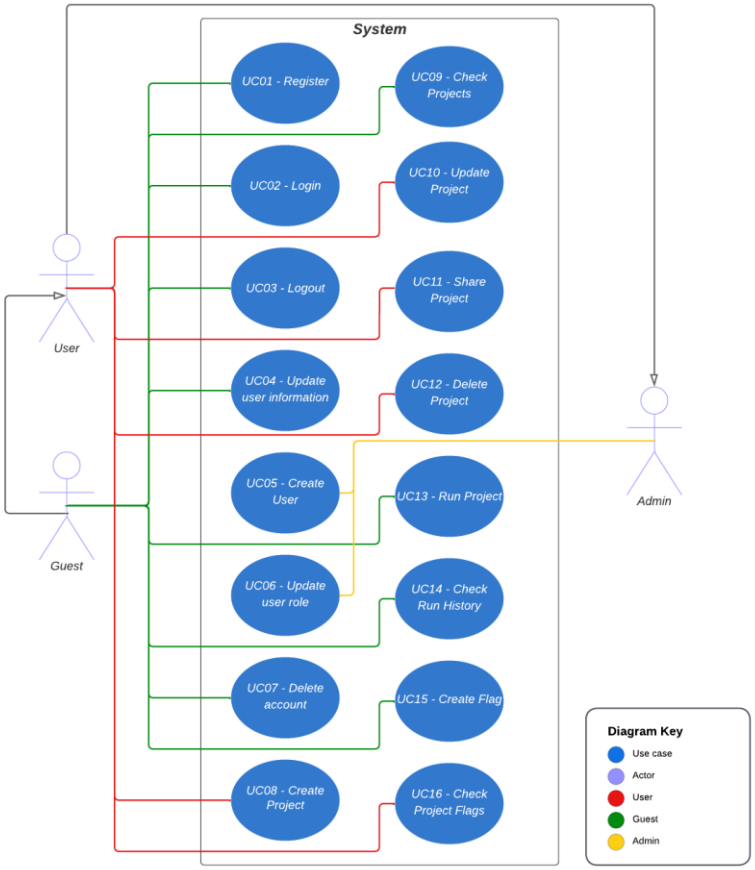



Figure 7 - Use Cases Diagram.

The diagram above serves as a high-level overview of the essential use cases within the system.

APPENDIX 2 - "Use Cases" of this dissertation, readers will find a description of each of the sixteen use cases in a tabular format. These tables provide comprehensive insights into each use case's functionalities, specifications, and interactions. These detailed descriptions serve as a valuable reference for readers and stakeholders, enhancing their understanding of the FMdeploy platform's operational aspects and potential applications.

## 3.8 REQUIREMENTS

Gathering requirements is a fundamental part of the development of any software. Quality requirement gathering, processing, and management are crucial to setting clear goals. With this in mind, the goal is to create a comprehensive but realistic list of the different usage requirements concerning the FMdeploy research platform.

The primary focus is on the user, shaped by examining the existing development system at the University of Minho and insights gained from the literature review. The goal was to pinpoint user needs and preferences with an understanding of the workflow and the ideal scenario. This understanding was crucial in identifying the requirements outlined in APPENDIX 3 - "Requirements".

APPENDIX 3 - "Requirements" provides comprehensive information regarding the system's functional and non-functional requirements. The functional requirements specify the various actions and functionalities available to users, such as user registration, authentication, project management, and more. Each requirement details the inputs, sources, outputs, and actions associated with a specific aspect of the system's functionality, ensuring a clear and precise understanding of how the system should operate. Additionally, the appendix includes non-functional requirements that outline the properties and constraints of the system, including aspects like performance, appearance, usability, and more. These non-functional requirements ensure the system is efficient, user-friendly, secure, and reliable, meeting user expectations and system performance standards. Together, the functional and non-functional requirements outlined in APPENDIX 3 - "Requirements" serve as a guide for the development and implementation of the system, ensuring that it aligns with user needs and operates effectively.

A detailed description of each of the following requirements can be found in Appendix 3.

**Functional Requirements:**

- Register User
- Authenticate User
- Update User Information
- Create User
- Update User Role
- Delete Account

- Create Project
- Update Project
- Python Script Upload
- Model Files Upload
- List Owned Projects
- List Shared Projects
- List Public Projects
- Share Project
- Make Project Public
- Delete Project
- Run Project
- Check Run History
- Create Flag
- Check Project Flags

**Non-Functional Requirements:**

- Appearance
- Style
- Ease of use
- Learning

# 4 FMDEPLOY - DESIGN AND DEVELOPMENT

**Key points:**

- FMdeploy is a platform for deploying pre-trained machine learning models.
- It utilizes FastAPI for the API and React for the web application, ensuring a straightforward user experience and responsive design.
- The entire codebase is containerized using Docker for efficient deployment and management.
- Security is prioritized with HTTPS communication enabled through Traefik and Let's Encrypt certificates.

This chapter focuses on the intricacies of the FMdeploy platform's design and development, forming the foundation of its functionality and efficiency. The exploration begins with an in-depth understanding of the system's rationale, providing insights into the design choices and structural underpinnings. The subsequent sections dissect the system architecture, detailing its essential components and interconnections. Testing and deploying machine learning models, pivotal aspects of FMdeploy, are thoroughly examined to underscore the platform's reliability and robustness.

Furthermore, the web application aspect of FMdeploy is presented, elucidating the user interface and interaction mechanisms. This chapter also delves into containerization and deployment, shedding light on the technology that allows FMdeploy to operate seamlessly across various environments. For a thorough understanding of the technologies and concepts discussed in this chapter, readers are encouraged to consult APPENDIX 1 - "Technologies and Concepts". These concepts and technologies are elaborated within this appendix, offering a deeper insight into their roles and integration within the FMdeploy platform.

## 4.1 SYSTEM REASONING

### 4.1.1 DEFINITION OF SYSTEM IDENTITY

The FMdeploy research platform aims to provide a user-friendly web interface for deploying and interacting with ML and DL models. The web interface offers intuitive interactions such as form filling and drag-and-drop file uploading, allowing users to engage with the models easily.

In FMdeploy, a project encompasses various elements, including the title, description, author, model files, and Python scripts. These scripts can range from simple Python functions to complex ML models. The platform simplifies running ML models by enabling users to submit an input file and await the output.

Additionally, the platform allows users to flag project runs if the outputs do not meet their expectations. The project authors can then review these flags and take appropriate actions. These functionalities encourage collaboration within the FMdeploy platform.

## 4.2 SYSTEM ARCHITECTURE

The system architecture aims to create a secure and user-friendly platform for the execution of ML models. Its foremost objective is to ensure that the requirements delineated in the preceding planning chapter are accomplished proficiently.

By leveraging well-established technologies known for their speed and adaptability, this architecture ensures the development of an intuitive interface accessible to users with varying technical expertise.

### 4.2.1 PROPOSED SOLUTION

Figure 8 presents a high-level diagram of the platform, comprising a full-stack application with a database, API, and frontend components. The platform is designed to minimize the setup required for users to interact with ML models in the web app. Assuming the user already has a prepared ML model and accompanying Jupyter Notebook, only minor modifications are needed to generate a Python script with suitable system integration parameters. The Python script and models can then be uploaded through the web app. Users can access various actions once a model is successfully uploaded, including search, run, edit, share, delete, and more. This straightforward architecture ensures compatibility across different hardware and operating systems by leveraging technologies that support cross-platform functionality.

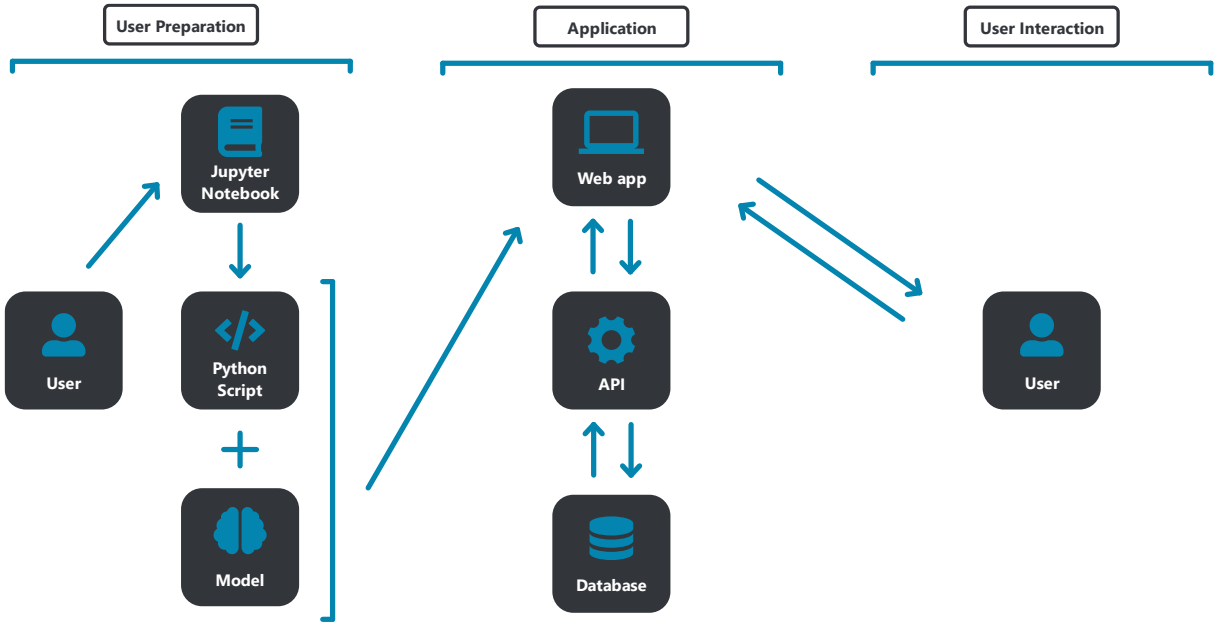



Figure 8 - User preparation and interaction with the application.

### 4.2.2 OVERVIEW

The platform will be available to users through web interfaces on any internet-connected browser. It will be hosted on a dedicated instance within servers at the University of Minho, with secure connections to both the web interface and API. The API is designed for integration with authorized endpoints, enabling interaction with third-party software. Using a REST API approach, applications such as mobile apps or programmatic access with Python can interact with the system once authorized. The system architecture, illustrated in Figure 9, starts with the server hardware (Intel CPU, NVIDIA GPU) and operating system (Ubuntu). The Docker host contains the database, which is connected to the API. The Gunicorn web server serves the API, with multiple Uvicorn workers running the FastAPI application. The web interface, built with React, is served by Nginx. HTTPS connections, facilitated by Traefik, ensure a secure browsing experience for both the API and UI.

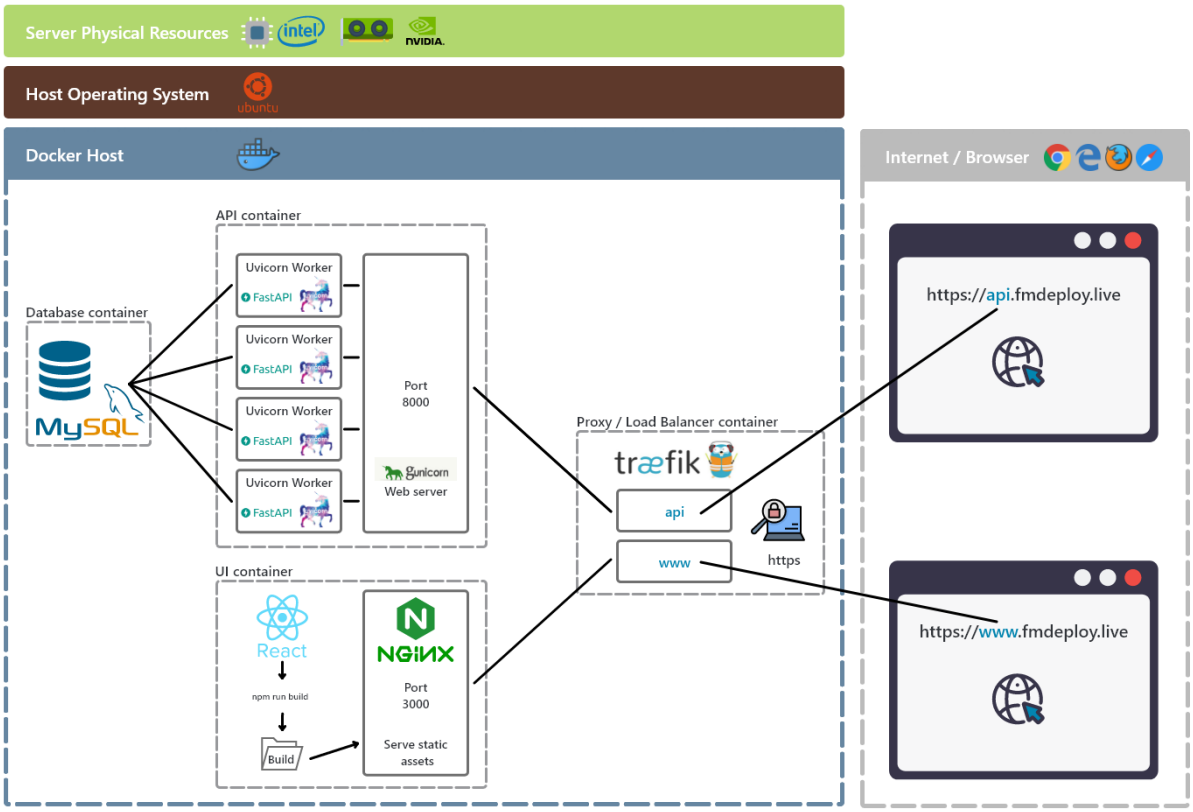



Figure 9 – FMdeploy System Architecture.

### 4.2.3 DATA SECURITY AND ACCESS CONTROL

The platform strongly emphasizes data security and access control to safeguard user information. Python scripts and ML models uploaded to the platform remain securely stored unless intentionally deleted by the user. Furthermore, the platform implements a customizable data retention policy, ensuring the timely deletion of inputs and outputs to enhance security and minimize storage requirements.

For secure data transfer, the platform uses HTTPS for all communication. This encrypts data transmission and ensures a secure channel for clients to connect with the trusted server.

Only authorized individuals can access and utilize the platform's features and functionalities. An admin must grant full access, and access control mechanisms are in place to regulate and manage user interactions within the system. These measures minimize the risks associated with unauthorized access and data breaches.

By implementing these comprehensive security measures, including HTTPS communication and controlled user access, the platform provides a secure environment for users to store, transfer, and manage their Python scripts, ML models, and associated data.

### 4.2.4 WORKFLOW

The workflow within the final app follows a structured process, allowing users to integrate their ML models developed in Jupyter notebooks. The assumption is that the models are already developed and tested within these notebooks, with the ideal scenario being the availability of the code in a Python script format.

The initial phase involves preparing the models for integration with the FMdeploy API. If the models are in a Jupyter Notebook, they must be converted into a Python script. To integrate the Python script, all that's needed is to establish "`load_models`" and "`run`" functions. The preparation phase is completed once the Python script and associated model files are ready.

With all the necessary preparations, the next step is to upload the files to the research platform. This can be achieved through the dedicated web application, programmatically using the API, or in the future, even via third-party applications that harness the platform's API, such as mobile applications.

The workflow starts with creating a new "Project," which serves as a container for the relevant data. This includes the project title, description, input and output types, the Python script, and the required model files. The web interface facilitates this process by providing everyday interactions, such as form filling and file dragging and dropping. Notably, the platform automatically validates the structure of the Python script during project creation, ensuring the presence of the essential functions.

Upon successful upload and project creation, authorized users gain access to the full range of functionalities the research platform offers through the web application. Access rights are carefully managed, allowing projects to be kept private, shared with specific users, or freely available to all platform users. The execution of models becomes a straightforward task, simply involving selecting and placing input files, followed by awaiting the generation of the corresponding output.

## 4.3 TESTING - MODEL DEPLOYMENT

An application's successful development and testing rely heavily on the quality and suitability of the models employed. In this dissertation, the application has been rigorously tested using

one segmentation and two classification models, which have also served as valuable testing assets throughout the development phase. These models have been carefully selected to ensure comprehensive functionality testing.

The following sections will provide a concise examination of each model, including brief overviews of its characteristics, capabilities, and application areas. By becoming familiar with these models, their functionalities for different classification assignments can be investigated, and a greater understanding of how the platform provides value can be achieved.

All three models will be publicly available on the research platform, allowing users to access and utilize them for their projects and research endeavors.

### 4.3.1 AUTOMATIC SEGMENTATION OF THE OLFACTORY BULB

The first model focuses on segmenting the olfactory bulb (OB), a crucial component of the human olfactory pathway. Changes in the volume of the OB have been linked to variations in olfactory function and have shown promise as prognostic indicators and biomarkers for neurodegenerative diseases such as Alzheimer's.

The segmentation algorithm employed a systematic four-step approach as outlined by Desser et al. [127]. It initially conducted multimodal data co-registration by aligning whole-brain T1 and T2 images. Subsequently, a template-based localization technique was utilized to identify the olfactory bulbs. This was followed by constructing a bounding box, facilitating the final step of segmenting the olfactory bulbs using a 3D-U-Net model.

The developed tool presented in the research paper [127] demonstrated the capability for automatic and reliable segmentation of olfactory bulbs. The algorithm achieved an efficient processing time of approximately 3-5 minutes per subject. Notably, the accuracy of the segmentations produced by this method closely paralleled the gold standard technique, which involves manual segmentations performed by experienced raters.

The model training process utilized the Monai Python Library, a powerful library specifically designed for medical imaging tasks. By leveraging multimodal data consisting of T1 whole-brain images and T2 coronal high-resolution images, the tool provides a ready-to-use solution for segmenting olfactory bulbs.

Lastly, the machine learning code and models related to this work are available in a dedicated GitHub repository[1].

### 4.3.2 DEEP LEARNING FOR REAL-TIME DECODING OF SOUND EVENTS RELATED TO AUTISM SPECTRUM DISORDER

The second model, developed in a dissertation by Diana Martins [128], focuses on classifying environmental sounds associated with auditory hypersensitivity in individuals with ASD. This model utilizes the XGBoost library and achieves an accuracy of 92.31%. The study begins with feature extraction from 519 sound files, extracting 44 features. To evaluate the impact of sounds on individuals with autism, the sounds are regrouped into three classes: "negative", "positive", and "unknown". The classification task is performed using XGBoost, resulting in an accuracy value of 92.31%. The study further explores the most important features for predicting each class using the plot importance function from the XGBoost library. By ordering the features based on their importance, it is observed that the top 20 features achieve the best accuracy value of 95.19%. The proposed XGBoost model for sound recognition and classification into three classes shows promise in addressing auditory hypersensitivity in individuals with ASD. It has the potential to aid in the identification of positive sounds that can be utilized in therapies targeting this issue.

Lastly, the machine learning code and models related to this work are available in a dedicated GitHub repository[2].

### 4.3.3 ENVIRONMENTAL SOUNDS AUTOMATIC CLASSIFICATION

The third model combines the fields of medical informatics and ambient sound analysis to develop intelligent systems capable of identifying sound sources and distinguishing different types of emotions conveyed in each sound wave.

The classification of environmental sounds, as proposed by Rafaela Machado [129], involves using techniques for feature extraction from sound data. The Python library Librosa was employed to facilitate this process, which offers a range of functionalities for audio analysis.

After extracting relevant features, two classification techniques were employed: Multilayer Perceptron (MLP) and XGBoost. MLP is an artificial neural network known for learning complex

[1] https://github.com/MarcelodeFreitas/Francisca-Assuncao-Model
[2] https://github.com/MarcelodeFreitas/Diana-Martins-Model

patterns and making predictions based on input data. XGBoost, on the other hand, is an optimized gradient-boosting framework that excels in handling large-scale datasets and achieving high prediction accuracy.

In order to classify the emotions conveyed in the sounds, clustering techniques were utilized to group sound waves with similar characteristics. This clustering process aimed to enhance the performance of the MLP model, which was later used to learn how to classify sound waves at an emotional level.

Additionally, the analysis of emotions in sound waves involved the application of the K-Means algorithm, which facilitated the grouping of sounds with similar characteristics based on their acoustic features. This clustering approach provided valuable insights into the emotional content of the sound data.

Lastly, the machine learning code and models related to this work are available in a dedicated GitHub repository[3].

## 4.4 MACHINE LEARNING MODEL DEPLOYMENT

The successful deployment of ML models requires a comprehensive understanding of how and where the models were developed. This initial investigation involved analyzing the development process to gain insights into implementing an effective deployment solution.

Students within personal Docker containers developed all three models discussed in the previous section. The first step was to examine the development environment thoroughly. Key findings from the analysis revealed that all containers are hosted on a server at the University of Minho, and students are granted access by a professor or the dissertation supervisor. Container management uses Portainer; all containers run the same Python and library versions. This standardized environment ensures consistency across all containers, especially those used for developing and testing the example models.

Having answered the question of "where" the models were developed – within Docker containers on a University of Minho server with strictly maintained library versions – the following question to address is "how". To find an answer, each Docker container was thoroughly reviewed. It was discovered that all three containers shared similar characteristics. Each container had one or more pre-trained models and Jupyter notebooks that utilized these models to perform their intended functions. In essence, the Jupyter notebooks took in input

[3] https://github.com/MarcelodeFreitas/Rafela-Machado-Model

files, validated and prepared the inputs, and then ran classification or segmentation tasks by importing the pre-trained models, ultimately generating one or more output files.

After a comprehensive review of the structure and functionality of the Jupyter notebooks, it was determined that combining Python with strict version control offered the most effective approach for serving machine learning models through an API. This approach ensures greater accuracy and consistency in delivering results by leveraging the same functions used in the Jupyter notebooks.

The subsequent subchapters will delve into preparing Jupyter notebooks and deploying machine learning models through an API, providing a comprehensive understanding of deploying and utilizing models successfully.

Note that this endeavor's true goal is to create a platform capable of deploying diverse ML models, as long as the versions used for its development align with those utilized in the deployment solution. This work has been structured to be highly adaptable, enabling the incorporation of new model types into the platform. The key considerations mainly revolve around specifying the accepted input and output types and determining the appropriate manner of displaying the outputs. By maintaining consistency in the underlying framework, the platform can effectively support deploying various ML models, expanding its versatility and utility.

### 4.4.1 JUPYTER NOTEBOOK PREPARATION

During the Jupyter Notebook preparation phase, a local Python environment was created for each test model to ensure compatibility with the server's Python version and library versions. The analysis focused on understanding the input and output mechanisms of each Jupyter notebook. Subsequently, the decision was made to convert the notebooks into simplified Python scripts to facilitate integration with the API, as elaborated in the following section. The development of two crucial functions, namely "`load_models`" and "`run`," aim to streamline the API's interaction with any model. While the models already had similar functions, they lacked generalization. Integrating a model with the API entails converting the Jupyter Notebook into a Python script and adapting the existing functions for loading and executing the models. Additionally, the Python logging library is implemented to provide comprehensive error logs, aiding users in troubleshooting issues such as mismatched model files or errors in any of the functions within the Python script.

All the code relating to the adaptation of the Jupyter Notebooks has been thoroughly documented in three GitHub repositories[4], one for each test model. These repositories serve as comprehensive resources, providing examples of the conversion process and the adapted code for each model.

The new "`load_models`" and "`run`" functions should adhere to the structure depicted in Code Snippet 1.

Diana-Martins-Model - sound_autism_fmdeploy.py

```python
def load_models(modelpaths):
    global model
    model = XGBClassifier()
    try:
        for i in modelpaths:
            name = i["name"]
            path = i["path"]
            model_list = ["sound_autism_model.json"]
            if name in model_list:
                model.load_model(path)
    except:
        return logging.exception("load_models: ")

def run(input_file_path, output_file_name, output_directory_path):
    try:
        input_validation(input_file_path)
        create_timestamps(input_file_path, output_directory_path)
        csv_header = create_csv_header()
        PATH_CSV_TIMESTAMPS = output_directory_path + 'features_timestamps.csv' #output csv file name
        PATH_TIMESTAMPS = output_directory_path + 'timestamps'
        generate_csv_music(csv_header, PATH_TIMESTAMPS, PATH_CSV_TIMESTAMPS)
        DATA_3=pd.read_csv(PATH_CSV_TIMESTAMPS)
        (X_TEST,Y_TEST) = load_data(DATA_3)
        series_prediction(model, output_file_name, output_directory_path, X_TEST, Y_TEST)
    except:
        return logging.exception("run: ")
```

Code Snippet 1 - "`load_models`" and "`run`" functions example.

The "`load_models`" function is responsible for handling a list of one or more models containing their file names and locations. The "`modelpaths`" list consists of the file names and paths of the models that have been uploaded to the platform. On the other hand, the "`model_list`" represents the expected model file names that the Python script expects.

During execution, the "`load_models`" function verifies if the provided model files in "`modelpaths`" match the expected file names in the "`model_list`" and have the correct file extensions. If any discrepancies are found, an error is logged for further examination. If the

[4] https://github.com/MarcelodeFreitas/Francisca-Assuncao-Model
https://github.com/MarcelodeFreitas/Diana-Martins-Model
https://github.com/MarcelodeFreitas/Rafela-Machado-Model

model file names and extensions align with the expected ones, the function utilizes the information from the respective model path to load the model.

Once the models are successfully loaded, they become available to use within the "`run`" function. The "`run`" function accepts the input file path, desired output file name, and output directory path as parameters. The remaining code within the "`run`" function mirrors the existing code in the original "`run`" present in the Jupyter Notebook. It is important to note that Code Snippet 1 is extracted from the Python script adapted explicitly for the automatic segmentation model of the olfactory bulb. With the pre-trained models and the new Python script in place, the subsequent step involved the development of the actual API.

### 4.4.2 API FOR ML DEPLOYMENT

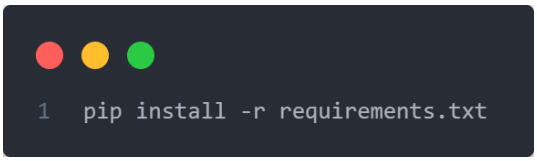
#### *4.4.2.1 PREPARATION OF THE DEVELOPMENT ENVIRONMENT*

In order to generate a comprehensive list of installed packages within the server containers, the Python package manager command freeze was utilized. Specifically, the following command (Code Snippet 2) was executed:


Code Snippet 2 - Freeze command.

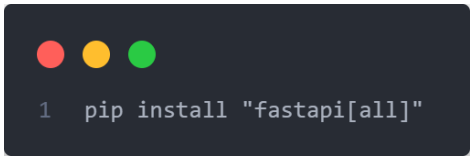
This command redirects the output of "`pip freeze`" to a file named "requirements.txt". The "`pip freeze`" command generates a list of all installed packages and their respective versions in the current Python environment. By capturing this list in the "requirements.txt" file, a snapshot of the installed packages, including both the direct dependencies and their sub-dependencies, was obtained.

Subsequently, the requirements.txt file was employed to recreate the same Python environment. By utilizing the file, all the required packages and their specific versions could be easily installed using the following command (Code Snippet 3):




Code Snippet 3 – Command to install requirements from a file.

Additionally, a few more packages necessary for FastAPI's functionality were installed. This was achieved by executing the following command (Code Snippet 4):




Code Snippet 4 - FastAPI install command.

By installing the "`fastapi[all]`" package, all the required dependencies for FastAPI, including additional optional packages and features, were installed in the Python environment.

It is essential to note that the Python versions captured using the "`freeze`" command originated from the Docker containers used for development, hosted on a University of Minho server. Subsequently, these versions were employed to recreate the identical environment on a personal/local machine, utilizing the "requirements.txt" file as the reference.

#### *4.4.2.2 ORM AND DATABASE CONFIGURATION*

This section focuses on leveraging the features of FastAPI and SQLAlchemy for Object-Relational Mapping (ORM) and database integration. APPENDIX 1 - "Technologies and Concepts" contains additional information detailing FastAPI and SQLAlchemy. SQLAlchemy, a versatile toolkit, empowers ORM techniques, streamlining the process of mapping Python classes to database tables, thereby simplifying database operations.

By adopting SQLAlchemy, adapting the API to different databases becomes straightforward. SQLAlchemy supports many databases, including PostgreSQL, MySQL, SQLite, Oracle, and Microsoft SQL Server. In this implementation, both MySQL and PostgreSQL are ready to use. To facilitate ease of use, relevant code for each database is clearly labeled with comments within files "`database.py`" and "`models.py`," as illustrated in Code Snippet 5 and Code Snippet 6. This labeling system enables flexibility in quickly switching databases; all that is required is

to comment out the sections related to the unused database. Notably, the changes needed to transition between these versions are minimal, as demonstrated below.

FMdeploy-FastAPI - database.py

```
# MYSQL
SQLALCHAMY_DATABASE_URL = (
    "mysql+mysqlconnector://root:password@localhost:3306/fmdeploy_db"
)
# POSTGRESQL
""" SQLALCHAMY_DATABASE_URL = 'postgresql://postgres:password@localhost/fmdeploy_db' """
```

Code Snippet 5 - Database connection.

FMdeploy-FastAPI - models.py

```
# add one admin the MySQL database after the table User is created
event.listen(
    User.__table__,
    "after_create",
    DDL(
        " INSERT INTO user (name, email, password, role) VALUES ('Admin Joe', 'fmdeploy@gmail.com', 'hashed_password', 'admin') "
    ),
)
# add one admin the PostgreSQL database after the table User is created
""" event.listen(
    User.__table__,
    "after_create",
    DDL(
        " INSERT INTO \"user\" (name, email, password, role) VALUES ('Admin Joe', 'fmdeploy@gmail.com', 'hashed_password', 'admin') "
    ),
) """
```

Code Snippet 6 - Add original admin to the database.

Code Snippet 5, the "`database.py`" file, defines the database connection. Additionally, the "`models.py`" file defines the database structure using SQLAlchemy's declarative syntax. Although SQL queries are not required, it is possible to incorporate SQL when needed, as exemplified by the creation of an admin in Code Snippet 6.

The subsequent analysis focuses on the models.py file, which provides a comprehensive understanding of how the database structure is defined.

When using SQLAlchemy, Python classes can be easily mapped to SQL tables. For instance, a SQL table named "`user`" can be represented by a Python class called "`User`." Each instance of this class corresponds to a row within the database. Along with table mapping, ORM tools establish connections and relationships between tables. It is worth noting that SQLAlchemy is utilized in this project, but alternative ORM frameworks could have been employed.

The provided Code Snippet 7 defines a Python class named "`User`," which corresponds to a SQL table in the database. The class inherits from the "`Base`" class, indicating its association with the SQLAlchemy ORM framework.

The "__tablename__" attribute specifies the name of the SQL table as "user." Each attribute within the class represents a column within the table. In this case, the "user_id" attribute is defined as an "Integer" type and serves as the primary key for the table. The "name", "email", "password", and "role" attributes are defined as "String" types and correspond to columns that store the user's name, email, password, and role, respectively.

Notably, the "role" attribute utilizes the "Enum" type provided by SQLAlchemy. This ensures that the role attribute can only have specific values defined within the "role_types" enumeration, namely "guest", "user", and "admin." The default value for the "role" attribute is set to "guest", and it is marked as non-nullable.

Additionally, the class establishes relationships with other tables by using the "relationship" function provided by SQLAlchemy. The "UserProject" and "RunHistory" tables are associated with the "User" table through the "userproject" and "run_history" attributes, respectively. These relationships allow for easier navigation and querying between related tables.

FMdeploy-FastAPI - models.py

```python
class User(Base):
    __tablename__ = "user"
    user_id = Column(Integer, primary_key=True, index=True)
    name = Column(String(length=255), nullable=False)
    email = Column(String(length=255), unique=True, nullable=False)
    password = Column(String(length=255), nullable=False)
    role = Column(
        Enum("guest", "user", "admin", name="role_types"),
        default="guest",
        nullable=False,
    )
    userproject = relationship("UserProject", backref="User", passive_deletes=True)
    run_history = relationship("RunHistory", backref="User", passive_deletes=True)
```

Code Snippet 7 - "User" class maps to "user" table.

The Code Snippet 8 defines a Python class named "UserProject" that maps to a corresponding SQL table called "userproject."

Within the "UserProject" class, the "__tablename__" attribute specifies the name of the SQL table as "userproject." This attribute serves as a mapping between the Python class and the corresponding table in the database.

The class has several attributes, each representing a column within the "userproject" table. The "user_project_list_id" attribute, defined as an Integer type, serves as the primary key for the table. It uniquely identifies each row within the table.

Furthermore, the "fk_user_id" attribute is an "Integer" column that establishes a foreign key relationship with the "user" table. This attribute references the "user_id" column in the "user" table and is marked as non-nullable to ensure data integrity. Similarly, the "fk_project_id" attribute, defined as a "String" column, establishes a foreign key relationship with the "project" table. It references the "project_id" column and is also marked as non-nullable.

Additionally, the "owner" attribute, represented by a "Boolean" column, indicates whether the user associated with a particular "UserProject" instance is the owner of the related project. It defaults to "False" and is marked as non-nullable.

FMdeploy-FastAPI - models.py

```
38  class UserProject(Base):
39      __tablename__ = "userproject"
40      user_project_list_id = Column(Integer, primary_key=True, index=True)
41      fk_user_id = Column(
42          Integer, ForeignKey("user.user_id", ondelete="CASCADE"), nullable=False
43      )
44      fk_project_id = Column(
45          String(length=255),
46          ForeignKey("project.project_id", ondelete="CASCADE"),
47          nullable=False,
48      )
49      owner = Column(Boolean, default=False, nullable=False)
```

Code Snippet 8 - "UserProject" class maps to "userproject" table.

The provided Code Snippet 9 showcases the definition of a Python class named "Project," representing a SQL table in the database. Similar to the previous examples, this class is based on the SQLAlchemy "Base" class, indicating its association with the ORM framework.

The "__tablename__" attribute specifies the name of the SQL table as "project." Each attribute within the class corresponds to a column in the table. The "project_id" attribute is defined as a "String" type and serves as the primary key for the table. The "title," "description," "input_type", and "output_type" attributes are defined as "String" types and represent columns storing the project's title, description, input type, and output type, respectively.

Additionally, the class includes attributes such as "python_script_name" and "python_script_path," which are "Strings" indicating the name and path of the associated Python script for the project. The "is_private" attribute is a "Boolean" that defaults to "True" and indicates whether the project is private. The "created_in" attribute is a "DateTime" that

stores the creation date of the project, while the "last_updated" attribute stores the date of the last update, allowing for tracking changes over time.

The class establishes relationships with other tables using the "relationship" function provided by SQLAlchemy. The "Project" table has associations with the "ModelFile" and "RunHistory" tables through the "modelfile" and "run_history" attributes, respectively. These relationships facilitate queries and navigation between the related tables.

Overall, this class definition demonstrates the structure and attributes of the "Project" table within the database, allowing for effective storage and retrieval of project-related information.

FMdeploy-FastAPI - models.py

```python
class Project(Base):
    __tablename__ = "project"
    project_id = Column(String(length=255), primary_key=True, index=True)
    title = Column(String(length=255), nullable=False)
    description = Column(String(length=255), nullable=False)
    input_type = Column(String(length=255), nullable=False)
    output_type = Column(String(length=255), nullable=False)
    python_script_name = Column(String(length=255), nullable=True)
    python_script_path = Column(String(length=255), nullable=True)
    is_private = Column(Boolean, default=True, nullable=False)
    created_in = Column(DateTime, nullable=False)
    last_updated = Column(DateTime, nullable=True)
    modelfile = relationship("ModelFile", backref="Project", passive_deletes=True)
    run_history = relationship("RunHistory", backref="Project", passive_deletes=True)
```

Code Snippet 9 - "Project" class maps to "project" table.

The provided Code Snippet 10 illustrates the definition of a Python class named "ModelFile," representing a SQL table in the database.

The "__tablename__" attribute specifies the SQL table's name as "modelfile." Each attribute within the class corresponds to a column within the table. The "model_file_id" attribute is defined as an "Integer" type and serves as the primary key for the table. The "fk_project_id" attribute is a "String" type and acts as a foreign key, establishing a relationship with the "project" table by referencing its "project_id" column. This relationship is defined with the "ForeignKey" function, ensuring that the "project_id" value is adequately linked and enabling cascading deletion when necessary.

Furthermore, the "name" attribute is defined as a "String" type and represents the name of the model file. The "path" attribute, also a "String" type, denotes the file path of the model.

These attributes provide crucial information about the model files associated with a particular project.

By defining the "ModelFile" class, the corresponding SQL table can effectively store and retrieve information regarding model files, linking them to their respective projects through the foreign key relationship.

FMdeploy-FastAPI - models.py

```python
class ModelFile(Base):
    __tablename__ = "modelfile"
    model_file_id = Column(Integer, primary_key=True, index=True)
    fk_project_id = Column(
        String(length=255),
        ForeignKey("project.project_id", ondelete="CASCADE"),
        nullable=False,
    )
    name = Column(String(length=255), nullable=False)
    path = Column(String(length=255), nullable=False)
```

Code Snippet 10 - "ModelFile" class maps to "modelfile" table.

The provided Code Snippet 11 showcases the definition of a Python class named "InputFile," representing a SQL table in the database.

The "__tablename__" attribute specifies the SQL table's name as "inputfile." Each attribute within the class corresponds to a column within the table. The "input_file_id" attribute is defined as a "String" type and serves as the primary key for the table. This attribute provides a unique identifier for each input file entry.

Additionally, the "name" attribute is defined as a "String" type and represents the name of the input file. The "path" attribute, also a "String" type, denotes the file path of the input file. These attributes store essential information about the input files associated with the corresponding records.

Moreover, the class establishes a relationship with the "RunHistory" table using the "relationship" function provided by SQLAlchemy. The "RunHistory" table is associated with the "InputFile" table through the "run_history" attribute, allowing for easier navigation and querying between these related tables.

By defining the "InputFile" class, the corresponding SQL table can effectively store and retrieve information about input files. The relationship with the "RunHistory" table facilitates the traceability and analysis of the input files in specific run history entries.

FMdeploy-FastAPI - models.py

```
80  class InputFile(Base):
81      __tablename__ = "inputfile"
82      input_file_id = Column(String(length=255), primary_key=True, index=True)
83      name = Column(String(length=255), nullable=False)
84      path = Column(String(length=255), nullable=False)
85      run_history = relationship("RunHistory", backref="InputFile", passive_deletes=True)
```

Code Snippet 11 - "InputFile" class maps to "inputfile" table.

The provided Code Snippet 12 defines a Python class named "OutputFile", representing a SQL table in the database.

The "__tablename__" attribute specifies the SQL table's name as "outputfile". Each attribute within the class corresponds to a column within the table. The "output_file_id" attribute is defined as a "String" type and serves as the primary key for the table. It provides a unique identifier for each output file entry.

Furthermore, the "name" attribute, defined as a "String" type, represents the name of the output file. The "path" attribute, also a "String" type, denotes the file path of the output file. These attributes store essential information about the output files associated with the corresponding records.

Moreover, the class establishes a relationship with the "RunHistory" table using the "relationship" function provided by SQLAlchemy. The "RunHistory" table is associated with the "OutputFile" table through the "run_history" attribute, facilitating easier navigation and querying between these related tables.

By defining the "OutputFile" class, the corresponding SQL table can effectively store and retrieve information about output files. The relationship with the "RunHistory" table allows for seamless tracking and analysis of the output files generated during specific run history entries.

FMdeploy-FastAPI - models.py

```python
class OutputFile(Base):
    __tablename__ = "outputfile"
    output_file_id = Column(String(length=255), primary_key=True, index=True)
    name = Column(String(length=255), nullable=False)
    path = Column(String(length=255), nullable=False)
    run_history = relationship("RunHistory", backref="OutputFile", passive_deletes=True)
```

Code Snippet 12 - "OutputFile" class maps to "outputfile" table.

The provided Code Snippet 13 showcases the definition of the last Python class named "RunHistory," representing a SQL table in the database.

The "__tablename__" attribute specifies the SQL table's name as "runhistory". Each attribute within the class corresponds to a column within the table. The "run_history_id" attribute, defined as an "Integer" type, serves as the primary key for the table. It provides a unique identifier for each run history entry.

Furthermore, the "fk_user_id" attribute references the "user_id" column in the "user" table, establishing a foreign key relationship. Similarly, the "fk_project_id" attribute references the "project_id" column in the "project" table, serving as a foreign key. These foreign key relationships enable the association of run history entries with specific users and projects.

Moreover, the "fk_input_file_id" and "fk_output_file_id" attributes establish foreign key relationships with the "inputfile" and "outputfile" tables, respectively. These relationships allow for linking the run history records to specific input and output files associated with each run.

Additionally, the "flagged" attribute, defined as a "Boolean" type, indicates whether a run history entry is flagged. The "flag_description" attribute, a "String" type, stores a description of the flag if applicable. The "timestamp" attribute, defined as a "DateTime" type, records the timestamp of the run history entry.

By defining the "RunHistory" class, the corresponding SQL table can effectively store and retrieve information about run histories. The foreign key relationships with other tables allow for comprehensive tracking and analysis of the relationships between users, projects, input files, output files, and their associated run history entries.

FMdeploy-FastAPI - models.py

```python
class RunHistory(Base):
    __tablename__ = "runhistory"
    run_history_id = Column(Integer, primary_key=True, index=True)
    fk_user_id = Column(
        Integer, ForeignKey("user.user_id", ondelete="CASCADE"), nullable=False
    )
    fk_project_id = Column(
        String(length=255),
        ForeignKey("project.project_id", ondelete="CASCADE"),
        nullable=False,
    )
    fk_input_file_id = Column(
        String(length=255),
        ForeignKey("inputfile.input_file_id", ondelete="CASCADE"),
        nullable=False,
    )
    fk_output_file_id = Column(
        String(length=255),
        ForeignKey("outputfile.output_file_id", ondelete="CASCADE"),
        nullable=True,
    )
    flagged = Column(Boolean, default=False, nullable=True)
    flag_description = Column(String(length=255), nullable=True)
    timestamp = Column(DateTime, nullable=False)
```

Code Snippet 13 - "RunHistory" class maps to "runhistory" table.

With Python classes accurately mapping to the database tables and a defined database connection, the FMdeploy API automatically creates the database if it does not exist. When utilizing MySQL, the reverse engineering feature generates the Enhanced Entity-Relationship (EER) diagram shown in Figure 10.

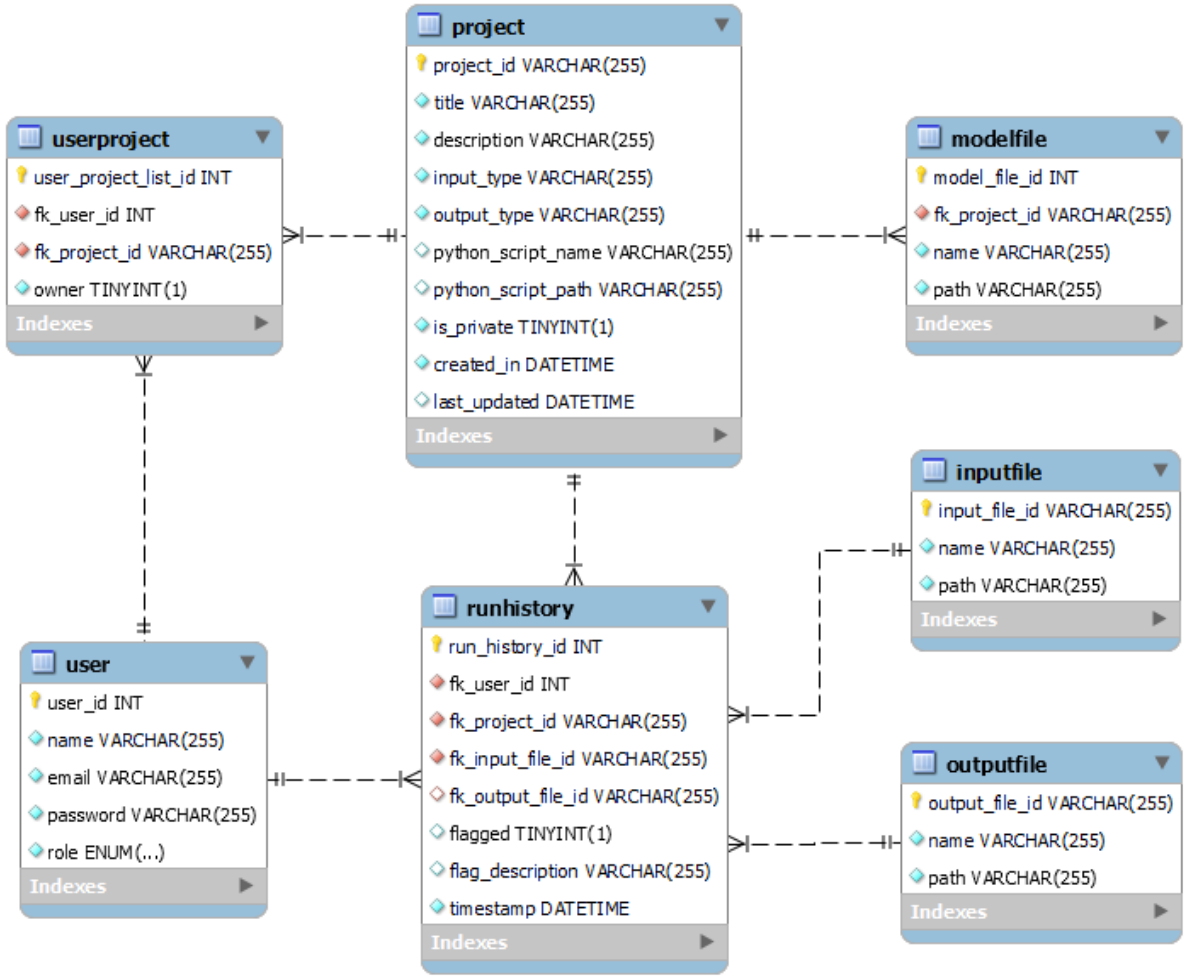



Figure 10 – MySQL Enhanced Entity-Relationship (EER) diagram.

The EER diagram in Figure 10 visually represents the structure and relationships of the database, concisely capturing the tables and their attributes. It offers a clear overview of the database schema and entity connections, illustrating interrelationships between the tables through foreign key constraints.

Developers and stakeholders benefit from the EER diagram as it provides a visual understanding of the database's structure, including entities, attributes, and relationships.

In conclusion, the EER diagram is a valuable resource, consolidating information from the database tables into a concise visual representation. It facilitates a better understanding of the database structure, identification of entity connections, and supports effective data management and analysis.

#### 4.4.2.3 API IMPLEMENTATION DETAILS

With the Python environment, Python scripts, and database prepared, the development of the API progressed smoothly. This section highlights the key implementation details, focusing on the API's essential functionalities and design considerations.

A significant decision in API development was the selection of the FastAPI framework, a modern, high-performance web framework offering advantages such as automatic request and response validation, asynchronous capabilities, and automatic API documentation.

In the upcoming sections, selected code snippets will be reviewed, explaining their purposes and functionalities in detail. These snippets will provide valuable insights into how the core functions of the API are achieved by leveraging FastAPI, showcasing its modern capabilities and performance.

The API endpoints will be presented, outlining their functionalities, input/output formats, and any relevant authentication or authorization mechanisms. Special attention will be given to the usability and functionality of the API from a user's perspective.

To start the exploration of the API, an analysis of the directory structure available in the GitHub repository[5] and illustrated in Figure 11 is warranted. The "app" directory is the central location for all API logic, containing request and response handling code. Dockerfiles pertinent to containerization are present and will be discussed in detail in the appropriate section. Additionally, the "requirements.txt" file in the same directory specifies the required dependencies for the Project, as mentioned earlier.

[5] https://github.com/MarcelodeFreitas/FMdeploy-FastAPI/tree/latest

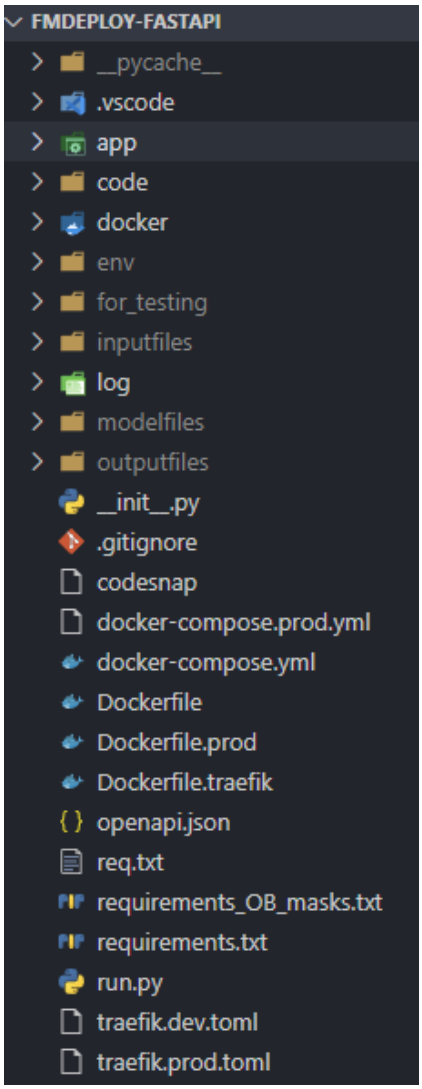



Figure 11 – FMdeploy complete API directory.

Within the repository, the "run.py" script is crucial in starting the API using a web server called Uvicorn. Uvicorn is a lightning-fast ASGI server that provides excellent performance and scalability for the API. When executed, "run.py" not only starts Uvicorn but also creates three directories: "`modelfiles`", "`inputfiles`", and "`outputfiles`". These directories store large API-related files, such as trained models, and input and output files. Storing these large files outside the database offers several benefits, such as reduced database size, improved API performance, and simplified file access and management. In the development environment, the "run.py" script enables hot reloading, a valuable feature FastAPI provides. Hot reloading automatically detects code changes and restarts the server, facilitating real-time testing and development.

Moreover, the "run.py" script includes code that leverages Gunicorn, a mature HTTP server commonly used in production environments. Gunicorn enhances API performance and

scalability, ensuring smooth operation even under high loads through features like multiple worker processes. This combination of Gunicorn and Uvicorn guarantees efficient handling of concurrent requests.

Code Snippet 14 provides an example of the "run.py" script, demonstrating the creation of directories and the execution of Uvicorn with different options. It showcases configuring the host, port, and other parameters based on specific deployment requirements.


Code Snippet 14 – FMdeploy API "run.py"

Code Snippet 14 illustrates the setup for the Uvicorn server, specifying the necessary directories to monitor for changes during hot reloading. It also provides examples of alternative configurations for the host and port settings, allowing flexibility in deployment options.

Overall, the "run.py" script and the usage of Uvicorn and Gunicorn facilitate the seamless execution and deployment of the API, enabling efficient handling of requests and delivering high-performance results.

Moving forward, the examination of the code within the "app" directory, as illustrated in Figure 12, is the next step. Within this directory, the "database.py" file is responsible for establishing the connection with the database, as explained earlier. The models.py" file, discussed previously, can also be found here. Additionally, the "schemas.py" file contains the pydantic schemas used for reading and returning data within the API.

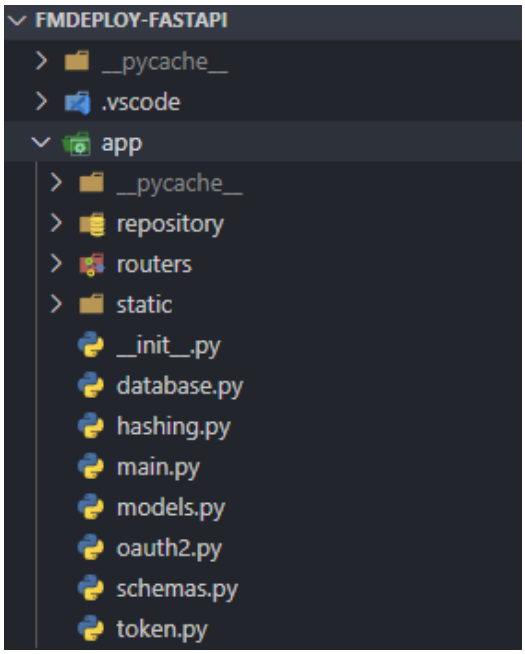



Figure 12 - FMdeploy API "app" directory.

The "schemas.py" file plays a crucial role in data manipulation while reading and returning data from the API. It is important to note that the "`ShowUser`" Pydantic model, utilized when reading a user (returning it from the API), excludes sensitive information such as passwords and IDs. In contrast, when displaying information to an admin, the user ID becomes relevant, including the "`user_id`" field in the "`ShowUserAdmin`" model.

Referring to Code Snippet 15 from the FMdeploy API "schemas.py" file, it is worth mentioning that the provided code excerpt demonstrates the usage of Pydantic schemas. These are examples of how the schemas define the structure and characteristics of the data models.

FMdeploy-FastAPI - schemas.py

```python
class ShowUser(BaseModel):
    name: str
    email: str

    class Config:
        orm_mode = True


class ShowUserAdmin(BaseModel):
    user_id: str
    name: str
    email: str
    role: str

    class Config:
        orm_mode = True
```

Code Snippet 15 - "ShowUser" and "ShowUserAdmin" Pydantic models from schemas.py

The Pydantic models in the provided code snippet include an internal "`Config`" class, which provides configurations to Pydantic. Within this Config class, the attribute "`orm_mode`" is set to "`True`". By enabling "`orm_mode`", Pydantic can access the necessary data through the model attributes, allowing the explicit declaration of the specific data to be returned, even when working with ORMs like SQLAlchemy that employ "lazy loading" as their default behavior.

In the same "app" directory, there are three additional files: "hashing.py", "oauth2.py", and "token.py". These files handle the application's security, authentication, and authorization aspects. They utilize standard libraries recommended by FastAPI for these purposes. While important for the functioning of the application, these files represent common security practices and will not be further discussed as they are not unique features of this specific implementation.

The "main.py" file serves as the entry point for a FastAPI application. It initializes the FastAPI instance, configures CORS (Cross-Origin Resource Sharing) to allow requests from specific origins, and defines routes and endpoints using routers. The "main.py" file also includes a route for the root URL "/", which returns an HTML landing page. This landing page provides information about the API and convenient links to access the automatically generated API documentation. Figure 13 illustrates the landing page:

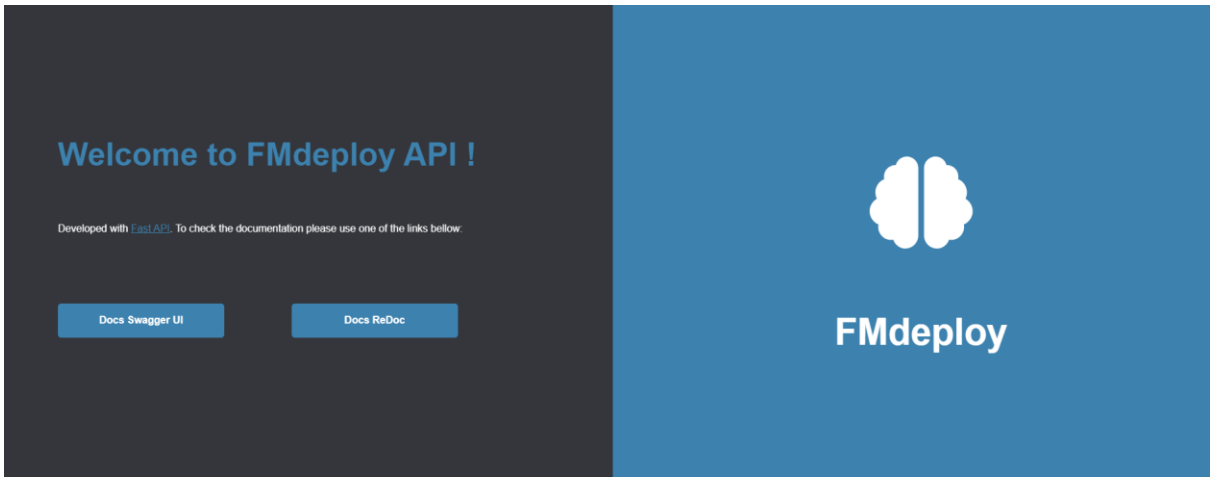



Figure 13 – FMdeploy API landing page.

Furthermore, "main.py" handles file cleanup by scheduling a task to delete files that haven't been accessed in 24 hours. This mechanism not only reduces the storage time of potentially sensitive data, enhancing security and privacy, but it also helps optimize storage needs and reduce the load on the server.

It is important to note that the specific time threshold for file deletion can be adjusted according to the implementation requirements. This flexibility allows customization to align with specific security policies and data retention practices.

In summary, "main.py" is a central file configuring the FastAPI application, defining routes and endpoints, and providing access to the automatic API documentation. It also includes a landing page and file cleanup functionality, contributing to a smooth and secure API experience.

Next, the directories "routers" and "repository" within the "app" directory merit attention. The "routers.py" file contains API routers that define various endpoints, categorized into "files.py", "project.py", "runhistory.py", "user.py", and "userproject.py". In contrast, the "repository" directory houses counterparts to the routers, with the same file names but encompassing the functions imported by the routers. These "repository" files hold the logic and operations required to process requests and provide the expected responses.

The "User" routers handle endpoints related to retrieving user roles, obtaining user information, and updating user details such as name, email, and password. User creation, without role selection, is accessible to all users. Administrative functionalities include obtaining a user list, creating users with specific roles, updating user roles, and deleting users by their ID and email.

Figure 14 showcases the components and interactions of the "User" routers and associated endpoints.


Figure 14 - User API documentation.

The "File" routers, depicted in Figure 15, offer functionalities for uploading input files, Python scripts, and model files. They also provide methods to check associated model files for a project and retrieve stored inputs and outputs.


Figure 15 - Files API documentation.

Figure 16 illustrates the "Project" routers, which manage project information updates such as title, description, input and output specifications, and privacy settings (public or private). These routers handle project creation, retrieval of public projects, retrieval of specific public projects based on ID and title, project deletion, and project execution.


Figure 16 - Project API documentation.

The "User Project" routers, shown in Figure 17, provide endpoints to retrieve project owners, list projects owned by users, share projects, cancel sharing, check project sharing status, list shared projects, and view the list of users with whom a project has been shared.

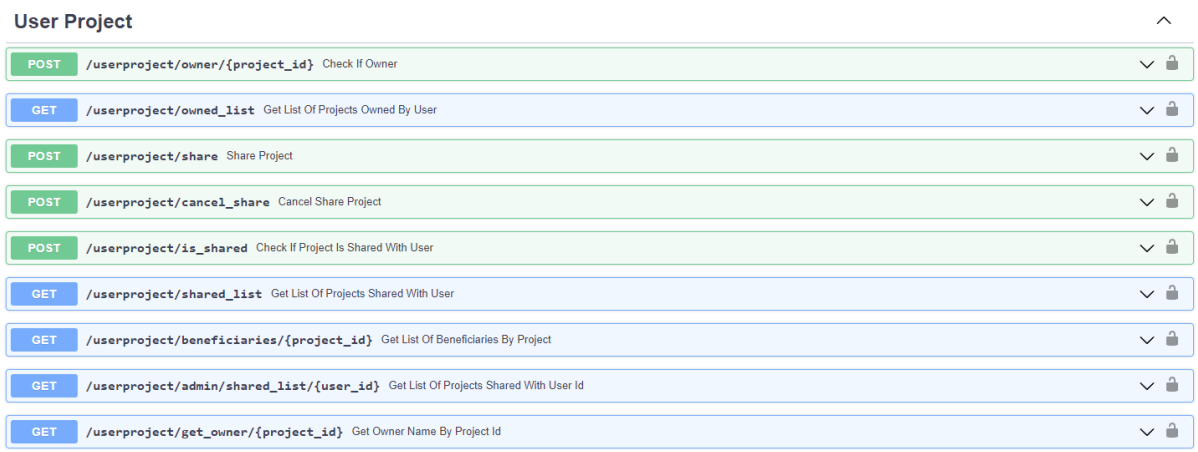



Figure 17 - User Project API documentation.

Lastly, the "Run History" routers, shown in Figure 18, are responsible for endpoints related to retrieving the run history, accessing project flags, and creating flags.

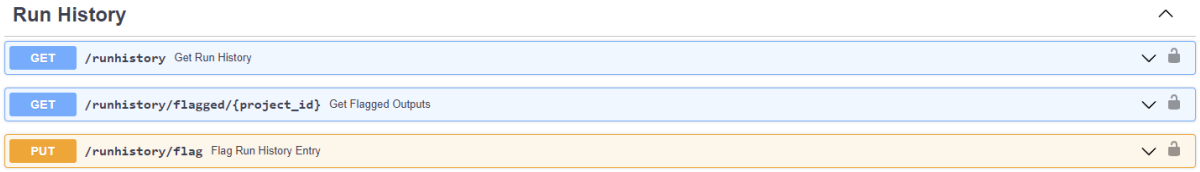



Figure 18 - Run History API documentation.

Figure 14 to Figure 18 display the automatic documentation generated by FastAPI, which utilizes the Pydantic response models. This documentation provides comprehensive information about available endpoints, their functionalities, and the structure of request and response data based on the defined Pydantic models.

The "`run`" function, defined as an asynchronous function in the "repository" "project.py" file, is the core component responsible for executing projects within the platform. Its asynchronous nature allows for efficient and concurrent processing of multiple requests, ensuring optimal performance and responsiveness.

In addition to projects with pre-trained models, an opportunity was identified to introduce projects without models, enhancing the platform's value. This led to the development of the "`run_simple_script`" function, enabling users to perform various Python functions within a project. This feature acts as a support mechanism for other projects, allowing the creation of auxiliary projects to address specific requirements.

For example, consider a project that exclusively accepts .wav files as input. A supporting project can be created solely for converting ".mp3" files to the required ".wav" format. This empowers individuals with commonly available ".mp3" files to utilize models that rely on .wav

files. While specific updates to the original project script require authorization from the project author, the platform encourages community-driven solutions to niche issues by enabling users to create support projects. This flexibility promotes the creation of on-demand solutions by anyone, fostering adaptability.

The "`run`" function performs a series of checks and validations. It verifies the existence of the provided user and project IDs, ensuring the necessary permissions for accessing the Project. Additionally, it checks the availability of input files and the presence of required Python script files associated with the Project.

Once the checks are completed, the function registers the project execution in the "`runhistory`" table. Depending on the Project's characteristics, it determines if pre-existing model files are required. If so, it ensures the presence and availability of these files before initiating the Project's execution using the "`run_script`" function. If no model files are present, the "`run_simple_script`" function is called instead.

Upon successful execution, the function generates an output file and performs additional checks to ensure its existence. It determines the appropriate media type for the output file based on the Project's output type. This enables the return of the output file as a response, facilitating access and further processing.

In cases where errors occur during the execution process, such as missing output files or exceptions, the function handles these scenarios and provides relevant error responses or log files.

The "`run_script`" function executes the Python script associated with a project with model files. It follows a series of steps to carry out this execution.

Firstly, the function retrieves necessary information, such as the name of the Python script, input file details, and the Project's output type. It then creates an output directory specific to the input file, ensuring that it exists or is created if not already present.

Next, the function checks if the Project has a valid file directory. If the directory is not found, an HTTP exception is raised to indicate the absence of the file directory.

The function adds the Project's file directory path to the system's search path using the "`sys.path`" to enable the import of the Python script as a "`module.append()`" method.

The Python script module is then imported using "`importlib.import_module()`", which allows access to its functions.

To capture relevant information during the execution of the script, the function sets up a logging system. It creates a log file, sets the log level, and defines the log format.

The function executes the "`load_models`" function from the Python script, which loads the required model files.

Following that, the "`run_script`" function from the Python script is executed. This function performs the core logic of the script, taking the input file path, output file name, and output directory path as arguments.

After executing the script, the function checks if the output file exists and verifies that it has the expected output type. If any issues are encountered, appropriate log entries are recorded.

The output file information is then added to the database and the run history table, associating it with the corresponding input file.

Finally, the function returns the path of the generated output file.

The "`run_simple_script`" function executes a Python script associated with a project that does not require model files. This function is similar to the "`run_script`" function but is designed to handle projects without models.

The function begins by retrieving relevant information such as the Python script name, input file details, and the Project's output type. It also creates the necessary output directory specific to the input file, ensuring its existence or creating it if needed.

Like the "`run_script`" function, "`run_simple_script`" checks if the Project has a valid file directory. If the directory is not found, an HTTP exception is raised to indicate the absence of the file directory.

The function appends the Project's file directory path to the system's search path using "`sys.path.append()`", allowing the Python script module to be imported.

A logging system is established to capture relevant information during script execution. A log file is created, log levels are set, and a log formatter is defined.

The function then executes the "run" function from the imported Python script module. This function performs the core logic of the script, taking the input file path, output file name, and output directory path as arguments.

After executing the script, the function checks if the output file exists and verifies that it has the expected output type. If any issues are encountered, appropriate log entries are recorded.

Output file information is added to the database and the run history table, associating it with the corresponding input file.

Finally, the function returns the path of the generated output file.

In this section, the implementation details of the API have been explored. The analysis focused on the directory organization and the functionality of various components within the "app" directory. The examination encompassed files such as "database.py", "models.py", and "schemas.py", which are integral in establishing database connections, defining data schemas, and facilitating database operations. The "run" function was discussed, highlighting its asynchronous nature that enables efficient handling of concurrent requests, optimizing the overall performance of the API. Additionally, the "`run_script`" and "`run_simple_script`" functions were examined, emphasizing their distinct features and how they handle projects with varying requirements. This comprehensive overview provides valuable insights into the API's implementation, shedding light on its internal mechanisms.

## 4.5 WEB APPLICATION

In modern software development, web applications are crucial in providing intuitive user interfaces and seamless interactions with backend systems. This section focuses on developing the FMdeploy web application, built using the powerful React library. React enables the creation of dynamic and responsive user interfaces, enhancing the overall user experience while leveraging the functionalities offered by the FMdeploy API.

React was selected as the foundation for web application development due to its extensive features and capabilities. Its component-based architecture facilitates the creation of reusable UI components, ensuring efficient code organization and maintenance. Additionally, React's virtual DOM manipulation and efficient rendering algorithms optimize performance, resulting in a smooth and seamless user interface.

While this chapter acknowledges the presence of standard web application features such as user registration, sign-in, and profile editing, the primary focus lies in highlighting the unique and essential features of the FMdeploy web application. This includes providing user interfaces that enable the deployment of pre-trained machine-learning models. The FMdeploy web application boasts support for various model types and possesses the flexibility to adapt and accommodate future deployment challenges.

Throughout this subchapter, the relevant components and pages of the FMdeploy web application will be reviewed in detail. The intended use of each page will be explained, and guidance will be provided on navigating and utilizing the application's features effectively. By

examining the design and functionality of the web application, readers will gain a comprehensive understanding of its purpose and capabilities.

This subchapter serves as a comprehensive guide to the web application development process using React within the context of the FMdeploy platform.

The subsequent section embarks on an in-depth exploration of the components, pages, and features comprising the FMdeploy web application. To ensure a comprehensive understanding of the FMdeploy web application, delving into its diverse components, pages, and features is highly recommended. This approach facilitates a thorough grasp of the application's structure, empowering users to navigate its interface effortlessly and effectively leverage its extensive functionalities.

### 4.5.1 SPECIFYING THE USER INTERACTION

In this section, the interfaces and functionalities of the FMdeploy web application, developed using the React library, will be examined. The application's codebase is available in a GitHub repository[6].

#### *4.5.1.1 LANDING PAGE*

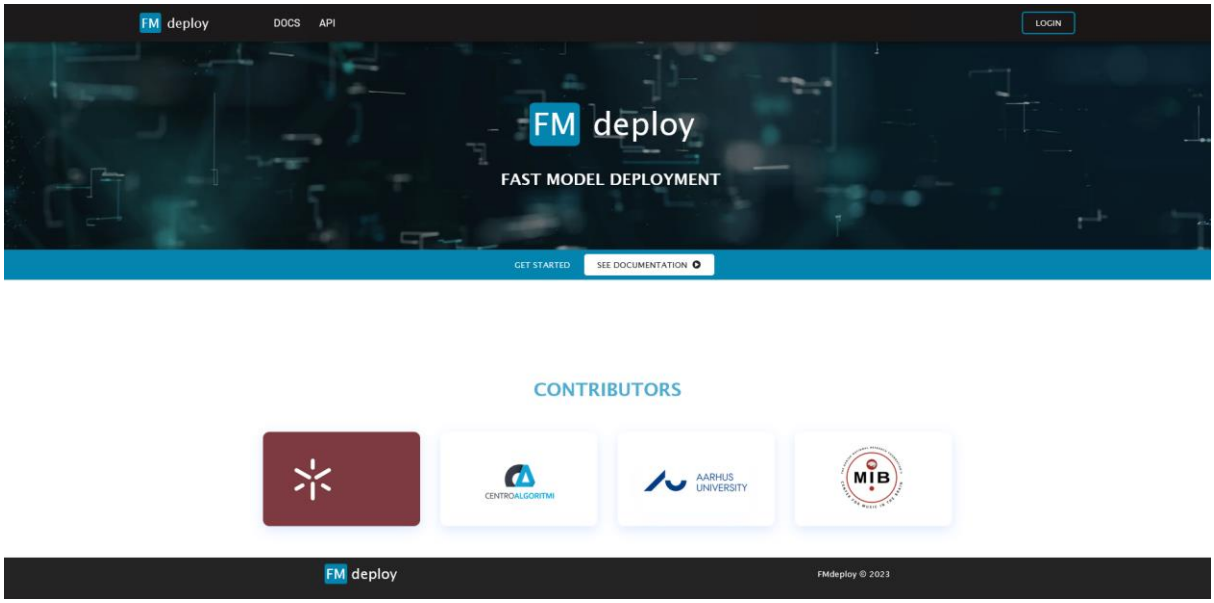



Figure 19 – FMdeploy web app - landing page.

Figure 19 showcases the landing page, featuring a top navigation bar with buttons for FMdeploy, "DOCS," "API," and "LOGIN." The FMdeploy button provides a means to return to the landing page. Clicking "DOCS" displays a page with information about the GitHub

[6] https://github.com/MarcelodeFreitas/FMdeploy-ReactJS

repositories where the API and web application code are available. "API" redirects users to the landing page for API documentation, as depicted in Figure 13. Clicking "LOGIN" directs users to a login or register page. Below, there are two additional buttons in a blue bar: "GET STARTED," which leads to the same page as "LOGIN," and "SEE DOCUMENTATION," redirecting to the same page as "DOCS." Additionally, the landing page and every other component feature four cards displaying contributor information. Clicking each card opens the corresponding website in a new tab.

#### 4.5.1.2 *LOGIN AND REGISTER*

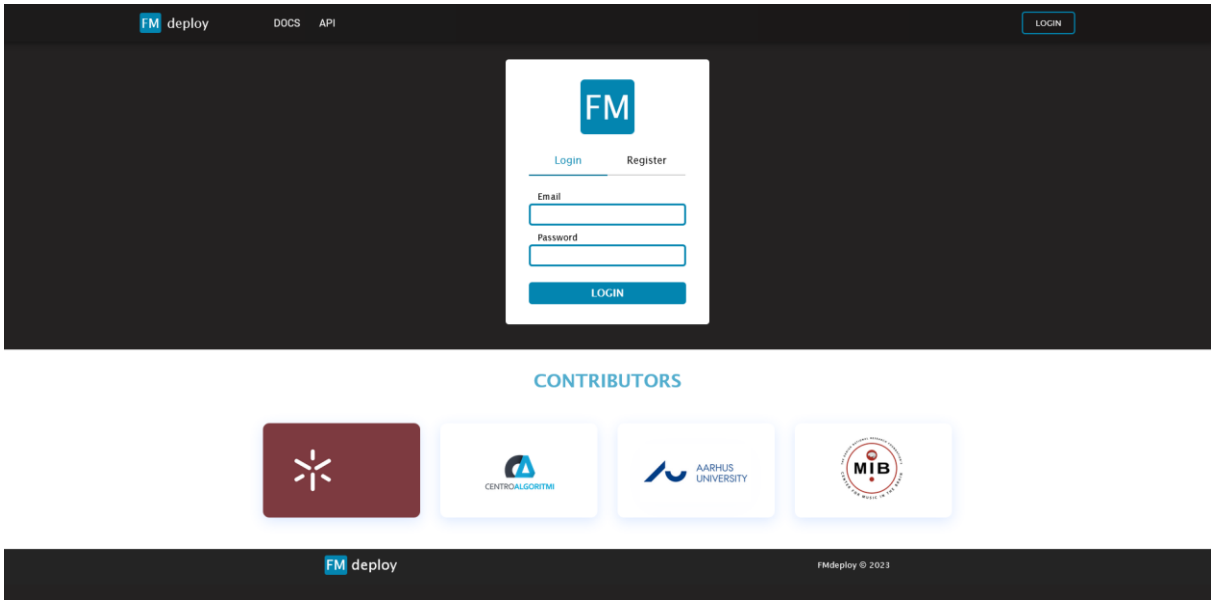



Figure 20 – FMdeploy web app - login page.

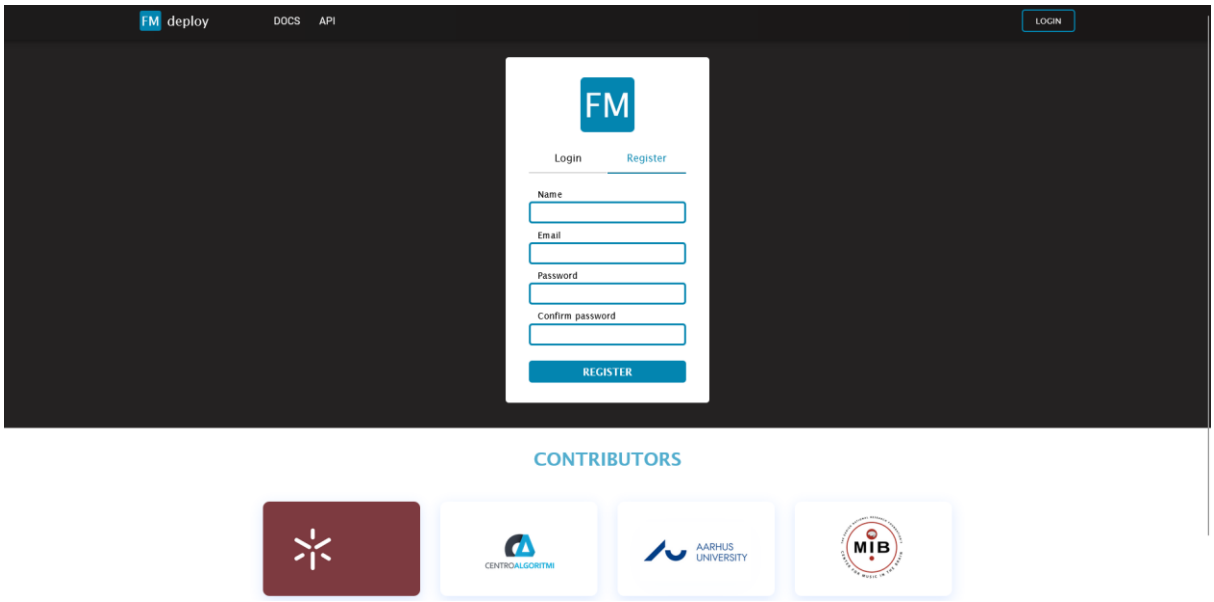



Figure 21 - FMdeploy web app - register page.

Moving on to the login and register screen, Figure 20 and Figure 21 illustrate the familiar top navigation component, contributor cards, and footnotes. On this screen, users can either log in or register an account. Upon registration, accounts are assigned the "Guest" role by default for security reasons. "Guest" accounts are restricted from creating projects to mitigate the risk of injecting malicious Python code directly into the API. However, "Guest" accounts are still valuable as they can run existing public or private projects specifically shared with them. Furthermore, "Guests" can provide valuable feedback, as will be further explored.

The login requires users to provide their email and password, as shown in Figure 20. When the register option is selected, additional fields are displayed, including the requirement for a name, email, password, and password confirmation, as seen in Figure 21.

After authentication, individuals are redirected to a specific component based on their role: "Admins" are redirected to the "User Management" component, "Users" are redirected to the My Projects component, and "Guests" are redirected to the "Shared Projects" component.

#### 4.5.1.3 NAVIGATION

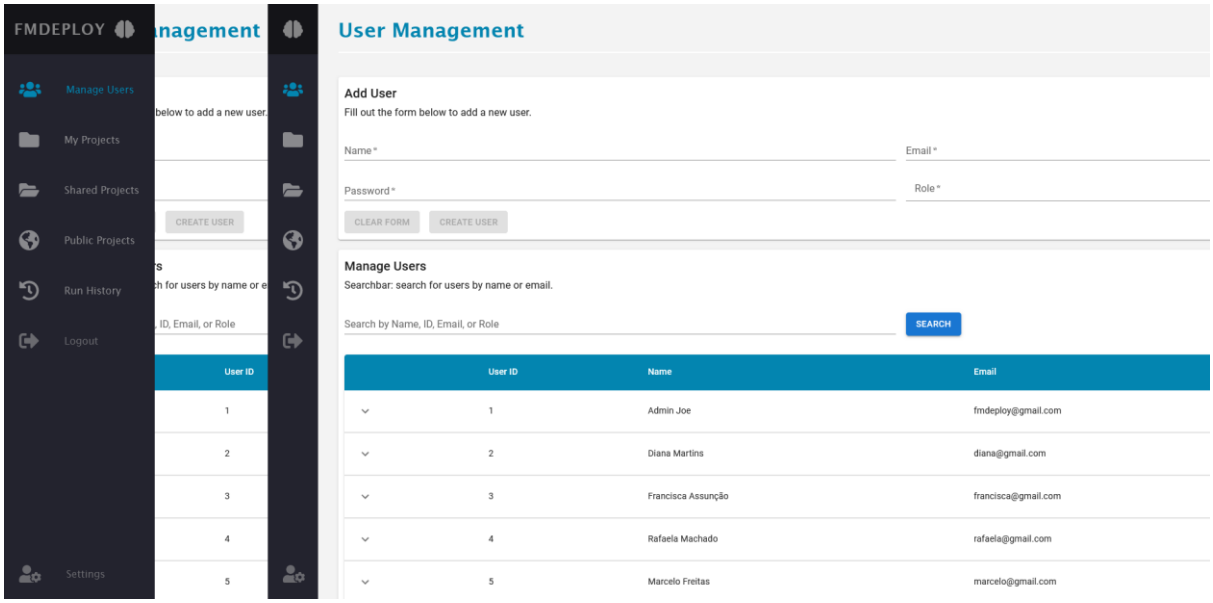



Figure 22 - FMdeploy web app - navigation sidebar open and closed.

After authentication, the web application offers a convenient sidebar navigation that expands when the cursor hovers over it, utilizing the CSS hover feature. At the top of the sidebar, users can find the clickable FMdeploy logo, which redirects to the landing page (Figure 19).

In addition to the logo, several other clickable options are available in the sidebar. Figure 22 showcases the options accessible to "Admins", granting them full access to all components analyzed in this section. "Users" have access to all components except for the management

components, which are exclusively reserved for administrators. On the other hand, "Guests" do not have access to either the "User Management" or "My Projects" components, as those are reserved for "Admins" and "Users".

#### 4.5.1.4 *USER MANAGEMENT*

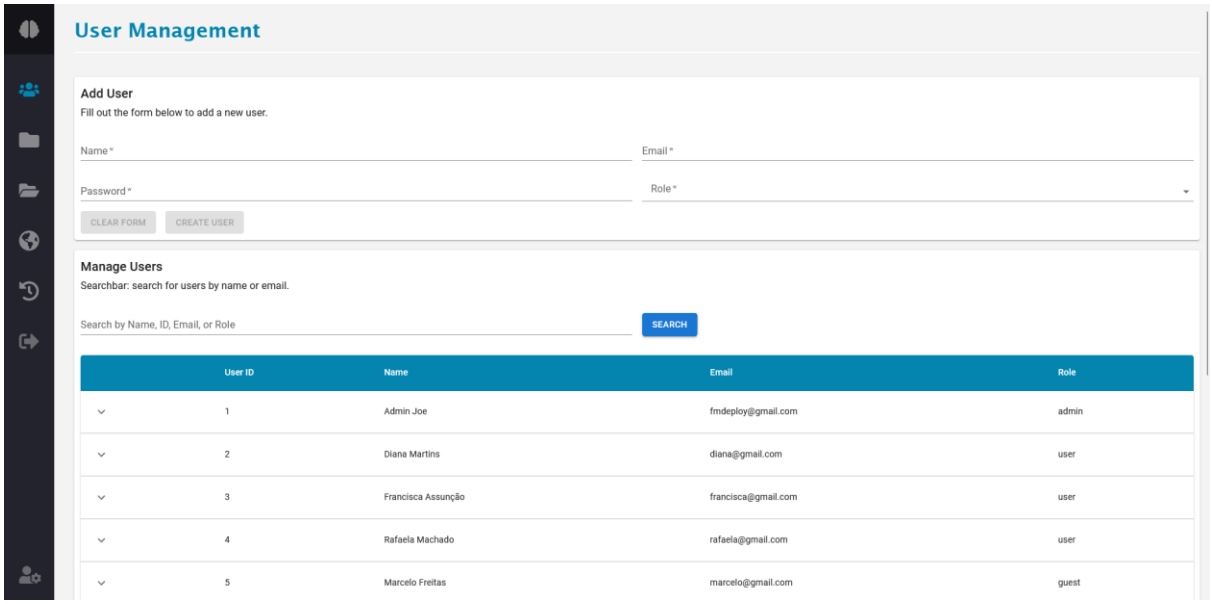



Figure 23 - FMdeploy web app - User Management - Add user.

The "User Management" component is exclusively accessible to administrators, encompassing two vital functionalities: the "Add User" and "Manage Users". At the top of this component, Figure 23 showcases the "Add User" form, which enables administrators to create new users by completing the required text boxes, including name, email, password, and role selection from the dropdown menu, offering options such as "Guest," "User," or "Admin."

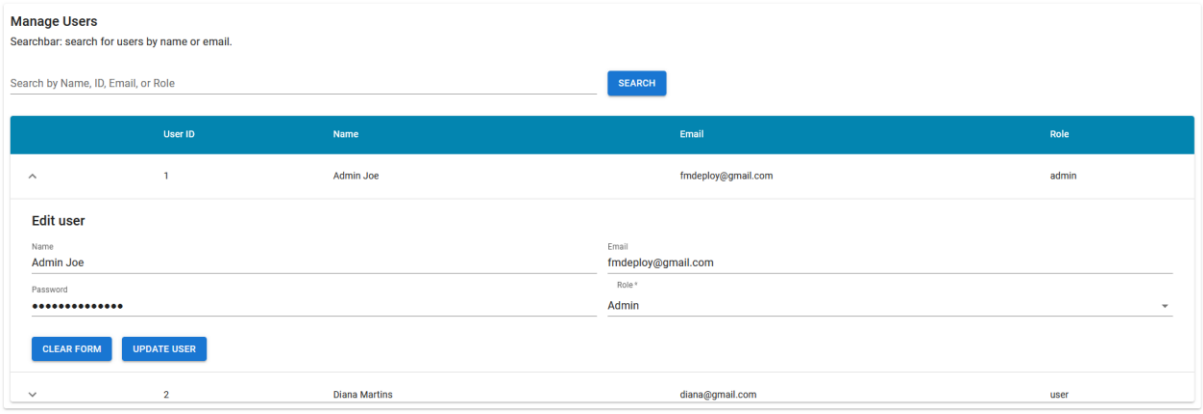



Figure 24 - FMdeploy web app - User management - Manage Users.

Directly below the "Add User" component is the "Manage Users" functionality. This component displays a comprehensive list of all users and a search bar that facilitates searching

by ID, name, email, or role. Notably, each item in the list is expandable, as depicted in Figure 24, revealing a form that allows administrators to edit user data. Administrators can modify fields such as name, email, password, and role within this form. Administrators can make the necessary modifications, submit the form, and the changes will be implemented accordingly.

#### *4.5.1.5 MY PROJECTS*

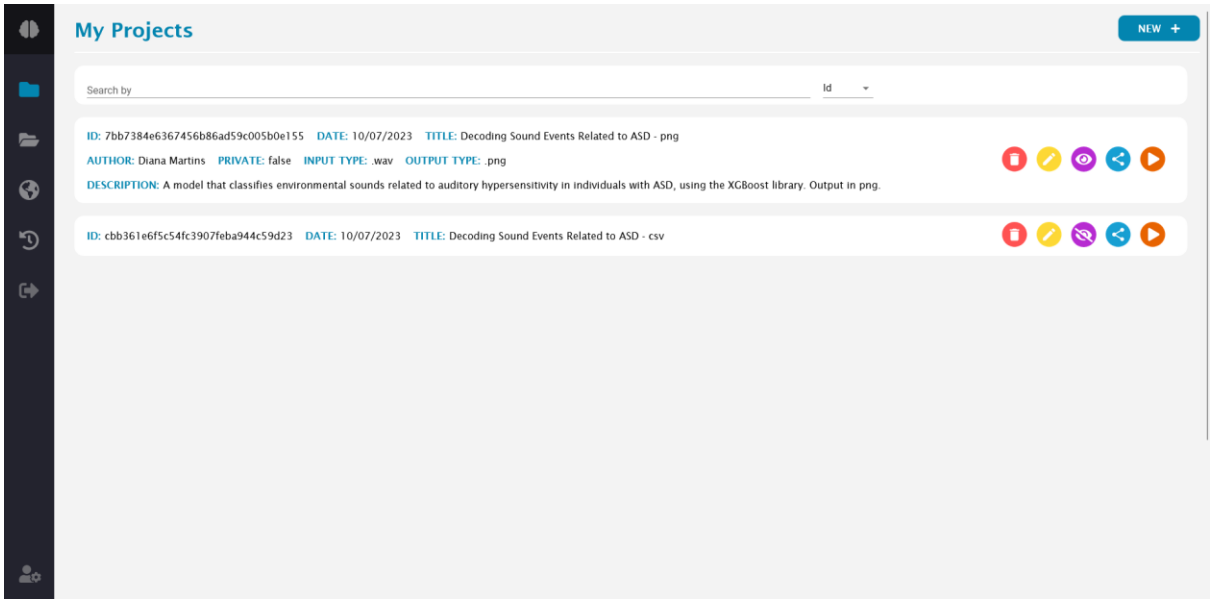



Figure 25 - FMdeploy web app - My Projects.

The "My Projects" section is accessible to both "User" and "Admin" roles. At the top of the page is a search bar that enables users to search for projects by ID or title, with the search type selectable from the dropdown menu. Located in the top right corner, the "NEW" button opens the "New Project" component. Notably, "Guests" cannot access this page, as it displays owned projects and offers the functionality to create new projects.

The page presents a list of owned projects, with each item corresponding to a specific project and clickable to reveal more detailed information. In Figure 25, the first item in the list is expanded to display additional details. When the item is not expanded, the visible information includes the ID, date, and title. However, upon expansion, the displayed fields include the author, privacy status, input and output types, and a description.

On the right side of each item in the list, five colorful buttons represent various actions associated with the project. Starting from the leftmost button, a red button with a trash icon allows users to delete a project. To prevent accidental deletions, selecting the delete button prompts the display of a confirmation component, as illustrated in Figure 26.

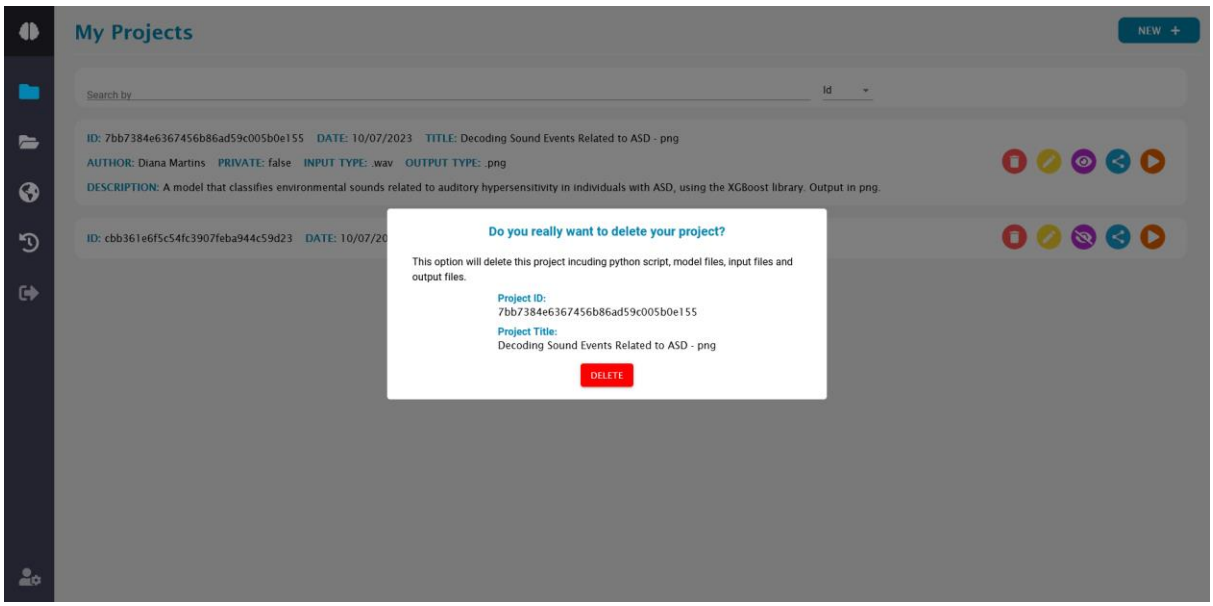



Figure 26 - FMdeploy web app - My Projects - Delete Confirmation.

Adjacent to the delete button, a yellow button with an edit symbol redirects users to the "Edit Project" component, enabling them to modify the project.

The purple button, adorned with an eye icon, allows users to toggle the project's visibility between private and public. Subsequently, a blue button with a share icon redirects users to the "Share Project" component, facilitating sharing projects with specific individuals.

Lastly, an orange button with a play icon redirects users to the "Run Project" component, enabling the execution of the selected project.

#### 4.5.1.6 NEW PROJECT

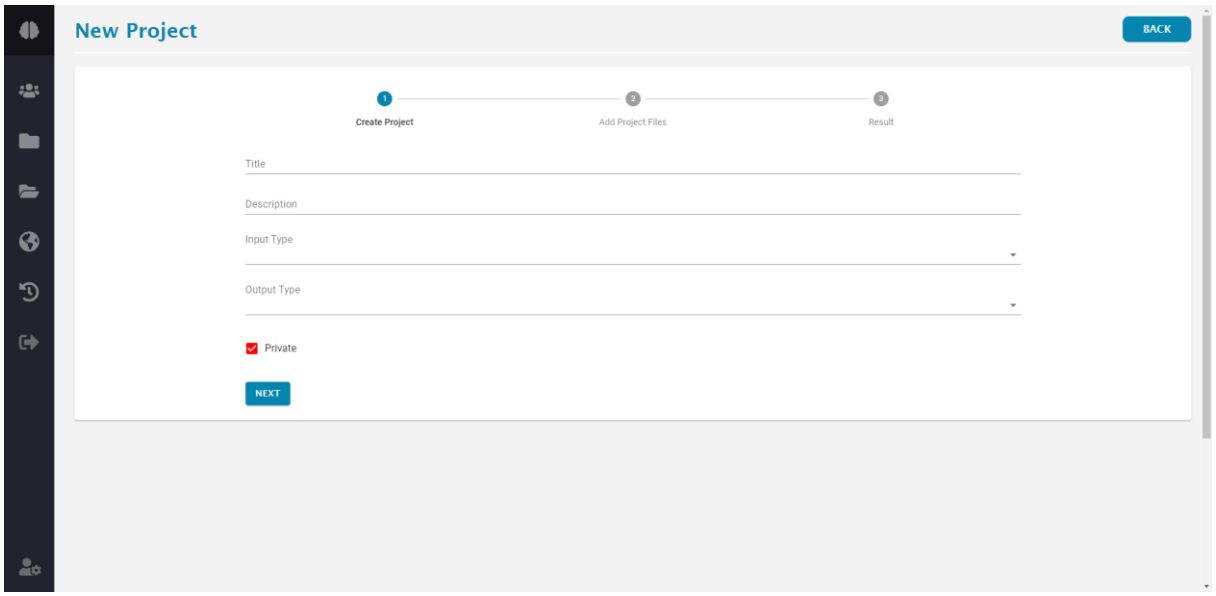



Figure 27 - FMdeploy web app - New Project – Create Project.

In the "New Project" section, users will follow a three-step process: "Create Project," "Add Project Files," and "Result." In Figure 27, the initial form requires users to input project details such as title, description (which supports clickable hyperlinks from embedded URLs), and selection of input and output types using dropdown menus. By default, the project is set as "Private," and users can uncheck the checkbox next to "Private" to make it "Public," granting access to all registered users.

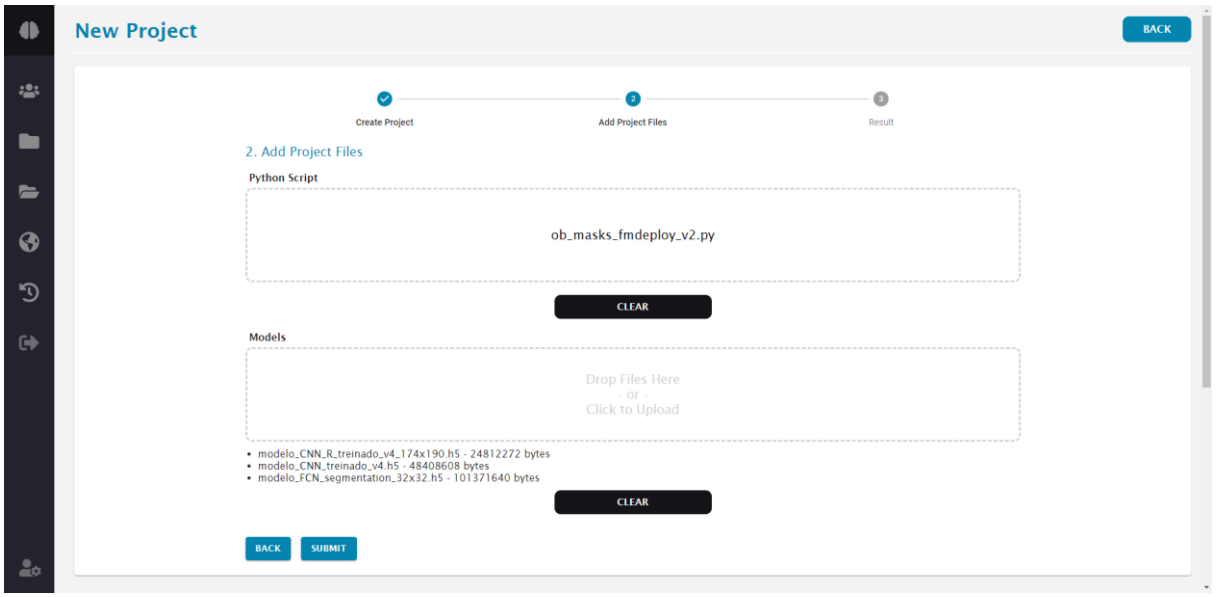



Figure 28 - FMdeploy web app - New Project – Add Project Files.

The "Add Project Files" component, illustrated in Figure 28, features two distinct drag-and-drop zones, allowing users to upload a single Python script and zero or more machine learning

models. These zones automatically verify file extensions and display error messages if a file, for instance, in the Python Script zone, lacks the ".py" file extension.

Upon dropping a file into the Python Script zone, the zone updates to reflect the uploaded file's name. Similarly, the Models drop zone displays a list of dropped file names beneath it. Each zone also includes a "Clear" button below it, allowing users to remove the submitted files for the respective zone easily.

At this stage, users can navigate back to modify any form details or proceed with submitting the files.

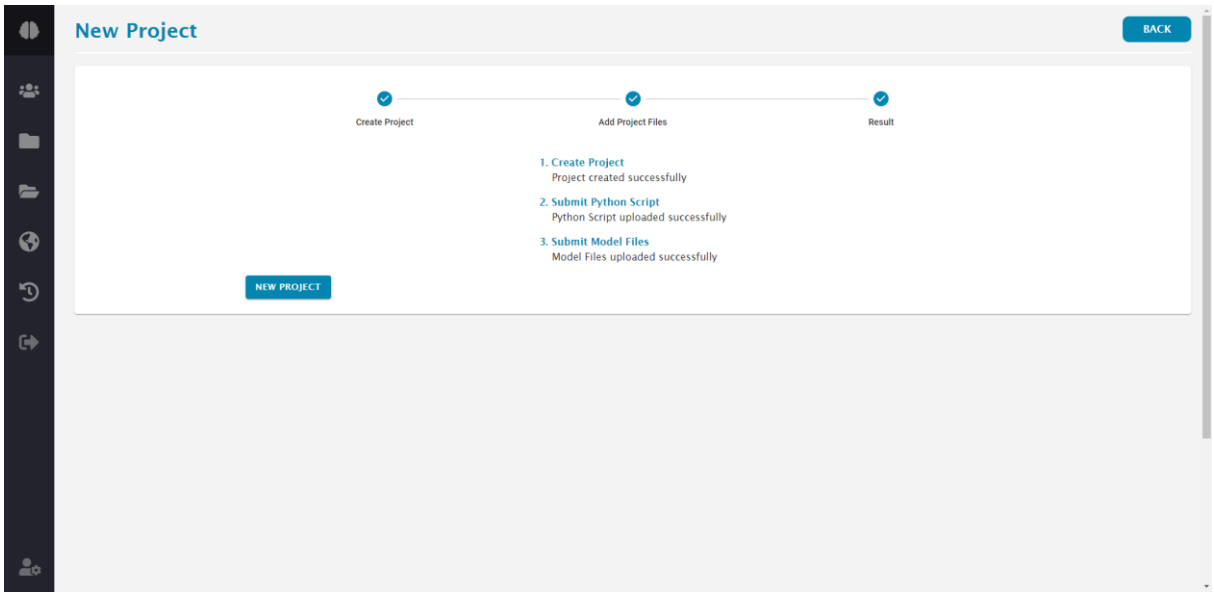



Figure 29 - FMdeploy web app - New Project – Result.

The "Result" component, displayed in Figure 29, is an essential feedback mechanism following project creation. In this section, users receive updates on the project creation process, including any encountered issues. Error messages are also displayed here if there are issues during project creation.

Python script submissions undergo validation by confirming the existence of the "load models" and "run" functions. If these functions are not present, an error message is presented to inform users of the issue. Moreover, in the case of an error, a "Back" button becomes available next to the "New Project" button. This "Back" button allows users to easily navigate, rectify the problem, and resubmit the project details.

Additionally, the "Next Project" button allows users to create subsequent projects one after the other.

#### 4.5.1.7 EDIT PROJECT

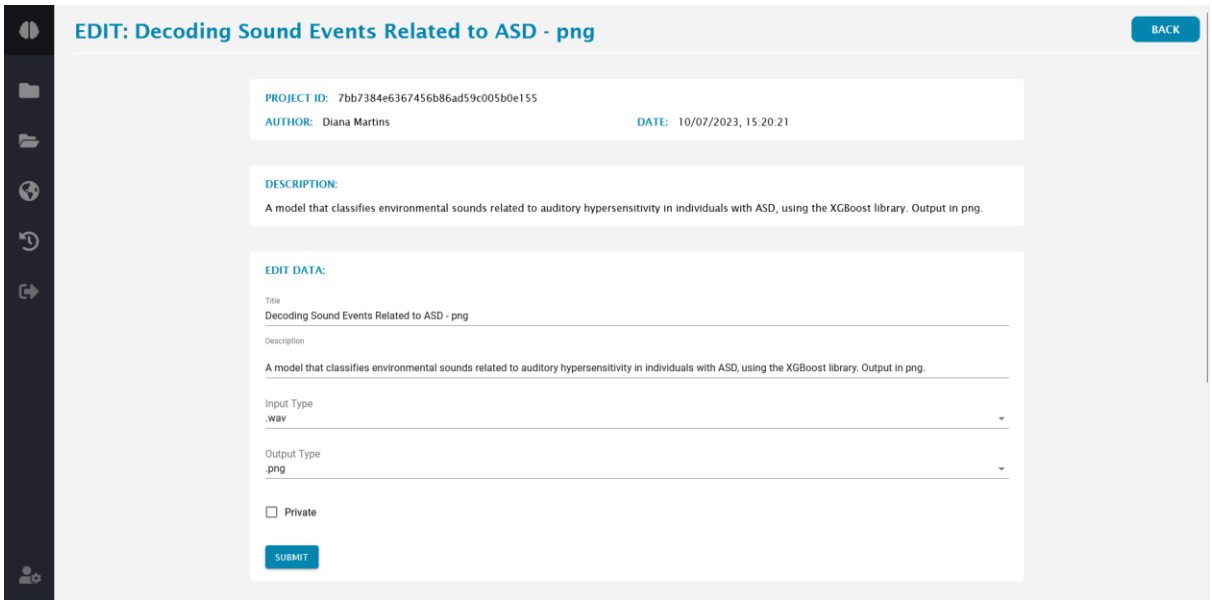



Figure 30 - FMdeploy web app - Edit Project.

The "Edit Projects" section is accessible to the "User" and "Admin" roles. The component provides information about the project, including the title, ID, author, date, and description. The primary function of this page is to enable users to edit project data, facilitated through the "Edit Data" form, as seen in Figure 30. Users can modify various project details within the form, such as the title, description, input and output types, and privacy status. Users wishing to update only specific fields can modify the desired field and submit the form. The displayed data above the form will be automatically updated to reflect the changes made.

#### 4.5.1.8 SHARE PROJECT

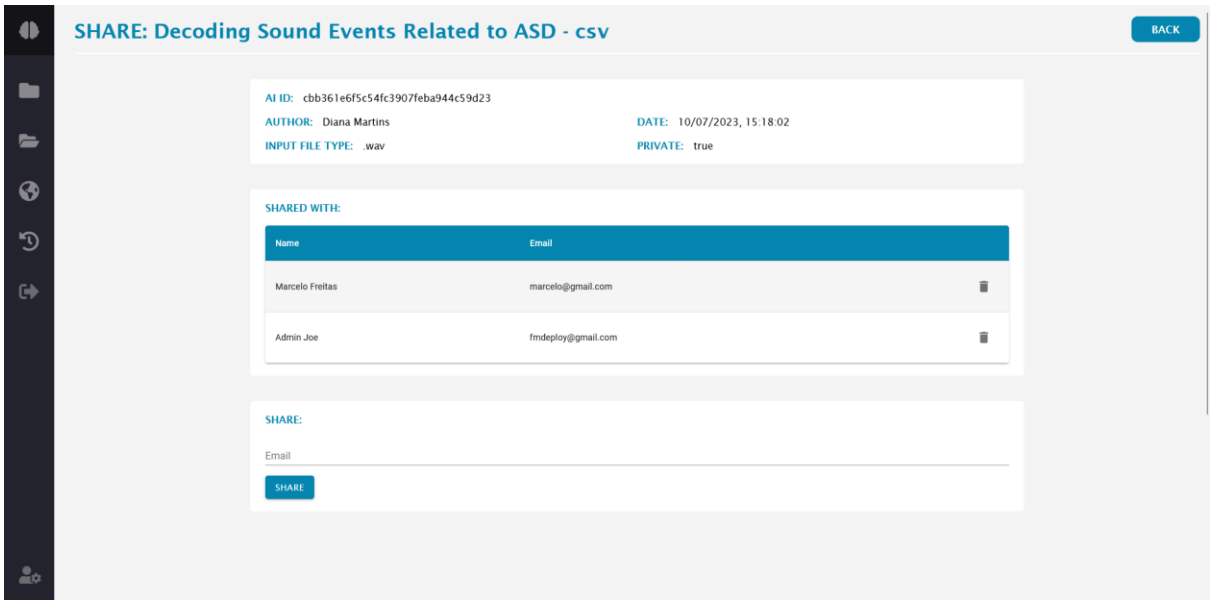



Figure 31 - FMdeploy web app - Share Project.

The "Share Project" section, illustrated in Figure 31, is not accessible to "Guests." The main features of this screen are the "Shared With" and "Share" components.

The "Shared With" component displays a table listing the users with whom the project has been shared. Each item in the list represents a user and provides their name and email information. A trash icon is also displayed next to each user entry, allowing the project owner to revoke sharing with that specific user.

Below the "Shared With" component, the "Share" component enables users to share the project with others. To share the project, enter the email address of the desired user and submit the form. The shared project will then become accessible to the recipient user in the "Shared Projects" tab.

#### 4.5.1.9 RUN PROJECT

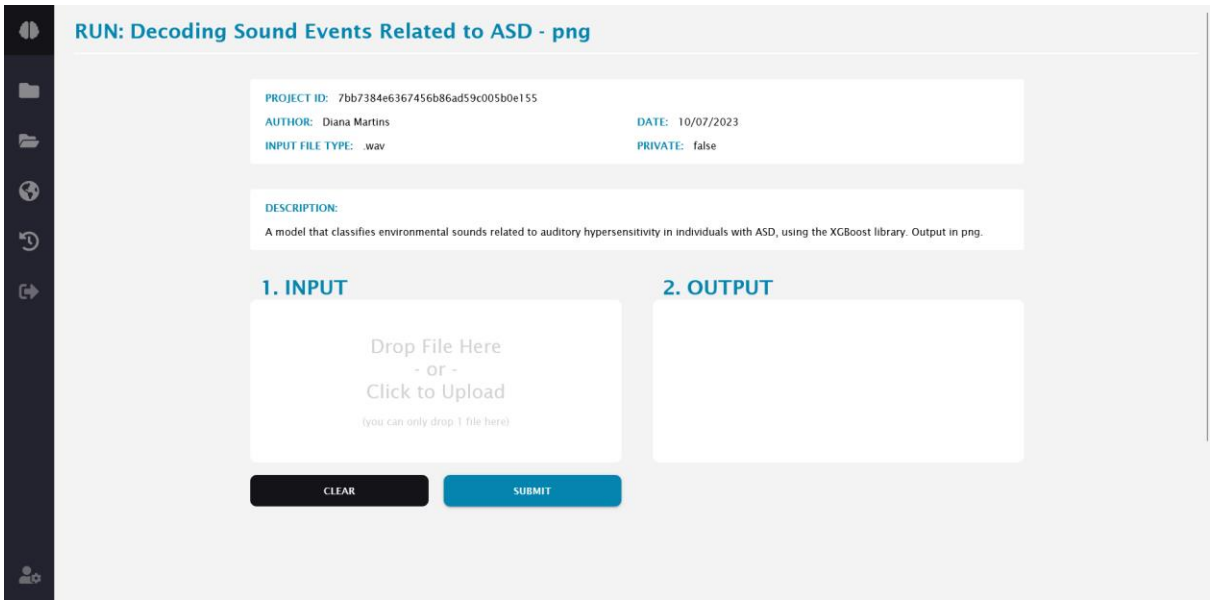



Figure 32 - FMdeploy web app - Run Project.

The "Run Project" component, depicted in Figure 32, is accessible to users with authorized access to the project, regardless of their role. The page provides project information at the top, followed by the run component. To execute the project, users can drag and drop an input file or click the input component to select a file from their device.

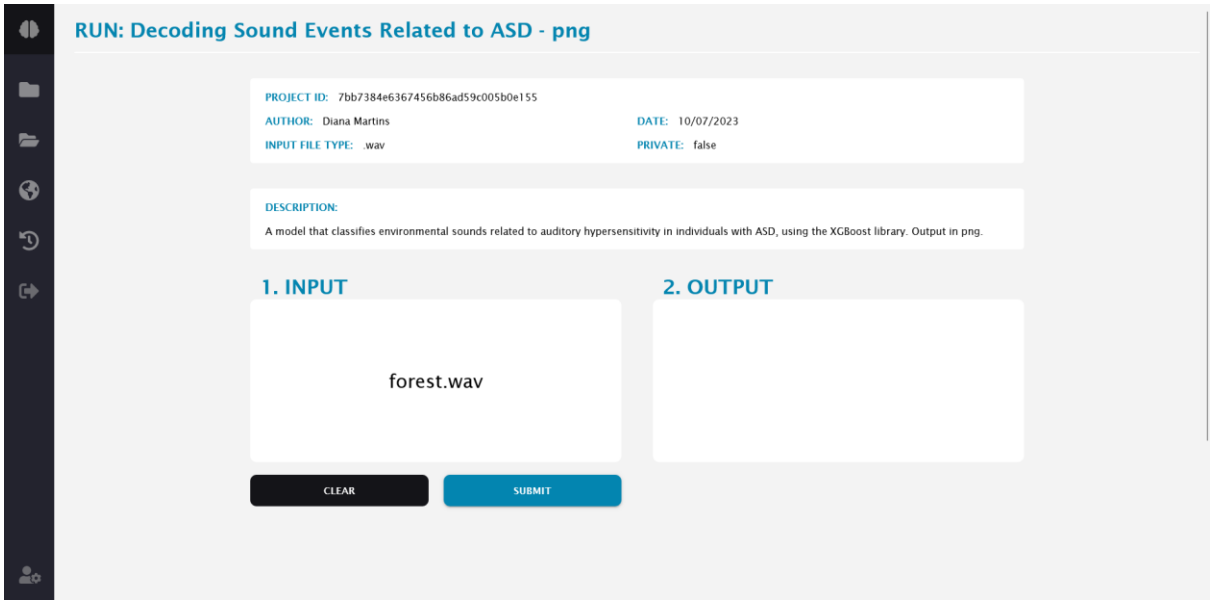



Figure 33 - FMdeploy web app - Run Project input selected.

Once the input file is selected, the input file name is presented in the input zone, as shown in Figure 33. Users have the option to clear it and choose another file. If the selected input file is correct, clicking the submit button initiates the project execution. The project utilizes pre-

trained models specified by the submitted Python script and the input file to generate an output file displayed on the page's right side. During the output generation process, the submit button is disabled, and a loading animation, as shown in Figure 34, is displayed.

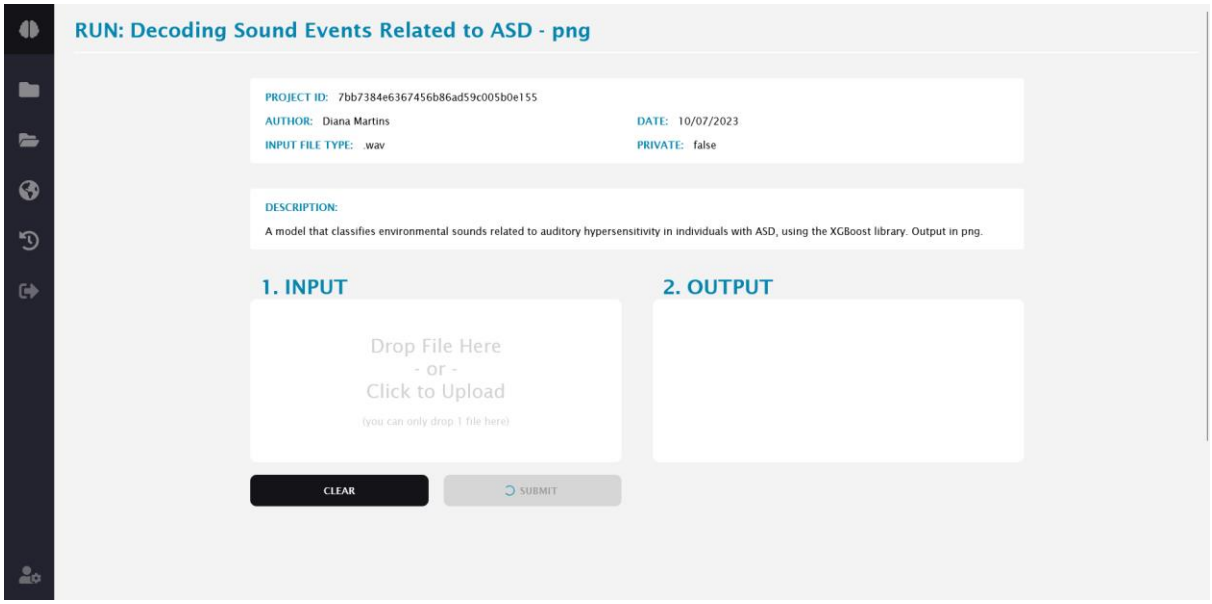



Figure 34 - FMdeploy web app - Run Project in execution.

Upon completing the project execution, users can download the output file by clicking its name or the "DOWNLOAD" button. In projects where the output type is ".png", a preview of the output is also provided, as seen in Figure 35.

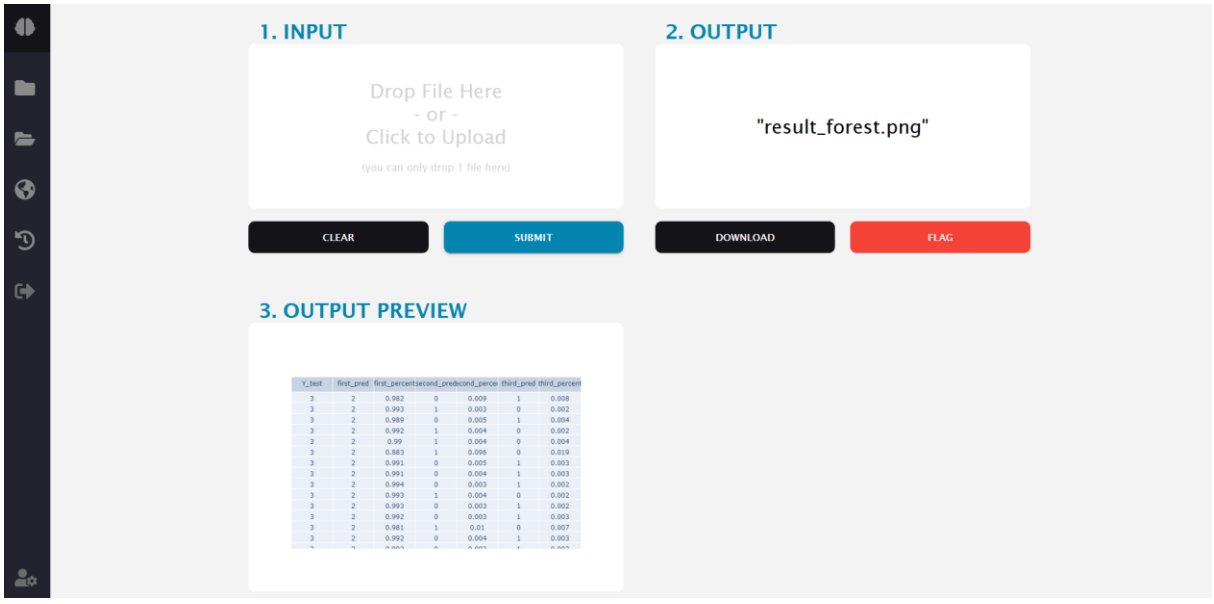



Figure 35 - FMdeploy web app - Run Project after execution.

The "Run Project" page also features a red "FLAG" button that allows users to flag the project run if they encounter any issues or unexpected results. Clicking the flag button displays a component, shown in Figure 36, where users can submit the flag and provide further details if desired.

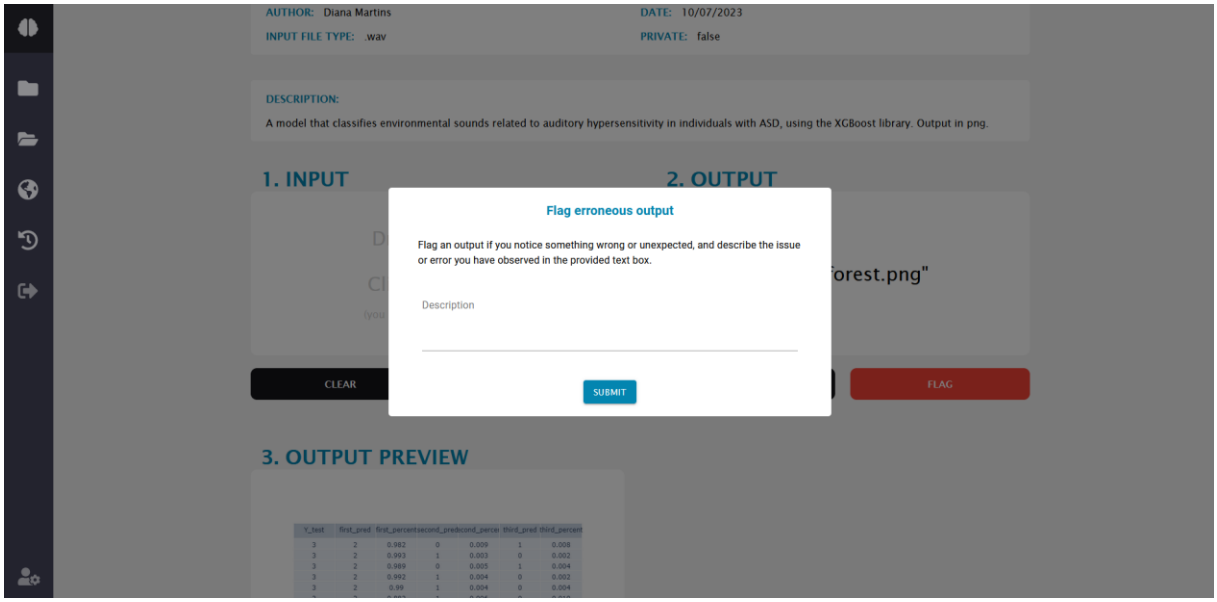



Figure 36 - FMdeploy web app - Run Project flag.

If outputs of a project run have been flagged, a list of flags will be visible exclusively to "User" and "Admin" roles on the run page. The flagged outputs table, as shown in Figure 37, displays information such as the user who created the flag (name and email), a download link for the input and respective output files, and a timestamp.

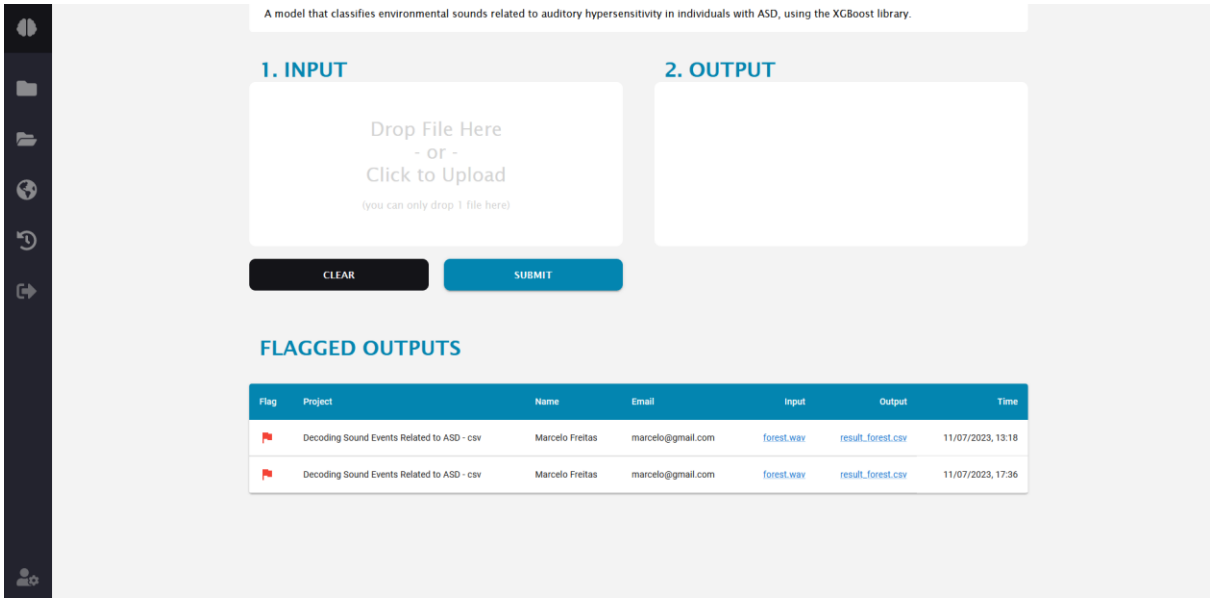



Figure 37 - FMdeploy web app - Run Project Flags table.

There is a clickable red flag icon to the left of each item in the list. Clicking the flag expands it, revealing the description of the flag, if provided, as depicted in Figure 38.

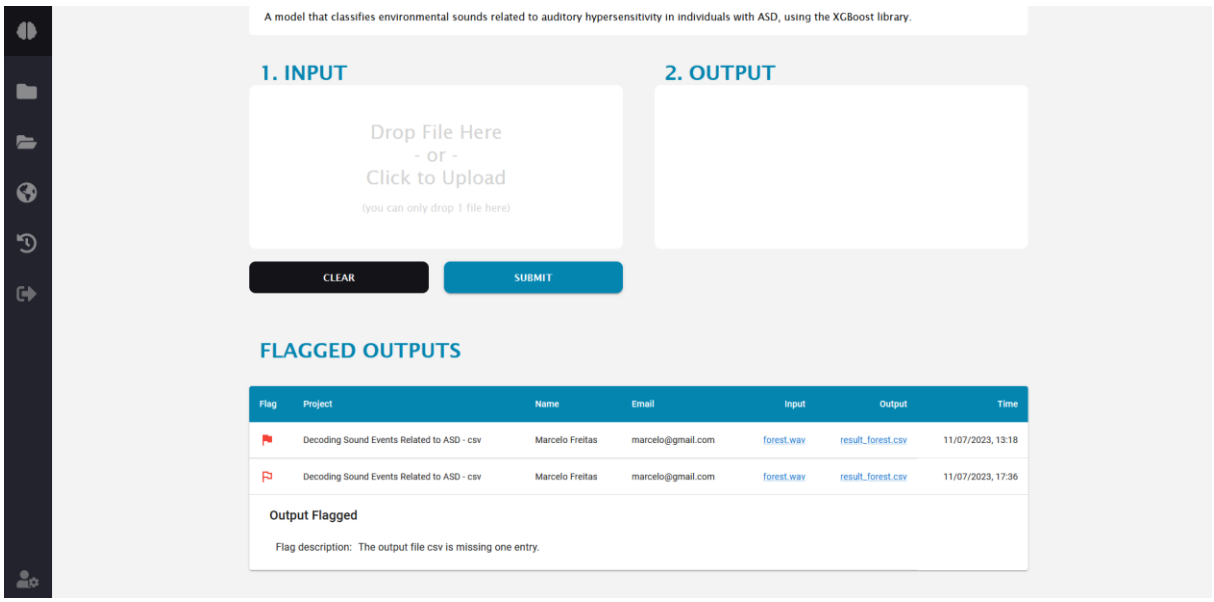



Figure 38 - FMdeploy web app - Run Project Flags table expanded.

Lastly, it is important to note that the Run Project page has a shareable URL. However, a login component, depicted in Figure 39, is displayed to ensure proper user access authorization. After successful login, users are redirected to the shared run project page.

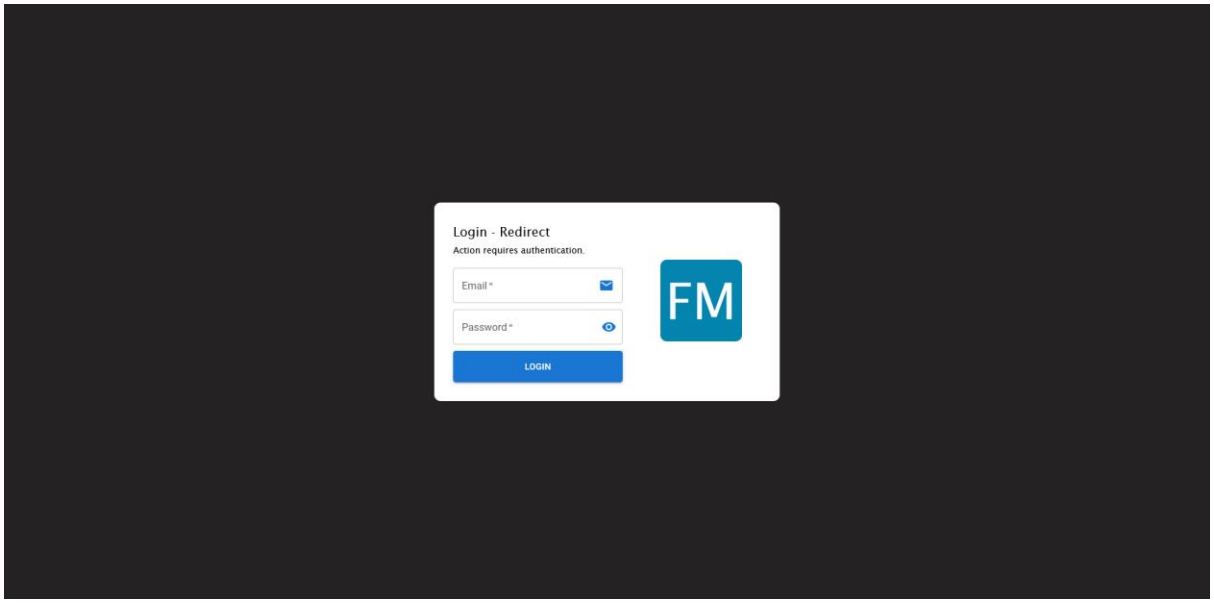



Figure 39 - FMdeploy web app - Run Project shareable URL.

#### 4.5.1.10 SHARED PROJECTS

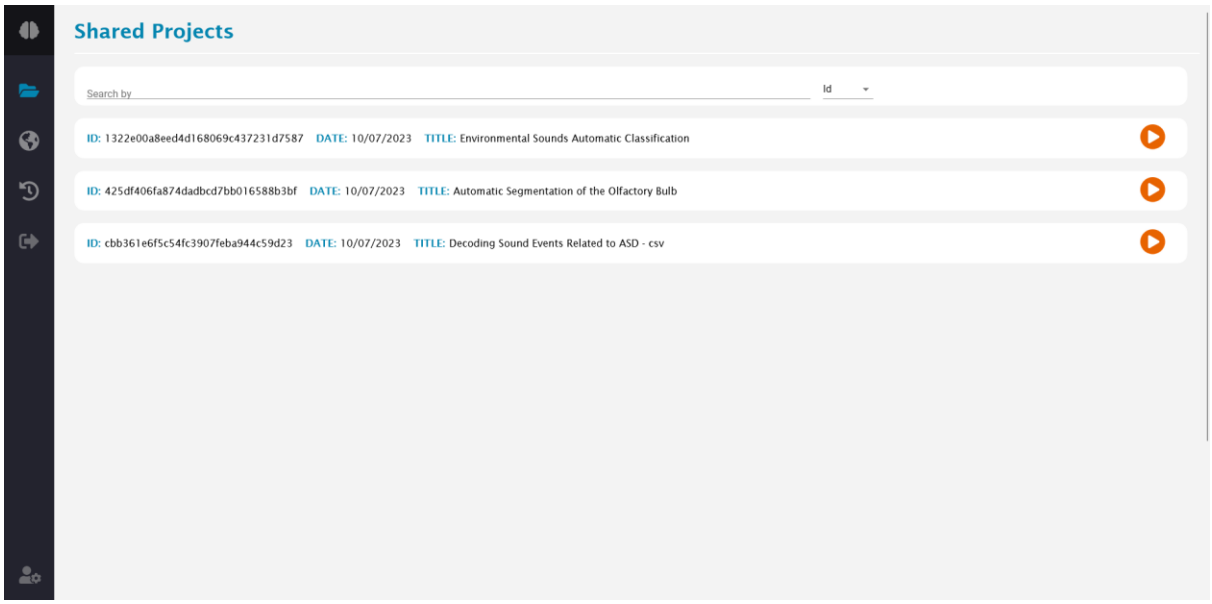



Figure 40 - FMdeploy web app - Shared Projects.

The "Shared Projects" component, as shown in Figure 40, is accessible to every user role, including "Guest" accounts. When "Guests" log in, they are redirected to this page since they cannot access "User Management" or "My Projects". The "Shared Projects" page shares similarities with the "My Projects" page, featuring a search bar at the top but without the "New Project" button in the top right corner.

Below the search bar, there is a list of projects that have been shared with the user. Each item on the list can be expanded to reveal more information about the project. However, the buttons on the right side of each item are limited, with only the orange button with the play icon remaining. This allows users to access the run page of projects that have been shared with them.

#### 4.5.1.11 PUBLIC PROJECTS

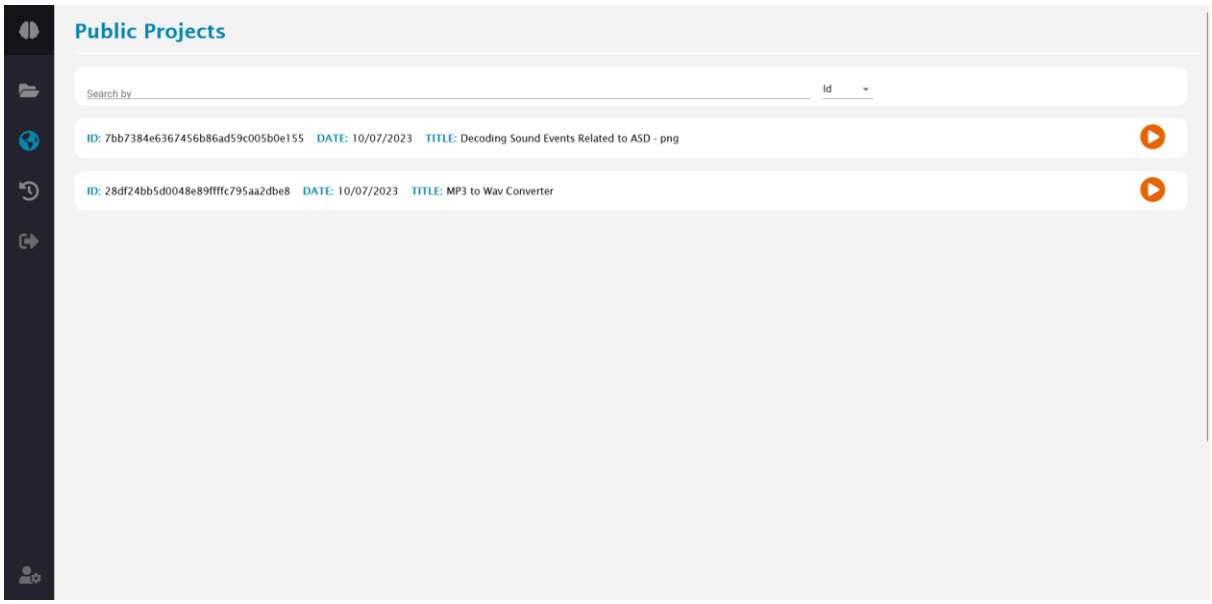



Figure 41 - FMdeploy web app - Public Projects.

The "Public Projects" component, accessible to users of any role, is identical to the "Shared Projects". The distinction lies in the showcased projects, as depicted in Figure 41, which explicitly displays projects that authors have designated as publicly accessible.

#### 4.5.1.12 RUN HISTORY

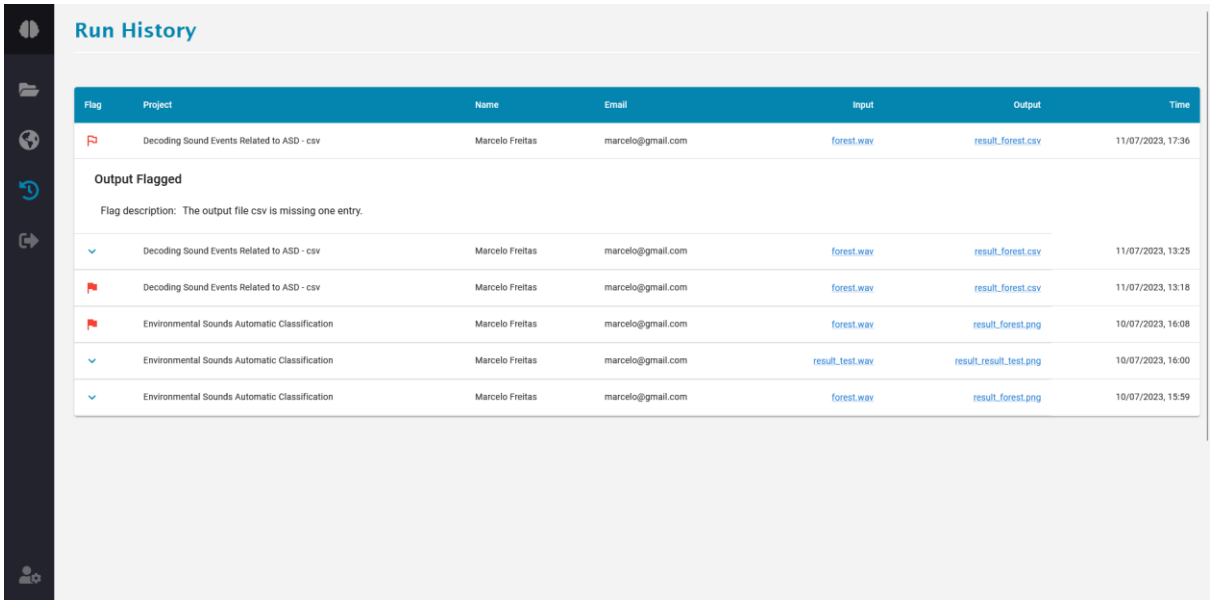



Figure 42 - FMdeploy web app - Run History flag expanded.

The "Run History" component, accessible to users of any role, provides a comprehensive overview of all project executions made by the user. Each item in the table, as depicted in Figure 42, corresponds to a specific project run and displays key information such as the

project title, clickable input and output names for easy download, and a timestamp indicating the execution time. The first column of the table includes a symbol that indicates if the run has been flagged, with red flags denoting the presence of a flag. Users can expand each list item by clicking the flag icon in the case of a flag or the down arrow icon if no flag exists. This allows users to view the flag description, as demonstrated in Figure 42.

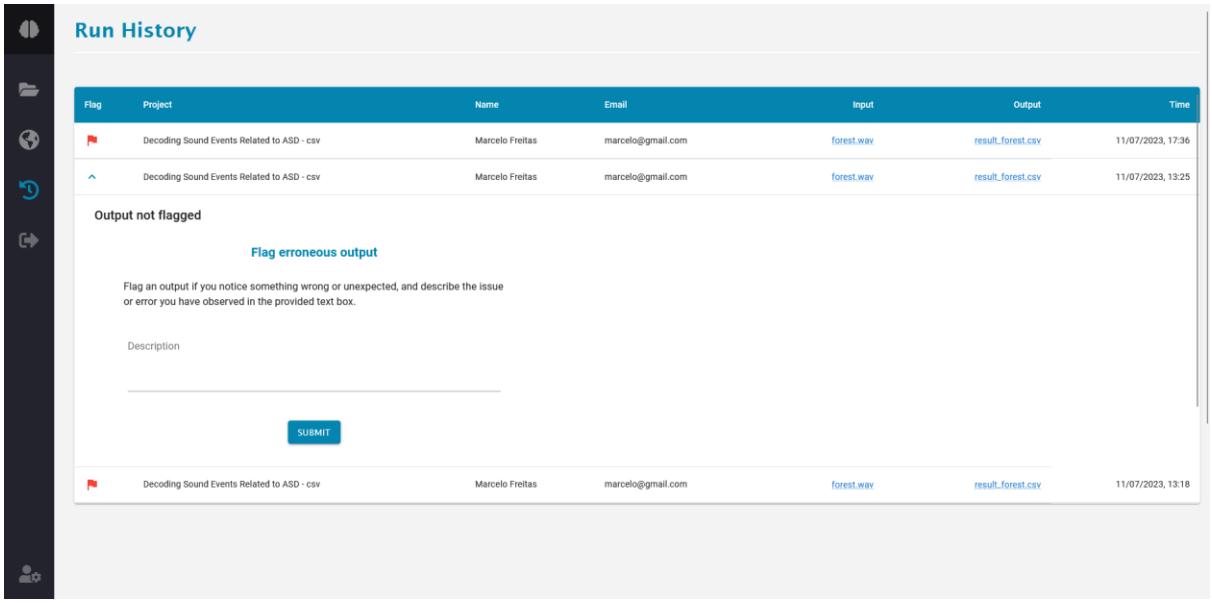



Figure 43 - FMdeploy web app - Run History no flag expanded.

Furthermore, in scenarios where no flag is present, clicking the down arrow icon reveals a form to create a new flag, as illustrated in Figure 43. The Run History component is a valuable resource for users to access project outputs when they cannot wait for them within the project run screen. If a user exits the project run screen before the output is ready, the files become available in the run history. Users can use the run history feature to create flags for specific project runs in case they forget to do so in the "Run Project" component. This functionality enhances the usability and versatility of the FMdeploy web application, enabling users to effectively manage their project executions and maintain a comprehensive record of their activities.

#### 4.5.1.13 USER SETTINGS

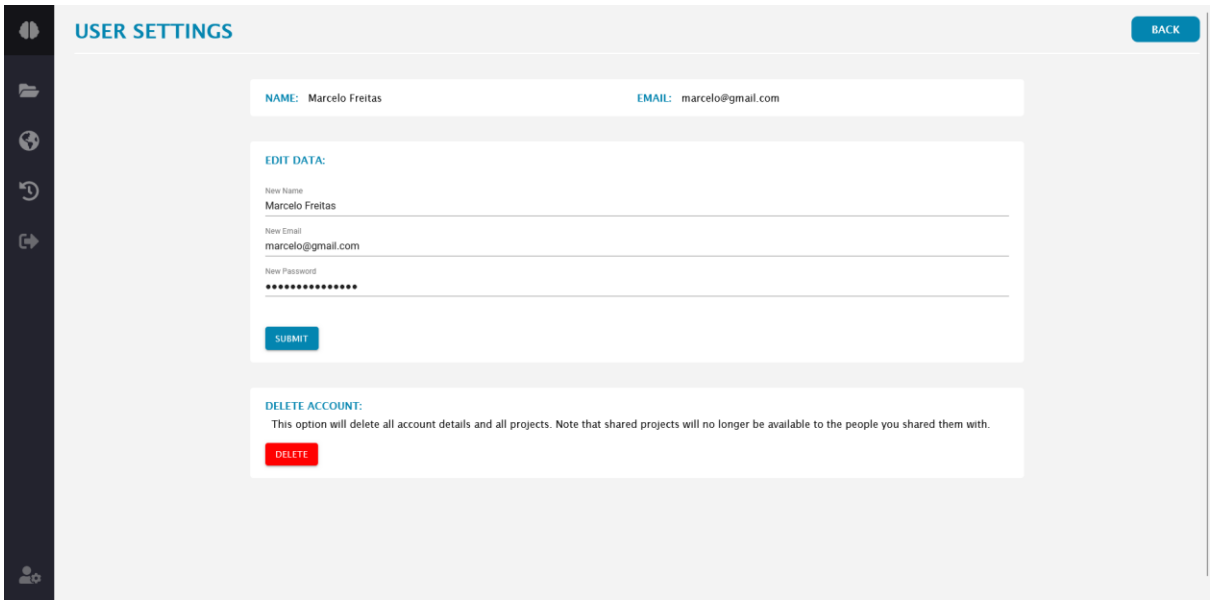



Figure 44 - FMdeploy web app - User Settings.

The "User Settings" component, accessible to users of any role, provides a convenient interface to manage their personal information and account settings. As depicted in Figure 44, this component includes an edit data form where users can easily update their name, email, and password. Users can ensure their account information remains accurate by modifying the desired fields and submitting the changes.

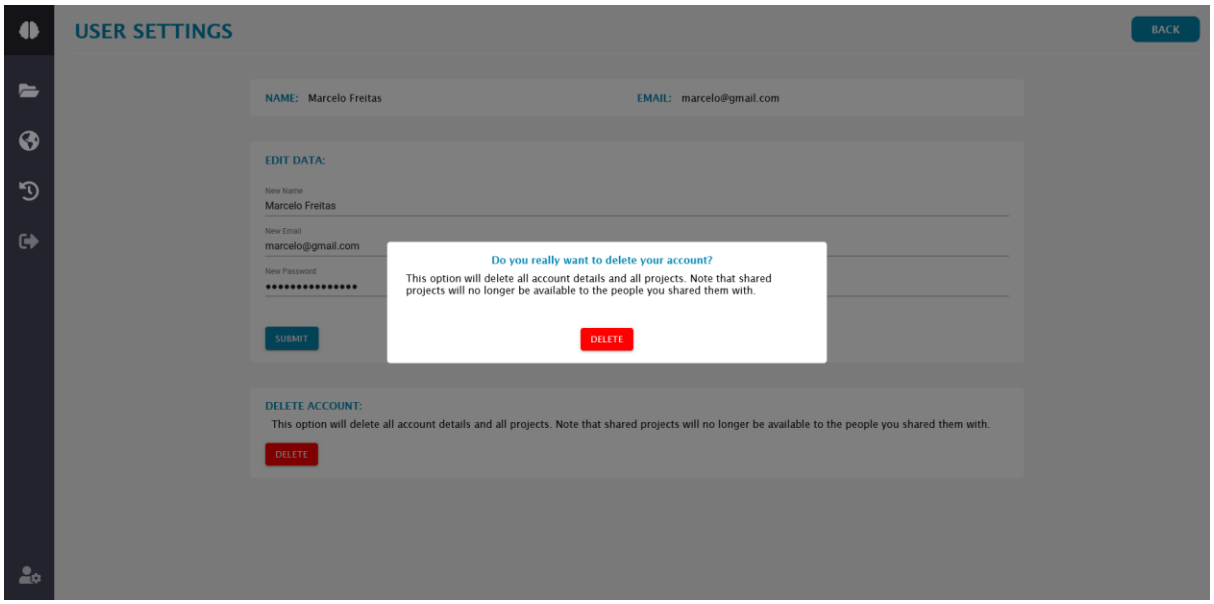



Figure 45 - FMdeploy web app - User Settings delete account confirmation.

In addition to updating information, the "User Settings" component allows users to delete their accounts. A delete button below the edit data form provides a straightforward way to permanently remove the account, along with all associated data created within the application. A confirmation component, illustrated in Figure 45, is displayed when the delete button is clicked to prevent accidental deletions. This additional step ensures that users can make informed decisions before proceeding with the deletion process.

### 4.5.2 OPTIMIZING USER EXPERIENCE WITH RESPONSIVE DESIGN

The FMdeploy web application is developed with a responsive design, ensuring an optimal user experience across diverse devices. This approach adapts the application's layout and functionality to different screen sizes and resolutions.

Responsive design offers several advantages. Firstly, it enables users to conveniently access and interact with the platform across various devices, including tablets and smartphones, regardless of their device preferences.

The automatic adjustment of the interface's layout, content, and navigation based on screen size enhances usability. Users can navigate the application, access features, and view information without excessive scrolling or zooming.

Figure 46, Figure 47, and Figure 48 visually depict the FMdeploy web application's responsive nature, showcasing its adaptability across different screen sizes. These examples demonstrate how the application optimizes its interface for diverse devices, ensuring an exceptional user experience.

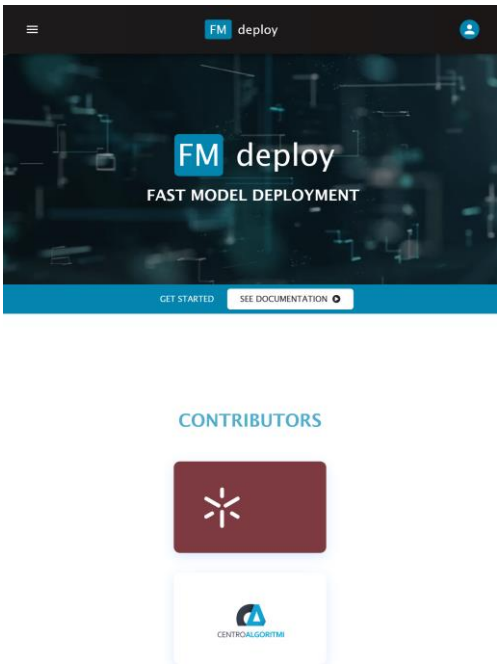



Figure 46 - UI iPad iPadOS 14.7.1.


Figure 47 - UI iPhone 11 Pro Max iOS 14.6.

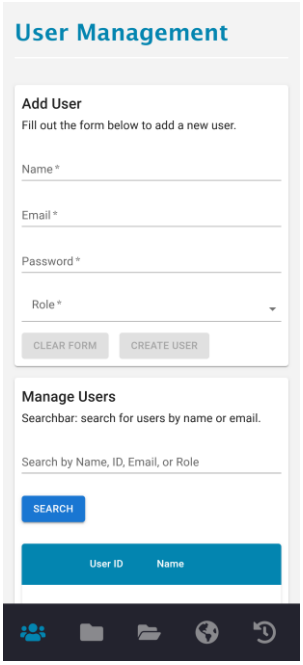



Figure 48 - UI Samsung Galaxy S20 Ultra Android 11.

By embracing responsive design, the FMdeploy web application prioritizes accessibility and usability, empowering users to leverage its functionalities from virtually any device with an internet connection.

## 4.6 CONTAINERIZATION AND DEPLOYMENT

### 4.6.1 SERVER INFRASTRUCTURE

The FMdeploy application is required to be tailored to fit the infrastructure of the University of Minho server, where an instance of the research platform will be hosted. The server configuration specifications include:

- OS: Ubuntu 14.04 LTS (64-bit)
- CPU: Intel Xeon E5-1650
- RAM: 64GB
- Secondary Memory: 2 Disks of 2TB and 1 Disk of 512GB
- GPU: NVIDIA P6000
    - Cuda Parallel-Processing Cores: 3840
    - GPU memory: 24 GB GDDR5X
    - FP32 Performance: 12 TFLOPS

It is important to emphasize that the platform will be containerized using Docker and will share resources with other containers on the server. This sharing of resources ensures efficient utilization of the server's capabilities.

### 4.6.2 CONTAINERIZATION AND DEPLOYMENT ON A REMOTE SERVER

The deployment of the FMdeploy API and web application to a remote server was undertaken to achieve one of the key objectives of this dissertation. This deployment offers accessibility beyond the local development environment, thereby extending the reach of the application. The server's specifications were detailed in the previous subchapter.

Docker was identified as the ideal approach for deploying to a remote server due to its advantages in containerization. By leveraging Docker, the code was containerized, and a new GitHub repository named "fmdeploy"[7] was created. This repository includes the code from

[7] https://github.com/MarcelodeFreitas/fmdeploy

the "FMdeploy-FastAPI"[8] repository and the "FMdeploy-ReactJS"[9] repository, along with all the necessary Docker files for containerization.

Security is a paramount concern for deployment on a remote server, necessitating HTTPS-secured communication. To achieve this, Traefik, a widely used reverse proxy and load balancer, was employed. Traefik enables HTTPS communication, enhancing the security and privacy of data transmission. Additionally, it simplifies the handling of Let's Encrypt certificates, which are essential for establishing secure HTTPS connections.

Let's Encrypt is a free and open certificate authority that facilitates the issuance of SSL/TLS certificates. These certificates encrypt communication between the server and clients, ensuring secure data transmission. Traefik automatically obtains and manages these Let's Encrypt certificates, simplifying the setup process and ensuring a secure HTTPS connection.

To enable HTTPS communication with Traefik, a domain name was required. A domain name serves as a unique identifier for a website or application. In this case, a domain name was registered through the GitHub Pro account, available to students for free. This account allowed registering one domain name for a year on Name.com.

Following the configuration of DNS, A records, a waiting period for DNS propagation was necessary, typically ranging from 24 to 48 hours. To confirm successful DNS propagation, a simple "`ping`" command to the A record could be used to verify the correct IP address resolution.

The remote server was already equipped with Docker and managed with Portainer, facilitating the deployment process. To deploy the application, a clone of the "fmdeploy"[10] GitHub repository was created on the server, and the Docker Compose file was utilized.

Docker Compose played a crucial role in defining and configuring the multi-container application. It facilitated the creation of services, networks, and volumes required for deployment. The containers were successfully deployed and operationalized by utilizing Docker commands, such as building the image based on the Dockerfiles and executing the "docker-compose up" command.

Through the combined use of Docker, Traefik, Let's Encrypt, and appropriate server configuration, the containerization and deployment of FMdeploy on the remote server were

[8] https://github.com/MarcelodeFreitas/FMdeploy-FastAPI/tree/latest
[9] https://github.com/MarcelodeFreitas/FMdeploy-ReactJS
[10] https://github.com/MarcelodeFreitas/fmdeploy

achieved. This approach ensured scalability, security, and efficient resource utilization, providing a robust and accessible platform for users.

# 5 Discussion and Conclusions

## 5.1 SYNOPSIS

The dissertation focuses on FMdeploy, a comprehensive platform for deploying pre-trained machine learning models. FMdeploy consists of two main parts: the FMdeploy API and the FMdeploy web application.

The FMdeploy API is the platform's backbone, handling all the logic and functionalities related to model deployment. Powered by the Python framework FastAPI, the API provides a robust and efficient solution for uploading models, configuring inputs and outputs, and executing the models to generate predictions. It ensures seamless integration and compatibility with various machine-learning libraries.

Complementing the FMdeploy API is the FMdeploy web application, which offers simple and interactive web-based user interfaces. Built with the React framework, the web application provides a user-friendly experience, allowing users to easily navigate the platform, manage their models, and visualize the results. Its responsive design ensures accessibility and usability across different devices.

For secure communication and streamlined deployment, FMdeploy leverages Docker and Docker Compose. Docker containers are created for the FMdeploy API and the web application, along with a database and Traefik (a reverse proxy and load balancer). This containerization approach encapsulates models and their dependencies, ensuring consistent and reproducible execution across different environments. It simplifies the deployment process, improves scalability, and facilitates resource allocation for concurrent model execution.

The research presented in this dissertation showcases the successful development of FMdeploy, offering a comprehensive solution for deploying pre-trained machine learning models. With the powerful FMdeploy API handling the backend logic and the user-friendly web application powered by React, the platform provides researchers, practitioners, and developers with an efficient and intuitive tool for managing and utilizing machine learning models effectively.

## 5.2 DISCUSSION

The developed platform, FMdeploy, sought to address several critical challenges in the field of ML deployment and usage, as identified in the literature review. The emphasis on user-friendly interfaces aligns with the recommendations of D. Baylor et al. [37], M. Madaio et al. [102], and K. Ackermann et al. [103]. Their work underscored the importance of usability and accessibility in ML tools, and FMdeploy's design strives to embody these characteristics. This focus on user-friendliness aligns with the perspective of L. Baier et al. [88], who noted that such tools could allow non-technical users and domain experts to utilize the platform effectively.

Moreover, FMdeploy was designed to avoid the limitations associated with highly specialized or overly customized tools, as noted by R. Elshawi et al. [27] and A. A. A. Abid et al. [45]. Instead, FMdeploy aimed to be a more general, flexible, and adaptable tool. As S. Klemm et al. [104] and D. Muthukrishna et al. [105] pointed out, highly customized tools often limit their applicability in different contexts. In contrast, FMdeploy's generalized approach makes it suitable for various applications.

The design of FMdeploy also considers the language barrier issue highlighted by P. Singh et al. [35]. By reducing the number of programming languages used in its backend (fully coded in Python), FMdeploy minimizes integration complexities associated with ML models.

However, despite the strengths of FMdeploy, it is not without limitations. For instance, while it includes some security measures, it is still susceptible to common adversarial attacks, as pointed out by F. Tramèr et al. [120]. This implies that further work is needed to enhance the platform's security.

There are several similarities and differences compared to other open-source tools like Gradio [45]. Similar to Gradio, FMdeploy provides web-based interfaces for ML models and supports a range of ML frameworks. However, unlike Gradio, which is a Python package, FMdeploy is powered by an all-Python-based API. This difference in implementation may have implications for ease of use and integration with other systems.

Further, FMdeploy, like Gradio, creates shareable links to ML models. However, it does not require an open SSH tunnel from a developer to a user, instead relying on the server it is hosted on. Despite these advantages, FMdeploy has room for improvement, especially in allowing collaborators to manipulate input data directly in the web interfaces.

Lastly, FMdeploy leverages Docker containers to address the reproducibility challenge highlighted by J. Gruendner et al. [125]. This approach also allows the platform to be adaptable to both on-premises and cloud-based solutions. As J. Y. Cheng et al. [49] and P. Singh et al. [35] mentioned, both solutions have pros and cons. The choice is thus left to the user.

FMdeploy holds significant promise in addressing several challenges in ML deployment, although further work is needed to enhance its security measures and improve certain features.

## 5.3 CONCLUSIONS

In conclusion, this dissertation aimed to promote collaboration in research settings and facilitate the deployment of pre-trained ML models by developing FMdeploy. FMdeploy consists of an API powered by FastAPI and a web application powered by React. It addresses accessibility issues and enables direct feedback on model performance.

FMdeploy leverages ML as a service through a RESTful API, ensuring interoperability in web browsers and data security with Docker containers and HTTPS communication. The API is designed for expandability and flexibility, supported by comprehensive documentation. The web application offers a simple and intuitive user experience.

The dissertation showcases FMdeploy's capabilities through the deployment of three models: Automatic Segmentation of the Olfactory Bulb [127], Decoding of Sound Events related to ASD [128], and Environmental Sounds Automatic Classification [129]. FMdeploy provides a research platform that promotes interdisciplinary collaboration and efficient model deployment with its accessible interface and feedback system.

In conclusion, FMdeploy simplifies machine learning model deployment through a FastAPI-driven API and React-powered web interfaces. It improves accessibility, facilitates direct model feedback, ensures secure HTTPS communication, and guarantees reproducibility by containerizing code with Docker. FMdeploy holds great potential for advancing ML deployment practices and promoting research endeavors.

## 5.4 FUTURE WORK

Although the successful development of a platform for deploying pre-trained machine learning models with web-based interfaces has been achieved, several areas warrant further

exploration and improvement. Machine learning model deployment offers vast opportunities for future work and advancements.

One potential avenue for future research involves testing the deployment of different ML models using various libraries, inputs, and outputs. By expanding the platform's support for a broader range of models, it can cater to diverse applications and domains. Examples of models that could be deployed using FMdeploy include ensemble models, reinforcement learning models, and natural language processing models.

Furthermore, it is crucial to conduct comprehensive evaluations of the developed API and web application in terms of performance and effectiveness. Comprehensive testing and assessments must involve targeted users who can provide valuable feedback. Stress tests can be performed to gauge the system's robustness and scalability under heavy workloads. User questionnaires and surveys can be utilized to gather insights on usability and performance, allowing for iterative improvements to the application.

Once the platform has been thoroughly tested and evaluated by a larger audience, it can be considered for permanent deployment in a production environment, including integration into the university research context and other educational institutions. Additionally, since the platform's code is open source, it can be adopted by other organizations and individuals who recognize its value and potential.

In addition to the web application, an exciting future direction would be developing a mobile application using React Native. Given that the web interfaces were built using React, a substantial portion of the codebase can be reused, enabling efficient development for both Android and iOS platforms. This expansion into mobile devices would enhance the accessibility and reach of the platform, enabling users to deploy and interact with ML models on the go.

By exploring these future avenues, the platform can continue to evolve, adapt, and contribute to advancing ML model deployment, ultimately empowering researchers, medical practitioners, and organizations in leveraging the potential of ML for diverse applications.

# REFERENCES

[1] R. Haux, “Medical informatics: Past, present, future,” *Int. J. Med. Inform.*, vol. 79, no. 9, pp. 599–610, 2010, doi: 10.1016/j.ijmedinf.2010.06.003.

[2] F. Sullivan, “What is health informatics?,” *J. Heal. Serv. Res. Policy*, vol. 6, no. 4, pp. 251–254, 2001, doi: 10.1258/1355819011927468.

[3] M. F. Collen, “Origins of medical informatics.,” *The Western journal of medicine*, vol. 145, no. 6. pp. 778–85, Dec. 1986, Accessed: Jan. 24, 2022. [Online]. Available: http://www.ncbi.nlm.nih.gov/pubmed/3544507.

[4] R. C. Deo, “Machine learning in medicine,” *Circulation*, vol. 132, no. 20, pp. 1920–1930, 2015, doi: 10.1161/CIRCULATIONAHA.115.001593.

[5] W. R. Hersh, “Medical Informatics,” *JAMA*, vol. 288, no. 16, p. 1955, Oct. 2002, doi: 10.1001/jama.288.16.1955.

[6] A. Jeklin, “Medical Imaging Informatics,” no. July, pp. XXII, 446, 2010, doi: 10.1007/978-1-4419-0385-3.

[7] H. P. Meinzer, M. Thorn, M. Vetter, P. Hassenpflug, M. Hastenteufel, and I. Wolf, “Medical imaging: Examples of clinical applications,” *ISPRS J. Photogramm. Remote Sens.*, vol. 56, no. 5–6, pp. 311–325, 2002, doi: 10.1016/S0924-2716(02)00072-2.

[8] X. Liu *et al.*, “Advances in Deep Learning-Based Medical Image Analysis,” *Heal. Data Sci.*, vol. 2021, pp. 1–14, Jun. 2021, doi: 10.34133/2021/8786793.

[9] E. Klang, “Deep learning and medical imaging.,” *J. Thorac. Dis.*, vol. 10, no. 3, pp. 1325–1328, Mar. 2018, doi: 10.21037/jtd.2018.02.76.

[10] B. J. Erickson, P. Korfiatis, Z. Akkus, and T. L. Kline, “Machine learning for medical imaging,” *Radiographics*, vol. 37, no. 2, pp. 505–515, 2017, doi: 10.1148/rg.2017160130.

[11] A. S. Lundervold and A. Lundervold, “An overview of deep learning in medical imaging focusing on MRI.,” *Z. Med. Phys.*, vol. 29, no. 2, pp. 102–127, May 2019, doi: 10.1016/j.zemedi.2018.11.002.

[12] J.-G. Lee *et al.*, “Deep Learning in Medical Imaging: General Overview,” *Korean J. Radiol.*, vol. 18, no. 4, p. 570, May 2017, doi: 10.3348/kjr.2017.18.4.570.

[13] M. Kim *et al.*, “Deep Learning in Medical Imaging.,” *Neurospine*, vol. 17, no. 2, pp. 471–472, Jun. 2020, doi: 10.14245/ns.1938396.198.c1.

[14] G. Currie, K. E. Hawk, E. Rohren, A. Vial, and R. Klein, “Machine Learning and Deep Learning in Medical Imaging: Intelligent Imaging.,” *J. Med. imaging Radiat. Sci.*, vol. 50,

no. 4, pp. 477–487, Dec. 2019, doi: 10.1016/j.jmir.2019.09.005.

[15] B. Sahiner *et al.*, "Deep learning in medical imaging and radiation therapy," *Med. Phys.*, vol. 46, no. 1, pp. e1–e36, Jan. 2019, doi: 10.1002/mp.13264.

[16] D. N. Sarah *et al.*, "AI Watch : AI Uptake in Health and Healthcare, 2020," 2020. doi: 10.2760/948860.

[17] Y. Guo, Z. Hao, S. Zhao, J. Gong, and F. Yang, "Artificial Intelligence in Health Care: Bibliometric Analysis.," *J. Med. Internet Res.*, vol. 22, no. 7, p. e18228, Jul. 2020, doi: 10.2196/18228.

[18] H. Hricak, "2016 New Horizons Lecture: Beyond Imaging—Radiology of Tomorrow," *Radiology*, vol. 286, no. 3, pp. 764–775, Mar. 2018, doi: 10.1148/radiol.2017171503.

[19] G. Affairs, "The Fourth Industrial Revolution," *Kartogr. i geoinformacije (Cartography Geoinformation)*, vol. 12, no. 20, pp. 126–129, Feb. 2017, doi: 10.17226/24699.

[20] E. Gómez González and E. Gómez Gutiérrez, "Artificial Intelligence in Medicine and Healthcare: applications, availability and societal impact." 2020, doi: 10.2760/047666.

[21] US Census Bureau, "2013 Information and Communication Technology Survey Publication." https://www.census.gov/library/publications/2015/econ/2013-icts.html (accessed Feb. 08, 2022).

[22] European Commission, "Accelerating the Development of the eHealth Market in Europe," 2007. doi: 10.2759/19946.

[23] IDC, "Worldwide Spending on Artificial Intelligence Is Expected to Double in Four Years, Reaching $110 Billion in 2024, According to New IDC Spending Guide." https://www.idc.com/getdoc.jsp?containerId=prUS46794720 (accessed Feb. 09, 2022).

[24] J. Bresnick, "Artificial Intelligence in Healthcare Spending to Hit $36B." https://healthitanalytics.com/news/artificial-intelligence-in-healthcare-spending-to-hit-36b (accessed Feb. 09, 2022).

[25] I. S. Hofer, M. Burns, S. Kendale, and J. P. Wanderer, "Realistically Integrating Machine Learning Into Clinical Practice: A Road Map of Opportunities, Challenges, and a Potential Future.," *Anesth. Analg.*, vol. 130, no. 5, pp. 1115–1118, May 2020, doi: 10.1213/ANE.0000000000004575.

[26] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in Deploying Machine Learning: a Survey of Case Studies," Nov. 2020, [Online]. Available:

http://arxiv.org/abs/2011.09926.

[27] R. Elshawi, M. Maher, and S. Sakr, “Automated Machine Learning: State-of-The-Art and Open Challenges,” Jun. 2019, [Online]. Available: http://arxiv.org/abs/1906.02287.

[28] J. vom Brocke, A. Hevner, and A. Maedche, “Introduction to Design Science Research,” no. September, pp. 1–13, 2020, doi: 10.1007/978-3-030-46781-4_1.

[29] D. L. Kappen, “Simplifying Design Science Research, Action Research and Design Research | by Dennis L. Kappen | Medium,” 2019. https://medium.com/@3D_Ideation/simplifying-design-science-research-action-research-and-design-research-bf564959402b (accessed Jan. 09, 2022).

[30] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *J. Manag. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[31] L. V. Lapão, M. M. Da Silva, and J. Gregório, “Implementing an online pharmaceutical service using design science research,” *BMC Med. Inform. Decis. Mak.*, vol. 17, no. 1, pp. 1–14, 2017, doi: 10.1186/s12911-017-0428-2.

[32] V. K. Vaishnavi, V. K. Vaishnavi, and W. Kuechler, “Design Science Research Methods and Patterns,” *Design Science Research Methods and Patterns*. 2015, doi: 10.1201/b18448.

[33] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Q. Manag. Inf. Syst.*, vol. 28, no. 1, pp. 75–105, 2004, doi: 10.2307/25148625.

[34] B. Pandey, D. Kumar Pandey, B. Pratap Mishra, and W. Rhmann, “A comprehensive survey of deep learning in the field of medical imaging and medical natural language processing: Challenges and research directions,” *J. King Saud Univ. - Comput. Inf. Sci.*, Jan. 2021, doi: 10.1016/j.jksuci.2021.01.007.

[35] P. Singh, “Deploy Machine Learning Models to Production,” *Deploy Machine Learning Models to Production*. Apress, Berkeley, CA, 2021, doi: 10.1007/978-1-4842-6546-8.

[36] C. Hill, R. Bellamy, T. Erickson, and M. Burnett, “Trials and tribulations of developers of intelligent systems: A field study,” in *2016 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, Sep. 2016, pp. 162–170, doi: 10.1109/VLHCC.2016.7739680.

[37] D. Baylor *et al.*, “TFX: A TensorFlow-Based Production-Scale Machine Learning

Platform," 2017, doi: 10.1145/3097983.3098021.

[38] R. Ashmore, R. Calinescu, and C. Paterson, "Assuring the Machine Learning Lifecycle: Desiderata, Methods, and Challenges," *ACM Comput. Surv.*, vol. 54, no. 5, 2021, doi: 10.1145/3453444.

[39] S. Amershi *et al.*, "Software Engineering for Machine Learning: A Case Study," in *Proceedings - 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2019*, 2019, pp. 291–300, doi: 10.1109/ICSE-SEIP.2019.00042.

[40] AHA, "AI and the Health Care Workforce | AHA." 2019, Accessed: Mar. 09, 2023. [Online]. Available: https://www.aha.org/center/emerging-issues/market-insights/ai/ai-and-health-care-workforce.

[41] E. J. Topol, "High-performance medicine: the convergence of human and artificial intelligence," *Nat. Med.*, vol. 25, no. 1, pp. 44–56, Jan. 2019, doi: 10.1038/s41591-018-0300-7.

[42] McKinsey & Company, "Transforming healthcare with AI: The impact on the workforce and organizations | McKinsey." https://www.mckinsey.com/industries/healthcare/our-insights/transforming-healthcare-with-ai#/ (accessed Mar. 09, 2023).

[43] F. Shan *et al.*, "Lung Infection Quantification of COVID-19 in CT Images with Deep Learning," no. C, pp. 2–5, Mar. 2020, doi: 10.1002/mp.14609.

[44] S. Jackson, M. Yaqub, and C.-X. Li, "The Agile Deployment of Machine Learning Models in Healthcare," *Front. Big Data*, vol. 1, Jan. 2019, doi: 10.3389/fdata.2018.00007.

[45] A. A. A. Abid, A. Abdalla, A. A. A. Abid, D. Khan, A. Alfozan, and J. Zou, "Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild," no. Ml, Jun. 2019, [Online]. Available: http://arxiv.org/abs/1906.02569.

[46] G. Stiglic, P. Kocbek, N. Fijacko, M. Zitnik, K. Verbert, and L. Cilar, "Interpretability of machine learning-based prediction models in healthcare," *WIREs Data Min. Knowl. Discov.*, vol. 10, no. 5, Sep. 2020, doi: 10.1002/widm.1379.

[47] D. S. Weld and G. Bansal, "The Challenge of Crafting Intelligible Intelligence," Mar. 2018, [Online]. Available: http://arxiv.org/abs/1803.04263.

[48] D. Gunning, "Explainable Artificial Intelligence (XAI)," *DARPA*, 2017, Accessed: Mar. 16, 2023. [Online]. Available: http://listverse.com/.

[49] J. Y. Cheng, J. T. Abel, U. G. J. Balis, D. S. McClintock, and L. Pantanowitz, “Challenges in the Development, Deployment, and Regulation of Artificial Intelligence in Anatomic Pathology.,” *Am. J. Pathol.*, vol. 191, no. 10, pp. 1684–1692, 2021, doi: 10.1016/j.ajpath.2020.10.018.

[50] A. S. Ahuja and C. E. Schmidt, “The impact of artificial intelligence in medicine on the future role of the physician,” *Subj. Ophthalmol. Radiol. Med. Imaging*, doi: 10.7717/peerj.7702.

[51] J. V. Tu, “Advantages and disadvantages of using artificial neural networks versus logistic regression for predicting medical outcomes,” *J. Clin. Epidemiol.*, vol. 49, no. 11, pp. 1225–1231, Nov. 1996, doi: 10.1016/S0895-4356(96)00002-9.

[52] F. Cabitza, R. Rasoini, and G. F. Gensini, “Unintended Consequences of Machine Learning in Medicine,” *JAMA*, vol. 318, no. 6, p. 517, Aug. 2017, doi: 10.1001/jama.2017.7797.

[53] A. Vellido, “The importance of interpretability and visualization in machine learning for applications in medicine and health care,” *Neural Comput. Appl.*, vol. 32, no. 24, pp. 18069–18083, Dec. 2020, doi: 10.1007/s00521-019-04051-w.

[54] R. Piltaver, M. Luštrek, M. Gams, and S. Martinčić-Ipšić, “What makes classification trees comprehensible?,” *Expert Syst. Appl.*, vol. 62, pp. 333–346, Nov. 2016, doi: 10.1016/j.eswa.2016.06.009.

[55] J. Petch, S. Di, and W. Nelson, “Opening the Black Box: The Promise and Limitations of Explainable Machine Learning in Cardiology,” *Can. J. Cardiol.*, vol. 38, no. 2, pp. 204–213, Feb. 2022, doi: 10.1016/j.cjca.2021.09.004.

[56] I. G. and Y. B. and A. Courville, “Deep Learning,” 2016, [Online]. Available: http://www.deeplearningbook.org.

[57] S. Shalev-Shwartz and S. Ben-David, “Understanding machine learning: From theory to algorithms,” *Understanding Machine Learning: From Theory to Algorithms*, vol. 9781107057. pp. 1–397, 2013, doi: 10.1017/CBO9781107298019.

[58] I. Nunes and D. Jannach, “A systematic review and taxonomy of explanations in decision support and recommender systems,” *User Model. User-adapt. Interact.*, vol. 27, no. 3–5, pp. 393–444, Dec. 2017, doi: 10.1007/s11257-017-9195-0.

[59] C. Rudin, K. L. Wagstaff, C. Rudin, and K. L. Wagstaff, “Machine learning for science and society,” *Mach Learn*, vol. 95, pp. 1–9, 2014, doi: 10.1007/s10994-013-5425-9.

[60] B. Baesens, R. Bapna, J. R. Marsden, J. Vanthienen, and J. L. Zhao, "Transformational Issues of Big Data and Analytics in Networked Business," *MIS Q.*, vol. 40, no. 4, pp. 807–818, Apr. 2016, doi: 10.25300/MISQ/2016/40:4.03.

[61] M. Shafique *et al.*, "Adaptive and Energy-Efficient Architectures for Machine Learning: Challenges, Opportunities, and Research Roadmap," in *2017 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, Jul. 2017, vol. 2017-July, pp. 627–632, doi: 10.1109/ISVLSI.2017.124.

[62] J. L. Guidi *et al.*, "Clinician Perception of the Effectiveness of an Automated Early Warning and Response System for Sepsis in an Academic Medical Center.," *Ann. Am. Thorac. Soc.*, vol. 12, no. 10, pp. 1514–9, Oct. 2015, doi: 10.1513/AnnalsATS.201503-129OC.

[63] A. Weller, "Transparency: Motivations and Challenges," 2019, pp. 23–40.

[64] U. Bhatt *et al.*, "Explainable machine learning in deployment," in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, Jan. 2020, pp. 648–657, doi: 10.1145/3351095.3375624.

[65] B. Lepri, N. Oliver, E. Letouzé, A. Pentland, and P. Vinck, "Fair, Transparent, and Accountable Algorithmic Decision-making Processes: The Premise, the Proposed Solutions, and the Open Challenges," *Philos. Technol.*, vol. 31, no. 4, pp. 611–627, Dec. 2018, doi: 10.1007/S13347-017-0279-X/METRICS.

[66] S. M. Lundberg *et al.*, "Explainable machine-learning predictions for the prevention of hypoxaemia during surgery," *Nat. Biomed. Eng.*, vol. 2, no. 10, pp. 749–760, Oct. 2018, doi: 10.1038/s41551-018-0304-0.

[67] M. Mitchell *et al.*, "Model Cards for Model Reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency*, Jan. 2019, pp. 220–229, doi: 10.1145/3287560.3287596.

[68] O. O'Neill, "Linking Trust to Trustworthiness," *Int. J. Philos. Stud.*, vol. 26, no. 2, pp. 293–300, Mar. 2018, doi: 10.1080/09672559.2018.1454637.

[69] A. Goldstein, A. Kapelner, J. Bleich, and E. Pitkin, "Peeking Inside the Black Box: Visualizing Statistical Learning With Plots of Individual Conditional Expectation," *J. Comput. Graph. Stat.*, vol. 24, no. 1, pp. 44–65, Jan. 2015, doi: 10.1080/10618600.2014.907095.

[70] D. V. Carvalho, E. M. Pereira, and J. S. Cardoso, "Machine Learning Interpretability: A

Survey on Methods and Metrics," *Electronics*, vol. 8, no. 8, p. 832, Jul. 2019, doi: 10.3390/electronics8080832.

[71] M. Reyes *et al.*, "On the Interpretability of Artificial Intelligence in Radiology: Challenges and Opportunities," *Radiol. Artif. Intell.*, vol. 2, no. 3, p. e190043, May 2020, doi: 10.1148/ryai.2020190043.

[72] R. Elshawi, M. H. Al-Mallah, and S. Sakr, "On the interpretability of machine learning-based model for predicting hypertension.," *BMC Med. Inform. Decis. Mak.*, vol. 19, no. 1, p. 146, Jul. 2019, doi: 10.1186/s12911-019-0874-0.

[73] C. Rudin, "Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead," Nov. 2018, [Online]. Available: http://arxiv.org/abs/1811.10154.

[74] J. L. Ribeiro, M. Figueredo, A. Araujo, N. Cacho, and F. Lopes, "A Microservice Based Architecture Topology for Machine Learning Deployment," in *2019 IEEE International Smart Cities Conference (ISC2)*, Oct. 2019, no. Isc2 2019, pp. 426–431, doi: 10.1109/ISC246665.2019.9071708.

[75] M. Van Lent, W. Fisher, and M. Mancuso, "An explainable artificial intelligence system for small-unit tactical behavior," *Proc. Natl. Conf. Artif. Intell.*, pp. 900–907, 2004.

[76] T. Miller, "Explanation in artificial intelligence: Insights from the social sciences," *Artif. Intell.*, vol. 267, pp. 1–38, Feb. 2019, doi: 10.1016/j.artint.2018.07.007.

[77] R. El Shawi, Y. Sherif, M. Al-Mallah, and S. Sakr, "Interpretability in HealthCare A Comparative Study of Local Machine Learning Interpretability Techniques," in *2019 IEEE 32nd International Symposium on Computer-Based Medical Systems (CBMS)*, Jun. 2019, pp. 275–280, doi: 10.1109/CBMS.2019.00065.

[78] L. H. Gilpin, D. Bau, B. Z. Yuan, A. Bajwa, M. Specter, and L. Kagal, "Explaining Explanations: An Overview of Interpretability of Machine Learning," in *2018 IEEE 5th International Conference on Data Science and Advanced Analytics (DSAA)*, Oct. 2018, pp. 80–89, doi: 10.1109/DSAA.2018.00018.

[79] S. Stumpf *et al.*, "Interacting meaningfully with machine learning systems: Three experiments," *Int. J. Hum. Comput. Stud.*, vol. 67, no. 8, pp. 639–662, Aug. 2009, doi: 10.1016/j.ijhcs.2009.03.004.

[80] S. Stumpf *et al.*, "Toward harnessing user feedback for machine learning," in *Proceedings of the 12th international conference on Intelligent user interfaces*, Jan.

2007, pp. 82–91, doi: 10.1145/1216295.1216316.

[81] T. Kulesza, S. Stumpf, M. Burnett, and I. Kwan, “Tell me more? the effects of mental model soundness on personalizing an intelligent agent,” *Conf. Hum. Factors Comput. Syst. - Proc.*, pp. 1–10, 2012, doi: 10.1145/2207676.2207678.

[82] A. Glass, D. L. McGuinness, and M. Wolverton, “Toward establishing trust in adaptive agents,” *Int. Conf. Intell. User Interfaces, Proc. IUI*, pp. 227–236, 2008, doi: 10.1145/1378773.1378804.

[83] M. T. Dzindolet, S. A. Peterson, R. A. Pomranky, L. G. Pierce, and H. P. Beck, “The role of trust in automation reliance,” *Int. J. Hum. Comput. Stud.*, vol. 58, no. 6, pp. 697–718, Jun. 2003, doi: 10.1016/S1071-5819(03)00038-7.

[84] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, “Power to the People: The Role of Humans in Interactive Machine Learning,” *AI Mag.*, vol. 35, no. 4, pp. 105–120, Dec. 2014, doi: 10.1609/AIMAG.V35I4.2513.

[85] C. Xiao, E. Choi, and J. Sun, “Opportunities and challenges in developing deep learning models using electronic health records data: a systematic review,” *J. Am. Med. Informatics Assoc.*, vol. 25, no. 10, pp. 1419–1428, Oct. 2018, doi: 10.1093/jamia/ocy068.

[86] B. Shickel, P. J. Tighe, A. Bihorac, and P. Rashidi, “Deep EHR: A Survey of Recent Advances in Deep Learning Techniques for Electronic Health Record (EHR) Analysis,” *IEEE J. Biomed. Heal. Informatics*, vol. 22, no. 5, pp. 1589–1604, Sep. 2018, doi: 10.1109/JBHI.2017.2767063.

[87] K. Wagstaff, “Machine Learning that Matters,” *Proc. 29th Int. Conf. Mach. Learn. ICML 2012*, vol. 1, pp. 529–534, Jun. 2012, [Online]. Available: http://arxiv.org/abs/1206.4656.

[88] L. Baier and S. Seebacher, “Challenges in the Deployment and Operation of Machine Learning in Practice,” *27th Eur. Conf. Inf. Syst.*, no. May, pp. 1–15, 2019, [Online]. Available: https://aisel.aisnet.org/ecis2019_rp/163/.

[89] S. Tonekaboni, S. Joshi, M. D. McCradden, and A. Goldenberg, “What Clinicians Want: Contextualizing Explainable Machine Learning for Clinical End Use,” May 2019, [Online]. Available: http://arxiv.org/abs/1905.05134.

[90] A. Hertzmann, “Can Computers Create Art?,” Jan. 2018, [Online]. Available: http://arxiv.org/abs/1801.04486.

[91] R. Bhardwaj, A. R. Nambiar, and D. Dutta, “A Study of Machine Learning in Healthcare,” in *2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, Jul. 2017, pp. 236–241, doi: 10.1109/COMPSAC.2017.164.

[92] A. Radovic *et al.*, “Machine learning at the energy and intensity frontiers of particle physics.,” *Nature*, vol. 560, no. 7716, pp. 41–48, Aug. 2018, doi: 10.1038/s41586-018-0361-2.

[93] J. Zou, M. Huss, A. Abid, P. Mohammadi, A. Torkamani, and A. Telenti, “A primer on deep learning in genomics.,” *Nat. Genet.*, vol. 51, no. 1, pp. 12–18, Jan. 2019, doi: 10.1038/s41588-018-0295-5.

[94] D. Sculley *et al.*, “Hidden technical debt in machine learning systems,” *Adv. Neural Inf. Process. Syst.*, vol. 2015-Janua, pp. 2503–2511, 2015.

[95] Z.-H. Zhou, “Machine learning challenges and impact: an interview with Thomas Dietterich,” *Natl. Sci. Rev.*, vol. 5, no. 1, pp. 54–58, Jan. 2018, doi: 10.1093/nsr/nwx045.

[96] A. L. Ferguson, “Machine learning and data science in soft materials engineering,” *J. Phys. Condens. Matter*, vol. 30, no. 4, Jan. 2018, doi: 10.1088/1361-648X/aa98bd.

[97] J. Dyck, “Machine learning for engineering,” in *2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*, Jan. 2018, pp. 422–427, doi: 10.1109/ASPDAC.2018.8297360.

[98] M. Sendak *et al.*, “‘The Human Body is a Black Box’: Supporting Clinical Decision-Making with Deep Learning,” Nov. 2019, [Online]. Available: http://arxiv.org/abs/1911.08089.

[99] ICO, “Project explAIn Interim report Public and industry engagement,” Accessed: Mar. 16, 2023. [Online]. Available: https://ico.org.uk/about-the-ico/research-and-reports/project-explain-interim-report/.

[100] K. Xu *et al.*, “ECGLens: Interactive Visual Exploration of Large Scale ECG Data for Arrhythmia Detection,” in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, Apr. 2018, pp. 1–12, doi: 10.1145/3173574.3174237.

[101] R. Soden, D. Wagenaar, D. Luo, and A. Tijssen, “Taking Ethics, Fairness, and Bias Seriously in Machine Learning for Disaster Risk Management,” Dec. 2019, [Online]. Available: http://arxiv.org/abs/1912.05538.

[102] M. Madaio *et al.*, “Firebird: Predicting Fire Risk and Prioritizing Fire Inspections in Atlanta,” Feb. 2016, [Online]. Available: http://arxiv.org/abs/1602.09067.

[103] K. Ackermann *et al.*, “Deploying Machine Learning Models for Public Policy,” in

*Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Jul. 2018, pp. 15–22, doi: 10.1145/3219819.3219911.

[104] S. Klemm, A. Scherzinger, D. Drees, and X. Jiang, "Barista - a Graphical Tool for Designing and Training Deep Neural Networks," Feb. 2018, [Online]. Available: http://arxiv.org/abs/1802.04626.

[105] D. Muthukrishna, D. Parkinson, and B. Tucker, "DASH: Deep Learning for the Automated Spectral Classification of Supernovae and their Hosts," Mar. 2019, doi: 10.3847/1538-4357/ab48f4.

[106] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-World Challenges and Research Innovations," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, May 2019, pp. 4–5, doi: 10.1109/ICSE-Companion.2019.00023.

[107] K. Hazelwood *et al.*, "Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb. 2018, pp. 620–629, doi: 10.1109/HPCA.2018.00059.

[108] N. Lopes and B. Ribeiro, "Novel Trends in Scaling Up Machine Learning Algorithms," in *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, Dec. 2017, pp. 632–636, doi: 10.1109/ICMLA.2017.00-90.

[109] C. Shea, A. Page, and T. Mohsenin, "SCALENet: A SCalable Low power AccELerator for Real-time Embedded Deep Neural Networks," in *Proceedings of the 2018 on Great Lakes Symposium on VLSI*, May 2018, pp. 129–134, doi: 10.1145/3194554.3194601.

[110] R. Boutaba *et al.*, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *J. Internet Serv. Appl.*, vol. 9, no. 1, p. 16, Dec. 2018, doi: 10.1186/s13174-018-0087-2.

[111] C. Parker, "Unexpected challenges in large scale machine learning," in *Proceedings of the 1st International Workshop on Big Data, Streams and Heterogeneous Source Mining: Algorithms, Systems, Programming Models and Applications*, Aug. 2012, pp. 1–6, doi: 10.1145/2351316.2351317.

[112] B. Langmead and A. Nellore, "Cloud computing for genomic data analysis and collaboration," *Nat. Rev. Genet.*, vol. 19, no. 4, pp. 208–219, Apr. 2018, doi: 10.1038/nrg.2017.113.

[113] N. Werts and M. Adya, “Data Mining in Health-Care : Issues and a Research Agenda,” *Amcis*, pp. 94–97, 2000.

[114] P. Domingos, “A few useful things to know about machine learning,” *Commun. ACM*, vol. 55, no. 10, pp. 78–87, Oct. 2012, doi: 10.1145/2347736.2347755.

[115] A. D. Sarwate and K. Chaudhuri, “Signal Processing and Machine Learning with Differential Privacy: Algorithms and Challenges for Continuous Data,” *IEEE Signal Process. Mag.*, vol. 30, no. 5, pp. 86–94, Sep. 2013, doi: 10.1109/MSP.2013.2259911.

[116] J. Heit, J. Liu, and M. Shah, “An architecture for the deployment of statistical models for the big data era,” in *2016 IEEE International Conference on Big Data (Big Data)*, Dec. 2016, pp. 1377–1384, doi: 10.1109/BigData.2016.7840745.

[117] G. Ateniese, G. Felici, L. V. Mancini, A. Spognardi, A. Villani, and D. Vitali, “Hacking Smart Machines with Smarter Ones: How to Extract Meaningful Data from Machine Learning Classifiers,” Jun. 2013, [Online]. Available: http://arxiv.org/abs/1306.4447.

[118] M. Fredrikson, S. Jha, and T. Ristenpart, “Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, Oct. 2015, pp. 1322–1333, doi: 10.1145/2810103.2813677.

[119] M. Fredrikson, E. Lantz, S. Jha, S. Lin, D. Page, and T. Ristenpart, “Privacy in Pharmacogenetics: An End-to-End Case Study of Personalized Warfarin Dosing.,” *Proc. ... USENIX Secur. Symp. UNIX Secur. Symp.*, vol. 2014, pp. 17–32, Aug. 2014, [Online]. Available: http://www.ncbi.nlm.nih.gov/pubmed/27077138.

[120] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, “Stealing Machine Learning Models via Prediction APIs,” *Proc. 25th USENIX Secur. Symp.*, no. Ml, pp. 601–618, Sep. 2016, [Online]. Available: http://arxiv.org/abs/1609.02943.

[121] A. Kurakin, I. Goodfellow, and S. Bengio, “Adversarial Machine Learning at Scale,” Nov. 2016, [Online]. Available: http://arxiv.org/abs/1611.01236.

[122] B. Biggio and F. Roli, “Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning,” Dec. 2017, doi: 10.1016/j.patcog.2018.07.023.

[123] G. E. Soto and J. A. Spertus, “EPOCH® and ePRISM®: A web-based translational framework for bridging outcomes research and clinical Practice,” *Comput. Cardiol.*, vol. 34, pp. 205–208, Sep. 2007, doi: 10.1109/CIC.2007.4745457.

[124] M. Khalilia, M. Choi, A. Henderson, S. Iyengar, M. Braunstein, and J. Sun, “Clinical

Predictive Modeling Development and Deployment through FHIR Web Services."

[125] J. Gruendner *et al.*, "Ketos: Clinical decision support and machine learning as a service – A training and deployment platform based on Docker, OMOP-CDM, and FHIR Web Services," *PLoS One*, vol. 14, no. 10, pp. 1–16, 2019, doi: 10.1371/journal.pone.0223010.

[126] E. Gibson *et al.*, "NiftyNet: a deep-learning platform for medical imaging," *Comput. Methods Programs Biomed.*, vol. 158, pp. 113–122, 2018, doi: 10.1016/j.cmpb.2018.01.025.

[127] D. Desser, F. Assunção, X. Yan, V. Alves, H. M. Fernandes, and T. Hummel, "Automatic Segmentation of the Olfactory Bulb," *Brain Sci.*, vol. 11, no. 9, p. 1141, Aug. 2021, doi: 10.3390/brainsci11091141.

[128] D. Martins, "Deep Learning for Real-time Decoding of Sound Events Related to Autism Spectrum Disorder," University of Minho, 2021.

[129] R. Machado, "Environmental Sounds Automatic Classification," University of Minho, 2021.

[130] Stack Overflow Ltd., "Stack Overflow Developer Survey 2020," *Stack Overflow Insights*, 2020. https://insights.stackoverflow.com/survey/2020 (accessed Apr. 30, 2022).

[131] Stack Overflow, "Stack Overflow Developer Survey 2021," *Stack Overflow Insights*, 2021. https://insights.stackoverflow.com/survey/2021 (accessed Apr. 30, 2022).

[132] "Overview — NumPy v1.21 Manual." https://numpy.org/doc/stable/ (accessed Dec. 13, 2021).

[133] "Package overview — pandas 1.3.5 documentation." https://pandas.pydata.org/pandas-docs/stable/getting_started/overview.html (accessed Dec. 13, 2021).

[134] "Matplotlib — Visualization with Python." https://matplotlib.org/ (accessed Dec. 13, 2021).

[135] "Neuroimaging in Python — NiBabel 3.2.1+100.g9fdf5e3e documentation." https://nipy.org/nibabel/ (accessed Dec. 13, 2021).

[136] "jiaaro/pydub @ GitHub." https://pydub.com/ (accessed Dec. 13, 2021).

[137] "librosa — librosa 0.8.1 documentation." https://librosa.org/doc/latest/index.html (accessed Dec. 13, 2021).

[138] "TensorFlow." https://www.tensorflow.org/ (accessed Dec. 13, 2021).

[139] F. Chollet and & O., “Keras: the Python deep learning API,” *Keras: the Python deep learning API*, 2020. https://keras.io/ (accessed Apr. 30, 2022).

[140] KerasSIG, “About Keras,” *Keras Special Interest Group*. 2020, Accessed: Apr. 30, 2022. [Online]. Available: https://keras.io/about/.

[141] “XGBoost Documentation — xgboost 1.5.1 documentation.” https://xgboost.readthedocs.io/en/stable/ (accessed Dec. 13, 2021).

[142] “logging — Logging facility for Python — Python 3.10.1 documentation.” https://docs.python.org/3/library/logging.html (accessed Dec. 13, 2021).

[143] G. Beattie, “Machine Learning,” no. July. Independently published (July 27, 2020), pp. 1–23, 2020.

[144] O. Soranson, “Python Machine Learning.” Independently published (November 7, 2019), p. 162, 2019.

[145] A. Coen, *Machine Learning for Beginners: The Ultimate Guide to Learn and Understand Machine Learning - A Practical Approach to Master Machine Learning to Improve and Increase Business Results*. Independently Published, 2020, 2020.

[146] J. Hugh, “Python Machine Learning : A Crash Course for Beginners to Understand Machine learning, Artificial Intelligence, Neural Networks, and Deep Learning with Scikit-Learn, TensorFlow, and Keras.” Independently published (November 13, 2019), p. 133, 2019.

[147] L. Visengeriyeva, A. Kammer, I. Bär, A. Kniesz, and M. Plöd, “End-to-end Machine Learning Workflow,” *MLOps*, 2021. https://ml-ops.org/content/end-to-end-ml-workflow (accessed Apr. 29, 2022).

[148] “Glossary — ML Glossary documentation.” https://ml-cheatsheet.readthedocs.io/en/latest/glossary.html (accessed Dec. 21, 2021).

[149] “Machine learning glossary - ML.NET | Microsoft Docs.” https://docs.microsoft.com/en-us/dotnet/machine-learning/resources/glossary (accessed Dec. 21, 2021).

[150] “Machine Learning Glossary | Google Developers.” https://developers.google.com/machine-learning/glossary (accessed Dec. 21, 2021).

[151] F. M. Mark L. Gillenson, Paulraj Ponniah, Alex Kriegel, Boris Trukhnov, Allen G. Taylor, Gavin Powell, “Introduction to Database Management Edition: 1.” Wiley, p. 504, 2007.

[152] A. M. Satinder Bal Gupta, “Introduction to Database Management System, 2nd Edition.”

University Science Press, p. 552, 2017.

[153] C. J. Date, “An Introduction to Database Systems Edition: 8th.” Pearson, p. 1024, 2003.

[154] “What Is a Database | Oracle Portugal.” https://www.oracle.com/pt/database/what-is-database/ (accessed Jan. 02, 2022).

[155] A. Uwase, “Here is the reason why SQLAlchemy is so popular | by Ange Uwase | Towards Data Science.” https://towardsdatascience.com/here-is-the-reason-why-sqlalchemy-is-so-popular-43b489d3fb00 (accessed Feb. 20, 2022).

[156] “Object-relational Mappers (ORMs) - Full Stack Python.” https://www.fullstackpython.com/object-relational-mappers-orms.html (accessed Feb. 22, 2022).

[157] “SQL (Relational) Databases - FastAPI.” https://fastapi.tiangolo.com/tutorial/sql-databases/?h=sql (accessed Feb. 22, 2022).

[158] “Databases — The Hitchhiker’s Guide to Python.” https://docs.python-guide.org/scenarios/db/#sqlalchemy (accessed Feb. 20, 2022).

[159] “Features - SQLAlchemy.” https://www.sqlalchemy.org/features.html (accessed Feb. 22, 2022).

[160] L. Amazon, X. Guo, and Z. Xia, “The Architecture of Open Source Applications, Volume II,” *University of California Berkeley*. lulu.com, pp. 291-314 (chapter 20), 2013, [Online]. Available: http://software-carpentry.org/2011/05/06/.

[161] “SQLAlchemy - Full Stack Python.” https://www.fullstackpython.com/sqlalchemy.html (accessed Feb. 20, 2022).

[162] M. Bayer, “SQLAlchemy - The Database Toolkit for Python.” https://www.sqlalchemy.org/ (accessed Feb. 20, 2022).

[163] “Dialects — SQLAlchemy 1.4 Documentation.” https://docs.sqlalchemy.org/en/14/dialects/ (accessed Feb. 22, 2022).

[164] P. Johnston, “10 Reasons to love SQLAlchemy.” http://pajhome.org.uk/blog/10_reasons_to_love_sqlalchemy.html (accessed Feb. 22, 2022).

[165] X. Gantan, “Python’s SQLAlchemy vs Other ORMs | Python Central.” https://www.pythoncentral.io/sqlalchemy-vs-orms/ (accessed Feb. 22, 2022).

[166] “What is PostgreSQL? | IBM.” https://www.ibm.com/cloud/learn/postgresql (accessed Mar. 03, 2022).

[167] “What is PostgreSQL? | DigitalOcean.” https://www.digitalocean.com/community/tutorials/what-is-postgresql (accessed Mar. 03, 2022).

[168] “PostgreSQL: About.” https://www.postgresql.org/about/ (accessed Mar. 03, 2022).

[169] “PostgreSQL: Documentation: 14: 1. What Is PostgreSQL?” https://www.postgresql.org/docs/current/intro-whatis.html (accessed Mar. 03, 2022).

[170] P. Luzanov, E. Rogov, and I. Levshin, “Postgres: The First Experience,” 2020, [Online]. Available: https://edu.postgrespro.ru/introbook_v6_en.pdf.

[171] “Web App Development in 2022: Everything You Need to Know | Trio Developers.” https://trio.dev/blog/web-app-development (accessed May 02, 2022).

[172] “How to Develop a Web Application? [Types, Steps, Challenges].” https://spdload.com/blog/how-to-develop-a-web-application/ (accessed May 02, 2022).

[173] “Website vs Web App: What’s the Difference? | by Essential Designs | Medium.” https://medium.com/@essentialdesign/website-vs-web-app-whats-the-difference-e499b18b60b4 (accessed May 02, 2022).

[174] “What is the Difference Between a Website and a Web Application?” https://www.freecodecamp.org/news/difference-between-a-website-and-a-web-application/ (accessed May 02, 2022).

[175] “Web Apps vs Websites: What’s the Difference? Does It Matter?” https://themeisle.com/blog/web-apps-vs-websites/ (accessed May 03, 2022).

[176] “What Is a Web Application? How It Works, Benefits and Examples | Indeed.com.” https://www.indeed.com/career-advice/career-development/what-is-web-application (accessed May 11, 2022).

[177] “The benefits of web applications in today’s technological era.” https://www.kcsitglobal.com/blogs/detail-blog/the-benefits-of-web-applications-in-todays-technological-era (accessed May 11, 2022).

[178] “The benefits of web-based applications.” https://www.magicwebsolutions.co.uk/blog/the-benefits-of-web-based-applications.htm (accessed May 11, 2022).

[179] “Advantages And Disadvantages - Web Apps - Objective.” https://objectiveit.com/blog/the-advantages-and-disadvantages-of-web-apps/

(accessed May 11, 2022).

[180] “What are the advantages and disadvantages of web applications?” https://www.itrobes.com/what-are-the-advantages-and-disadvantages-of-web-applications/ (accessed May 11, 2022).

[181] “5 Advantages and Disadvantages of Web Application | Drawbacks & Benefits of Web Application.” https://www.hitechwhizz.com/2021/04/5-advantages-and-disadvantages-drawbacks-benefits-of-web-application.html (accessed May 11, 2022).

[182] MOZILLA, “SPA (Single-page application) - MDN Web Docs Glossary: Definitions of Web-related terms | MDN,” *developer.mozilla.org*, 2021. https://developer.mozilla.org/en-US/docs/Glossary/SPA (accessed May 02, 2022).

[183] “Single-page application vs. multiple-page application | by Neoteric | Medium.” https://medium.com/@NeotericEU/single-page-application-vs-multiple-page-application-2591588efe58 (accessed May 11, 2022).

[184] “Top 5 Single Page Application Frameworks To Use in 2022.” https://www.monocubed.com/blog/top-single-page-application-frameworks/ (accessed May 02, 2022).

[185] “What Is a Single Page Application (SPA)? | OutSystems.” https://www.outsystems.com/glossary/what-is-single-page-application/ (accessed May 02, 2022).

[186] “Advanced Load Balancer, Web Server, & Reverse Proxy - NGINX.” https://www.nginx.com/ (accessed May 23, 2022).

[187] “Uvicorn.” https://www.uvicorn.org/ (accessed May 23, 2022).

[188] “Gunicorn - Python WSGI HTTP Server for UNIX.” https://gunicorn.org/ (accessed May 23, 2022).

[189] “HTML: Linguagem de Marcação de Hipertexto | MDN.” https://developer.mozilla.org/pt-BR/docs/Web/HTML (accessed May 22, 2022).

[190] “Understanding APIs.” https://www.redhat.com/en/topics/api (accessed May 23, 2022).

[191] Rehan Haider, “Web API Development with Python: A Beginner’s Guide using Flask and FastAPI (Intermediate Python).” 2021, [Online]. Available: https://www.amazon.in/Web-API-Development-Python-Intermediate-ebook/dp/B09BJLKM6F.

[192] “What is an API?” https://www.redhat.com/en/topics/api/what-are-application-programming-interfaces (accessed Jun. 07, 2022).

[193] “What is an Application Programming Interface (API) | IBM.” https://www.ibm.com/cloud/learn/api (accessed Jun. 07, 2022).

[194] “What is an API? - API Beginner’s Guide - AWS.” https://aws.amazon.com/what-is/api/?nc1=h_ls (accessed Jun. 07, 2022).

[195] “REST APIs must be hypertext-driven » Untangled.” https://roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven (accessed Feb. 19, 2022).

[196] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” 2000.

[197] “Understanding Flask vs FastAPI Web Framework | by Sue Lynn | Towards Data Science.” https://towardsdatascience.com/understanding-flask-vs-fastapi-web-framework-fe12bb58ee75 (accessed Aug. 11, 2022).

[198] Z. Ahmed, “High-performing Apps with Python - A FastAPI Tutorial,” *Toptal*. https://www.toptal.com/python/build-high-performing-apps-with-the-python-fastapi-framework (accessed Aug. 11, 2022).

[199] “Choosing between Django, Flask, and FastAPI | Engineering Education (EngEd) Program | Section.” https://www.section.io/engineering-education/choosing-between-django-flask-and-fastapi/ (accessed Aug. 11, 2022).

[200] “Moving from Flask to FastAPI | TestDriven.io.” https://testdriven.io/blog/moving-from-flask-to-fastapi/ (accessed Aug. 11, 2022).

[201] “FastAPI vs Flask: Comparison Guide for Data Science Enthusiasts -.” https://analyticsindiamag.com/fastapi-vs-flask-comparison-guide-for-data-science-enthusiasts/ (accessed Aug. 11, 2022).

[202] “Python Flask versus FastAPI: Which Should You Choose?” https://www.netguru.com/blog/python-flask-versus-fastapi (accessed Aug. 11, 2022).

[203] “Revisiting Flask vs FastAPI in 2022 | by Mike Wolfe | Python in Plain English.” https://python.plainenglish.io/revisiting-flask-vs-fastapi-in-2022-650fb76baf08 (accessed Aug. 11, 2022).

[204] “FastAPI.” https://fastapi.tiangolo.com/ (accessed Aug. 11, 2022).

[205] “Introducing FastAPI. FastAPI is a modern, fast… | by Sebastián Ramírez | Medium.”

https://tiangolo.medium.com/introducing-fastapi-fdc1206d453f (accessed Aug. 11, 2022).

[206] "FastAPI: The Modern Age API Framework For Pythonista | by Shritam Kumar Mund | Analytics Vidhya | Medium." https://medium.com/analytics-vidhya/fastapi-the-modern-age-api-framework-for-pythonista-4b2cd1e6652 (accessed Aug. 11, 2022).

[207] OpenAPI Institute, "GitHub - OAI/OpenAPI-Specification: The OpenAPI Specification Repository," 2020. https://github.com/OAI/OpenAPI-Specification (accessed Aug. 11, 2022).

[208] json-schema-org, "JSON Schema | The home of JSON Schema," 2020. https://json-schema.org/ (accessed Aug. 11, 2022).

[209] "Features - FastAPI." https://fastapi.tiangolo.com/features/ (accessed Aug. 11, 2022).

[210] Tonya Sims, "Vonage API Developer." https://developer.vonage.com/blog/21/08/10/the-ultimate-face-off-flask-vs-fastapi (accessed Aug. 11, 2022).

[211] "FastAPI vs Flask - The Complete Guide." https://christophergs.com/python/2021/06/16/python-flask-fastapi/ (accessed Aug. 11, 2022).

[212] "TechEmpower Framework Benchmarks." https://www.techempower.com/benchmarks/#section=test&runid=7464e520-0dc2-473d-bd34-dbdfd7e85911&hw=ph&test=query&l=zijzen-7 (accessed Aug. 11, 2022).

[213] "Best Frontend Frameworks for Web Development of 2021–2022 | SaM Solutions." https://www.sam-solutions.com/blog/best-frontend-framework/ (accessed Sep. 26, 2022).

[214] "10 Best Front end Frameworks for Web Development in 2022." https://www.monocubed.com/blog/best-front-end-frameworks/ (accessed Sep. 26, 2022).

[215] "15 Best Front end Frameworks for Web Development in 2022." https://www.knowledgehut.com/blog/web-development/front-end-development-frameworks (accessed Sep. 26, 2022).

[216] D. Karczewski, "What are the best frontend frameworks to use in 2021?," *Ideamotive*, 2022. https://www.ideamotive.co/blog/best-frontend-frameworks (accessed Sep. 26, 2022).

[217] “10 Best Frontend Frameworks in 2022 | Technostacks.” https://technostacks.com/blog/best-frontend-frameworks/ (accessed Sep. 26, 2022).

[218] “What is a Web Framework, and Why Should I use one? · We Learn Code.” https://welearncode.com/what-are-frontend-frameworks/ (accessed Sep. 26, 2022).

[219] “Stack Overflow Developer Survey 2022.” https://survey.stackoverflow.co/2022/#most-popular-technologies-language (accessed Sep. 26, 2022).

[220] “What is JavaScript? - Learn web development | MDN.” https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_is_JavaScript (accessed Sep. 26, 2022).

[221] “The State of JS 2021: Front-end Frameworks.” https://2021.stateofjs.com/en-US/libraries/front-end-frameworks/ (accessed May 03, 2022).

[222] “12 Best Javascript Frameworks In 2021 [Updated].” https://technostacks.com/blog/javascript-frameworks/ (accessed Sep. 26, 2022).

[223] “Top 6 Frontend Frameworks To Use in 2022.” https://www.telerik.com/blogs/top-6-frontend-frameworks-2022 (accessed Sep. 26, 2022).

[224] “React vs Angular - Which is Best For You in 2022?” https://technostacks.com/blog/react-vs-angular/ (accessed Sep. 26, 2022).

[225] “The Most Popular Front-end Frameworks in 2022 - Stack Diary.” https://stackdiary.com/front-end-frameworks/ (accessed May 03, 2022).

[226] “Using Vue with TypeScript | Vue.js.” https://vuejs.org/guide/typescript/overview.html (accessed Sep. 26, 2022).

[227] “Angular Usage Statistics.” https://trends.builtwith.com/framework/Angular (accessed Nov. 09, 2022).

[228] “Vue Usage Statistics.” https://trends.builtwith.com/javascript/Vue (accessed Nov. 09, 2022).

[229] “React Usage Statistics.” https://trends.builtwith.com/javascript/React (accessed Nov. 09, 2022).

[230] “Most used web frameworks among developers 2022 | Statista.” https://www.statista.com/statistics/1124699/worldwide-developer-survey-most-used-frameworks-web/ (accessed Nov. 10, 2022).

[231] “Stack Overflow Developer Survey 2022.”

https://survey.stackoverflow.co/2022/#section-most-loved-dreaded-and-wanted-web-frameworks-and-technologies (accessed Nov. 10, 2022).

[232] "2021 JavaScript Rising Stars." https://risingstars.js.org/2021/en#section-framework (accessed Jan. 16, 2023).

[233] M. T. Chung, N. Quang-Hung, M.-T. Nguyen, and N. Thoai, "Using Docker in high performance computing applications," in *2016 IEEE Sixth International Conference on Communications and Electronics (ICCE)*, Jul. 2016, pp. 52–57, doi: 10.1109/CCE.2016.7562612.

[234] Docker Inc., "What is a Container | Docker," *Docker.Com*. 2017, Accessed: Apr. 07, 2022. [Online]. Available: https://www.docker.com/resources/what-container/.

[235] A. Kropp and R. Torre, "Docker: Containerize your application," *Computing in Communication Networks: From Theory to Practice*. Elsevier Inc., pp. 231–244, 2020, doi: 10.1016/B978-0-12-820488-7.00026-8.

[236] "What is Docker | Oracle Portugal." https://www.oracle.com/pt/cloud-native/container-registry/what-is-docker/ (accessed Mar. 27, 2022).

[237] J. Fink, "Docker: A Software as a Service, Operating System-Level Virtualization Framework," *Code4Lib J.*, no. 25, [Online]. Available: https://journal.code4lib.org/articles/9669.

[238] Docker, "Why Docker? | Docker," *why-docker*. https://www.docker.com/why-docker/ (accessed Apr. 07, 2022).

[239] "What is Docker? | IBM." https://www.ibm.com/cloud/learn/docker (accessed Mar. 27, 2022).

[240] C. Anderson, "Docker [Software engineering]," *IEEE Softw.*, vol. 32, no. 3, pp. 102-c3, May 2015, doi: 10.1109/MS.2015.62.

[241] "Home - Docker." https://www.docker.com/ (accessed Mar. 27, 2022).

[242] "What is Docker? | AWS." https://aws.amazon.com/docker/ (accessed Mar. 27, 2022).

[243] "What is Docker? | Opensource.com." https://opensource.com/resources/what-docker (accessed Mar. 27, 2022).

[244] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Mar. 2015, pp. 171–172, doi: 10.1109/ISPASS.2015.7095802.

[245] Docker, “Install Docker Engine,” *Docker.com*, 2021. https://docs.docker.com/engine/ (accessed Apr. 07, 2022).

[246] A. M. Potdar, N. D G, S. Kengond, and M. M. Mulla, “Performance Evaluation of Docker Container and Virtual Machine,” *Procedia Comput. Sci.*, vol. 171, no. 2019, pp. 1419–1428, 2020, doi: 10.1016/j.procs.2020.04.152.

[247] “Docker overview | Docker Documentation.” https://docs.docker.com/get-started/overview/#docker-architecture (accessed Apr. 07, 2022).

[248] “Develop with Docker Engine API | Docker Documentation.” https://docs.docker.com/engine/api/ (accessed Apr. 10, 2022).

[249] “Docker Desktop for Linux (Beta) | Docker Documentation.” https://docs.docker.com/desktop/linux/ (accessed Apr. 10, 2022).

[250] “Docker Desktop overview | Docker Documentation.” https://docs.docker.com/desktop/ (accessed Apr. 10, 2022).

[251] “3 Reasons you need Portainer in your life.” https://www.portainer.io/blog/3-reasons-you-need-portainer-in-your-life (accessed Apr. 10, 2022).

[252] “Stack Overflow Developer Survey 2022.” https://survey.stackoverflow.co/2022/#section-most-popular-technologies-web-frameworks-and-technologies (accessed Nov. 10, 2022).

[253] “React, Angular, Vue.js - Explorar - Google Trends.” https://trends.google.com/trends/explore?date=today 5-y&geo=PT&q=%2Fm%2F012l1vxv,%2Fg%2F11c6w0ddw9,%2Fg%2F11c0vmgx5d (accessed Sep. 26, 2022).

[254] “GitHub Star History.” https://star-history.com/#facebook/react&vuejs/vue&angular/angular (accessed Nov. 09, 2022).

[255] “GitHub - angular/angular.js: AngularJS - HTML enhanced for web apps!” https://github.com/angular/angular.js (accessed Jan. 16, 2023).

[256] “GitHub - angular/angular: The modern web developer’s platform.” https://github.com/angular/angular (accessed Jan. 16, 2023).

[257] “GitHub - vuejs/vue: Vue.js is a progressive, incrementally-adoptable JavaScript framework for building UI on the web.” https://github.com/vuejs/vue (accessed Jan. 16, 2023).

[258] “GitHub - facebook/react: A declarative, efficient, and flexible JavaScript library for

building user interfaces." https://github.com/facebook/react (accessed Jan. 16, 2023).

[259] "angular vs react vs vue | npm trends." https://npmtrends.com/angular-vs-react-vs-vue (accessed Jan. 16, 2023).

[260] "npm-stat: angular, react, vue." https://npm-stat.com/charts.html?package=angular&package=react&package=vue&from=2018-12-30&to=2020-07-07 (accessed Jan. 16, 2023).

# 6 APPENDIX 1 - "TECHNOLOGIES AND CONCEPTS"

This chapter contains a detailed description of the technologies used to build the proposed research platform. It covers the essential concepts required to understand the technologies and justifications for the choices made. Generally, there was a preference for open-source, well-documented technologies.

First, Python is introduced as the most popular language in AI, and some critical libraries are discussed. Then, ML and DL are discussed, including a comparison between traditional and ML programming and a distinction between supervised and unsupervised ML. Lastly, the concepts of databases, web applications and frameworks, APIs, and software virtualization are explored.

## 6.1 MACHINE LEARNING DEVELOPMENT FRAMEWORK

An AI project differs in many ways from a regular programming project. Most of the differences lie in the skills required for this type of project, the technology, and the necessity of ML algorithms. Most AI projects today use Python because it is stable, flexible, and supports an extensive list of libraries and frameworks fundamental in development. Python's simplicity, consistency, and platform independence, when paired with a large and growing community, add to the overall popularity of this language in the AI field.

Python is appealing to numerous developers because it is easy to learn. The language's inherent simplicity liberates developers from the shackles of wrestling with intricate syntax, allowing them to channel their efforts into the core task of addressing ML challenges. Additionally, the availability of many frameworks, libraries, and extensions simplifies the implementation process, making it straightforward to bring solutions to life.

The deployment of ML algorithms is a nuanced endeavor, often demanding a meticulously structured and rigorously tested environment. It necessitates a considerable investment of time to arrive at optimal coding solutions. Python's rich ecosystem of frameworks and libraries plays a pivotal role in expediting development cycles. Notably, as reusable code repositories, software libraries offer solutions to recurrent programming conundrums. Python boasts an extensive library ecosystem tailored to the unique demands of AI and ML. Key players include Keras, TensorFlow, Scikit-learn, PyTorch, NumPy, SciPy, Pandas, and Seaborn.

Python's platform independence is another invaluable facet of its appeal. This attribute underscores the versatility of Python, allowing the same codebase to seamlessly function across diverse operating systems, encompassing Windows, Linux, and macOS, with minimal,

if any, modifications. Additionally, this quality reduces potential training costs since data scientists can harness their hardware resources, namely GPUs, for ML model training, thereby reducing the need for external computing services from entities like Google or Amazon.

In the 2020 Stack Overflow developer survey, Python was among the top five most popular programming languages, with 30% of developers not developing with the language expressing interest in developing with it [130]. Furthermore, 66.7% of developers develop with the language and expressed interest in continuing to develop with it [130]. In the 2021 developer survey, Python passed SQL, becoming the third most popular technology [131]. Ultimately, these surveys confirm Python's robust community, escalating popularity, and sustained growth.

### 6.1.1 JUPYTER NOTEBOOK

Jupyter Notebook is an open-source web application widely adopted for data science projects. Jupyter supports over forty programming languages, including Python. The Jupyter Notebook is a non-profit, open-source web-based environment for creating and sharing documents with code, equations, visualizations, and text. Uses include data science, scientific computing, computational journalism, ML, and more. Jupyter Notebooks support Python ML work and provide a great environment to develop code and communicate results.

A Jupyter Notebook provides an easy-to-use, interactive data science environment that works as an integrated development environment (IDE), presentation, and educational tool. Jupyter provides a way to work with Python inside a "virtual" notebook, combining code, images, plots, comments, and more. This flexibility aligns with the data science process, which explains the growing popularity of the tool with data scientists.

### 6.1.2 PYTHON LIBRARIES

This project benefits from various Python libraries. In the following paragraphs lies a brief description of some of the most relevant libraries:

**NumPy**

NumPy is an essential package for scientific computing in Python. It provides a multidimensional array object used to perform various mathematical operations. This package is one of the most used, and its popularity comes from its superior performance compared to

regular Python arrays. NumPy is compatible with and used by several popular Python packages, particularly Pandas and Matplotlib [132].

**Pandas**

Pandas is a Python package frequently used for data science, analysis, and ML. It offers fast, flexible, and expressive data structures geared towards working with "relational" or "labeled" data. This package takes data from, for example, CSV, TSV, or SQL databases and creates a Python object with rows and columns called "DataFrame" that are easy to manipulate [133].

**Matplotlib**

Matplotlib is a comprehensive package commonly used for data visualization and graphical plotting. It uses an object-oriented API that allows developers to embed plots in JupyterLab and Graphical User Interfaces (GUIs). This package enables the creation of visual plots with concise code [134].

**NiBabel**

NiBabel is a vital package that provides read and write access to an ever-growing collection of standard medical and neuroimaging file formats. Since each file format has features and peculiarities, NiBabel delivers high-level format-independent access to neuroimages and an API with diverse levels of format-specific access to available information [135].

**Pydub**

Pydub is a helpful package that provides audio manipulation with a simple and easy interface. In fact, with this library, it is possible to play, split, merge, and edit audio files [136].

**Librosa**

Librosa is one of many packages for music and audio analysis. It provides the building blocks necessary to create music information retrieval systems. This package provides the tools to visualize audio signals and extract features using several signal-processing techniques [137].

**TensorFlow**

TensorFlow is an open-source artificial intelligence library developed by Google for numerical computation and ML. It has an exceptional performance by using Python to provide a frontend API while executing in C++. Essentially, TensorFlow offers accessible model building. In other words, building and training ML models become simple with intuitive high-level APIs such as Keras, which make for quick model iteration and easy debugging.

TensorFlow can be used for Classification, Perception, Understanding, Discovering, Prediction, and Creation [138].

**Keras**

Keras is a high-level deep learning API written in Python for ML development. Keras is a free, open-source Python library that provides an easy-to-use interface to TensorFlow. This library provides the necessary abstraction and reduces cognitive overload, empowering researchers to design and build ML models faster, which is crucial to doing good research [139], [140].

**XGBoost**

XGBoost or Extreme Gradient Boosting is an optimized distributed gradient boosting package devised to be highly efficient, flexible, and portable. It implements ML algorithms under the Gradient Boosting framework, which is popular for structured or tabular data. XGBoost delivers a parallel tree boosting (also known as GBDT, GBM) that solves many data science problems quickly and accurately [141].

**Logging**

Logging is a crucial Python package that provides a functional and flexible framework for issuing log messages from Python programs. Since it is a standard library module, all Python modules can participate in logging so that the application log will include the messages defined plus all the messages from third-party modules used [142].

## 6.2 MACHINE LEARNING

ML is an application of AI that confers computers with the capacity for autonomous learning, adaptation, and the attainment of instructional autonomy. A system with ML can enhance and refine its performance based on inputs, creating more adaptable and versatile programs. Typically, a human provides a continuous influx of quality data to guarantee the algorithms adjust and optimize the models, which leads to automatic programming [143]–[145].

Unlike traditional programming, ML operates by harnessing the capabilities of data itself to accomplish tasks, rendering it an increasingly vital component in contemporary lifestyles and business practices. ML can lessen the workload by empowering devices to learn how to do jobs quickly and efficiently instead of writing programs for all potential functions a machine may need to perform. In traditional programming, a human must perform several tasks to make a computer complete an activity. However, a new program is needed for new tasks to be completed. New activities will require a new comprehensive list of rules, leaving an opening for errors that will disturb operations. Focusing on ML to accomplish a similar result would be quicker and have fewer chances of developing errors. Additionally, the traditional update

method never leaves the feeling that the job is complete. Nonetheless, there will always be the need to update and check compatibility with the latest technologies [143], [144].

The learning process derived from human biology is currently being applied to machines through complex systems of neural networks and algorithms that work in unison to provide the correct solutions. This indicates that computers are learning to think like humans, looking at data and comparing it to previous data to find patterns, adapt, and refine results. It starts with observations or data to look for patterns and make finer future decisions based on the examples provided. The main objective is to allow computers to learn without assistance or intervention [143], [146].

It is critical to input suitable data and algorithms and clearly define the corresponding analysis rules for pattern recognition in a data inventory. These preparations empower ML to implement multiple functions. It can employ identified patterns to optimize processes, use data analysis to make predictions, and independently adapt to developments. Moreover, it can also find, extract, and summarize essential data in addition to calculating the probabilities for specific results. The computer scientist consistently provides the software with feedback, and then the algorithms utilize the resulting signals to modify and refine the model. The more data is provided, the better optimized the model becomes, and the better the distinctions between the objects of study. For example, a scientist inputs data informing the computer that a specific item is a healthy brain while another is not. Continually feeding the software and constantly updating data input optimizes the model until it can differentiate correctly between healthy and unhealthy brains [144].

ML can promote accurate diagnoses and predictions in the medical field. This kind of learning helps medical professionals identify high-risk patients, make better diagnoses, and select the best medicines for each case while mainly remaining based on anonymous datasets of patient records. Hence, ML can help doctors and other medical professionals do their jobs more efficiently [146].

ML projects frequently follow a workflow. There are many descriptions of what phases such a workflow should include since there is always room for improvement. The diagram below (Figure 49) gives a high-level overview of the three main steps: Data Engineering, Model Engineering, and Model Deployment [147].

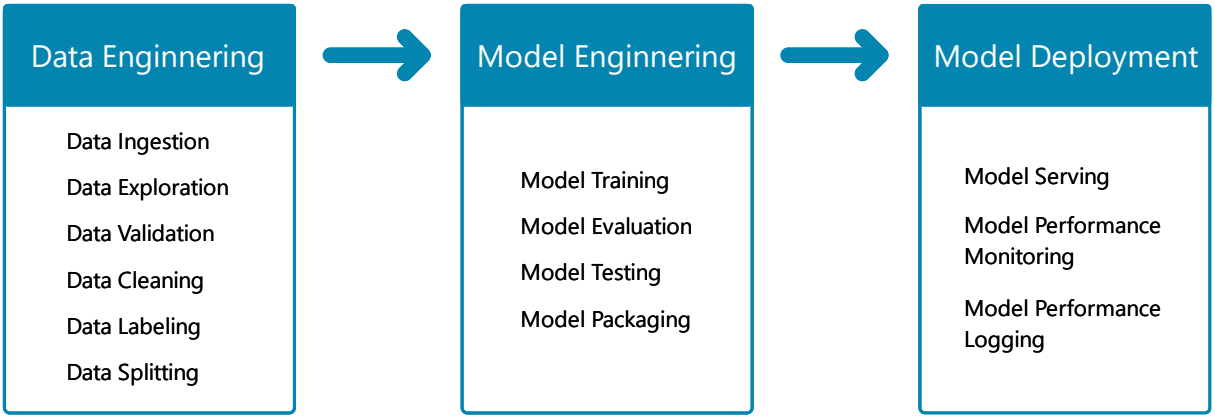



Figure 49 - Machine Learning Workflow adapted from [147].

The following is a brief description of each step.

**Data Engineering**

First, data is collected and prepared to be analyzed. This stage is critical to the workflow because the quantity and quality of the data will determine the quality of the ML model obtained.

**Model Engineering**

The ML model obtained in this phase heavily depends on the first phase's quality since errors can propagate between steps. Model engineering constitutes the core of the ML workflow, which includes writing and executing ML algorithms to obtain an ML model. This phase contains training, evaluation, testing, and packaging of the best ML model into a specific format.

**Model Deployment**

The last phase of the ML workflow integrates the model engineered in the previous stage into an application, such as a web application.

### 6.2.1 TERMS AND DEFINITIONS

**Algorithm**

An Algorithm is a method, function, or set of assumptions that a machine applies to input to produce output. Algorithms are equivalent to program instructions and are used to generate machine-learning models. In the training stage, particular algorithms must be followed to get specific outcomes. Examples include linear regression, neural networks, decision trees, and support vector machines [144], [148].

**Feature**

A Feature in ML is equal to the input variables of a program. It is a quantifiable property of the measured phenomenon, generally a numeric value. Regarding a dataset, a feature combines an attribute and a value. For example, length is an attribute, and "length is 10 cm" is a feature [144], [148], [149].

**Label**

Labels are the opposite of features. The final output or "result" from machine learning is called label data in supervised learning. For example, in a dataset used to classify brain health, the features might include symmetry and perimeter, while the label would probably be either "brain healthy" or "brain not healthy." [144], [148], [150]

**Model**

A Model is a data structure that stores the real-world interpretation in a mathematical format that the machine can analyze (weights and biases). Models represent what the ML system has learned from the training data. Essentially, a model is a product of training an algorithm on a dataset. Therefore, the performance of a model is heavily dependent on the quality of the training data [144], [148], [150].

**Classification**

Classification involves grouping data into predefined classes based on relevant factors [144]. In binary classification, the task is to predict one of two outcomes [149], such as distinguishing between "normal" and "abnormal" medical images. Multiclass classification deals with predicting among multiple categories [148], like classifying skin lesions into "benign nevus," "melanoma," or "dermatofibroma." Classification enables the prediction of categorical outputs and plays a crucial role in various domains.

**Deep Learning**

Deep Learning is an ML algorithm that has gained popularity for its success in various domains, such as computer vision, signal processing, and medical diagnosis. It is a powerful approach based on artificial neural networks, known for its superior accuracy and robustness compared to traditional methods. With the availability of large datasets and advancements in computing power, deep learning has achieved state-of-the-art performance and is widely used in modern applications [148].

**GPU**

Graphics Processing Units (GPUs) are specialized processors that neural networks and ML systems use. They offer significantly more processing power than CPUs, with hundreds or

thousands of processors per chip. GPUs excel in performing arithmetic operations on vectors, enabling them to carry out tasks in parallel and deliver faster computations compared to CPUs [144].

## 6.3 DATABASE AND DATABASE MANAGEMENT SYSTEMS

The foundation lies within data, and in the present era, one can confidently affirm that humanity resides in an age defined by information. To an unprecedented extent, individuals are constantly inundated with a ceaseless influx of data. However, amassing an immense amount of data due to the increase in new data types to monitor does not translate to value. It is the extraction of information hidden in the data that brings value [151], [152].

Even though the terms "data" and "information" are similar, it is significant to understand the distinction between them in this context. Raw data holds limited intrinsic value in its unprocessed form, comprising unorganized and challenging-to-interpret facts. Data refers to the organized and refined version of raw data, particularly what is stored in a database. Conversely, "information" denotes the meaningful and valuable interpretation of data as perceived by the user. In essence, information represents organized data that is useful to individuals [151], [153].

The following section will revolve around database and database management systems (DBMS). On the one hand, a database is where data is stored. On the other hand, the DBMS includes components that work alongside the operating system to manipulate data, making the database a viable storage and retrieval tool [151], [152].

### 6.3.1 DATABASE SYSTEM OR DATABASE SYSTEM ENVIRONMENT

A database system is a computerized archive system that aims to enable users to retrieve and update information on demand. This information can be anything relevant that assists the processes that run in a specific context. Figure 50 illustrates a simplified database system. As the figure depicts, such a system comprises four fundamental components: data, hardware, software, and users [153].

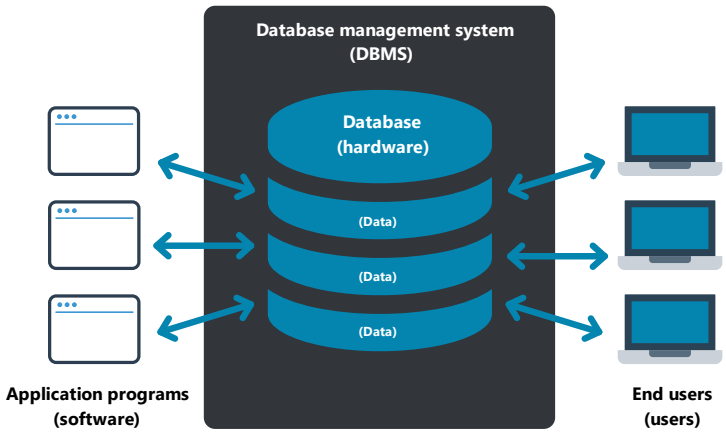



Figure 50 - Simplified database management system.

**Data**

Database systems are available on all kinds of machines, from smartphones to laptops. However, each system's features rely on the underlying device's size and power. Generally, bigger and more powerful machines are multi-user systems, and smaller devices are single-user systems. This distinction is irrelevant to users because one of the main objectives of multi-user systems is to behave towards users in the same way a single-user system does. In practice, even in small systems, there might be good reasons to split data across different databases. For the sake of simplicity, the assumption is made that the entirety of system data resides in a single database [153].

Typically, the data in a database will be integrated and shared. Data integration and data sharing portray significant benefits of database systems. "Integration" refers to the database's role in reducing or eliminating redundancy by unifying different files. "Sharing" signifies the capacity of the database to be accessible to diverse users, enabling multiple users to access identical data, possibly even simultaneously [153].

An additional consequence of an integrated and shared database is that a user will generally be interested in a small piece of a database. Furthermore, different users may try to access portions of data that overlap. Simply put, users perceive a database in a variety of different ways [153].

**Hardware**

There are two types of hardware. First, a database resides in secondary storage volumes. Secondary storage may include hard disk drives (HDDs) and solid-state drives (SSDs). These devices hold the stored data. Second, the hardware processor(s) and associated main

memory, also known as random access memory (RAM), support the execution of the database system software [152], [153].

The problems that arise from hardware are not specific to databases. Consequently, exploring them in this context is out of scope [153].

**Software**

There is a layer of software between the physical database and the users. This layer is commonly known as a database management system (DBMS) but can also be referred to as a database manager or database server.

The DBMS is an interface that handles all requests for access to the database. It also provides the facilities to access, create, update, and delete data/files/tables. Essentially, a DBMS shields database users from hardware-level details.

Thus, elevating the perception above the hardware level and consequently facilitating user operations such as SQL operations [152], [153].

Although the DBMS is an essential software in the overall system, it is not the only one. Others include utilities, application development tools, and, most importantly, the transaction manager or TP monitor [153].

**Users**

Users are the people who interact with a database system. Users fit into several classes depending on the range of interaction and level of competence. There are three types of users [152], [153].

First, application programmers or software developers are capable of writing database application programs in various programming languages, such as Assembly, COBOL, C++, Java, or some higher-level language. These programs access the database by making the appropriate request corresponding to an SQL statement sent to the DBMS. Such programs or user interfaces empower end-users to access the database interactively. Users frequently access these programs through online applications [152], [153].

Next, the end-users can access the database with the online applications mentioned above or through the interface provided as an integral part of the system. Such interfaces are built-in and can also be supported by online applications. Most systems provide no less than one built-in application, a designated query language processor, that enables the user to interactively make database requests such as SELECT and UPDATE to the DBMS. An example of a database query language is SQL. Systems frequently provide additional built-in interfaces, such as

menus and forms, that allow end-users to interact with the database without making explicit requests. Menu-driven and form-driven interfaces are generally easier for people with no formal training. Conversely, command-driven interfaces that use query languages are more flexible and include more features but require a higher level of mastery [153].

Thirdly, the database administrator or DBA is the technical person responsible for creating the database. The DBA must execute system operations with acceptable performance while providing various other technical services [152], [153].

A database is a collection of organized data stored in a computer system. Such data, when interpreted by people, becomes information. Managing data with increasing volume and access demand is a challenge. Hence, software, faster hardware, and specialized personnel are crucial to help answer these requirements. The software that controls the database is the database management system (DBMS). In conjunction with other tools, the DBMS is used by specialized personnel such as the database manager (DBA). Hardware has progressed to be faster and more affordable when considering its performance level.

In conclusion, all these elements add up to much more than the sum of their parts. The amalgamation of data, software, hardware, and users culminates in what is commonly referred to as the database system or database environment, often abbreviated as "database" [151], [154].

### 6.3.2 DATABASE

#### *6.3.2.1 DEFINITION*

There are many definitions for the term database. The following descriptions are shared to bring some clarity to this complex concept. Firstly, a database is an assortment of persistent data wielded by the application systems of an enterprise [153]. Secondly, a database is an ordered collection of interrelated data stored with controlled redundancy, intended to meet the information needs of an enterprise, and designed to be shared by multiple users [151], [152]. Thirdly, a database is an ordered assortment of structured information, or data, usually stored in a computer system [153], [154]. Finally, a database can also be defined as an electronic filing or record-keeping system [152], [153].

To truly understand these definitions, two more concepts need to be developed. Those concepts are persistent data and enterprise. The data inside a database is persistent. On the contrary, other types of data are more ephemeral, such as input data, output data, SQL

statements, intermediate results, and, in a broader sense, any transient data. Data is persistent in a database since once the DBMS stores it, it cannot be deleted unless there are specific instructions to do so [153]. Lastly, the term "enterprise" is the generic word used in this context to represent commercial, scientific, technical, or other organizations. An enterprise may range from a single individual to a complete corporation such as a hospital or a university [153].

#### *6.3.2.2 BASIC CONCEPTS*

The following concepts help better understand databases:

**Data Repository**

A data repository contains all the data in a database. The data repository consists of a storage unit where the physical data files are kept and the central storage location for the data content. The physical design of the data repository varies according to each database [151].

**Data Dictionary**

A data dictionary contains metadata or information about the structure of the database. This information describes field names, sizes, data types, constraints, applications, authorization, and so forth. On the other end of this structural information lays the actual data values stored in the fields. These data values reside in the data repository.

A data dictionary comprises data item structures and the relationships between data items. The data dictionary and the data repository act coherently to supply information to users. A DBMS frequently consults the data dictionary to perform accurate database operations [151], [152].

**Database Software**

Products like SQL Server, MySQL, PostgreSQL, and equivalent products are occasionally called databases. However, they are specialized software that manage data and databases. These products are database software or database management systems (DBMS), not the actual database. Their function provides crucial database actions like storing, retrieving, and updating data in the database [151].

**Data Access**

All database management systems provide a set of essential operations to interact with data. These operations are:

- READ data present in the database.

- ADD data to the database.
- UPDATE data fields in the database.
- DELETE chunks of data in the database.

Database designers and programmers commonly refer to these operations by the abbreviation CRUD:

- C: create or add data.
- R: read data.
- U: update data.
- D: delete data [151].

**Transaction Support**

A set of statements executed as a group is called a transaction. A transaction, as well as a set of steps, are illustrated in Figure 51.

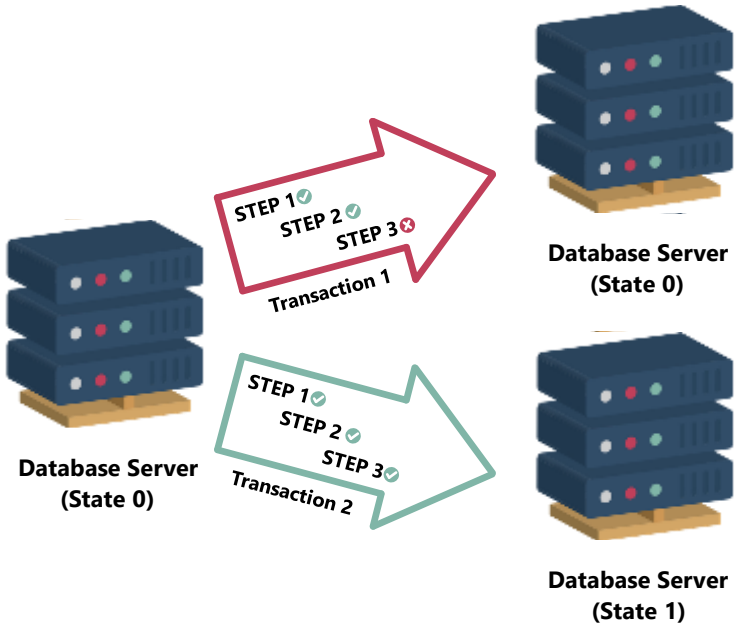



Figure 51 - Database transaction.

A successful transaction depends on every statement that makes up the transaction and leaves the database in a consistent state (State 1 in Figure 51). In other words, it only succeeds if every step runs without errors (Transaction 2 in Figure 51). If one of the steps fails, the transaction fails.

Successful steps in a failed transaction must rollback to leave the database in the same state as before the transaction attempt (State 0 in Figure 51). This action restores the balance in the database when errors occur during transaction execution. This type of all-or-nothing transaction is known as an atomic transaction [151].

**Database Use**

There are various types of databases with different uses. However, considering the project at hand, the focus is on production databases. This type of database supports the core operational systems of an enterprise. Some of the uses range from online transaction processing to CRUD transactions. Lastly, the following features are characteristic of this type of database: concurrency, transaction processing, and security [151].

#### *6.3.2.3 DATABASE MODELS*

A data model is a theoretical, independent, rational definition of objects and operators that collectively establish the theoretical machine with which the user interacts.

Firstly, objects facilitate the modeling of data structure. Secondly, operators enable the modeling of data behavior. This data model definition points to a valuable distinction between model and implementation [153].

The implementation of a data model is a physical realization on a tangible machine of the elements of the theoretical machine that combined form that model. This refers to the fundamental distinction between logical and physical [153].

Database models are classified by the data structures and operators they provide to the user [153]. The primary data models are:

- Hierarchical database model.
- Network database model.
- Relational database model.
- Object-oriented database model.

There are differences between relational and nonrelational systems. In a relational model, the data is presented in tables. However, in a nonrelational system, other data structures, like trees, may be used independently or in addition to tables. Moreover, nonrelational models, due to their nature, include pointers. Then again, pointers are a major distinguishing factor since relational systems do not involve pointers (visible to the user; the physical implementation might require pointers) [151].

Attention is directed toward relational databases and hybrid databases. The first are databases that prescribe to the relational model. The latter are databases founded on combining different models into one database. The relational database is applied to everything from simple desktop storage to enterprise-level applications. At the same time, the

object-oriented method proves to be valuable in specific contexts. The importance of the object-oriented approach stems from the implementation of some of its concepts into relational systems generating hybrid object/relational databases [151].

**Relational Database Model**

In contrast to hierarchical and network models, the relational database model does not use physical pointers to link related data. This model sets up the relations between related data with logical connections carried out by foreign keys. A foreign key establishes and sustains relationships using logical pointers.

The relational model is based on two-dimensional tables composed of rows and columns. A table is a central object in a relational database, and each table represents an individual entity. All entities are depicted by their attributes, the trackable and recordable information that makes up such an entity. Columns contain the attribute data, while rows represent a unique instance of an entity. Primary keys impose this uniqueness on a key column or set of key columns. Which, logically, enforces table uniqueness.

Additionally, the primary key also defines relationships. When a foreign key is created in a referencing table, it must point to a column with the same content in the referenced table. A foreign key can point to a primary key in the referenced table or a column where every attribute is unique.

A relationship is an association between two data entities. A foreign key field is inserted in the data structure to define a relationship. The foreign key enforces and sustains the relationship, replacing the physical pointers used in hierarchical and network models [151].

**Object-relational Database Model**

The leading database system applications, notably Oracle and Microsoft SQL Server, follow an object-relational database model. This was not always the case; historically, they are founded on the relational database system but have come to merge multiple features from the object database model in particular data types with increased flexibility and the ability to customize data types and functions. The object-relational model compounds the capabilities of object technology to manage advanced data types with data integrity, reliability, and recovery, characteristic of the relational model [151].

Ultimately, database systems can be predicated on several diverse methods. Relational systems are founded on a formal theory called the relational model, in which data is bound to rows in tables, and operators are given to assist in the procedure of extracting new knowledge

from existing data. From both an economic and theoretical viewpoint, relational systems are the primary option [153].

### 6.3.3 QUERY LANGUAGES

#### *6.3.3.1 QUERY PROCESSOR*

A DBMS is a group of specialized software modules; one of these modules is the query processor. The query processor or query optimizer is behind the following crucial actions: parsing, optimizing, and compiling queries for execution. SQL statements are employed to produce these queries [151].

#### *6.3.3.2 QUERY LANGUAGE*

Structured query language or SQL is the standard relational system language endorsed by most DBMS available today. It was initially designed by IBM Research in the early 1970s and has since evolved by being implemented repeatedly in various commercial products. Each relational database product improves upon the SQL standard command set and adds new features. The fact remains that the essential SQL features are present in every DBMS. However, there needs to be clarity as to what is part of the SQL language since DBMS manufacturers keep making new variations by adding new features. Some manufacturers that have produced versions of SQL are DB2, Oracle, MySQL, Informix, and Microsoft SQL Server [151], [153].

**Data Definition and Data Manipulation**

SQL includes data management operations comprised of data definition and data manipulation. Data definition needs a data definition language (DDL) that informs the DBMS of what tables exist in the database, what attributes are contained in a table, which attributes are indexed, and so forth. SQL includes four basic manipulative operations executed on data stored in a DBMS: data retrieval, data update, insertion of new records, and deletion of existing records. These commands are transmitted from users to a DBMS through a specialized data manipulation language (DML). SQL is recognized as the standard by the American National Standards Institute (ANSI) and the International Organization for Standardization (ISO). In addition, it is also a Federal Information Processing Standard [151]–[153].

**SQL Features and Characteristics**

The SQL standard specifies the features supported by “standard” SQL implementations.

As previously mentioned, most manufacturers extend upon the standard features by providing added functionalities. These extensions generally consist of new commands and command options. Additionally, the implementation of SQL features needs to be better defined since the standard may vary from one DBMS solution to another [151].

The following are the categories upon which the basic SQL features fall [151]:

- Data definition language (DDL): statements employed to produce and sustain database objects.
- Data manipulation language (DML): statements used to recover and manage data.
- Command operators: operator symbols and keywords manipulated to handle arithmetic, comparison, and logical operations.
- Functions: specialized executables that produce values.
- Transaction control: possess statements operated to start and finalize or abort transaction processing.

With this in mind, it is common to consider some things part of SQL when, in reality, they are just commonly available in relational database system implementations. When considered valuable, these features or commands are introduced in a DBMS solution and naturally propagate to most other systems [151].

The following statements describe relevant SQL characteristics [152]:

- SQL is incredibly adaptable.
- SQL utilizes free-form syntax that allows the user to structure SQL suitably.
- SQL is a free-formatted language, which means the positioning of SQL statements in a specific column or the need to be completed in a single line is not applicable.
- SQL has a reasonably moderate number of commands.

SQL is a non-procedural language. In other words, SQL statements transmit to the DBMS the end goal without detailing how to achieve it.

**SQL Advantages**

Some advantages of using SQL are [152]:

- SQL is a high-level language that delivers higher abstraction than procedural languages. It is only necessary to define what is needed without detailing how to perform it.
- SQL is a unified language. A single syntax can define data structures, queries, access control, inserts, deletes, and updates.

- Programs written in SQL are portable. These programs can be moved from one database to another with just a few modifications. This ability comes in handy when the DBMS is upgraded or replaced.
- The syntax is straightforward and effortless to learn. It can manage tricky problems remarkably efficiently.
- The language has a solid theoretical basis with no ambiguity regarding how a query is interpreted. Thus, the results are well defined and according to expected.
- SQL is independent of implementation since it defines what is required but not how it should be done.
- SQL is transversal across many DBMS systems. This means users who learn SQL can interact with most of the DBMS products commercially available.

### 6.3.4 SQL ALCHEMY

There are two levels of approach when connecting a Python application to a database: low-level and high-level. The low-level strategy involves installing a relational database management system and writing SQL commands for database operations. Conversely, the high-level method resorts to an object-relational mapper (ORM) [155].

An ORM is a database abstraction layer between the developer and the database engine [155]. It allows a developer to write Python code that translates into low-level SQL instructions to read, create, update, and delete data and schemas in a relational database [155], [156]. ORMs have tools to automate data transfer from database tables into objects used in the code [157]. Writing only Python code can speed up development since switching between Python and SQL is no longer required. Additionally, ORMs allow developers to write database-agnostic code that makes it possible to switch between various databases [156], [158]. There are numerous ORM implementations in Python, but SQL Alchemy seems most preferred amongst software engineers [158].

SQL Alchemy is an open-source Python database toolkit and an ORM with a mature and high-performing architecture [159], first introduced in 2005 and written by Michael Bayer [160]. SQL Alchemy is a well-regarded [161] and commonly used [158] toolkit that gives developers the full power and flexibility of SQL [162]. Many organizations in various fields use this toolkit, for example, Yelp, Reddit, and Dropbox [162]. Many considered it the standard for working with relational databases in Python since it leads to more straightforward, cleaner, and faster

code writing [155], [160]. SQL Alchemy includes dialects for PostgreSQL, SQLite, MySQL, Oracle, and Microsoft SQL Server [163].

SQL Alchemy is unique to other SQL and ORM tools because it has a complementary approach. It does not hide SQL and object-relational details behind the guise of automation. All processes are exposed to ensure the developer fully controls the database and the SQL commands. Unlike many other database libraries, it provides a generalized API for writing database-agnostic code without SQL, known as "Core", separate from the ORM [159].

Other key features include forming complex SQL queries, object mapping, and the "unit of work" pattern [160]. Firstly, the Unit of Work system is central to the ORM. This system is responsible for organizing pending insert, update, and delete operations into queues flushed in batches to deliver maximum security and transaction safety and minimize deadlocks [159]. Secondly, the query construction is function-based, allowing Python functions and expressions to build SQL statements. Thirdly, other benefits of using SQL Alchemy are summarized below:

- The database schema can be defined using a Python model.
- The add-on Alembic synchronizes the Python model with the database schema (convenient for the development phase).
- Application and database code in the same language, specifically Python, makes it easier to read.
- Seamless integration with web frameworks such as Fast API [164].

Lastly, SQL Alchemy has good documentation that includes thorough tutorials and a comprehensive API reference [164].

PROS:

- Enterprise-level APIs that makes the code robust and adaptable.
- Flexible design that supports complex queries.

CONS:

- Uncommon unit of work concept.
- Heavyweight API entails a steep learning curve [165].

In summary, this toolkit empowers developers to write Python code that maps the database schema to the Python objects used in the application. Although SQL is not required, raw SQL queries are still supported. Without using SQL, developers can create, maintain, and query the database. The mapping of Python objects to the database frees developers to focus on these

objects instead of writing the code required to manipulate data in and out of relational tables [161].

### 6.3.5 POSTGRESQL

As previously mentioned, SQL Alchemy provides a toolkit that makes writing database-agnostic code possible. Accordingly, most databases supported by SQL Alchemy would answer the simple needs of the research platform to be proposed. MySQL and PostgreSQL implementations can be consulted in the GitHub repositories[11] shared. Ultimately, the recommendation for PostgreSQL is based on the reasons outlined in the subsequent section.

The recommended DBMS (database management system) is PostgreSQL. PostgreSQL is a powerful enterprise-class open-source ORDBMS (Object Relational Database System). PostgreSQL, commonly referred to as Postgres [166], [167], is a database system with over thirty years of diligent development that has earned it a strong reputation for its proven architecture, reliability, flexibility, performance, feature robustness, and support of open technical standards. The origins of this DBMS data back to 1986, as it is an open-source descendant of the POSTGRES project at the University of California at Berkeley led by Michael Stonebraker [168], [169]. PostgreSQL has become an enterprise-level product that is a realistic alternative to commercial databases [170].

PostgreSQL continues to evolve, maintained by a worldwide team devoted to regularly improving this free and open-source database project [166]. Some of the key benefits of using this database management system are:

**Reliability and Stability**

PostgreSQL provides support for hot standby servers, point-in-time recovery, and various types of replications (synchronous, asynchronous, cascade) to ensure enterprise-level reliability [170]. One of the distinguishing features is PITR (Point-In-Time-Recovery), which essentially makes it easy to restore file systems to a stable starting point [166].

**Security**

PostgreSQL supports secure SSL connections and provides various authentication methods [170].

[11] https://github.com/MarcelodeFreitas/FMdeploy-FastAPI/tree/latest
https://github.com/MarcelodeFreitas/FMdeploy-ReactJS
https://github.com/MarcelodeFreitas/fmdeploy

**Scalability**

PostgreSQL has high scalability in the quantity of data to manage and the number of concurrent users to support [168].

**Performance**

PostgreSQL supports performance optimizations typically found in proprietary database systems, such as unrestricted concurrency [166]. Concerning hardware, it takes advantage of modern multicore processor architecture to achieve a performance that grows almost linearly with the number of cores [170].

**Extensibility**

PostgreSQL architecture supports extensions that provide additional functionality without changing the core system code [170].

**Transaction Support**

Postgres is ACID-compliant (Atomic, Consistent, Isolated, Durable) and supports transaction isolation using another of its distinguishing features, the MVCC (multi-version concurrency control method). This method avoids the lockout that occurs in traditional database systems to avoid read/write conflicts. The MVCC method enables efficient concurrency where reads don't block writes and vice versa [166], [170].

**Deep Language Support**

PostgreSQL is compatible and supports multiple programming languages, such as Python, JavaScript, C/C++, and Ruby [166].

**Cross-platform support**

PostgreSQL supports both Unix and Windows operating systems. Its portable open-source C code allows building PostgreSQL in the platform desired [170].

**Availability and Independence**

PostgreSQL is a 100% open-source database management technology with a liberal license that allows unlimited use, code modification, and integration with other products. The investment in PostgreSQL is safe since it does not belong to any company and is developed by the international community [166], [170].

## 6.4 WEB APPLICATIONS

A web application, commonly known as a web app, is a subset of web development distinct from websites and mobile applications. Web application development focuses on

functionality and interactivity by providing customizable user interfaces (UI) and seamless user experience (UX). Despite the differences between traditional websites and web apps, the untrained eye may not tell them apart since both are accessible by typing a URL in a browser connected to the internet [171], [172].

Web applications are interactive, whereas websites present information (Figure 52). Unlike traditional websites that primarily function as a read-only interface, web applications highlight user interaction. The user interaction with a website is limited to a few exchanges to present more information and the occasional newsletter signup. On the other hand, web app development involves a more comprehensive scope of possibilities. This development demands an understanding of web application architecture, user experience, and authentication.

The following is a summary of the differences between websites and web apps. As shown in Figure 52, a web application typically has a different layout and functionality compared to a traditional website.

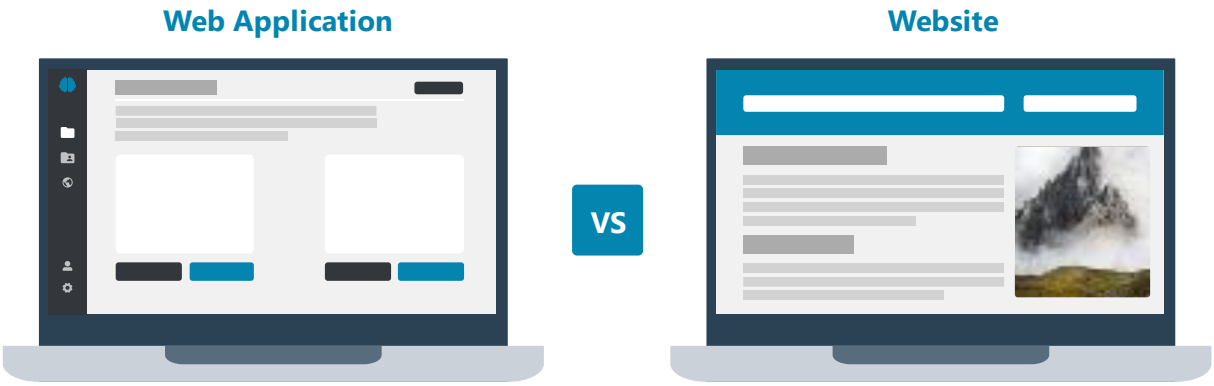



Figure 52 - Web application vs. website.

**Websites**

Websites are navigable and display public information such as digital content, documents, videos, images, and audio. Websites are static, meaning they do not update dynamically. Websites are read-only feeds that do not allow users to interact or communicate back to the site. Most websites use HTML, CSS, and sometimes JavaScript [173], [174]. If no user interaction results in a response, it is a website and not a web application [175].

**Web applications**

Web apps have functionality and interactive components that may be restricted to registered users. Web apps are dynamic, extremely customizable, built for user engagement, and

perform a wide range of functionality. A distinguishing characteristic is the ability to reference, store, and access data according to user needs through customizable interfaces that simplify information delivery. Forms are one example of how users may manipulate data. Generally, web apps are more complex and, therefore, more challenging to develop when compared to websites [173], [174]. Most web applications use additional languages besides HTML. These languages are CSS and JavaScript, which are heavily relied on to provide dynamic abilities, for example, search bars, forms, drag-and-drop documents, and many more features [175]. Table 3 summarizes some advantages and disadvantages of web applications [171], [172], [174], [176]–[181].

Table 3 - Advantages and Disadvantages of Web Applications

| **Advantages** | **Disadvantages** |
|---|---|
| Multiplatform, accessible from any device and operating system. The minimum requirement is a web browser such as Chrome, Firefox, or Safari and an internet connection. | Not as fast as an application hosted on a local server. |
| No download and installation, taking up less space in the device. | Require an internet connection. |
| Communication through HTTPS requests and securely storing data on a server. | Can't use the hardware and OS of each specific device as effectively as a native application. |
| Easy to maintain and update. Updates are applied centrally so all users access the same version, eliminating compatibility issues. | Larger web apps perform slower than native desktop applications. |
| Minimal technical requirements from the user's device. | While most browsers are supported due to an increasing number of web browsers, some may not work with specific web applications. |
| Extremely customizable. It is easy to create new designs and features according to user needs. | Not available in the App Store or Play Store. They can only be accessed by searching a URL in a browser. |
| To increase processor capacity, only the server needs to be upgraded. Additionally, web apps can run on multiple servers simultaneously. When the workload increases, new servers can be added. | |

### 6.4.1 SINGLE PAGE APPLICATIONS

A Single-page application (SPA) is a dynamic web app that loads a single web document and then updates the content using JavaScript [182]. SPAs deliver an exceptional UX with a single web page that does not require reloads [183]. When users interact with SPAs, only a few elements are dynamically updated with data from the web server, while the rest of the components remain unchanged [184]. These characteristics increase performance and create a dynamic user experience similar to a native environment [182]. Single-page applications are a cross-platform, user-centric solution preferred for their speed and relatively simple development, high security, fast debugging, testing, and deployment [184]. Table 4 summarizes some advantages and disadvantages of SPAs [183], [185].

Table 4 - Advantages and Disadvantages of Single-page applications

| Advantages | Disadvantages |
|---|---|
| Fast, most resources load once, and only data is exchanged. | JavaScript needs to be present and enabled unless rendering occurs on the server. |
| Reduced load times. | Susceptible to Cross-Site Scripting attacks (XSS). |
| Simplified development and easy to start coding locally without needing a server. | JavaScript memory leaks can degrade performance. |
| Easy debugging. | |
| The same backend code can be used to develop a mobile application. | |
| Effective caching of local storage. | |

## 6.5 WEB FRAMEWORKS AND OTHER TECHNOLOGIES

The development of a full-stack web application involves various frameworks and tools. A full-stack application comprises a frontend focused on the interface with users and a backend invisible to users that drives all the application logic. The technologies described below are essential to creating a web app [171].

### 6.5.1 BACKEND

The backend development features a wide range of technologies consistent with the number of components involved in server-side programming. The backend includes web servers, APIs,

and databases, which create the application's core infrastructure. In essence, the backend defines how the application will work [171], [172].

Firstly, the language used for backend programming must handle the functionality and empower the application to perform as intended. Some popular server-side languages include Python, Java, Ruby, and PHP. Python is a favored option when machine learning is involved. Some Python frameworks are Django, Flask, and FastAPI.

Secondly, databases store application data. As mentioned in a previous chapter, working with databases involves using SQL directly or ORMs to request and retrieve data using queries.

Thirdly, servers answer requests from clients. These requests, made using the internet connection, instruct the server on what to answer the client [171]. Some examples of web servers include Nginx [186], Uvicorn [187], and Gunicorn [188].

Lastly, the communication between the client and server is facilitated by an API.

### 6.5.2 FRONTEND

The frontend development defines the look and feel of the application, in other words, the user experience.

JavaScript is at the core of client-side web development. This programming language is the most popular and it allows developers to build dynamic websites. JavaScript is responsible for scrolling bars, clickable buttons, and much more. Since its conception, this programming language and its frameworks have kept evolving and improving [171].

Frameworks serve as a foundation or starting point that eliminates starting from scratch and provides tools to facilitate development. The most popular web frameworks are React, Vue, and Angular.

Besides JavaScript, there are two other core technologies of web development: HTML and CSS. HTML is responsible for the structure, and CSS is responsible for the style of a web page.

Hypertext Markup Language (HTML) is the most fundamental building block on the web. It defines the meaning and structure of the content. Cascading Style Scripts (CSS) is a language used to describe appearance and style [189]. CSS invokes essential characteristics like colors, layouts, and fonts.

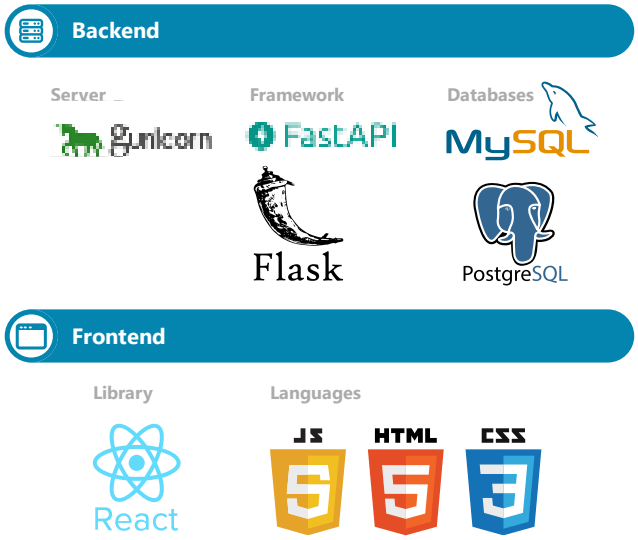



Figure 53 - Backend and frontend technologies.

Web app development is a subset of web development that uses a combination of frontend and backend technologies. Figure 53 represents an overview of common technologies in web application development considered for the creation of fmdeploy.

## 6.6 APPLICATION PROGRAMMING INTERFACE

An acronym for Application Programming Interface is API. This term refers to tools, definitions, and protocols for building, communicating, and integrating application software and services [190]. They offer a way to interact with applications whose inner workings are invisible to external users [191]. Furthermore, they provide the infrastructure that allows products and services to communicate with each other without knowing how they are implemented [190], [192].

Application programming interfaces (APIs) enable easy and secure data and functionality exchange between applications. The standardization of communication simplifies software development and promotes innovation.

In summary, APIs provide a well-documented interface that allows services and products to communicate and leverage each other's data and functionality [193].

### 6.6.1 API FUNCTIONING

APIs are an intermediary layer that processes data transfer between a web server and an application, thus the term interface [193]. The word application in the context of APIs relates to any software with a specific function. The application that sends the requests is a client, and the web server is the application that sends the responses [194]. An API can be thought of as a contract where the documentation represents an agreement that defines the structure for requests and responses between a client and a server [192]. The API documentation informs developers on how to structure requests and responses [194]. One does not need to know the server's inner workings to interact with the API [191].

Figure 54 represents the steps present in interactions between client and server through an API.


Figure 54 - API communication.

The steps illustrated are concisely presented below.

1. a client makes a request
2. once a valid request is received, the API calls the external program or web server
3. the server sends a response to the API
4. the API transfers the answer to the client

The process of request and response happens through APIs. Unlike user interfaces designed for humans, API design is adapted for use by other applications [193].

### 6.6.2 API SECURITY

The design of APIs increases security by facilitating the abstraction of functionality between two systems. API security can be further bolstered by including authorization credentials like authentication tokens, API keys, and API gateways to limit access, minimizing threats. Additionally, communication can be made safer with Transport Layer Security (TLS), HTTP headers, cookies, and query string parameters, which provide security layers to data [193].

Authentication tokens authorize users to make a specific API call by verifying identity and validating access rights [194]. Similarly, API keys validate the access rights required to access the API, but they authorize a program or application instead of a user. Further, tokens are more secure than API keys. However, API keys allow monitoring to gather data on usage [194]. To conclude, APIs provide a layer of protection between data and servers. API security can be further strengthened by using tokens, signatures, and Transport Layer Security (TLS) encryption, implementing API gateways that manage and authenticate traffic, and practicing effective API management. APIs open access to resources while maintaining security and controlling who and how users may have access [192].

### 6.6.3 API TYPES

The many different types of APIs are classified according to the systems they were designed to accommodate. APIs can be open or public (promoting innovation by allowing third parties to interact with the API and develop apps), partner (only exposed to strategic business partners), and internal or private (hidden from external users and only intended for internal use) [190], [193], [195].

Some of those types are briefly described as follows [193], [194]:

- Open APIs: open source and publicly accessible.
- Partner APIs: only exposed to strategic business partners.
- Internal APIs: remain hidden from external users, only intended for internal use.
- Composite APIs: combine numerous data or service APIs.

Presently, the most common are web APIs that give access to data and functionality through the web [193]. Public APIs are especially valuable since they can simplify and expand how the API connects with developers and open a path to data monetization.

### 6.6.4 API PROTOCOLS AND ARCHITECTURES

Before personal computers emerged, people used APIs as libraries for operating systems. By the 2000s, APIs finally broke out of their local environments and became important technology for remote data integration. Engineers designed remote APIs to communicate through a network. The term remote refers to external resource manipulation external to the client making the request. Nowadays, the internet is the most popular communication network, which makes most APIs designed according to web standards. Web APIs commonly

use HTTP for requests, also referred to as API calls, and define the structure expected from response messages. These response messages may be XML or JSON, depending on the protocol or architecture adopted. Both XML and JSON have been preferred formats because the way they structure data is easy for other applications to operate [192].

A Web Service API, or simply Web API, is a specific type of API that interfaces between a web browser (client) and a web server (server) and is used to interact with and manipulate data or resources over the internet [191]. Every web service is an API, but not all APIs are web services. REST is a popular type of Web API explained below [194].

The increased use of web APIs brought about the development of various protocols and architectures, like REST, SOAP, XML-RPC, and JSON-RPC. These API protocols facilitate standardized information exchange, empowering applications with different programming languages and technologies to communicate seamlessly. Each protocol is explained below [190], [192]–[194]:

**REST (Representational State Transfer):** is a set of architectural principles for web APIs. A Web API that adheres to the REST architectural constraints is called a RESTful API. Currently, this is the most flexible and prevalent API.

**SOAP (Simple Object Access Protocol):** is a protocol built with XML that enables users to exchange data through SMTP and HTTP. SOAP APIs promote data sharing between apps or components running in different environments or coded in distinct languages. This API is less flexible and used to be favored in the past.

**XML-RPC:** is a Remote Procedure Call protocol that depends on XML to transfer data.

**JSON-RPC:** this protocol uses JSON instead of XML to transfer data.

SOAP and REST have made APIs simpler in their design and more suitable in their implementation. As the web gained popularity, developers created SOAP to standardize communication formats. Following this protocol made it easier for apps developed in distinct ways to communicate. Unlike SOAP, REST is an architectural style that is ultimately simpler to follow than a protocol, making it more popular by comparison [190].

### 6.6.5 REST API

REST, or Representational State Transfer, is a term coined by Roy Fielding at the beginning of the 21st century in his dissertation "Architectural Styles and the Design of Network-based

Software Architectures" [196]. In the meantime, it has become the most popular and flexible API. As defined by Roy Fielding, a RESTful API must follow six guiding constraints [192], [196]:

- **Client-Server:** a client-server architecture composed of clients and servers and handles requests through HTTP
- **Stateless:** an additional constraint on the client-server architecture prohibiting session state in the server component. A request made by the client to the server must contain all the necessary information to understand the request without relying on any stored context.
- **Cacheable:** If a system is considered cacheable, a response to a prior request can be reused if the request is equivalent and would likely result in an identical response from the server.
- **Uniform Interface:** The REST architectural style is characterized by four fundamental principles: the identification of resources, the manipulation of resources through representations, self-describing messages, and the use of hypermedia to drive the application state. These features distinguish REST from other architectural styles.
- **Layered system:** A layered system allows mediation between client and server interactions. These additional layers can offer proxies, load balancing, gateways, shared caches, or secure firewalls.
- **Code on demand:** COD allows clients to access functionality without the know-how to process. The server response is code that the client then executes locally. COD is a way for servers to extend functionality to clients by providing executable code.

## 6.7 MACHINE LEARNING AS A SERVICE

Data science does not end with Model Engineering. The next step is Model Deployment, a crucial step where results are shared with others. They might need help understanding coding or ML, but everyone must be able to interact with the final applications and retrieve results quickly [197]. Firstly, the code will be wrapped in a Rest API and deployed as microservices. Secondly, an interactive web interface will use those microservices. Frameworks will be used to achieve the steps above, produce and deliver a quality product faster, and make the development experience more enjoyable [198]. Frameworks are collections of modules and packages used to develop software and help developers handle low-level details [199].

There is no best framework since each one has its importance and features. The following chapters thoroughly evaluate a few frameworks to select the most suitable for the project at hand.

### 6.7.1 BACKEND FRAMEWORKS

Python is widely considered to be one of the most popular programming languages. Its uses range from scripting to API development and ML. Python offers exceptional tools and overall experience [200]. Two of these tools are Fast API and Flask. Both are microframeworks used to work with Python efficiently [199], [200]. Flask is a micro web framework widespread in the ML field and used for API development. On the other hand, Fast API is a newer framework for building APIs and is suitable for ML-based applications [200], [201].

#### *6.7.1.1 FLASK*

Flask was released in 2010 [197], [202]. Flask is a lightweight, open-source, Python micro web framework with a concise and extensible core that supports the deployment of web applications and Rest APIs [197], [202]. This framework is easy to set up, fast, and capable of scaling up complex microservices and apps [199]. The framework uses WSGI (Web Server Gateway Interface) as the synchronous interface between web apps and servers. The synchronous framework requires one worker per request, and new tasks must wait in a queue for previous ones to finish. Flask combines Werkzeug and Jinja2 to offer a minimalistic but extensible framework where missing features are added with third-party plugins. Some of these extensions aid the development of Rest APIs: Flask-RESTful, Flask-Classful, and Flask-RESTPlus [202].

**Features**

**Features-Scalable**

Flask was designed to facilitate the growth of tech projects at speed. Flask applications are readily and extensively scalable. The framework allows the creation of advanced applications by adding new functionalities and use cases as needed. [202]

**Features-Ample resources available**

Since Flask has been in use for over a decade, it has many supporting resources. Flask is a well-established community with lots of extensions. Readily available educational resources make it very easy to learn for self-starters. [202]

**Features-Flexible**

Most parts of Flask can be changed, which is a rare feature of a development framework. This framework has a flexible and minimalist nature that accounts for changes while maintaining a stable overall structure. [202]

**Features-Security**

Although Flask is a minimalist framework, it is equipped to handle common security concerns like CSRF (Cross-Site Request Forgery), XSS, and JSON security. Additionally, as with many other features, developers can benefit from third-party extensions like Flask-security. Nevertheless, extensions may negatively impact performance, and developers must carefully evaluate each. Since these extensions can introduce security concerns, developers must remember to update extensions regularly or when vulnerabilities are exposed. [202]

**Pros and Cons**

**Pros**

1. Scalable. [202]
2. Flexible and comfortable. Most parts have the potential to change. [199]
3. Because of its extensibility, it is very flexible. Allowing unit testing and many other features, such as integrated support. [199], [203]
4. Beginner-friendly due to its simplicity. It enables developers to build apps quickly and effortlessly. [199]
5. A large amount of documentation and resources are available for learning. [203]
6. Minimalistic but powerful. [203]

**Cons**

1. Lack of support for asynchronicity. A singular source will handle each request in turns, one after the other, which takes more time [199].
2. Flask is developed for WSGI, so it does not offer native async support. Using WSGI means that long queries may block the application. A REST API built with Flask will only be able to handle a small number of users. Requests are handled in turns, which takes more time [202].
3. Flask uses third-party modules that increase the risk of security breaches. Implementing a malicious module can have grave consequences [199], [202].
4. Lack of data validation. Flask does not validate data formats. This way, developers can pass any data type, including strings and integers. Validation scripts that require

additional effort or extensions such as Flask-Marshmallow or Flask-inputs can solve the data validation problems [202].

5. Although it is possible to build APIs, it is not the main focus of Flask [203].

**Users and Use Cases**

This framework is helpful for a quick start but not advisable for high-load enterprise software [199]. Despite not being appropriate for heavy loads, there are many reasons to use this framework for personal or commercial projects. Some examples would be e-commerce systems, social media bots for Twitter and Facebook, online social networks, or static websites [202], [203]. Finally, Flask deemed the most popular Python development framework for beginners, and is currently used by companies such as Netflix, Lyft, and Zillow [202].

#### *6.7.1.2 FAST API*

FastAPI was released near the end of 2018 [197], [202]. FastAPI is a modern, open-source, high-performance Python micro web framework for building Rest APIs and fast web applications with Python versions 3.6+ based on standard Python type hints [199], [204], [205]. This framework is easy to learn, quick to code, and ready for production, ultimately allowing developers to build production-ready APIs efficiently with the best practices by default [204], [206]. The framework uses ASGI (Asynchronous Server Gateway Interface) as the asynchronous interface between web apps and web servers. The high performance and speed of FastAPI come from supporting asynchronous tasks that don't have to wait in a queue for the previous jobs to finish [203]. FastAPI brings together Uvicorn, Starlette, Pydantic, OpenAPI, and JSON Schema to address three core concerns: speed, developer experience, and open standards [197], [201].

**Features**

Fast API has exceptionally high performance and speed. This very high performance is at the level of Go and NodeJS. Starlette and Pydantic are the cornerstones that make FastAPI one of the fastest Python frameworks available. FastAPI promotes fast coding, claiming to increase the development speed of features by 200% to 300% based on internal testing [204]. These tests also point to a 40% reduction in human-induced errors, ultimately reducing overall bugs. Debugging time is also reduced by the intuitive nature of the framework, which provides strong editor support and code completion. The developers of FastAPI intended for it to be simple to use and learn, with a minimal amount of time spent reading documentation.

Although easy to learn and use, it is still robust and capable of production-ready code. A distinguishing feature is automatic interactive documentation that allows developers to test without resorting to additional tools like Postman to explain to the user how the API is expected to be used and eliminates the tedious process of creating documentation after the fact. Other features include minimizing code duplication, supporting multiple features from each parameter declaration, and fewer bugs. FastAPI is based on open API standards, OpenAPI [207], and JSON Schema [208]. [204]

The following list summarizes the most significant features of this framework [198], [209].

- **Based on open standards:** OpenAPI for API creation and JSON Schema for automatic data model documentation.
- **Automatic docs:** As the framework is based on OpenAPI, interactive API web interfaces are automatically created.
- **Modern Python:** doesn't require learning new syntax since it is all based on standard Python 3.6+ type declarations.
- **Editor support:** the framework design accounts for one of the most used features, auto-completion. Autocompletion reduces the need to go back to documentation.
- **Short:** Adequate defaults and the ability to fine-tune all the parameters to define the API.
- **Validation:** Validation for most Python data types is handled by the well-established and robust Pydantic.
- **Security and authentication:** Include all the security features from Starlette plus security schemes from OpenAPI, such as HTTP basic Oauth2 and API keys.
- **Dependency injection:** Offers a powerful and easy-to-use dependency injection system automatically handled by the framework.
- **Unlimited plugins:** There is no need for plugins. Import and use code as needed.
- Since it is based on Starlette and Pydantic, it includes their features.

The following paragraphs give a more detailed view of the most relevant features.

**Features - Running in Development**

FastAPI supports Hot Reloading. The app keeps running while the code changes, eliminating the need to restart the development server. While similar functionality is possible with Flask, it requires extra steps [210].

**Features - HTTP Methods**

The process of requesting the information is very similar to Flask and FastAPI. However, Flask needs a tool like Postman to act as a client, while FastAPI has interactive automatic documentation that allows users to make requests. Flask can also use automatic documentation but involves installation and configuration [210].

**Features - Automatic Documentation**

Automatic documentation comes included with FastAPI. Therefore, there is no need to install external tools to test requests. Out-of-the-box automatic documentation is one of the many ways FastAPI speeds up development time [210]. FastAPI offers handy documentation and a browser-based user interface powered by SwaggerUI or ReDoc that interactively documents the API. This interactive documentation allows developers to easily use and test API endpoints and explain the program to others [197], [202].

**Features - Data validation**

FastAPI includes data validation by default with Pydantic, a powerful data validation package [210]. FastAPI allows developers to customize validation on the parameters received [197]. The Pydantic library detects invalid datatypes during a run and returns the reason for inadequate input in JSON format. Pydantic ensures faster typing, which is hard to achieve manually, and reduces bugs, a 40% decrease according to FastAPI [201], [202].

**Features - Portable**

FastAPI code can be ported to other frameworks that use standard Python-type hinting [210].

**Features - Production Server**

Flask uses WSGI, which has been the Python standard for several years. It has the downside of being synchronous. If there are many requests, they must wait in a queue [210].

FastAPI uses Asynchronous Server Gateway (ASGI), which is very fast due to its asynchronous nature. If there are many requests, they don't have to wait for the ones that arrived first to be processed [210].

**Feature - Performance and Asynchronous tasks**

FastAPI is much faster than Flask because it is built over ASGI instead of WSGI and consequently supports concurrency and asynchronous code [201], [202], [210].

FastAPI delivers performance on par with traditionally faster languages like GO and NodeJS. FastAPI is one of the fastest Python-based frameworks available [198], [200], [206], [211]. Only

Starlette and Uvicorn, on which FastAPI is built, are faster. Verify independent performance tests at [212].

**Pros and Cons**

**Pros**

1. High performance [202].
2. Concurrency support [202].
3. Easy-to-use dependency injection system [202].
4. Data validation even in deeply nested JSON requests [199].
5. Built on standards: JSON Schema, OAuth 2.0, and OpenAPI [199].
6. Interactive API documentation [203].
7. Editor completion [203].
8. FastAPI offers a rich API interface, especially for REST API [203].

**Cons**

1. The absence of an inbuilt security system instead offers the "fastapi.security" module, which includes the security modules [202].
2. A relatively small community of developers [202]. Since FastAPI is eight years younger than Flask, it has a smaller community [199].
3. Detailed documentation but few external educational materials [199], [203].

**Users and Use Cases**

FastAPI works perfectly if the concern is speed. Scales well for deploying production-ready models since ML models work best in production when wrapped around a RestAPI and deployed in microservices [199], [203].

Tech giants like Microsoft, Netflix, Uber, and Explosion AI, amongst many other corporations, have started to build their APIs with FastAPI [202], [206]. Netflix uses this framework for its internal crisis management [203].

#### *6.7.1.3 FASTAPI VS FLASK*

Both FastAPI and Flask are lightweight Python micro frameworks. Nonetheless, there are fundamental differences that make FastAPI the best choice for building an API in the context of this project. The ensuing section will delve into the disparities between FastAPI and Flask.

**Asynchronicity**

The first central difference is that FastAPI uses ASGI while Flask uses WSGI. As mentioned, ASGI handles requests asynchronously. Tasks do not have to wait for others to finish, and there is no queue to process requests. On the other hand, WSGI handles requests synchronously, meaning a task can only start once the previous one finishes [203]. Fast API supports concurrency and asynchronous code, while Flask lacks support for this feature [197].

**Performance**

FastAPI is better than Flask regarding speed and performance because FastAPI is built on Starlette, an ASGI framework [202]. Conversely, WSGI constrains Flask, reserving a worker for each request. Meanwhile, FastAPI behaves in a non-blocking way throughout the stack and applies concurrency at the request/response level [211]. This feature makes Fast API the better option for larger-scale projects as it is better at handling multiple requests much faster [203]. With FastAPI, a developer can use a framework built with cutting-edge technology and benefit from the time savings that come from using a project template [202].

**Data validation**

Flask has no built-in data validation support, but there are ways to install Pydantic despite requiring additional effort [210]. On the contrary, FastAPI includes Pydantic, a library for data validation that takes the idea of classes to a new level. FastAPI reduces errors with type declarations and Pydantic schemas. In FastAPI, Python-type hints and Pydantic models define the data schemas expected by the endpoint. Pydantic is deeply integrated into FastAPI as it is used to determine the API config and the API requests and responses [211].

**Automatic documentation**

With FastAPI, the API documentation is automatically created by writing the endpoints. FastAPI is built around the OpenAPI specification standards, which means that good documentation is created just by writing the application. FastAPI offers two types of documentation UI displays: SwaggerUI and ReDoc. Both offer interactive documentation that allows developers to input request data and trigger responses [211]. By comparison, Flask needs a tool like Postman to achieve to act as the client. Alternatively, Flask can use automatic documentation by adding an extension, for example, "flask-swagger". The downside is that the installation process requires extra work and involves lots of configuration [210].

**Tools**

Flask can be used to build an API, but it is not the main focus of the framework. On the other hand, FastAPI is best for backend REST APIs while maintaining the option to serve web pages.

FastAPI offers many tools often needed when building APIs. Such tools are leveraged from Starlette, on which the framework is based. Some of these features are in-process background tasks, WebSocket support, GraphQL support, CORS, GZip, Static Files, Streaming responses, and Session and Cookie support [211].

**Community and Popularity**

Flask is more mature than FastAPI due to being around eight years older. Throughout the years, Flask has developed a large community and become a well-established framework. Flask has many external educational materials, while FastAPI has fewer online materials. Nonetheless, FastAPIs popularity is increasing, and the community keeps growing. This growth may change the lack of external online learning materials. A simple indicator of community size and popularity is the number of GitHub stars and forks. As illustrated in Figure 55, while FastAPI has been released more recently than Flask, the number of stars and forks on GitHub for both technologies are not far apart, indicating that both are popular choices among developers.

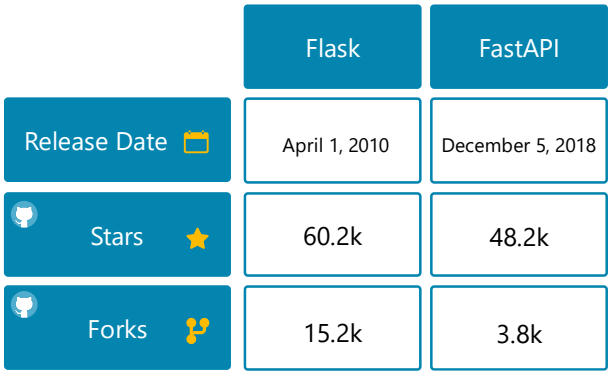



Figure 55 - Github stars and forks for Flask and FastAPI at 12/08/2022.

Flask and FastAPI have several differences, including maturity, community, performance, support for data validation, asynchronicity, and automatic documentation [203].

Flask is minimalistic and lighter with less out-of-the-box. Nevertheless, it is a robust framework that leaves most feature choices to the developer's discretion. With Flask, developers can add extensions to their code as they see fit. On the other hand, there are several standard features frequently needed. Therefore, deep integration with the framework results in less code for developers to create and maintain. Following this approach, FastAPI includes multiple desired features and implements strict standards, making code production

ready and painless to maintain [200]. Flask is tested, widely used, slightly more reliable, and has more learning resources available due to its earlier release [200], [210]. While FastAPI is not as battle-tested as Flask, it is a newer, more modern framework focused on speed, with an increasing number of developers using it to serve machine learning models and develop Restful APIs [200], [210]. In the context of machine learning and APIs, FastAPI is a good choice since it saves a lot of time building and is perfect for speed and scalability, which align with the goal of ML to test models in a production environment [199], [201]. By comparison, Flask takes more time to set up the same helpful documentation.

In conclusion, FastAPI is the better choice for building a RESTful API when working to deploy machine learning models as microservices. Given all these reasons, FastAPI was chosen to leverage speed both in performance and development time. Ultimately allowing the construction of an API from scratch while reducing the number of bugs and errors in the code [202].

### 6.7.2 FRONTEND FRAMEWORKS

In web development, the frontend is the primary interface through which users interact with a web application, encompassing the graphical user interface [213]. The focus of frontend development is to deliver an intuitive, user-friendly experience that not only loads quickly but also presents data in a manner that facilitates user comprehension and engagement [214]. This is achieved by providing readily available features and functionalities that cater to user needs [215].

Frontend frameworks are software tools that enable the creation of optimal user experiences by offering reusable code modules, standardized frontend technologies, and interface blocks [213]. These frameworks expedite the development process by enabling developers to create standard features more efficiently and provide tools for creating context-specific elements. Examples of these common features include user authentication, page rendering, database connections, stylized components, and development tools such as grids for UI design components, font settings, and website standard building blocks like side panels, buttons, and navigation bars [216].

Beyond these, frontend frameworks may also contain utility programs, code libraries, scripting languages, and other software that enhance the development and integration of components [213], [215]. These features form the foundation for developing critical project-specific

elements, accelerating the development process, improving productivity, and ensuring the final product's reliability while allowing design flexibility [213], [217].

Often referred to as CSS frameworks, frontend frameworks are instrumental in creating interactive tools and responsive components and building consistent products to enhance the overall aesthetic of web applications [213], [214]. Furthermore, these frameworks contain essential packages with convenient code segments that developers utilize to address common problems, such as managing requests or defining file structures [213].

In essence, frontend frameworks provide a structured approach to application architecture, offering a reusable framework that can be manipulated according to set prerequisites. This framework code structure standardizes how developers write code, thereby reducing the workload and saving time by offering a general format that can be reused [218].

In backend development, many alternatives exist depending on the purpose. However, regarding graphical user interfaces on the web, there is only one native language, JavaScript. According to the 2022 Stack Overflow developer survey, JavaScript is the most commonly used programming language for the tenth year, as shown in Figure 56 [219].

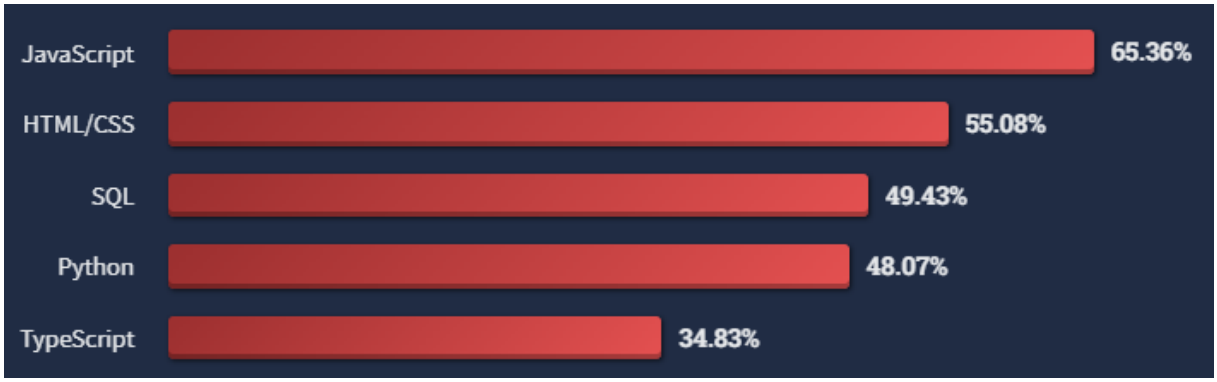



Figure 56 - 2022 Stack Overflow Developer Survey retrieved from [219].

There are two core web technologies besides JavaScript: HTML and CSS. HTML is a markup language employed to shape web content, CSS is a language of style rules that apply the style to HTML content, and JavaScript is a scripting language that enables the creation of dynamically updating content, multimedia, animations, and much more [220]. These three technologies are at the core of all web applications [218].

The quest for the best framework rages on, and it is as hard as ever to name the best one. There are several available to developers, and each offers unique features. Developers must choose the best one with the goals of the web app in mind. Accordingly, after researching recent studies and surveys, the three most popular will be discussed [217].

In most cases, frontend frameworks are written in JavaScript and help set up the functionality and interactivity of a website [218]. The most popular frontend frameworks are React, Angular, and Vue [217].

As reported by the 2021 State of JS survey, React, Angular, and Vue dominate the frontend framework market share based on usage for the fifth year, as illustrated in Figure 57. This report is confirmed by the 2022 Stack Overflow Developer Survey, as shown in Figure 58. React, Angular, and Vue are once again seen at the top of the most common web technologies used.

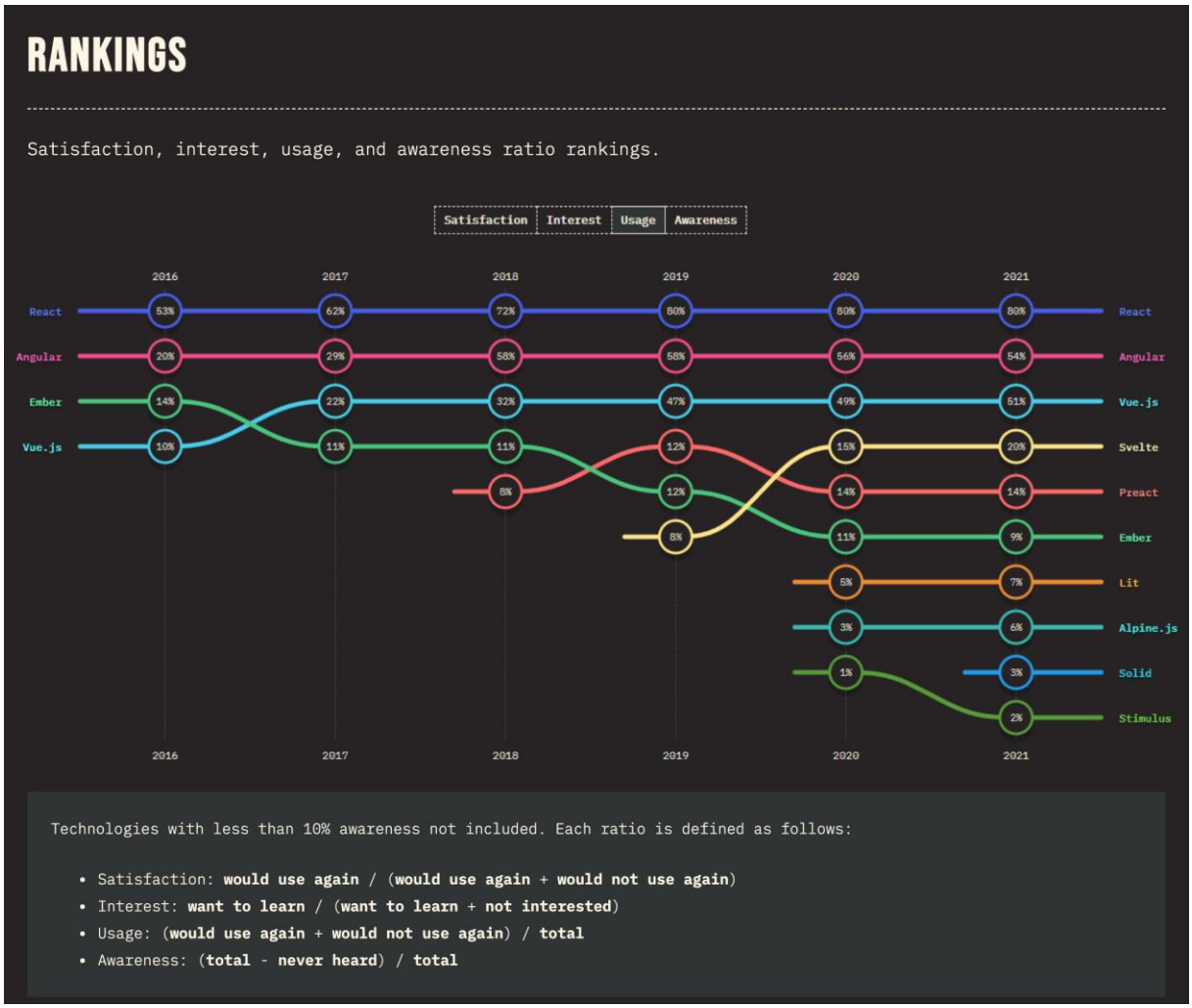



Figure 57 – 2021 State of JS retrieved from [221].

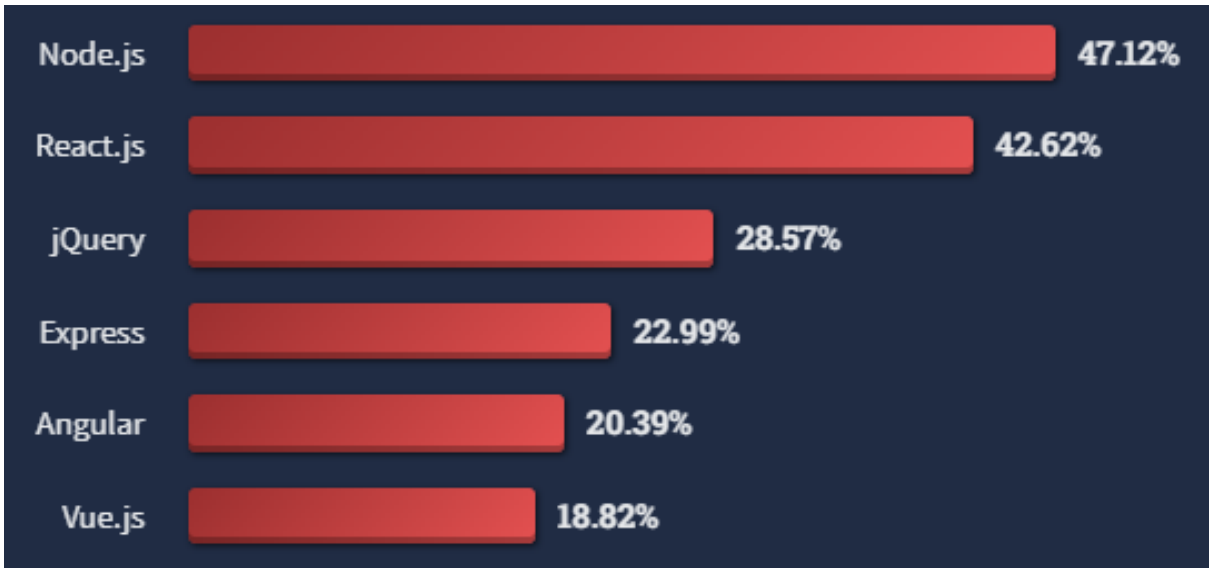



Figure 58 - 2022 Stack Overflow Developer Survey results retrieved from [252].

#### *6.7.2.1 REACT*

React is an open-source JavaScript library developed by Facebook. React developers use JavaScript and JSX with various HTML quotes and tag syntax. React is a library, not a framework, because the core package only offers tools to create user interfaces. React is unopinionated, meaning that the developer is free to choose from various tools and packages. Building a complete React application requires additional React-specific libraries. A framework includes many more tools, all bundled within the core package.

The primary concerns in the development of React were the performance with useful UI, code maintainability, and user-friendliness [214]. Component reusability and component-based architecture make maintenance easier and increase productivity [222], [223]. The reusable components follow a one-way data binding tree-like structure, which means all the information flows in one direction from the stem to the branches, in other words, from component to view [213]. Furthermore, React differentiates itself because of its virtual DOM with a low learning curve, which presents excellent functionality [214], [224]. Its virtual DOM promotes the faster creation of web apps and ensures steady performance at scale [223], [225].

This framework is considered one of the best UI development tools in 2022 [214]. Since it became open-source, more and more big businesses have begun to use React in frontend development, which attests to this tool's high quality, stability, and trustworthiness [213].

Lastly, thanks to the mass adoption of React, developers can enjoy a robust community and dedicated debugging tools and extensions [223].

**Features and functionalities**

**Components**

React is component and UI-based. This function is helpful for code maintenance [222].

**JSX**

JSX is a syntax extension of JavaScript that helps describe how the UI should look. JSX can make the syntax needed to transform coding more concise and straightforward when paired with JavaScript [222].

**Data Binding**

React has one-way data binding [222]. Data binding helps connect the UI to the business logic. The type of data binding affects the performance of an app. With React, debugging becomes more efficient as the data flows in one direction, particularly for more extensive applications [224].

**Event Handling**

React provides a simple cross-browser interface to the native event, promoting event name and field compatibility. React reduces memory as the event system is executed and implemented by event delegation and has diverse event objects [222].

**Performance**

The one-way data binding does not impact performance when an application's complexity increases. Furthermore, the built-in virtual DOM feature results in less loading time on the browser [224].

**Advantages**

- Backed by a powerful company, Facebook.
- Open-source.
- Cross-platform.
- Easy to learn and use.
- Component-based architecture and reusable components.
- Virtual DOM.
- Frequently updated and appropriate version control.
- Easy integration with other libraries.
- Extensive community and a wide diversity of online educational resources.

**Limitations**

- React only offers a solution for UI development, so it must rely on other tools.
- JSX is a syntax with some complexity.
- Documentation quality tends to decline due to fast-paced development.
- There are better choices for small projects.

**When to use it?**

React is the right choice for building rich user interfaces for cross-platform applications with dynamic content. The virtual DOM capability also makes it suitable for more complex applications. React also helps developers make efficient use of their limited time and becomes even more efficient and productive when used in conjunction with other libraries, such as Redux and Axios.

**When not to use it?**

React is not the best choice for developers unfamiliar with JavaScript or new to JSX since it can be challenging to learn.

**Companies and Brands**

The following are some of the brands that have used React: Uber, Dropbox, Netflix, Instagram, PayPal, Flipkart, Facebook, Pinterest, Airbnb, Reddit, Discord, Walmart, Tesla, Microsoft, LinkedIn, and Google.

#### *6.7.2.2 ANGULAR*

Angular is an open-source TypeScript-based framework backed by Google. Angular is a component-based framework for developing cross-platform scalable web applications, a set of tools for developers to create, build, test, reuse, and modify code, and a collection of well-integrated libraries [215]. This framework offers excellent performance, including component-based development, directives, filters, two-way data binding, dependencies, and more [213]. Angular also delivers a platform for building applications for all platforms simultaneously by reusing code in web, mobile, and desktop apps [223].

Despite the many great features, Angular is challenging to learn. Moreover, this steep learning curve may be one of the reasons for its decreasing popularity. According to the 2021 State of JS Survey, Angular retains a high usage percentage. However, as shown in Figure 59 and Figure 60, satisfaction and interest have dropped considerably, reaching an all-time low [221].

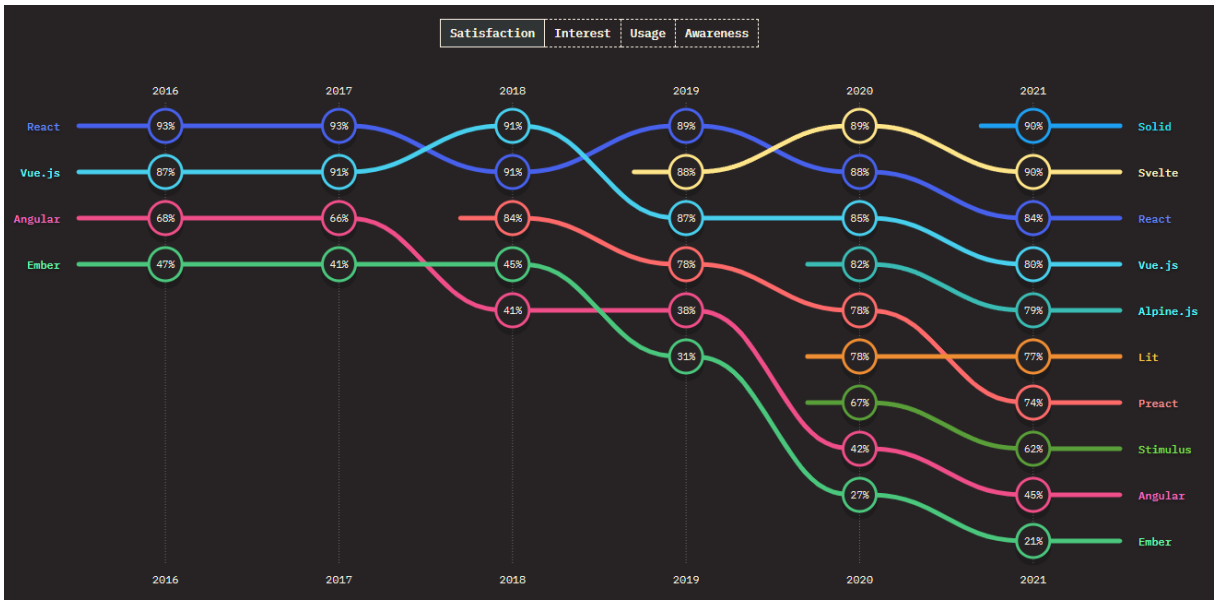



Figure 59 - 2021 State of JS user satisfaction retrieved from [221].

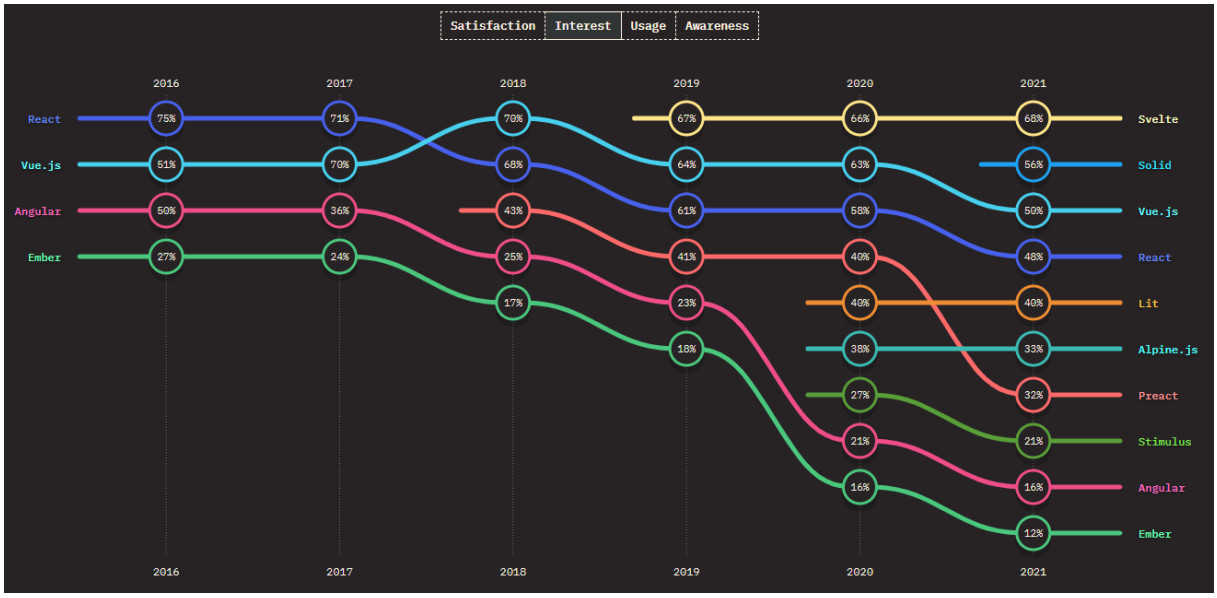



Figure 60 - 2021 State of JS user interest retrieved from [221].

### Features and functionalities

#### Data Binding

Angular has two-way data binding that lets the framework connect the model data to the DOM via the controller [224]. The two-way data binding ensures that any change in the view will immediately synchronize with the model and vice versa [216]. This type of data binding may negatively influence performance as app complexity increases [224].

#### Templates

The browser parses templates and directly passes them into the DOM.

#### Dependency Injection

Angular has the necessary dependency injection to improve development and testing [222].

**Directives**

Directives support the creation of new HTML tags that execute like new custom widgets. They can also be used to manipulate DOM attributes [222].

**Code generation**

Templates transform into code that is then optimized for JavaScript virtual machines. This process provides some of the benefits of hand-written code [222].

**Advantages**

- Backed by a powerful company, Google.
- Open-source.
- Cross-platform.
- Two-way data-binding and real-time model-view synchronization.
- Dependency injectors that allow component reuse.
- Component-based architecture.
- Large community.

**Limitations**

- Steep learning curve.
- The documentation is lacking.
- Difficult to debug.
- Verbose and large package size
- Dynamic applications may underperform.

**When to use it?**

Angular is the right option when compatibility with all browsers is paramount. It helps with app consistency across older browsers. Angular is a good option for enterprise-scale applications with multiple features as it provides a scalable and reliable framework. Experienced developers familiar with creating complex web solutions are recommended when choosing Angular.

**When not to use it?**

Angular as a framework offers a comprehensive dynamic solution not best suited for small-scale web applications. Small projects may only partially take advantage of the vast resources provided. For simple applications, it is best to avoid Angular. Not opting for Angular in this

context avoids unnecessary complexities by choosing a more straightforward framework with a simpler syntax.

**Companies and Brands**

The following are some of the brands that have used Angular: NASA, HBO, Nike, YouTube, Forbes, Lego, Autodesk, UPS, BMW, Deutsche, Bank, Mixer, Santander, Gmail, Xbox, Upwork, and PayPal.

#### *6.7.2.3 VUE*

Vue is an open-source framework written in TypeScript and developed by Evan You, who was previously involved in the development of Angular [223], [226]. Accordingly, no big corporation manages this framework. Vue combines features of existing frameworks to deliver a straightforward and lightweight solution [217]. In fact, Vue is lighter than React and Angular. Compared to Angular, it eliminates software intricacies and improves performance and usability [214]. Vue combines the template style from Angular with the component-based properties of React [222]. Additionally, it employs a virtual DOM and two-way data binding [214]. The two-way data binding underlays its high-speed performance, making it easier to update related components and track data changes, which are crucial for any application requiring real-time updates. Vue is beginner-friendly but comes with a smaller community. Vue has various tools, including end-to-end testing tools, plugin installation systems, browser debugging tools, server renderers, and state managers [216]. All these features make it a versatile framework for designing web applications with a beautiful UI [222]. Despite all these favorable characteristics, Vue has less support from the general market and less popularity with the most prominent companies [214], [216].

**Functionalities and Features**

**Templates**

It uses syntax based on HTML. Templates are parsed with HTML parsers and spec-compliant browsers [222].

**Transitions**

It enables transition effects while components are updated, inserted, and removed from the DOM [222].

**Components**

Components expand on fundamental HTML elements by making them reusable [222].

**Reactivity**

The view is automatedly updated when the models in JavaScript objects are custom-made [222].

**Data Binding**

Similarly to Angular, it employs a bi-directional data binding feature [222].

**CSS Transitions and Animations**

Vue offers a built-in feature that envelops the elements responsible for producing the transition effect [222].

**Advantages**

- Open-source.
- Component-based architecture.
- Virtual DOM.
- Simple syntax and beginner friendly.
- Light package.
- Two-way data binding.

**Limitations**

- Language barriers since various resources are in Chinese.
- Less widely adopted.
- Lack of plugins.
- A robust company does not back it.
- Relatively new and developed by private individuals.
- Small developer community.

**When to use?**

Vue's simplicity and flexibility make it the right choice for most project sizes. Vue offers a broad spectrum of powerful features that support the development of both small and large-scale web and mobile applications.

**When not to use?**

Although Vue offers many features, component stability is still questionable since the framework has presented difficulties with the firmness of parts [214], [217]. Therefore, a newer framework that cannot ensure a high level of steadiness, support, and quick problem-solving is not the most appropriate for enterprise-level applications [214]. Another reason not

to opt for Vue is the smaller community mostly active in China. This may prove a significant obstacle when problems appear and help is needed [217].

Finally, if the developers do not have much experience with the framework, it is best not to choose Vue since the community may not always be ready to help.

**Companies and Brands**

The following are some of the brands that have used Vue: Alibaba, 9gag, Xiaomi, Nintendo, and Adobe.

#### *6.7.2.4 REACT VS. ANGULAR VS. VUE*

Frameworks and libraries offer various features and characteristics that make it very hard to claim that one is better than the other for every scenario. The best framework for a specific application must consider many factors. Some determining factors are application requirements, developer experience, project size, complexity, and scope. Therefore, in search of the most suitable framework for the project, there are a few crucial facets to consider: Architecture, Components, Syntax, Data Binding, Size, Performance, Learning Curve, Libraries and ecosystem, and Popularity and community.

**Architecture**

React as a UI library does not enforce a specific project structure. React offers the view and leaves the rest up to the developer to decide. Nevertheless, it defines two building blocks: React Elements and Components.

Angular is a full-fledged framework that offers all the necessary features out of the box. These features include HTML templating, Dependency injection, ajax requests, routing, CSS encapsulation, testing utilities, and form creation, among others.

The core of Vue focuses on the view and model layer. However, since it is a progressive framework, functionality can be extended with additional packages.

**Components**

React Components combine appearance and logic. A component has all the code needed to define its appearance and behavior. Firstly, React elements differ from standard DOM elements as React DOM ensures efficient updates upon detecting changes. Secondly, Components are independent and reusable fragments of code that accept inputs, update accordingly, and present elements to users.

In contrast, Angular components are called directives and separate UI in the form of HTML tags and behavior in JavaScript.

Similarly to React, Vue combines UI and behavior in components.

**Syntax**

React and Vue use JavaScript but also support Typescript. Additionally, React has a syntax extension, JavaScript XML (JSX), which allows the creation of elements that simultaneously contain HTML and JavaScript.

Angular has the most complex syntax since its design entirely takes advantage of Typescript.

**Data Binding**

React uses one-way data binding, while Angular and Vue use two-way data binding. One-way data binding is one-directional because the information flows from a code file to HTML. Alternatively, two-way data binding is bi-directional since information flows from a code file to HTML and vice-versa.

**Size**

Angular is the heaviest, followed by React and, finally, Vue. Package size may influence performance. Therefore, less complex applications with fewer sophisticated components can benefit from smaller frameworks.

**Performance**

Performance is not a big concern if the project is small. However, achieving good performance becomes more problematic as project size and complexity increase.

In the context of performance, it is more critical to have quality development that follows good practices rather than the choice of framework. Therefore, performance is not the main deciding factor.

**Learning Curve**

Angular has the steepest learning curve out of the three since it presents a complete, full-featured solution that requires learning TypeScript and Model-View-Controller (MVC) architecture. TypeScript, a defined design structure, and other concepts required to use Angular as a complete frontend solution make it the most complex option. Although more complex to learn, the result can be more structured and thus easier to maintain at scale.

React is the easiest to learn if only the core library is considered. However, most React applications require additional libraries since the core does not include many features.

Ultimately, the learning curve depends on the path chosen by the developer. Different third-party libraries offer crucial advanced features that require different levels of proficiency.

Something that does not depend on third-party libraries is JSX. It is possible to use only JavaScript, but the benefits of JSX make learning it unavoidable. JSX is the only factor that enhances the challenge of learning React.

In contrast with Angular, React does not follow a specific structure, meaning learning to use React is insufficient. React developers must make sure to follow the best practices.

Vue is simple to learn due to its high customizability. One syntax challenge specific to Vue is its template syntax, which is easy to grasp. However, it does not impose a strict structure. This high flexibility can foment poor code that is difficult to test and debug.

Concisely, React and Vue have a similar level of complexity, and Angular is the hardest to learn.

**Libraries and ecosystem**

All three solutions benefit from open-source packages and libraries that speed up development by providing robust and battle-tested tools. Based on popularity alone, React has more components, themes, and tools than Vue. In contrast, Angular may have fewer options, but since it offers a complete set of tools, third-party libraries may not be needed and certainly not as frequently used compared with React and Vue. These extensions have many functions, from easing the start of a project, developing mobile applications, and providing design components to administrating states.

**Popularity and community**

Popularity matters since popular frameworks have a higher chance of being maintained and updated in the future. The size of the community is also directly related to popularity. A more popular framework tends to have a more extensive community and, consequently, complementary packages, tools, and help in the form of tutorials, forum threads, documentation, and more.

There are a variety of methods to ascertain the popularity of frameworks. The present work's sources are Google Trends, GitHub statistics, BuiltWith, NPM downloads, Statista, JavaScript Rising Stars, State of JavaScript, and Stack OverFlow developer survey results from 2021.

Angular, Vue, and especially React are common words that can make it hard to gauge popularity from Google trends, Figure 61. React is the most searched, followed by Angular, and Vue is the least searched. As previously mentioned, and confirmed by the results by region in Figure 62, Vue is primarily relevant in China. However, this search strictly measures popularity, which may not translate into actual usage. Other sources are required to determine usage.

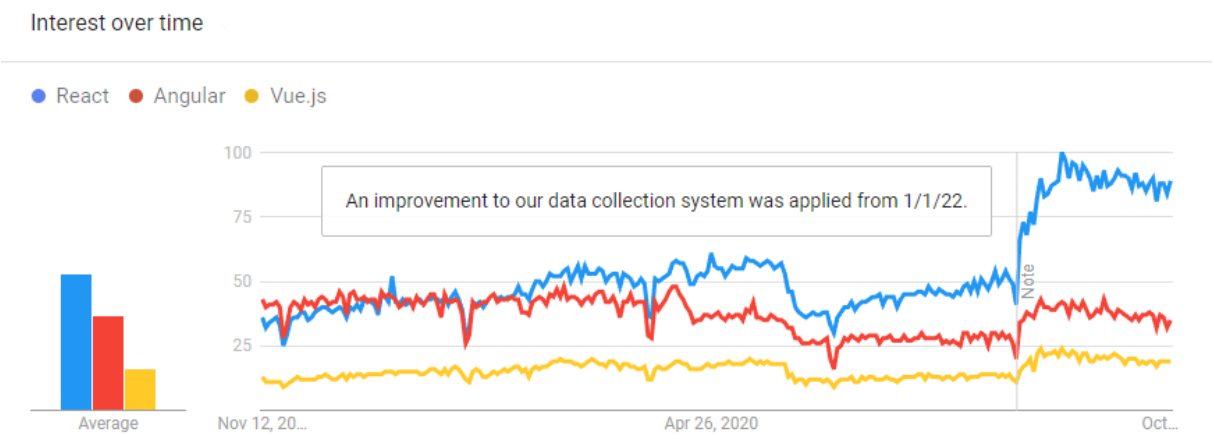



Figure 61 - Google Trends interest over time retrieved from [253].

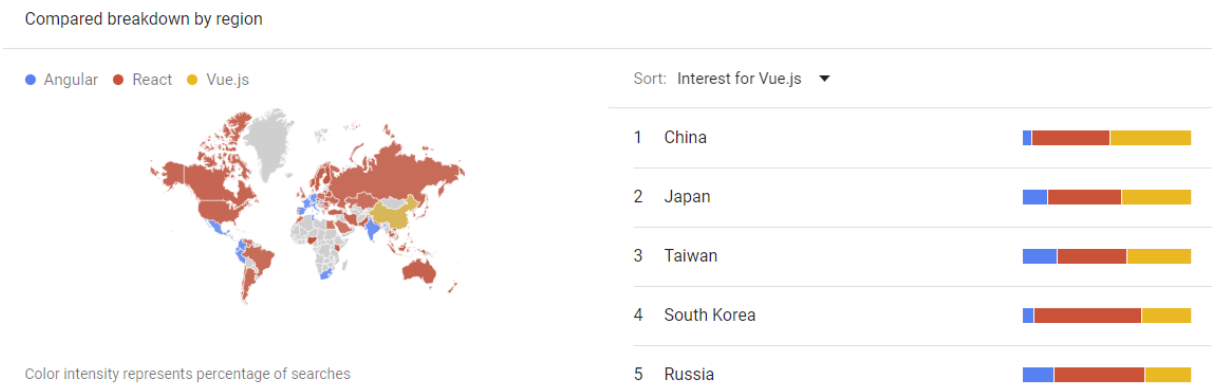



Figure 62 - Google trends interest for Vue.js by region retrieved from [253].

A good indicator of popularity is the GitHub star history from each repository, Figure 63. In addition to GitHub stars, watchers, forks, contributors, and issues help estimate popularity and usage, Figure 64.

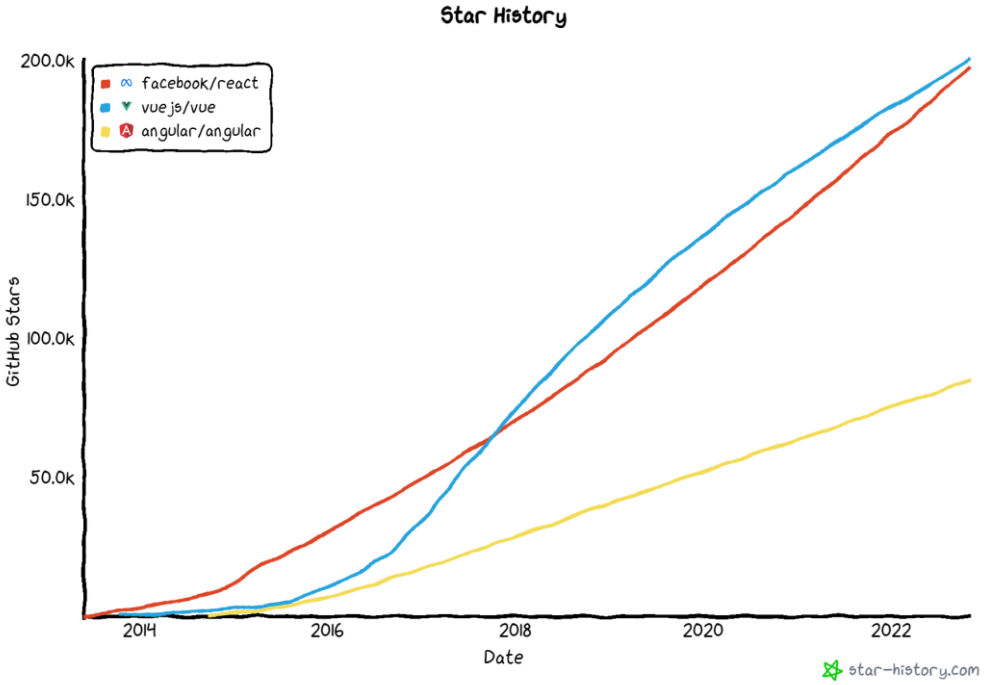



Figure 63 - GitHub star history for React, Angular and Vue retrieved from [254].

| | React | Angular | Vue |
|---|---|---|---|
| Stars | 197k | 84.7k + 59.4k* = 144.1k | 201k |
| Forks | 40.9k | 22.5k + 28.2k* = 50.7k | 33k |
| Watchers | 6.6k | 3.1k + 3.8k* = 6.9k | 6.1k |

* The Angular stats are the sum of the repositories angular and angular.js because the move from angular.js to the new repository reset the stats.

Figure 64 - React, Angular and Vue Github statistics 10/09/2022 adapted from [255]–[258].

A straightforward way to measure the popularity is by conferring the number of applications built with each technology. According to BuiltWith, React is by far the most used.

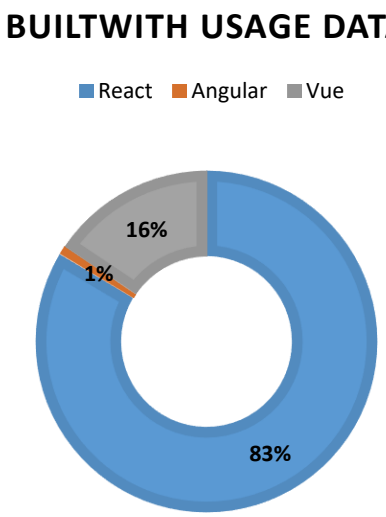



Figure 65 - BuiltWith usage data adapted from retrieved from [227]–[229].

NPM trends provide a way to compare package download counts over time. Analyzing the download numbers for each tool on NPM trends shows that React is the most popular, followed by Vue, and Angular is the least downloaded among the three, as shown in Figure 66.

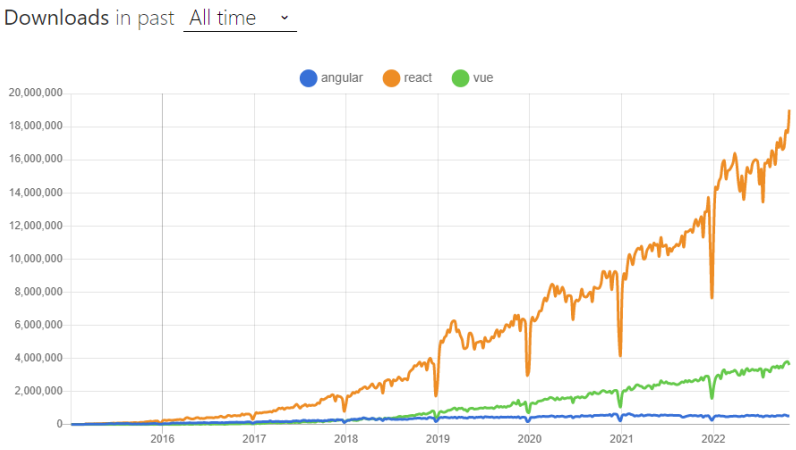


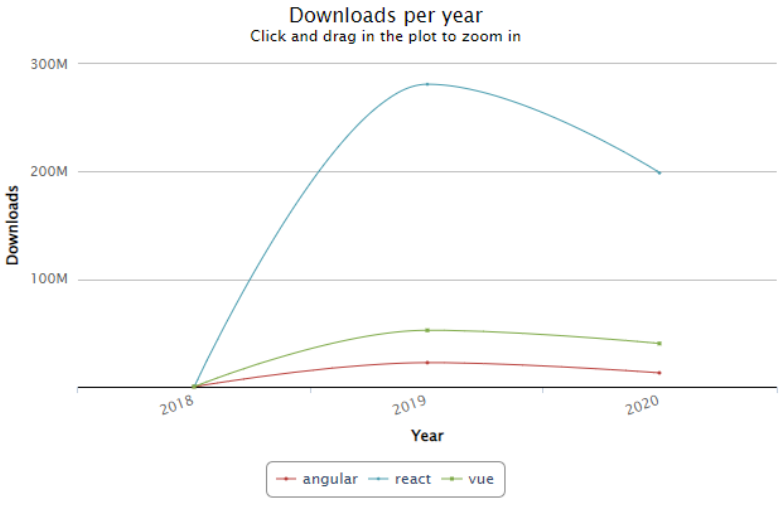

Figure 66 - NPM trends download statistics retrieved from [259].

As depicted in Figure 67, the number of downloads of NPM packages confirms the results from Google Trends, with React being the most downloaded, followed by Vue, and Angular being the least downloaded among the three. It is worth noting that Angular distributes cached

packages to enterprise customers, which comprise a significant portion of its users. This means that the actual number of Angular downloads may be higher. Despite this, it is clear that React is still more popular than Angular and Vue.

Statista shows that the most commonly used web framework among software developers in 2022 was React, with 42.62%. Angular and Vue are closer, with 20.39% and 18.82%, respectively [230]. This is further confirmed by the Stack Overflow Developer Survey Results 2022. React is the most common web technology used by developers (42.62%), followed by Angular (20.39%), and only then Vue (18.82%) [219]. However, Angular is in its third year as the most dreaded. Conversely, React completes its fifth year as the most wanted, and Vue occupies third place in the list of most wanted web frameworks and technologies [231]. Additionally, the 2021 JavaScript Rising Stars, which estimates yearly GitHub star growth, shows that React takes the first place with 18.5k stars, leaving Vue with 14.3k stars in second place and Angular with 9.3k in fourth place [232].

Overall, there are active communities in all these libraries, as proved by downloads and GitHub activity. There is no doubt that developers are using these libraries. Nevertheless, there exists a disproportionate split between GitHub stars and the actual usage of Vue [221]. Concerning




Total number of downloads between *2018-12-30* and *2020-07-07*:

| package | downloads |
|---|---|
| react | 480,016,008 |
| vue | 92,653,799 |
| angular | 35,266,285 |

Figure 67 - NPM stat downloads per year [260].

usage, Vue does not compete with Angular and React. Lastly, the growth for React, and especially Angular, is starting to slow down a bit.

#### 6.7.2.5 CONCLUSION

In conclusion, this subchapter on frontend technologies has demonstrated that all three technologies discussed are viable options for use in a frontend application. Angular may be the most suitable for big projects, but with proper knowledge of architecture and web development concepts, any framework can have well-organized code. Moreover, they all have active communities, documentation support, and third-party educational resources.

Each of these libraries has its benefits and drawbacks. Based on the project's goals and requirements, one of these will be more suitable than the others.

- React is mature, extremely popular, and widely adopted.
- React is backed by Facebook, which adds a level of confidence that is missing with Vue.
- React has active development with the regular release of new versions and maintenance of existing ones.
- React is flexible and lightweight since it does not adhere to a specific structure and lets the developers integrate the most appropriate libraries to reach a full-fledged React application.

A barrier to using React would be the complexity of JSX, but since there was previous experience with React, JSX is not an issue. In addition to the comparisons made, by considering critical factors such as experience, project scope, and complexity, React is the most appropriate technology for developing the platform.

## 6.8 SOFTWARE VIRTUALIZATION

When deploying virtualized instances, there are two main approaches to consider. One is based on hypervisors and machine-level virtualization, while the other centers around software-level isolation via containers. Virtual Machines (VMs) create a virtualized environment by layering a guest operating system over a host operating system (OS). Containers, on the other hand, utilize the resources of a host operating system to isolate applications [233].

### 6.8.1 VIRTUAL MACHINES

VMs are an abstraction of the hardware layer that provides virtual environments where a guest runs with a full copy of an operating system, the application, necessary binaries, and libraries that can take up tens of gigabytes. An essential part of this hypervisor-based

virtualization is the hypervisor. Practically, the hypervisor allows a single machine to run multiple VMs. It does this by instantiating, removing, and assigning resources to VMs. A guest system can only access and control the virtualized hardware provided by the hypervisor. Besides taking up a sizable amount of storage, virtual machines often supply more resources than the application requires [234], [235].

Figure 68 illustrates the structure of a VM. Generally, a VM is a copy of an operating system running on top of a virtualization technology implementation, such as VMware, VirtualBox, or KVM, that provides the hypervisor. Both the guest operating system and hypervisor run on top of the host operating system and the physical hardware. Lastly, at the top of all these layers resides the application. The structure described above and visible in Figure 68 presents speed, performance, boot time, and manageability challenges.

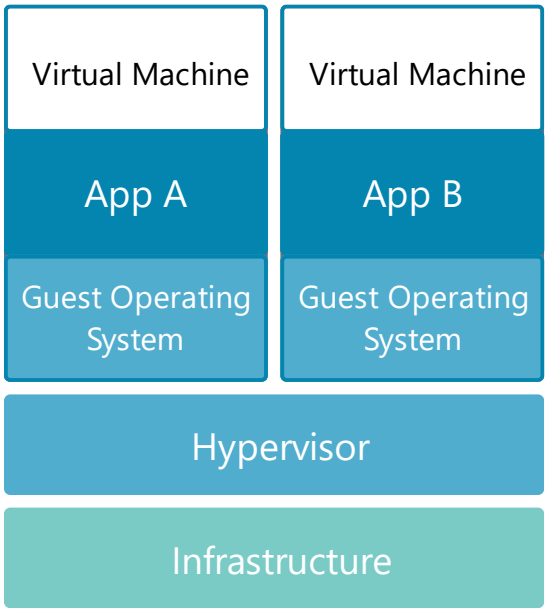



Figure 68 - Virtual Machine Architecture adapted from [234].

### 6.8.2 CONTAINERS

Containers are an abstraction of the operating system at the app layer that packages code and dependencies. Although a container does not have to boot an operating system, it still has a file system, network, IP address, process space, and defined resource limitations for CPU and memory [233], [234], [236].

A single machine can run multiple containers, but unlike VMs, these containers execute natively on the host OS using the host kernel and run in isolated processes in the user space, ultimately reducing the required resources by utilizing system memory as any other

executable [235], [236]. Consequently, containers take up less space than VMs, handle more applications, have faster boot times, and are more portable and efficient [234], [236].

Figure 69 illustrates the structure of a container. Generally, a container runs on top of the host OS that runs an engine provided by Docker or LXC natively. The configuration described above and visible in Figure 69 translates to better speed and performance, faster boot times, and fewer operating systems.

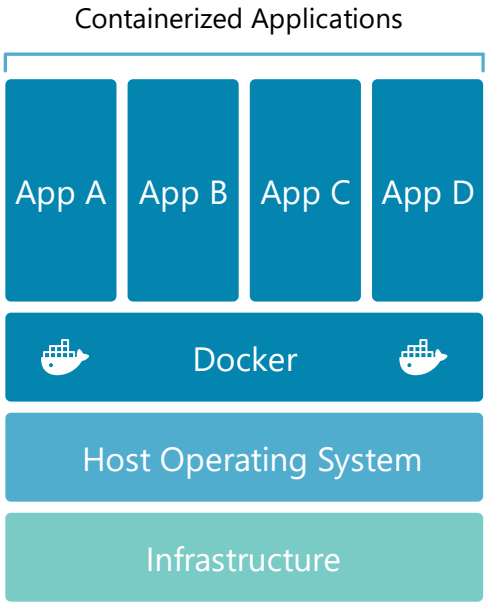



Figure 69 - Container architecture adapted from [234].

Table 5 comprehensively compares virtual machines and containers in terms of isolation process level, operating system, boot-up time, resource usage, pre-built images, customized preconfigured images, size, mobility, creation time, security, diversity, and ease of migration.

Table 5 - Comparison of Virtual Machines and Containers

| | **Virtual Machines** | **Containers** |
|---|---|---|
| **Isolation Process Level** | Hardware | Operating System |
| **Operating system** | Separated | Shared |
| **Boot up time** | Long | Short |
| **Resource usage** | More | Less |
| **Pre-built images** | Hard to find and manage | Readily available |
| **Customized preconfigured images** | Hard to build | Easy to build |

| | | |
|---|---|---|
| **Size** | Bigger because they contain full-sized OS | Smaller with only an engine over the host OS |
| **Mobility** | Easy to move to a new host OS | Destroyed and recreated instead of moving |
| **Creation Time** | Longer | Within seconds |
| **Security** | Each VM runs its OS | Kernel shared |
| **Diversity** | Multiple OS | Host and guest OS must be the same |
| **Booting time** | Slow | Fast, around milliseconds (ms) |
| **Size** | Large, difficulties to copy and push | Lightweight |
| **Migration** | Long migration time, heavy copy, low downtime | Short migration time, a light copy, high downtime, pre-copy not supported |
| **Resources** | Different flavors, shared resources | Ability to provide the number of cores when booting. Normally, more containers than VMs can coexist in a host |

### 6.8.3 DOCKER

The virtualization method chosen for this project was Docker. Docker is an open-source implementation of operating-system-level virtualization regarded as the industry standard for containerization [237], [238]. Docker was introduced in 2013 [238] as a container-based virtualization solution that leveraged existing Linux technologies such as copy-on-write union file systems, Linux Containers (LXC), and control groups (Cgroups), and namespaces along with new functionality and a focus on developers and app development. Making the application the priority is a subtle but fundamental distinguishing factor from traditional machine-level virtualization that revolves around the faithful recreation of hardware. In most cases, the critical asset is the applications, not the specific real or virtual hardware environment [237].

The Docker framework helps developers build applications inside containers, isolating the app and separating its dependencies from infrastructure [234], [238]. Furthermore, Docker's developer-centric toolkit facilitates building, testing, deploying, and managing containerized

applications [239]. With Docker, developers quickly package applications into standard, executable, portable, lightweight, and self-sufficient units named containers that combine everything needed to run them: source code, operating system libraries, system tools, dependencies, settings, and runtime [234]. Docker containers ensure applications run reliably across computing environments, eliminating the classic "works on my machine" problem since containerized software runs the same regardless of infrastructure [238], [240].

Today, Docker is an industry-standard adopted by most cloud providers, making it easier to build, ship, and run containerized applications [239]. Docker provides a containerization method that is more lightweight, agile, and even capable of launching containers in a subsecond. Generally, it is possible to run many more simultaneous Docker instances than virtual machines. Thus, the potential to pack lots of containers onto a physical or a virtual machine translates to immense scalability benefits. According to IBM, the average container is about 26 times faster than VMs [240]. For these reasons, Docker's popularity is such that "containers" and "docker" are used interchangeably [239]. Currently, Docker reports 13 million developers and 13 billion monthly image downloads [241].

### 6.8.4 DOCKER TERMS AND DEFINITIONS

The following paragraphs address terms, definitions, and tools encountered when using Docker. Docker consists of two core concepts: images and containers. A container is a running instance of an image. In the case of Docker, images become containers at runtime when containers run on the Docker engine [234]. Thus, a container in execution can be viewed as a modified instance of the original image [235]. Several containers can come from the original image [237]. Unlike traditional virtualization, a Docker container runs the kernel of the host OS, and a Docker image does not include a separate operating system [236].

#### *6.8.4.1 DOCKERFILE*

A script known as a Dockerfile, which is a text file, is used to define the necessary commands to build a Docker image. The Dockerfile automates the creation of Docker images by providing a list of command-line interface (CLI) instructions interpreted by the Docker engine to assemble the Docker image [239].

#### 6.8.4.2 DOCKER IMAGE

A Docker image contains everything necessary to run an application as a container, including source code, libraries, and dependencies [239]. If an image needs changes, a new one is required because they are immutable [242]. Docker images can be built in two ways: created or pulled from a Docker registry. It's possible to create images from scratch, but pulling suitable ones for the application from registries is more common. Finally, when the Docker run command is executed, the image is created with all the configurations and dependencies necessary and detailed in the Dockerfile.

#### 6.8.4.3 DOCKER CONTAINER

Docker containers are nonpersistent instances spun up from Docker images [242]. Containers are standardized units of software that package code and dependencies, made possible by process isolation and virtualization capabilities built into Linux. The three main Linux kernel capabilities that make Docker containers possible are Linux Containers (LXC), control groups (Cgroups), and namespaces. Firstly, the Linux Containers introduced in 2008 fully enabled virtualization for a single instance of Linux. Secondly, control groups allowed the allocation of resources to processes. Thirdly, namespaces made it possible to restrict the access or visibility of processes to other resources or areas of a system [239].

Docker containers present many benefits, one of their biggest ones being the ability to shut down and respawn gracefully on command. Independently of the reason to go down, be it either crashes or other reasons, they are lightweight, which means starting them has a low impact on resources. Thus, containers can seamlessly appear and disappear and be quickly multiplied to run multiple services or instances of the same service. Further, containers provide both the application and configuration, making installation faster than from a traditional source [243]. Lastly, research shows that Docker containers deliver performance that equals or surpasses VMs [244].

#### 6.8.4.4 DOCKER REGISTRY

Docker registry is a scalable open source-based repository under the Apache license. The registry addresses the challenge of managing, storing, and distributing Docker images. The registry also enhances access control and security of the images it stores. It provides image tracking by version and identification tags using the version control tool git. There are two

ways to utilize the Docker registry: hosting one or by using a hosted service such as Docker Hub, Oracle Container Registry, or Azure Container Registry.

Docker Hub is a public Docker registry hosted by Docker. According to Docker Hub, the platform has "over 100.000 container images from software vendors, open-source projects, and the community" [244]. Most importantly, Docker Hub contains software and applications from official repositories such as Python, Node, Traefik, Nginx, and others that can be downloaded and used as a starting point for any containerization projects. Furthermore, users can create images, upload them into public or private repositories, and share them with collaborators [236], [239].

#### *6.8.4.5 DOCKER ENGINE*

Docker engine is the open-source software containerization technology for building and containerizing applications [242], [245]. This essential part of the Docker system acts as a client-server application with the following:

- Docker daemon: a long-running process dockerd that creates and manages Docker objects, such as images, containers, networks, and volumes [245].
- Docker APIs: RESTful APIs that specify interfaces programs can use to interact with the Docker daemon [245].
- CLI: command-line interface client docker that uses Docker APIs to control or interact with Docker daemon through CLI commands or scripting [245].

Figure 70 shows the Docker architecture. The Docker client interfaces with the Docker Daemon responsible for building, running, and distributing Docker containers. The Docker daemon and client can exist on the same or different machines. An alternative to Docker client is Docker Compose, used in applications consisting of sets of containers [245], [246].

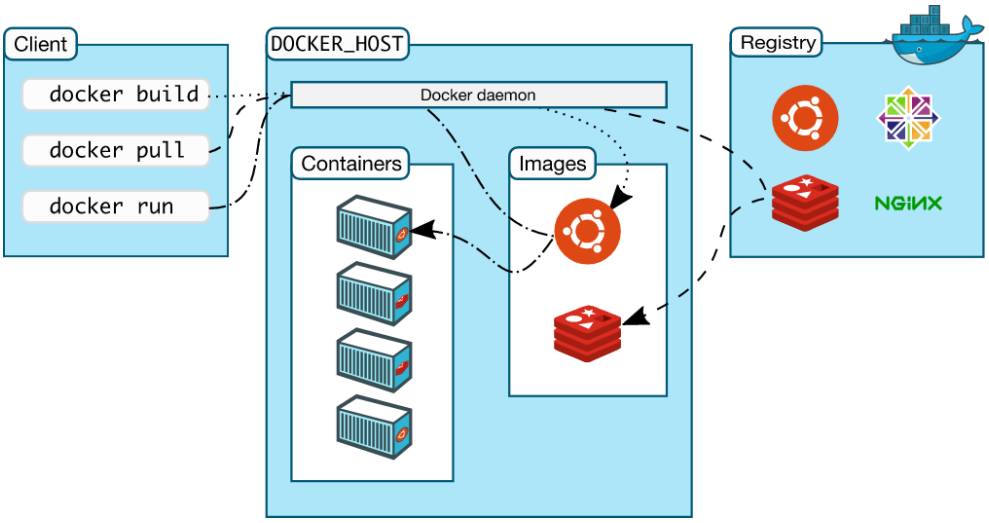



Figure 70 - Docker architecture retrieved from [247].

**Docker daemon**

Docker daemon (dockerd) is a long-running service on an operating system that acts as a control center for a Docker implementation. This service listens to Docker API requests and manages Docker objects. A daemon can communicate with other daemons [247].

**Docker client**

Docker client (docker) is the primary method for many users to engage with Docker. For instance, when using command line instructions such as docker run, the client sends the command to dockerd that subsequently carries them out. The docker command utilizes the Docker API. A Docker client can communicate with multiple daemons [247].

**Docker Objects**

Docker objects include images, containers, networks, volumes, plugins, and other objects created when operating Docker [247].

### 6.8.5 DOCKER FEATURES

Container technology delivers all the functionality and advantages of VMs, such as isolation, scalability, disposability, and further additional benefits listed below. These benefits achieved from enhancing the Linux containerization capabilities make Docker containers the leaders in virtualization technologies:

**Lightweight**

Containers don't include an entire OS and hypervisor, making them lighter than VMs. Containers shine in lightweight applications because they share the host kernel. This characteristic reduces container size, hardware resource impact, and, consequently, the startup and processing times [235], [239].

**Efficient**

Containers can run several instances of an application on the same hardware, maximizing utilization by running more code on the same server and saving money by reducing cloud spending [239], [242].

**Portable**

Docker containers can seamlessly move from development to production machines, unlike Linux containers, which frequently need machine-specific configurations [239]. The Docker environment enables building, uploading, downloading, deploying, and executing Docker images anywhere [235].

**Productive**

Containers offer faster and more efficient deployment, provisioning, and restart capabilities than Virtual Machines. According to Amazon Web Services, Docker users ship software seven times more frequently, on average, than non-Docker users [242].

**Automated**

Docker can automatically build containers based on app source code [239].

**Trackable**

Docker enables container image version tracking, rollbacks to previous versions, and uploads of deltas between current and new versions [239].

**Reusable**

Containers can serve as building blocks for creating new containers by using them as templates [239].

**Flexible**

Containers can vary broadly from lightweight instances with few libraries to heavy ones with images that can reach gigabytes in size [235].

**Interchangeable**

Each Docker container runs only one process, making it possible to build applications that can continue running while one of its components is updated or repaired [239]. Updates and

upgrades can even be done while the application is running. After deploying a new version, it can seize the tasks of the older one [235].

**Scalable**

A variation in the number of instances is easily defined, automated, and even dynamically adjusted to user requirements [235].

**Stackable**

Docker creates stacks when multiple services share dependencies and orchestration is possible. Stacks save time, resources, and container size by grouping services together. Creating stacks while the application is running is possible [235].

**Standard**

Deployment, issue identification, and rollbacks are made more accessible by the Docker container standard [242].

**Shareable**

Open-source Docker registries allow developers to access thousands of container images [239].

### 6.8.6 MULTI-CONTAINER DEPLOYMENT

Docker delivers tools to simplify the creation of complex applications with several containers linked to each other. Docker allows multi-container applications to deploy as smoothly as a single logical application using Docker Compose or Docker Swarm. An example of such an application could be a database, an API, a web app, a load balancer, and a proxy. Docker Compose enables the creation of single-host applications and is particularly efficient for development because it can create images on the fly. On the other hand, Docker Swarm is better suited for larger deployments with multiple hosts but requires already compiled images [235].

#### *6.8.6.1 DOCKER COMPOSE*

While a Dockerfile configures individual images, Docker Compose uses a YAML file to define and run multi-container Docker applications with a single command. This YAML file includes, for example, Dockerfiles or images to use, network configurations, and dependencies. Firstly, a network allows containers to communicate. Secondly, a dependency parameter ensures container startup in the order defined [235]. Lastly, Docker Compose can also define volumes for persistent storage [239].

### 6.8.7 GUI TOOLS FOR MANAGING DOCKER ENVIRONMENTS

Docker API allows many options to interact with the Docker engine, containers, and images [248]. These range from web applications like Portainer to desktop applications like Docker Desktop.

#### *6.8.7.1 DOCKER DESKTOP*

Docker Desktop is an easy-to-install application available for Mac, Windows, and Linux (beta) [249], enabling the ability to build, manage and share containerized applications. Docker Desktop includes Docker Engine, Docker CLI client, and Docker Compose [250]. The dashboard from Figure 71 gives access to basic container operations, logs, stats, and inspection of containers without needing the CLI.

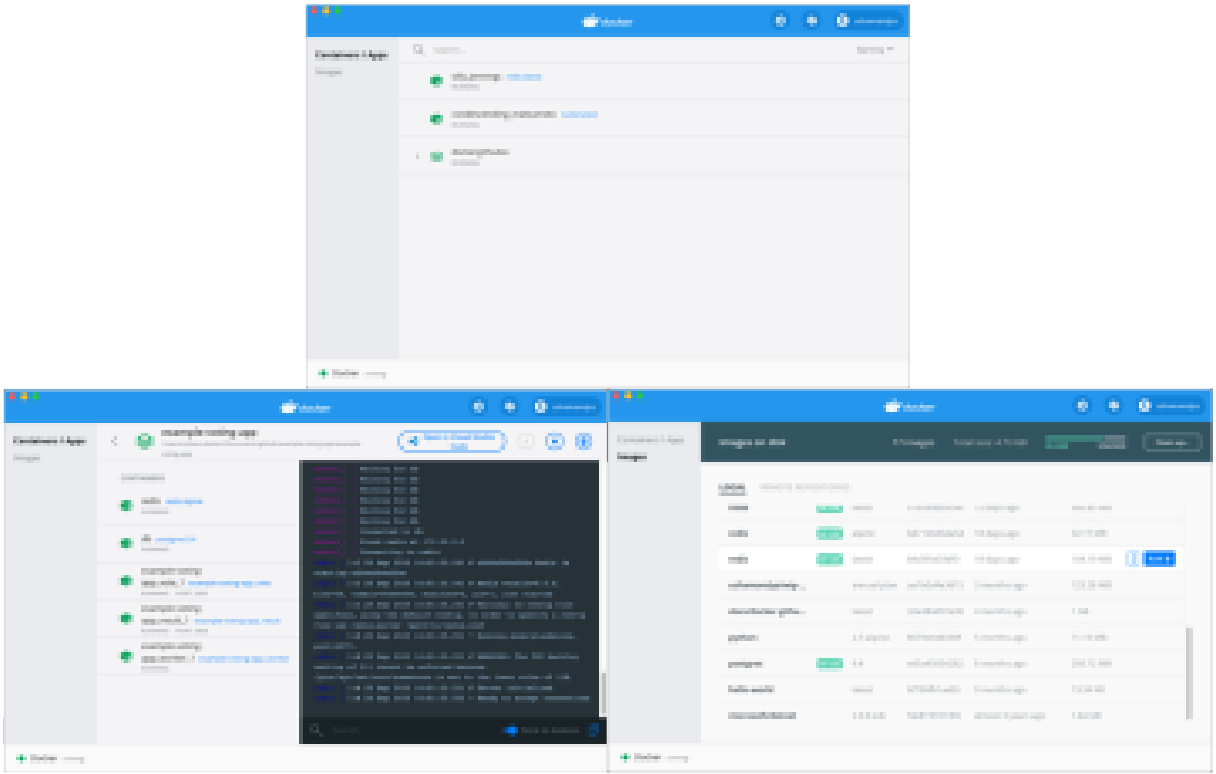


Figure 71 - Docker desktop dashboard.

#### *6.8.7.2 PORTAINER*

Portainer is an open-source tool for managing containerized applications. Portainer is a web GUI (Figure 72) for Docker, Docker Swarm, Kubernetes, and more, that removes the complexities of using the CLI and gives a visual representation of a container environment [251].

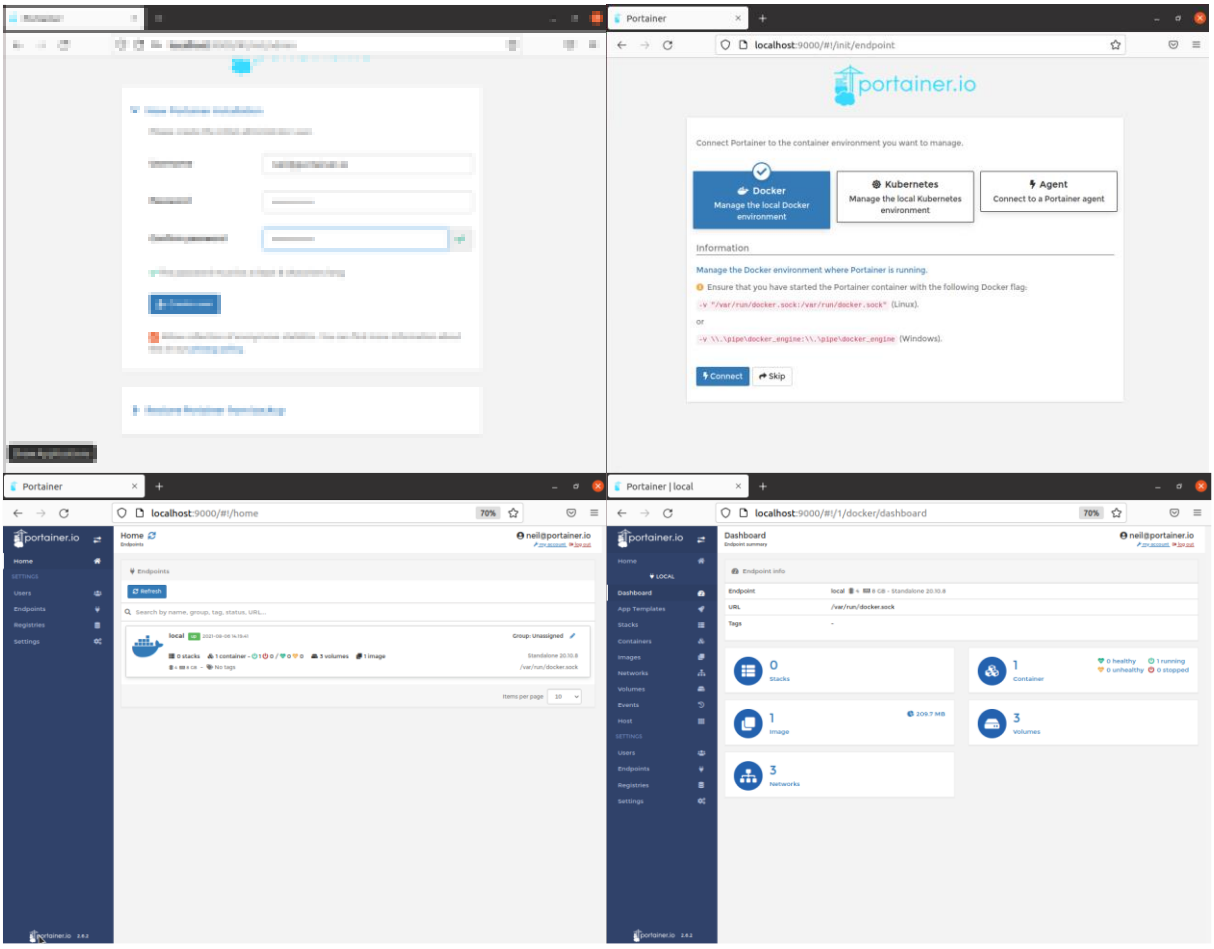



Figure 72 - Portainer dashboard.

# 7 APPENDIX 2 - "USE CASES"

## 7.1 USE CASE 1 - REGISTER

Table 6 - Use Case 1 - Register

| | | |
|---|---|---|
| **Use Case** | UC01 - Register | |
| **Description** | The user provides information to complete the registration process. | |
| **Precondition** | The user is not registered. | |
| **Postcondition** | The user is registered. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** User provides name, email, and password. | |
| | | **2.** System validates the information. |
| | | **3.** System stores new user information |
| | | **4.** System sends a success message. |
| **Exception 1 [invalid email] (step 2)** | | **2.1** System informs that the email is already registered. |
| | | **2.2** System cancels registration process. |

## 7.2 USE CASE 2 - LOGIN

Table 7 - Use Case 2 - Login

| | | |
|---|---|---|
| **Use Case** | UC02 - Login | |
| **Description** | User completes the authentication challenge. | |
| **Precondition** | The user is not authenticated. | |
| **Postcondition** | The user is authenticated. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** User provides an email and password. | |
| | | **2.** System validates the information. |
| | | **3.** System authenticates the user. |
| **Exception [invalid credentials] (step 2)** | | **2.1** System informs that credentials are invalid. |
| | | **2.2** System cancels login. |

## 7.3 USE CASE 3 - LOGOUT

Table 8 - Use Case 3 - Logout

| | | |
|---|---|---|
| **Use Case** | UC03 - Logout | |
| **Description** | The user exits the application. | |
| **Precondition** | The user is authenticated. | |
| **Postcondition** | The user is not authenticated. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** User ends the session. | |
| | | **2.** System disconnects the user from the application. |

## 7.4 USE CASE 4 – UPDATE USER INFORMATION

Table 9 - Use Case 4 - Update User Information

| | | |
|---|---|---|
| **Use Case** | UC04 - Update user information | |
| **Description** | User updates personal information (name, email, or password). | |
| **Precondition** | The user is authenticated. | |
| **Postcondition** | User information is updated. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** User provides the name and/or email and/or password. | |
| | | **2.** System validates the information. |
| | | **3.** System updates user information. |
| | | **4.** System sends a success message. |
| **Exception 1 [empty fields] (step 1)** | | **1.1** System informs that no update information was provided. |
| | | **1.2** System cancels update. |
| **Exception 2 [invalid email] (step 2)** | | **2.1** System informs that the email is already registered. |
| | | **2.2** System cancels update. |

## 7.5 USE CASE 5 – CREATE USER

Table 10 - Use Case 5 - Create User

| | | |
|---|---|---|
| **Use Case** | UC05 – Create user | |
| **Description** | Admin creates a user with any role. | |
| **Precondition** | Admin is authenticated. | |
| **Postcondition** | A user is created. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** Admin provides a name, email, password, and role. | |
| | | **2.** System validates the information. |
| | | **3.** System stores new user information. |
| | | **4.** System sends a success message. |
| **Exception 1 [invalid email] (step 2)** | | **2.1** System informs that the email is already registered. |
| | | **2.2** System cancels registration process. |

## 7.6 USE CASE 6 – UPDATE USER ROLE

Table 11 - Use Case 6 - Update User Role

| | | |
|---|---|---|
| **Use Case** | UC06 – Update user role | |
| **Description** | Admin updates the role of a user. | |
| **Precondition** | Admin is authenticated. | |
| **Postcondition** | The role of a user is updated. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** Admin selects a role. | |
| | | **2.** System validates the information. |
| | | **3.** System updates the role of the user. |
| | | **4.** System sends a success message. |

## 7.7 USE CASE 7 – DELETE ACCOUNT

Table 12 - Use Case 7 - Delete Account

| | |
|---|---|
| **Use Case** | UC07 - Delete account |
| **Description** | A user deletes the account and all data submitted and created since registration. |

| Precondition | The user is registered and authenticated. | |
|---|---|---|
| Postcondition | The user is not registered. | |
| | Actor | System |
| Normal scenario | **1.** User communicates an intention to delete the account. | |
| | | **3.** System requires confirmation. |
| | **3.** User confirms action to delete account. | |
| | | **4.** System deletes projects owned by the user. |
| | | **5.** System revokes access to shared projects. |
| | | **6.** System deletes all user information. |
| | | **6.** System sends a success message. |

## 7.8 USE CASE 8 – CREATE PROJECT

Table 13 - Use Case 8 - Create Project

| Use Case | UC08 - Create Project | |
|---|---|---|
| Description | User or Admin provides the necessary information and files to create a project. | |
| Precondition | User or Admin is authenticated. | |
| Postcondition | User or Admin owns a project. | |
| | Actor | System |
| Normal scenario | **1.** User or Admin provides title, description, input type, output type, and privacy status. | |
| | **2.** User or Admin provides Python script. | |
| | **3.** User or Admin provides 0 or more model files. | |
| | | **4.** System validates the information. |
| | | **5.** System stores project data. |
| | | **6.** System sends a success message. |

## 7.9 USE CASE 9 – CHECK PROJECTS

Table 14 - Use Case 9 - Check Projects

| Use Case | UC09 – Check Projects | |
|---|---|---|
| Description | The user consults a list of projects. | |
| Precondition | The user is authenticated. | |
| Postcondition | The user sees the list of owned projects. | |
| | Actor | System |
| Normal scenario | **1.** A user asks to see a list of projects. | |
| | | **2.** System shows the list of projects. |
| Alternative Scenario [owned projects] (step1) | **1.1** User or Admin asks to see a list of owned projects. | |
| | | **1.2** Go to 2. |
| Alternative Scenario [shared projects] (step1) | **1.1** A user asks to see a list of shared projects. | |
| | | **1.2** Go to 2. |
| Alternative Scenario [public projects] (step1) | **1.1** A user asks to see a list of public projects. | |
| | | **1.2** Go to 2. |

## 7.10 USE CASE 10 – UPDATE PROJECT

Table 15 - Use Case 10 - Update Project

| Use Case | UC10 - Update Project | |
|---|---|---|
| Description | User or Admin edits a project. | |
| Precondition | User or Admin is authenticated. | |
| Postcondition | User or Admin updates project. | |
| | Actor | System |
| Normal scenario | **1.** User or Admin provides new information and/or files. | |
| | **2.** User or Admin confirms action to delete the project. | |
| | | **4.** System validates the information. |
| | | **5.** System updates the project. |
| | | **6.** System sends a success message. |

## 7.11 USE CASE 11 – SHARE PROJECT

Table 16 - Use Case 11 - Share Project

| | | |
|---|---|---|
| **Use Case** | UC11 - Share Project | |
| **Description** | User or Admin shares a project. | |
| **Precondition** | User or Admin is authenticated. | |
| **Postcondition** | The project selected is shared. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** User or Admin selects a project to share. | |
| | **2.** User or Admin provides the email with whom to share the project. | |
| | | **3.** System validates the information. |
| | | **4.** System shares the project. |
| | | **5.** System sends a success message. |
| **Alternative Scenario [publish project] (step2)** | **2.1** User publishes project to share it with all users. | |
| | | **2.2** Go to 4. |
| **Exception 1 [invalid email] (step 3)** | | **3.1** System verifies the email is not registered. |
| | | **3.2** System sends an error message. |
| **Exception 2 [public project] (step 4)** | | **3.1** System verifies the project is already public. |
| | | **3.2** System sends an error message. |

## 7.12 USE CASE 12 – DELETE PROJECT

Table 17 - Use Case 12 - Delete Project

| | | |
|---|---|---|
| **Use Case** | UC12 - Delete Project | |
| **Description** | User or Admin deletes a project. | |
| **Precondition** | User or Admin is authenticated. | |
| **Postcondition** | Project selected by the user is eliminated. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** User or Admin communicates an intention to delete the project. | |
| | | **2.** System requires confirmation. |

| | | |
|---|---|---|
| | **2.** User or Admin confirms action to delete the project. | |
| | | **4.** System deletes project. |
| | | **5.** System sends a success message. |

## 7.13 USE CASE 13 – RUN PROJECT

Table 18 - Use Case 13 - Run Project

| | | |
|---|---|---|
| **Use Case** | UC13 - Run Project | |
| **Description** | The user runs a project. | |
| **Precondition** | The user is authenticated. | |
| **Postcondition** | The user receives an output. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** A user provides an input file. | |
| | | **2.** System validates the information. |
| | | **3.** System runs the project. |
| | | **4.** System returns an output file. |
| **Exception [validation error] (step 2)** | | **2.1** Input file extension differs from the type expected. |
| **Exception [runtime error] (step 3)** | | **3.1** The output provided by the system is a log file with errors. |

## 7.14 USE CASE 14 – CHECK RUN HISTORY

Table 19 - Use Case 14 - Check Run History

| | | |
|---|---|---|
| **Use Case** | UC14 – Check Run History | |
| **Description** | The user consults the Run History List. | |
| **Precondition** | The user is authenticated. | |
| **Postcondition** | The user sees the run history list. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** A user asks to see the Run History List. | |
| | | **2.** System shows the Run History List. |

## 7.15 USE CASE 15 – CREATE FLAG

Table 20 - Use Case 15 - Create Flag

| | | |
|---|---|---|
| **Use Case** | UC15 – Create flag. | |
| **Description** | The user creates a flag. | |
| **Precondition** | The user is authenticated. | |
| **Postcondition** | A flag is created. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** A user decides to flag a project run where the output file is incorrect, unexpected, etc. | |
| | **2.** A user submits a flag. | |
| | | **3.** System stores flag data. |
| | | **4.** System returns a success message. |
| **Alternative Scenario [flag description] (step2)** | **2.1** A user also submits an optional flag description. | |
| | | **2.2** Go to 3. |

## 7.16 USE CASE 16 – CHECK PROJECT FLAGS

Table 21 - Use Case 16 - Check Project Flags

| | | |
|---|---|---|
| **Use Case** | UC16 – Check Project Flags | |
| **Description** | User or Admin consults the project flags. | |
| **Precondition** | User or Admin is authenticated. | |
| **Postcondition** | User or Admin sees the project flags. | |
| | **Actor** | **System** |
| **Normal scenario** | **1.** User or Admin asks to see the project flags. | |
| | | **2.** System shows the project flags. |

# 8 APPENDIX 3 - "REQUIREMENTS"

# 8.1 FUNCTIONAL REQUIREMENTS

The following requirements (FR) explain the functionalities that will be available to the users of the system. They describe what is expected as an answer to a specific input. However, this form of requirement is free of design and implementation considerations to expand the potential pool of solutions.

### 8.1.1 REGISTER USER

**Function:** Allow the user to register a "Guest" account.

**Description:** Allow the user to navigate the app after creating an account with email and password.

**Inputs:** Email and password provided by the user.

**Source:** Inputs provided by the user.

**Output**: Success/failure message.

**Action**: The necessary data to register in the system must be requested and stored in the database.

**Requirements**: The email entered by the user cannot already be present in the database.

**Precondition**: None.

**Post-Condition**: The user is registered in the system.

| | |
|---|---|
| **Requirement:** FR01 | **Use Case:** UC01 |
| **Description:** The user signs up by providing a name, email, and password. | |
| **Reasoning:** The system requires new users to register an account to access platform functionalities. | |
| **Fit Criterion:** A user must be able to create an account with personal data. It's impossible to create an account with an email already registered in the system. | |

### 8.1.2 AUTHENTICATE USER

**Function**: Allow the user to log into the application.

**Description**: Allows the user to operate the application after validating their access data.

**Inputs**: Email and password provided by the user.

**Source**: Inputs provided by the user.

**Output**: Success/failure message and security token.

**Action**: The system should request the necessary data to log in to the system, verify the access data matches a user present in the system, and validate the credentials. The authentication fails if this validation is incomplete.

**Requirements**: The credentials provided by the user must already be present in the system.

**Precondition**: The user is not logged in.

**Post-Condition**: The user is authenticated in the system.

| | |
|---|---|
| **Requirement:** FR02 | **Use Case:** UC02 |
| **Description:** A registered user completes the authentication challenge. | |
| **Reasoning:** The system requires authentication. | |
| **Fit Criterion:** A registered user must be able to complete authentication by providing valid credentials. | |

### 8.1.3 UPDATE USER INFORMATION

**Function**: Update user information.

**Description**: Allows the user to edit the information provided when the account was created.

**Inputs**: User data.

**Source**: Inputs entered by the user.

**Output**: Success/failure message.

**Action**: The system must present the editable data to the user. The system asks for confirmation of data change. Data is changed and stored in case of confirmation and discarded otherwise.

**Requirements**: Inputs are expected to update user data.

**Precondition**: The user is logged in.

**Post-Condition**: User data is updated.

| | |
|---|---|
| **Requirement:** FR03 | **Use Case:** UC04 |
| **Description:** A user updates the information provided at registration. | |
| **Reasoning:** The system allows the update of user information. | |
| **Fit Criterion:** A registered user must be able to edit their name, email, and password. | |

### 8.1.4 CREATE USER

**Function**: Create a user account.

**Description**: Allows the "Admin" to create an account with any role for a user.

**Inputs**: User name, email, password, and role.

**Source**: Inputs entered by the "Admin."

**Output**: Success/failure message.

**Action**: The system must present a form to fill in with user data. Upon submission, the system must store the information.

**Requirements**: The email entered by the user cannot already be present in the database.

**Precondition**: The "Admin" is logged in.

**Post-Condition**: A user account with any desired role is created.

| | |
|---|---|
| **Requirement:** FR04 | **Use Case:** UC05 |
| **Description:** An "Admin" creates a user account by providing a name, email, password, and role. | |
| **Reasoning:** The system requires an "Admin" to create users with roles other than "Guest." | |
| **Fit Criterion:** An "Admin" must be able to create an account with user data. Creating an account with an email already registered in the system is impossible. | |

### 8.1.5 UPDATE USER ROLE

**Function**: Update user role.

**Description**: Allows the "Admin" to update a user's role.

**Inputs**: User role.

**Source**: Inputs entered by the "Admin."

**Output**: Success/failure message.

**Action**: The system must present the role options. Upon submission, the system must update the role information.

**Requirements**: None.

**Precondition**: The "Admin" is logged in.

**Post-Condition**: A user with an updated role.

| | |
|---|---|
| **Requirement:** FR05 | **Use Case:** UC06 |
| **Description:** An "Admin" updates the role of a user. | |
| **Reasoning:** The system requires an "Admin" to create users with roles other than "Guest." | |
| **Fit Criterion:** An "Admin" must be able to update user roles. | |

### 8.1.6 DELETE ACCOUNT

**Function**: Delete user account.

**Description**: Allows the user to delete the account.

**Inputs**: User data.

**Source**: Inputs entered by the user.

**Output**: Success/failure message.

**Action**: The system asks for confirmation to delete the account. User data and all projects owned by the user are deleted.

**Requirements**: Confirmation to delete the user account.

**Precondition**: The user is logged in.

**Post-Condition**: The user account and all projects owned by the user are deleted.

| | |
|---|---|
| **Requirement:** FR06 | **Use Case:** UC07 |
| **Description:** A user deletes their account with all data provided (personal information, projects, project files, input, and output files). | |
| **Reasoning:** The system allows user to delete their accounts. | |
| **Fit Criterion:** An authenticated user must be able to delete every piece of data shared with the system. | |

### 8.1.7 CREATE PROJECT

**Function**: Upload the projects, python code required to run the models, a title, and a description of how the project is used.

**Description**: Allows a "User" or "Admin" to choose a title and description and upload the code needed to run the project. The user must also choose the type of input and the type of output.

**Inputs**: Title, description, models, python code, type of input, and type of output.

**Source**: Inputs entered by the "User" or "Admin".

**Output**: Success/failure message and AI model ID.

**Action**: The system must present the title and description fields that the "User" or "Admin" can fill in. The system must store models and Python code.

**Requirements**: Inputs are expected to create a new project.

**Precondition**: The "User" or "Admin" is logged in.

**Post-Condition**: A new project is created.

| | |
|---|---|
| **Requirement:** FR07 | **Use Case:** UC08 |
| **Description:** A "User" or "Admin" creates a project. | |
| **Reasoning:** The system allows the "User" or "Admin" to create projects. | |

**Fit Criterion:** An authenticated "User" or "Admin" must be able to create a project by completing a form.

### 8.1.8 UPDATE PROJECT

**Function**: Update project information.

**Description**: This allows the "User" or "Admin" to edit the information provided when the project was created.

**Inputs**: Project data.

**Source**: Inputs entered by the "User" or "Admin".

**Output**: Success/failure message.

**Action**: The system must present the editable data to the "User" or "Admin". The system asks for confirmation of data change. Data is changed and stored in case of confirmation and discarded otherwise.

**Requirements**: Inputs are expected to update project data.

**Precondition**: The "User" or "Admin" is logged in.

**Post-Condition**: Project data is updated.

**Requirement:** FR08 **Use Case:** UC 10

**Description:** A "User" or "Admin" updates project information.

**Reasoning:** The system allows users to update project information.

**Fit Criterion:** An authenticated user must be able to update project information.

### 8.1.9 PYTHON SCRIPT UPLOAD

**Function**: Upload a Python script that will be associated with a specific project.

**Description**: Allows the "User" or "Admin" to choose one Python script and upload it to the server where it will be associated with a project. More specifically, it will be stored in the folder created to store all files of a model.

**Inputs**: Project ID, Python script.

**Source**: Inputs entered by the "User" or "Admin" and ID created from “CREATE PROJECT”.

**Output**: Success/failure message.

**Action**: The system must present an area where the user can drop the file. The system must store the Python code in the folder associated with the model specified.

**Requirements**: Drop a file in the designated area and submit that file.

**Precondition**: The "User" or "Admin" is logged in and is the owner of the project.

**Post-Condition**: A Python script is uploaded to the server.

| | |
|---|---|
| **Requirement:** FR09 | **Use Case:** UC08 |
| **Description:** A "User" or "Admin" uploads a Python script. | |
| **Reasoning:** The system allows the "User" or "Admin" to upload a Python script associated with the specified project. This file is required to run the project later. | |
| **Fit Criterion:** An authenticated user must be able to upload a Python script that becomes part of the specified project. | |

### 8.1.10 MODEL FILES UPLOAD

**Function**: Upload model files that will be associated with a specific project.

**Description**: Allows the "User" or "Admin" to choose model files and upload them to the server where they will be associated with a project. More specifically, it will be stored in the folder created to store all files of a project.

**Inputs**: Project ID, model files.

**Source**: Inputs entered by the "User" or "Admin".

**Output**: Success/failure message.

**Action**: The system must present an area where the user can drop the files. The system must store the model files in the folder associated with the project specified.

**Requirements**: Drop a file in the designated area and submit that file.

**Precondition**: The "User" or "Admin" is logged in and is the owner of the project.

**Post-Condition**: Model files are uploaded to the server.

| | |
|---|---|
| **Requirement:** FR10 | **Use Case:** UC08 |
| **Description:** A "User" or "Admin" uploads model files. | |
| **Reasoning:** The system allows the "User" or "Admin" to upload model files associated with the specified project. This file may be required to run the project later. | |
| **Fit Criterion:** An authenticated "User" or "Admin" must be able to upload model files that become part of the specified project. | |

### 8.1.11 LIST OWNED PROJECTS

**Function**: Lists all the projects owned by the "User" or "Admin".

**Description**: This allows the "User" or "Admin" to view the list of the projects that he owns.

**Inputs**: None.

**Source**: None.

**Output**: Success/failure message and if a successful list of projects.

**Action**: The system must present the ID, creation date, title, author, privacy, input type, output type, and description of each project listed.

**Requirements**: None.

**Precondition**: The "User" or "Admin" is logged in.

**Post-Condition**: A list of owned projects is presented.

| | |
|---|---|
| **Requirement:** FR11 | **Use Case:** UC09 |
| **Description:** A user sees the list of owned projects. | |
| **Reasoning:** The system allows users to see a list of owned projects that makes project actions accessible. | |

**Fit Criterion:** An authenticated user must be able to see a list of owned projects.

### 8.1.12 LIST SHARED PROJECTS

**Function**: Lists all the projects shared with the user.

**Description**: This allows the user to view the list of the projects that were shared with him.

**Inputs**: None.

**Source**: None.

**Output**: Success/failure message and if a successful list of projects.

**Action**: The system must present the ID, creation date, title, author, privacy, input type, output type, and description of each project listed.

**Requirements**: None.

**Precondition**: The user is logged in.

**Post-Condition**: A list of shared projects is presented.

| | |
|---|---|
| **Requirement:** FR12 | **Use Case:** UC09 |
| **Description:** A user sees the list of shared projects. | |
| **Reasoning:** The system allows users to see a list of shared projects that makes project actions accessible. | |
| **Fit Criterion:** An authenticated user must be able to see a list of shared projects. | |

### 8.1.13LIST PUBLIC PROJECTS

**Function**: Lists all the public projects.

**Description**: This allows the user to view the list of public projects.

**Inputs**: None.

**Source**: None.

**Output**: Success/failure message and if a successful list of projects.

**Action**: The system must present the ID, creation date, title, author, privacy, input type, output type, and description of each project listed.

**Requirements**: None.

**Precondition**: The user is logged in.

**Post-Condition**: A list of public projects is presented.

| | |
|---|---|
| **Requirement:** FR13 | **Use Case:** UC09 |
| **Description:** A user sees the list of public projects. | |
| **Reasoning:** The system allows users to see a list of public projects that makes project actions accessible. | |
| **Fit Criterion:** An authenticated user must be able to see a list of public projects. | |

### 8.1.14 SHARE PROJECT

**Function**: Share project.

**Description**: Allows the "User" or "Admin" to share a project he created with another registered user.

**Inputs**: Target user email.

**Source**: Inputs entered by the "User" or "Admin".

**Output**: Success/failure message.

**Action**: The system must share a project with the target user. The system asks for confirmation of project sharing. The project is shared.

**Requirements**: Inputs are expected to identify the target user.

**Precondition**: The "User" or "Admin" is logged in.

**Post-Condition**: The project is shared with the target user.

| | |
|---|---|
| **Requirement:** FR14 | **Use Case:** UC11 |
| **Description:** A "User" or "Admin" shares a project with another user by providing an email. | |

**Reasoning:** The system allows the "User" or "Admin" to share projects with other registered users by providing their email.

**Fit Criterion:** An authenticated "User" or "Admin" must be able to share a project. The user with whom the project has been shared can access project information and run the project.

### 8.1.15 MAKE PROJECT PUBLIC

**Function**: Make a project public.

**Description**: Allows a "User" or "Admin" to share a project with every registered user in the system.

**Inputs**: None.

**Source**: None.

**Output**: Success/failure message.

**Action**: The system must make the project available to every registered user. The project is made public.

**Requirements**: Inputs are expected to identify the target project.

**Precondition**: The "User" or "Admin" is logged in.

**Post-Condition**: The project is made public.

**Requirement:** FR15 **Use Case:** UC11

**Description:** A "User" or "Admin" toggles privacy to make a project public.

**Reasoning:** The system allows the "User" or "Admin" to share a project with every registered user.

**Fit Criterion:** An authenticated "User" or "Admin" must be able to toggle project privacy between private and public. A public project can be accessed and run by any registered user.

### 8.1.16 DELETE PROJECT

**Function**: Deletes the target project.

**Description**: Allows the "User" or "Admin" to delete a project.

**Inputs**: None.

**Source**: None.

**Output**: Success/failure message.

**Action**: The system must search the database for a project that matches the one to be deleted.

**Requirements**: Inputs are expected to delete the target project.

**Precondition**: The "User" or "Admin" is logged in. The "User" or "Admin" owns the project.

**Post-Condition**: The project is deleted.

| | |
|---|---|
| **Requirement:** FR16 | **Use Case:** UC12 |
| **Description:** A "User" or "Admin" deletes a project. | |
| **Reasoning:** The system allows the "User" or "Admin" to delete a project that includes project information and Python script, and may also include model files. | |
| **Fit Criterion:** An authenticated "User" or "Admin" must be able to delete a project. Shared projects deleted are no longer accessible to the users with access. Public projects deleted are no longer accessible by users. | |

### 8.1.17 RUN PROJECT

**Function**: Runs a project.

**Description**: Allows the user to upload an input file, run the project, and retrieve the result.

**Inputs**: Input file.

**Source**: File provided by the user.

**Output**: Success/failure message and the output from the project.

**Action**: The system receives a file to input into the Python script and responds with the output.

**Requirements**: File input by the user.

**Precondition**: The user is logged in.

**Post-Condition**: The project runs and provides an output.

| | |
|---|---|
| **Requirement:** FR17 | **Use Case:** UC13 |
| **Description:** A user runs a project. | |
| **Reasoning:** The system allows users to provide an input file with the required input type for a specific project. The project processes the input file with the Python script and associated model files. Finally, the system provides an output file. | |
| **Fit Criterion:** An authenticated user must be able to run a project. In other words, a user must be able to submit an input file and receive the appropriate output. | |

### 8.1.18 CHECK RUN HISTORY

**Function**: Lists all the project runs created by the user.

**Description**: This allows the user to view the list of the projects that were shared with him.

**Inputs**: None.

**Source**: None.

**Output**: Success/failure message and if a successful list of project runs.

**Action**: The system should display the project name, owner's name and email, input and output data, time of execution, and any flagged issues with a description, if applicable.

**Requirements**: None.

**Precondition**: The user is logged in.

**Post-Condition**: A list of project runs is presented.

| | |
|---|---|
| **Requirement:** FR18 | **Use Case:** UC14 |
| **Description:** A user sees the list of project runs. | |
| **Reasoning:** The system allows users to see a list of project runs. | |
| **Fit Criterion:** An authenticated user must be able to see a list of project runs. | |

### 8.1.19 CREATE FLAG

**Function**: Create a flag for a specific project run.

**Description**: Allows users to create a flag and an optional flag description for a specific project run.

**Inputs**: Flag, flag description.

**Source**: Inputs provided by the user.

**Output**: Success/failure message.

**Action**: The system should display a text box with a submission button. The system must store the information.

**Requirements**: None.

**Precondition**: The user is logged in.

**Post-Condition**: A project run with a flag and optional flag description.

| | |
|---|---|
| **Requirement:** FR19 | **Use Case:** UC15 |
| **Description:** A user creates a flag. | |
| **Reasoning:** The system allows users to create flags. | |
| **Fit Criterion:** An authenticated user must be able to create a flag in association with a project run. | |

### 8.1.20 CHECK PROJECT FLAGS

**Function**: Lists all the flags associated with a project.

**Description**: This allows the "User" or "Admin" to review the list flags created by other users for the projects they own.

**Inputs**: Project id.

**Source**: Inputs provided by the "User" or "Admin".

**Output**: Success/failure message and if a successful list of project flags.

**Action**: The system should display a list of flags and respective flag descriptions.

**Requirements**: The "User" or "Admin" owns the project.

**Precondition**: The user is logged in.

**Post-Condition**: A list of project runs is presented.

| | |
|---|---|
| **Requirement:** FR20 | **Use Case:** UC16 |
| **Description:** A "User" or "Admin" sees the list of project flags. | |
| **Reasoning:** The system allows users to see a list of project flags. | |
| **Fit Criterion:** An authenticated "User" or "Admin" must be able to see a list of project flags for projects they own. | |

## 8.2 NON-FUNCTIONAL REQUIREMENTS

These non-functional requirements (NFR) define the properties and constraints of the system, e.g., reliability, response time, and storage requirements. Constraints are the device's ability to input/output, system representations, etc. The following non-functional requirements were defined:

1. In terms of performance, the application must be as efficient as possible to reduce any delay that might exist. Project outputs must be sent in the shortest time possible.
2. The software must be supported by any browser.
3. The system must guarantee minimum security/privacy requirements. Any information entered by the user must be strictly used for app operations and never disclosed.
4. Finally, the application must be easily accessible, user-friendly, and always available.

### 8.2.1 LOOK AND FEEL REQUIREMENTS

**Appearance**

| |
|---|
| **Requirement:** NFR01 |
| **Description:** The system's appearance must conform to the user's expectations. |

**Reasoning:** The system shall look professional and be attractive to users.

**Fit Criterion:** The dissertation supervisor shall approve that the platform complies with current standards.

**Style**

**Requirement:** NFR02

**Description:** The system must look trustworthy.

**Reasoning:** The system shall inspire users to trust the system to handle sensitive data.

**Fit Criterion:** There must be a transparent reference to the organizations that support the project. This connection will allow users to transfer the existing trust in those organizations to FMdeploy.

### 8.2.2 USABILITY REQUIREMENTS

**Ease of use**

**Requirement:** NFR03

**Description:** The system must be accessible to users without great technological experience.

**Reasoning:** This shall promote a fluid user experience, minimize the time required to use core features, and reduce unintended use cases that may lead to errors.

**Requirement:** NFR04

**Description:** All core system functionalities must be accessible in less than five clicks.

**Reasoning:** The system shall not create time constraints for the user. The system shall have easy handling so as not to bore users.

**Learning**

**Requirement:** NFR05

**Description:** The system shall be able to be used without any training.

**Reasoning:** The system shall be intuitive to users and constructed so that all its functionality is apparent on the first encounter.